Date Published: April 01, 2026

Visit the Zebra Product Support page for the most up-to-date version of this documentation.

M3dim functions

Synopsis

The functions prefixed with M3dim are responsible for 3D image processing containers that hold 3D data. The main components of a container (M_CONTAINER) are M_COMPONENT_RANGE and M_COMPONENT_CONFIDENCE. Optional components are M_COMPONENT_REFLECTANCE, M_COMPONENT_INTENSITY, M_COMPONENT_NORMALS_AIL and M_COMPONENT_MESH_AIL.

Functions

M3dimAlloc
M3dimAllocResult
M3dimArith
M3dimCalculateMapSize
M3dimCalibrateDepthMap
M3dimControl
M3dimConvertDepthMap
M3dimCopy
M3dimCopyResult
M3dimCrop
M3dimDraw3d
M3dimFillGaps
M3dimFilter
M3dimFindTransformation
M3dimFix
M3dimFree
M3dimGet
M3dimGetList
M3dimGetResult
M3dimInquire
M3dimLattice
M3dimMatrixTransform
M3dimMatrixTransformList
M3dimMerge
M3dimMesh
M3dimNormals
M3dimOutliers
M3dimProfile
M3dimProfileEx
M3dimProject
M3dimProjectEx
M3dimRemapDepthMap
M3dimRemovePoints
M3dimRestore
M3dimRotate
M3dimSample
M3dimSave
M3dimScale
M3dimStat
M3dimStream
M3dimTranslate

M3dimAlloc

Synopsis

Allocate a 3D image processing context.

Syntax

```
AIL_ID M3dimAlloc(
    AIL_ID SysId,            //in
    AIL_INT64 ContextType,   //in
    AIL_INT64 ControlFlag,   //in
    AIL_ID *Context3dimIdPtr //out
)
```

Description

This function allocates a 3D image processing context on the specified system.

When the 3D image processing context is no longer required, release it using M3dimFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the 3D image processing context's lifetime and you must not manually free it.

Parameters

SysId

Specifies the identifier of the system on which to allocate the 3D image processing context.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ContextType

Specifies the type of 3D image processing context to allocate. This parameter must be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the context type																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_CALCULATE_MAP_SIZE_CONTEXT	Specifies to allocate a 3D image processing context that can be used with M3dimCalculateMapSize().	c		e	f	g	a		i		j	k	l	m	n	b
M_FILL_GAPS_CONTEXT	Specifies to allocate a 3D image processing context that can be used with M3dimFillGaps().	c		e	f	g	a		i		j	k	l	m	n	b
M_FILTER_CONTEXT	Specifies to allocate a 3D image processing context that can be used with M3dimFilter().	c		e	f	g	a		i		j	k	l	m	n	b
M_LATTICE_CONTEXT	Specifies to allocate a 3D image processing context that can be used with M3dimLattice().	c		e	f	g	a		i		j	k	l	m	n	b
M_MESH_CONTEXT	Specifies to allocate a 3D image processing context that can be used with M3dimMesh().	c		e	f	g	a		i		j	k	l	m	n	b
M_NORMALS_CONTEXT	Specifies to allocate a 3D image processing context that can be used with M3dimNormals().	c		e	f	g	a		i		j	k	l	m	n	b

M_OUTLIERS_CONTEXT	Specifies to allocate a 3D image processing context that can be used with M3dimOutliers().	c		e	f	g	a		i		j	k	l	m	n	b
M_PROFILE_CONTEXT	Specifies to allocate a 3D image processing context that can be used with M3dimProfileEx().	c		e	f	g	a		i		j	k	l	m	n	b
M_PROJECT_CONTEXT	Specifies to allocate a 3D image processing context that can be used with M3dimProjectEx().	c		e	f	g	a		i		j	k	l	m	n	b
M_REMAP_CONTEXT	Specifies to allocate a 3D image processing context that can be used with M3dimRemapDepthMap().	c		e	f	g	a		i		j	k	l	m	n	b
M_STATISTICS_CONTEXT	Specifies to allocate a 3D image processing context that can be used with M3dimStat().	c		e	f	g	a		i		j	k	l	m	n	b
M_STITCH_CONTEXT	Specifies to allocate a 3D image processing context that can be used with M3dimMerge().	c		e	f	g	a		i		j	k	l	m	n	b
M_SUBSAMPLE_CONTEXT	Specifies to allocate a 3D image processing context that can be used with M3dimSample() to perform subsampling.	c		e	f	g	a		i		j	k	l	m	n	b
M_SURFACE_SAMPLE_CONTEXT	Specifies to allocate a 3D image processing context that can be used with M3dimSample() to perform surface sampling.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Context3dimIdPtr

Specifies the address of the variable in which to write the 3D image processing context identifier or specifies the data type that the function should use to return the 3D image processing context identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated 3D image processing context ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated 3D image processing context ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_3DIM_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the 3D image processing context (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the calculate map size 3D image processing context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated map size 3D image processing context. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Address in which to write the fill gaps 3D image processing context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated fill gaps 3D image processing context. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the filter 3D image processing context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated filter 3D image processing context. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the lattice 3D image processing context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated lattice 3D image processing context. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the mesh 3D image processing context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated mesh 3D image processing context. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the normals 3D image processing context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated normals 3D image processing context. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the profile 3D image processing context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated profile 3D image processing context. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the project 3D image processing context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated project 3D image processing context. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the remap 3D image processing context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated remap 3D image processing context. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the statistics 3D image processing context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated statistics 3D image processing context. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the stitch 3D image processing context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated stitch 3D image processing context. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the subsample 3D image processing context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated subsample 3D image processing context. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the surface sample 3D image processing context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated surface sample 3D image processing context. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the 3D image processing context identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_3DIM_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.

Library	Use ail11.lib; ail3dim11.lib.
DLL	Requires ail11.dll; ail3dim11.dll.

M3dimAllocResult

Synopsis

Allocate a 3D image processing result buffer.

Syntax

```
AIL_ID M3dimAllocResult(
    AIL_ID SysId,           //in
    AIL_INT64 ResultType,   //in
    AIL_INT64 ControlFlag,  //in
    AIL_ID *Result3dimIdPtr //out
)
```

Description

This function allocates a 3D image processing result buffer on the specified system to store results obtained from a 3D image processing operation.

When the 3D image processing result buffer is no longer required, release it using M3dimFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the 3D image processing result buffer's lifetime and you must not manually free it.

Parameters

SysId

Specifies the system on which to allocate the result buffer. This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ResultType

Specifies the type of result buffer to allocate. This parameter must be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of result buffer to allocate																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FIND_TRANSFORMATION_RESULT	Specifies to allocate a 3D image processing result buffer used to store M3dimFindTransformation() results.	c		e	f	g	a		i		j	k	l	m	n	b
M_LATTICE_RESULT	Specifies to allocate a 3D image processing result buffer used to store M3dimLattice() results.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROFILE_RESULT	Specifies to allocate a 3D image processing result buffer used to store M3dimProfile() and M3dimProfileEx() results.	c		e	f	g	a		i		j	k	l	m	n	b
M_STATISTICS_RESULT	Specifies to allocate a 3D image processing result buffer used to store M3dimStat() results.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Result3dimIdPtr

Specifies the address of the variable in which to write the 3D image processing result buffer identifier or specifies the data type that the function should use to return the 3D image processing result buffer identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated 3D image processing result buffer ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated 3D image processing result buffer ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_3DIM_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the 3D image processing result buffer (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the find transformation 3D image processing result buffer identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated find transformation 3D image processing result buffer. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the lattice 3D image processing result buffer identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated lattice 3D image processing result buffer. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the profile 3D image processing result buffer identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated profile 3D image processing result buffer. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the statistics 3D image processing result buffer identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated statistics 3D image processing result buffer. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the 3D image processing result buffer identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_3DIM_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dim11.lib.
DLL	Requires ail11.dll; ail3dim11.dll.

M3dimArith

Synopsis

Perform a point-to-point arithmetic operation on depth maps and 3D geometries.

Syntax

```
void M3dimArith(
    AIL_ID Src1ImageBufOrGeometry3dgeoId, //in
    AIL_ID Src2ImageBufOrGeometry3dgeoId, //in
    AIL_ID DstImageBufId,                 //out
    AIL_ID MaskBufId,                     //in
    AIL_INT64 Operation,                  //in
    AIL_INT64 SignMode,                   //in
    AIL_INT64 ControlFlag                 //in
)
```

Description

This function performs the specified point-to-point arithmetic operation on two depth maps or one depth map and one 3D geometry, storing the resulting depth map in the specified destination container or image buffer. Note that **M3dimArith**() cannot be called with two 3D geometry operands. Alternatively, you can use this function to invert a depth map's Z-values, in which case only a single source depth map is required.

> Note that 3D geometries of type M_BOX and M_POINT are not supported for this function.

All image operands must be fully corrected depth maps for this operation. To determine if you have a fully corrected depth map, call McalInquire() with M_DEPTH_MAP and ensure that it returns M_TRUE. Note that if the two operands are images, their respective camera calibration information can vary in Z, but not in X or Y.

All operations are performed using the Z-world coordinates corresponding to the gray levels of the image operands or the surface point of a 3D geometry operand. The resulting Z-world coordinates are then converted back to gray levels, according to the camera calibration information of the destination buffer. If the Z-scale and Z-offset of the destination are such that the 3D point falls outside the vertical span of the destination buffer, the destination pixel will be saturated to either 0 or the maximum value of the buffer, minus 1 (since the maximum value of a depth map represents missing data).

Some pixels of the destination buffer can be left unchanged by providing a mask image buffer with the corresponding pixels set to 0.

By default, the destination buffer does not have to be calibrated before calling **M3dimArith**(). The function will fill the destination buffer and associate it with the necessary camera calibration information. You can, however, specify to use the existing camera calibration information of the destination buffer, by passing M_USE_DESTINATION_SCALES to the ControlFlag parameter. If the destination is a container and you pass M_USE_DESTINATION_SCALES to the ControlFlag parameter, the container must be a 3D-processable depth map container.

If either or both operands contain missing data (represented using the maximum value of the depth map buffer), missing data might be written to the destination buffer, depending on the value passed to the Operation parameter.

Parameters

Src1ImageBufOrGeometry3dgeoId

Specifies the first source container, image buffer, or 3D geometry object.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the first source container, image buffer, or 3D geometry object identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_XY_PLANE	Specifies the first operand to be the XY (Z=0) plane.	c		e	f	g	a		i		j	k	l	m	n	b
3D geometry object identifier	Specifies the first operand to be a 3D geometry object. The 3D geometry object must have been previously allocated on the required system using M3dgeoAlloc() and must have been successfully defined. Note that 3D geometries of type M_BOX and M_POINT are not supported for this function.	c		e	f	g	a		i		j	k	l	m	n	b

Depth map container identifier	Specifies the first operand to be a depth map container. The container must store data in a 3D-processable depth map format (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE_DEPTH_MAP, the function returns M_TRUE), and must have been previously allocated using MbufAllocContainer() with M_PROC. This container must not have a region component. Using a container with a region component will cause an error.	c		e	f	g	a		i		j	k	l	m	n	b
Image buffer identifier	Specifies the first operand to be an image buffer that contains a fully corrected depth map. The image buffer must be a 1-band, 8-bit, 16-bit, or 32-bit unsigned buffer and must contain a fully corrected depth map (that is, if you call McalInquire() with M_DEPTH_MAP, the function returns M_TRUE). It must also have the same dimensions and bit depth as the destination buffer (DstImageBufId). This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.	c		e	f	g	a		i		j	k	l	m	n	b

Src2ImageBufOrGeometry3dgeoId

Specifies the second source container, image buffer, or 3D geometry object.

If a second source is not required, set this parameter to **M_NULL**.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the second source container, image buffer, or 3D geometry object identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_XY_PLANE	Specifies the second operand to be the XY (Z=0) plane.	c		e	f	g	a		i		j	k	l	m	n	b
3D geometry object identifier	Specifies the second operand to be a 3D geometry object. The 3D geometry object must have been previously allocated on the required system using M3dgeoAlloc() and must have been successfully defined. > Note that 3D geometries of type M_BOX and M_POINT are not supported for this function.	c		e	f	g	a		i		j	k	l	m	n	b
Depth map container identifier	Specifies the second operand to be a depth map container. The container must store data in a 3D-processable depth map format (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE_DEPTH_MAP, the function returns M_TRUE), and must have been previously allocated using MbufAllocContainer() with M_PROC. This container must not have a region component. Using a container with a region component will cause an error.	c		e	f	g	a		i		j	k	l	m	n	b
Image buffer identifier	Specifies the second operand to be an image buffer that contains a fully corrected depth map. The image buffer must be a 1-band, 8-bit, 16-bit, or 32-bit unsigned buffer and must contain a fully corrected depth map (that is, if you call McalInquire() with M_DEPTH_MAP, the function returns M_TRUE). It must also have the same dimensions and bit depth as the destination buffer (DstImageBufId). This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error. When both operands are depth map image buffers, their calibration information can vary in Z, but not in X and Y. That is, the Src2ImageBufOrGeometry3dgeoId parameter must have the same camera calibration information for X and Y as the image buffer specified with Src1ImageBufOrGeometry3dgeoId. The operation is performed point-to-point; therefore, each respective pixel must represent the same area in the real world in X and Y.	c		e	f	g	a		i		j	k	l	m	n	b

DstImageBufId

Specifies the identifier of the destination container or image buffer in which to put the result of the arithmetic operation. The container must have been previously allocated using MbufAllocContainer() with M_PROC, and must not be a child container. If you pass M_USE_DESTINATION_SCALES to the ControlFlag parameter, the container must be a 3D-processable depth map container (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE_DEPTH_MAP, the function returns M_TRUE).

The image buffer must be a 1-band, 8-bit, 16-bit, or 32-bit unsigned buffer. The image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

MaskBufId

Specifies the identifier of the image buffer to use as a mask, if needed.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the mask identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies to write to all pixels in the destination buffer; no mask will be used.	c		e	f	g	a		i		j	k	l	m	n	b
Image buffer identifier	Specifies the identifier of the image buffer to use as a mask in the arithmetic operation. Pixels in the destination buffer corresponding to pixels set to 0 in the mask buffer are left unchanged, while the other destination pixels will be the result of the arithmetic operation. The image buffer must be a 1-band, binary, 8-bit, or 16-bit buffer, and must have the same dimensions as the destination buffer (DstImageBufId). This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.	c		e	f	g	a		i		j	k	l	m	n	b

Operation

Specifies the operation to perform.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the operation to perform																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DIST_NN(AIL_INT NeighborhoodSize)	Calculates the absolute difference between the Z-coordinates of corresponding pixels of a depth map and an operand, considering the neighborhood pixels. Typically, this is performed when comparing two depth maps of highly similar objects or features, which are slightly misaligned. M_DIST_NN() compensates for the misalignment. This is a more robust version of the M_SUB_ABS operation, which calculates the Z-coordinate difference without considering the neighborhood of pixels. If either operand is missing data for a certain pixel, the corresponding destination pixel is set to the value for missing data (255, 65535, or 4294967295 for an 8-, 16-, or 32-bit depth map, respectively).	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	NeighborhoodSize Specifies the size of the neighborhood.															

			0 <= *Value* <= 255 Specifies the size of the height and width of the neighborhood, in pixels, rounded up to the next odd number. Typically, this is a value between 3 and 7. Specifying 2 or 3 will create a 3x3 neighborhood, specifying 4 or 5 will create a 5x5 neighborhood, specifying 6 or 7 will result in a 7x7 neighborhood. The larger the value, the longer the operation will take and the more robust it will be to misalignment. Note that very large values can return insignificant results. > Note that specifying 0 or 1 will create a 1x1 neighborhood, which is no neighborhood other than the pixel. This renders this macro equivalent to the M_SUB_ABS operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_DIST_NN_SIGNED(AIL_INT NeighborhoodSize)	Calculates the signed difference between the Z-coordinates of corresponding pixels of a depth map and an operand, considering the neighborhood pixels. Typically, this is performed when comparing two depth maps of highly similar objects or features, which are slightly misaligned. M_DIST_NN_SIGNED() compensates for the misalignment. This is a more robust version of the M_SUB operation, which calculates the Z-coordinate difference without considering the neighborhood of pixels. M_DIST_NN_SIGNED() is equivalent to M_DIST_NN(), except that it returns a signed distance. If operand 1 has a higher Z-coordinate than operand 2, the returned value is positive. Otherwise the returned value is negative. If either operand is missing data for a certain pixel, the corresponding destination pixel is set to the value for missing data (255, 65535, or 4294967295 for an 8-, 16-, or 32-bit depth map, respectively).															c		e	f	g	a		i		j	k	l	m	n	b
		Parameters																	
		NeighborhoodSize Specifies the size of the neighborhood.																	

			0 <= *Value* <= 255 Specifies the size of the height and width of the neighborhood, in pixels, rounded up to the next odd number. Typically, this is a value between 3 and 7. Specifying 2 or 3 will create a 3x3 neighborhood, specifying 4 or 5 will create a 5x5 neighborhood, specifying 6 or 7 will result in a 7x7 neighborhood. The larger the value, the longer the operation will take and the more robust it will be to misalignment. Note that very large values can return insignificant results. > Note that specifying 0 or 1 will create a 1x1 neighborhood, which is no neighborhood other than the pixel. This renders this macro equivalent to the M_SUB operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ADD			Adds the Z-coordinate of the first operand to the Z-coordinate of the second operand. *Destination* = (*Operand 1* + *Operand 2*) If either operand is missing data, the destination is set to the value for missing data.	c		e	f	g	a		i		j	k	l	m	n	b
M_FLIP_Z			Inverts a depth map's Z values; bright values become dark, and dark values become bright. Note that any missing values are not inverted. *Destination* = (*Maximum value of source buffer* - *Operand 1*) If *Operand 1* is missing data, the destination is set to the value for missing data. > Note that you cannot use a mask when performing this operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX			Compares the Z-coordinate of the first operand with the Z-coordinate of the second operand, and takes the greater of the two. *Destination* = max(*Operand 1*, *Operand 2*) If both operands are missing data, the destination is set to the value for missing data. If a single operand is missing data, the destination is set to the value of the other operand.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN			Compares the Z-coordinate of the first operand with the Z-coordinate of the second operand, and takes the lesser of the two. *Destination* = min(*Operand 1*, *Operand 2*) If both operands are missing data, the destination is set to the value for missing data. If a single operand is missing data, the destination is set to the value of the other operand.	c		e	f	g	a		i		j	k	l	m	n	b
M_REPLACE_MISSING_DATA			Returns an image that is the same as the first source depth map, except that any missing values are replaced by their corresponding values from the second source. > This operation is available only when the first source is a depth map.	c		e	f	g	a		i		j	k	l	m	n	b
M_SUB			Subtracts the Z-coordinate of the second operand from the Z-coordinate of the first operand. *Destination* = (*Operand 1* - *Operand 2*) If either operand is missing data, the destination is set to the value for missing data.	c		e	f	g	a		i		j	k	l	m	n	b

M_SUB_ABS	Subtracts the Z-coordinate of the second operand from the Z-coordinate of the first operand, and takes the absolute value of the result. This operation is equivalent to taking the absolute value of the M_SUB operation. *Destination = | Operand 1 - Operand 2|* If either operand is missing data, the destination is set to the value for missing data.	c		e	f	g	a		i		j	k	l	m	n	b
M_VALIDITY_MAP	Returns an image corresponding to the validity of each pixel in the two operands.	c		e	f	g	a		i		j	k	l	m	n	b

Pixel in operand 1	Pixel in operand 2	Returned to corresponding pixel in the destination	
		Value	Constant
M_INVALID_POINT	**M_INVALID_POINT**	0	**M_NO_SRC_VALID_LABEL**
Valid point	**M_INVALID_POINT**	85	**M_ONLY_SRC1_VALID_LABEL**
M_INVALID_POINT	Valid point	170	**M_ONLY_SRC2_VALID_LABEL**
Valid point	Valid point	255	**M_BOTH_SRC_VALID_LABEL**

Note that the DstImageBufId parameter must be given an image buffer identifier.

SignMode

Specifies which Z-coordinate to select for geometries with more than 1 Z-coordinate for a given pair of X- and Y-coordinates. When the two sources are depth maps, or when performing an M_FLIP_Z operation, the SignMode parameter must be set to M_DEFAULT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the sign mode																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_MAX_Z.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX_Z	Specifies to use the surface point with the largest Z-coordinate value.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_Z	Specifies to use the surface point with the smallest Z-coordinate value.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Specifies how to handle the camera calibration information of the destination image buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying how to handle the camera calibration information of the destination image buffer																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_FIT_SCALES. This is the only possible value for M_FLIP_Z and M_VALIDITY_MAP.	c		e	f	g	a		i		j	k	l	m	n	b

M_FIT_SCALES	Specifies to choose the Z-scale and Z-offset so that the values can be represented in the destination image buffer without saturation. This mode depends only on the possible range of the data, and not on the actual source depth map contents. Note that the X- and Y-scales and offsets of the destination image buffer will be set to the same values as those of the source image(s). After calling **M3dimArith**(), the destination image buffer will be fully corrected. > Note that in extreme cases, M_DIST_NN() and M_DIST_NN_SIGNED() can return saturated pixels because the Euclidean distance is calculated and not just the Z-coordinate distance.	c		e	f	g	a		i		j	k	l	m	n	b
M_SET_WORLD_OFFSET_Z	Specifies to keep the same Z-scale as the first source image and to set the Z-offset so that the destination image buffer Z-range covers the center of the possible range of the data. Since the Z-scale remains unaltered, it is possible for saturation to occur. The X- and Y-scales and offsets of the destination image buffer will be set to the same values as those of the source image(s). After calling **M3dimArith**(), the destination image buffer will be fully corrected.	c		e	f	g	a		i		j	k	l	m	n	b
M_USE_DESTINATION_SCALES	Specifies to use the camera calibration information of the destination image buffer. This means the destination must be fully corrected before calling **M3dimArith**(), and have the same scales and offsets in X and Y as the source image(s). This can be done by copying the camera calibration information from another image using McalAssociate(), or by providing scales and offsets manually using McalUniform() or using McalControl() with M_GRAY_LEVEL_SIZE_Z and M_WORLD_POS_Z. **M3dimArith**() will leave the camera calibration information of the destination image buffer unchanged.	c		e	f	g	a		i		j	k	l	m	n	b
M_USE_SOURCE1_SCALES	Specifies to use the calibration information of the first source. > This option is not available when the Src1ImageBufOrGeometry3dgeoId parameter is set to M_XY_PLANE.	c		e	f	g	a		i		j	k	l	m	n	b
M_USE_SOURCE2_SCALES	Specifies to use the calibration information of the second source. > This option is not available when the Src2ImageBufOrGeometry3dgeoId parameter is set to M_XY_PLANE.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dim11.lib.
DLL	Requires ail11.dll; ail3dim11.dll.

M3dimCalculateMapSize

Synopsis

Estimate a recommended depth map image buffer size from a source container's point cloud.

Syntax

```
void M3dimCalculateMapSize(
    AIL_ID CalculateMapSizeContext3dimId, //in
    AIL_ID SrcContainerBufId,             //in
    AIL_ID AABox3dgeoId,                  //in
    AIL_INT64 ControlFlag,                //in
    AIL_INT *DepthMapSizeXPtr,            //out
    AIL_INT *DepthMapSizeYPtr             //out
)
```

Description

This function estimates a recommended depth map image buffer size in X and Y, using the dimensions of the source container's point cloud. You can optionally limit the calculation to consider only points inside a specified axis-aligned 3D box geometry.

The recommended size is useful for allocating a depth map image buffer (using MbufAlloc2d()) before calling M3dimProject().

Using M3dimControl(), you can set the depth map's required pixel aspect ratio. You can also set a fixed size along 1 dimension, and the function will calculate the necessary size of the other dimension (for either pixel size or image buffer size). Default settings let **M3dimCalculateMapSize**() do all calculations.

Parameters

CalculateMapSizeContext3dimId

Specifies a calculate map size 3D image processing context.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the calculate map size 3D image processing context identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default calculate map size 3D image processing context of the current Aurora Imaging Library application. > The operation will use default values for all calculate map size context control types (M3dimControl()).	c		e	f	g	a		i		j	k	l	m	n	b
Calculate map size 3D image processing context identifier	Specifies the identifier of a calculate map size 3D image processing context, previously allocated using M3dimAlloc() with M_CALCULATE_MAP_SIZE_CONTEXT. > If a calculate map size context is specified, the function applies the calculate map size control settings specified using M3dimControl().	c		e	f	g	a		i		j	k	l	m	n	b

SrcContainerBufId

Specifies the identifier of the source container containing a 3D-processable point cloud. The container must be 3D-processable (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE, the function returns M_PROCESSABLE).

The container must have been previously allocated using MbufAllocContainer() with M_PROC.

AABox3dgeoId

Specifies the identifier of the axis-aligned 3D box geometry object to use to limit the calculation to points within the box. The 3D box geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a box.

If a 3D box geometry object is not required, set this parameter to **M_NULL**.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

DepthMapSizeXPtr

Specifies the address of the variable in which to write the estimated depth map image buffer size in X.

DepthMapSizeYPtr

Specifies the address of the variable in which to write the estimated depth map image buffer size in Y.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dim11.lib.
DLL	Requires ail11.dll; ail3dim11.dll.

M3dimCalibrateDepthMap

Synopsis

Calibrate a buffer based on the bounding box of a container's point cloud, or an axis-aligned 3D box geometry, to prepare the buffer to hold a depth or intensity map.

Syntax

```
void M3dimCalibrateDepthMap(
    AIL_ID SrcContainerBufOrGeometry3dgeoId,    //in
    AIL_ID DepthMapImageBufId,                  //out
    AIL_ID IntensityMapImageBufId,              //out
    AIL_ID CalibrationAilObjectOrMatrix3dgeoId, //in
    AIL_DOUBLE PixelSizeAspectRatio,            //in
    AIL_INT64 ZSign,                            //in
    AIL_INT64 ControlFlag                       //in
)
```

Description

This function calibrates a buffer based on the bounding box of a container's point cloud, or an axis-aligned 3D box geometry. For a point cloud, the source container's M_COMPONENT_RANGE and M_COMPONENT_CONFIDENCE components are used to establish the bounding box. The calibration prepares the buffer to hold depth map or intensity map information. To generate a depth map from a point cloud or 3D geometry, or to generate an intensity map, use M3dimProject().

M3dimCalibrateDepthMap() sets the real-world to pixel unit scale for the image buffer, and sets the real-world to gray level scale (M_GRAY_LEVEL_SIZE_Z) for depth values.

You can set this function's PixelSizeAspectRatio to M_NULL to ensure an unconstrained pixel aspect ratio, so that source data fits completely into the depth map.

If the PixelSizeAspectRatio parameter is not set to M_NULL, there can be leftover, unused pixel rows or columns in the destination. You can specify where to place valid values in the destination buffer with the ControlFlag parameter and M_DEFAULT or M_CENTER. In the M_DEFAULT case, the data aligns to the top left of the buffer, and the function calculates a pixel size such that the data extends at least the breadth of one dimension, and no data is lost along the other dimension. In the M_CENTER case, valid data is centered in the buffer and extends the full breadth of one dimension with no loss of data along the other dimension. M3dimProject() will set pixels in unused rows or columns to the invalid pixel value. Note that the calculated pixel size does not distort the world data and the overall aspect ratio of source information is maintained. Also note that a projected point retains its source coordinate values, regardless of whether the corresponding pixel is centered (M_CENTER) or not (M_DEFAULT). This means that the depth map's top-left pixel will have different world XY-coordinates, depending on the ControlFlag setting. See the Specifying data placement in the destination buffer subsection of the Generating fully corrected depth and intensity maps section of Chapter 35: 3D image processing for more information.

By default, the relative coordinate system is set to the same location as the absolute coordinate system. You can optionally set the relative coordinate system to a different location by specifying (with CalibrationAilObjectOrMatrix3dgeoId) a calibration context, a calibrated Aurora Imaging Library object, or a transformation matrix object. If the destination is a depth map container, matrix information will be added to the M_COMPONENT_MATRIX_AIL component of the container. Note that the pixel-to-world mapping of the specified calibration context or calibrated Aurora Imaging Library object is not used.

Parameters

SrcContainerBufOrGeometry3dgeoId

Specifies the identifier of the source point cloud container or 3D geometry object.

The point cloud container must be 3D-processable (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE, the function returns M_PROCESSABLE). The container must have been previously allocated using MbufAllocContainer() with M_PROC.

The 3D geometry object must be an axis-aligned box, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a box.

DepthMapImageBufId

Specifies the identifier of the destination image buffer or container with which to associate the calibration information for a depth map.

The destination image buffer must be a 1-band, 8-bit, 16-bit or 32-bit unsigned image buffer.

The destination container must have been previously allocated using MbufAllocContainer() with M_PROC, must store data in a 3D-processable depth map format (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE_DEPTH_MAP, the function returns M_TRUE), and must not be a child container.

IntensityMapImageBufId

Specifies the identifier of the destination image buffer with which to associate the calibration information for an intensity map. This image buffer

must be a 1-band buffer and must have the same dimensions as the buffer specified for DepthMapImageBufId.

If an intensity map is not required, set this parameter to **M_NULL**.

CalibrationAilObjectOrMatrix3dgeoId

Specifies the identifier of the Aurora Imaging Library object that defines the relationship between the relative and absolute coordinate systems in the depth map. The Aurora Imaging Library object can be a camera calibration context, a 3D reconstruction context, a 3D reconstruction result buffer, or any object that has camera calibration information, such as an image or result buffer. It can also be a transformation matrix object, previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX. If the destination is a depth map container, matrix information will be added to the M_COMPONENT_MATRIX_AIL component. If no calibration context or object associated with a camera calibration context is required, set this parameter to **M_NULL**.

PixelSizeAspectRatio

Specifies how to adjust the X and Y pixel sizes.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the pixel aspect ratio																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies to apply a pixel aspect ratio such that the source container's bounding box or the axis-aligned box fits exactly inside the dimensions of the destination image buffer or container.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies an aspect ratio for the destination pixels. The pixel size is computed such that the pixel's length in X divided by its length in Y equals the specified PixelSizeAspectRatio value.	c		e	f	g	a		i		j	k	l	m	n	b

ZSign

Specifies the sign of the Z-scale. The sign applies to M_GRAY_LEVEL_SIZE_Z, which sets the length, in world units, of one gray level in a fully-corrected depth map.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the Z-scale sign																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_POSITIVE.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEGATIVE	Specifies a negative value for M_GRAY_LEVEL_SIZE_Z. A negative value indicates that larger Z-values are represented by darker depth map pixels, and the smaller the Z-value the brighter the pixel.	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITIVE	Specifies a positive value for M_GRAY_LEVEL_SIZE_Z. A positive value indicates that larger Z-values are represented by brighter depth map pixels, and the smaller the Z-value the darker the pixel.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Specifies where to place valid values in the depth map. When PixelSizeAspectRatio is set to M_NULL, ControlFlag must be set to M_DEFAULT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the control flag																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that, once M3dimProject() is called, the point cloud or 3D geometry data is projected onto the depth map buffer such that it aligns to the top left of the buffer, and the data extends at least the breadth of one dimension, leaving space at the bottom or right. M3dimProject() will set pixels in unused rows or columns to the invalid pixel value. The source data maintains its aspect ratio.	c		e	f	g	a		i		j	k	l	m	n	b
M_CENTER	Specifies that, once M3dimProject() is called, the point cloud data is projected onto the depth map buffer such that it is centered horizontally or vertically in the buffer. M3dimProject() will set pixels in unused rows or columns to the invalid pixel value. The source data maintains its aspect ratio.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dim11.lib.
DLL	Requires ail11.dll; ail3dim11.dll.

M3dimControl

Synopsis

Control a 3D image processing context or result buffer.

Syntax

```
void M3dimControl(
    AIL_ID Context3dimId,   //out
    AIL_INT64 ControlType,  //in
    AIL_DOUBLE ControlValue //in
)
```

Description

This function controls a general setting of a 3D image processing context or lattice 3D image processing result buffer. You can typically inquire these settings using M3dimInquire().

Parameters

Context3dimId

Specifies the identifier of the 3D image processing context or result buffer to control. The context or result buffer must have been previously allocated on the system using M3dimAlloc() or M3dimAllocResult().

See the Parameter associations section for possible values that can be returned.

ControlType

Specifies the type of control to set.

See the Parameter associations section for possible values that can be specified.

ControlValue

Specifies the required value for the control.

See the Parameter associations section for possible values that can be specified.

Parameter associations

The table below lists possible values for the **ControlType** and **ControlValue** parameters and possible values returned to the **Context3dimId** parameter.

- For specifying the control type and control value for a 3D image processing context or result buffer

The following Context3dimId, ControlType, and ControlValue parameter settings can be specified for different types of 3D image processing contexts or a lattice 3D image processing result buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the control type and control value for a 3D image processing context or result buffer																
Context3dimId - Possible values returned ControlType ControlValue	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Calculate map size context ID	Specifies a calculate map size 3D image processing context, allocated using M3dimAlloc() with M_CALCULATE_MAP_SIZE_CONTEXT, and used in M3dimCalculateMapSize() operations. The recommended size is useful for allocating a depth map image buffer (using MbufAlloc2d()) before calling M3dimProject().															
M_CALCULATE_MODE	Sets whether the source point cloud container is organized. M3dimCalculateMapSize() calculations are faster with organized point clouds.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_UNORGANIZED.	c		e	f	g	a		i		j	k	l	m	n	b

	M_ORGANIZED	Specifies an organized point cloud. > Note that using this value with an unorganized point will cause an error.	c		e	f	g	a		i		j	k	l	m	n	b
	M_UNORGANIZED	Specifies an unorganized point cloud.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL_ASPECT_RATIO		Sets the pixel aspect ratio of the depth map. This control type must be set to M_NULL if you have already specified a size in X with M_PIXEL_SIZE_X or M_SIZE_X and a size in Y with M_PIXEL_SIZE_Y or M_SIZE_Y. Otherwise, an error will occur. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NULL	Specifies that other data determines the pixel aspect ratio, such as the values set with M_PIXEL_SIZE_X and M_PIXEL_SIZE_Y.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies an aspect ratio for the depth map's pixels. The pixel size is computed such that the pixel's length in X divided by its length in Y equals the specified M_PIXEL_ASPECT_RATIO value.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL_SIZE_CONSTRAINT		Sets how to calculate the depth map pixel sizes to fit the required pixel aspect ratio (M_PIXEL_ASPECT_RATIO). This control type is supported only when M_PIXEL_ASPECT_RATIO is not set to M_NULL and M_PIXEL_SIZE_X, M_PIXEL_SIZE_Y, M_SIZE_X, and M_SIZE_Y are all set to M_NULL. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_PRESERVE_WORLD_AREA.	c		e	f	g	a		i		j	k	l	m	n	b
	M_INCREASE_PIXEL_SIZE	Specifies to calculate the pixel sizes such that the pixel size in one dimension is the same as in the source container and the pixel size in the other dimension is increased to fit the required pixel aspect ratio. The dimension chosen from the point cloud lattice is the one that results in the largest pixels given the required pixel aspect ratio. This generates a smaller depth map that might have missing data. > Note that this mode might overestimate the pixel size when an organized image has some pixel rotation, especially if the pixel aspect ratio is large.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PRESERVE_WORLD_AREA	Specifies to calculate the pixel sizes such that the area of a pixel is the same as in the source container. For point clouds whose lattice aspect ratio is much different than the required depth map pixel aspect ratio, this can result in data loss in one dimension and gaps in the other dimension.	c		e	f	g	a		i		j	k	l	m	n	b
	M_REDUCE_PIXEL_SIZE	Specifies to calculate the pixel sizes such that the pixel size in one dimension is the same as in the source container and the pixel size in the other dimension is decreased to fit the required pixel aspect ratio. The dimension chosen from the point cloud lattice is the one that results in the smallest pixels given the required pixel aspect ratio. This generates a larger depth map that might have gaps. > Note that this mode might overestimate the pixel size when an organized image has some pixel rotation, especially if the pixel aspect ratio is large.	c		e	f	g	a		i		j	k	l	m	n	b

M_PIXEL_SIZE_X	Sets the length in X of one pixel in the depth map. Note that you cannot set M_PIXEL_SIZE_X at the same time as M_PIXEL_SIZE_Y or M_SIZE_Y, unless M_PIXEL_ASPECT_RATIO is set to M_NULL. Note that you cannot specify a specific value for M_PIXEL_SIZE_X if you have done so for M_SIZE_X (and vice versa). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_NULL.	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies that M3dimCalculateMapSize() will compute the pixel size in X.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the length in X of one pixel in the depth map, in world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL_SIZE_Y	Sets the length in Y of one pixel in the depth map. Note that you cannot set M_PIXEL_SIZE_Y at the same time as M_PIXEL_SIZE_X or M_SIZE_X, unless M_PIXEL_ASPECT_RATIO is set to M_NULL. Note that you cannot specify a specific value for M_PIXEL_SIZE_Y if you have done so for M_SIZE_Y (and vice versa). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_NULL.	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies that M3dimCalculateMapSize() will compute the pixel size in Y.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the length in Y of one pixel in the depth map, in world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIZE_X	Sets the image size of the depth map along the X dimension. Note that you cannot set M_SIZE_X at the same time as M_PIXEL_SIZE_Y or M_SIZE_Y, unless M_PIXEL_ASPECT_RATIO is set to M_NULL. Note that you cannot specify a specific value for M_SIZE_X if you have done so for M_PIXEL_SIZE_X (and vice versa). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_NULL.	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies that M3dimCalculateMapSize() will compute the image size of the depth map in X.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the image size of the depth map in X, in pixel units.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIZE_Y	Sets the image size of the depth map along the Y dimension. Note that you cannot set M_SIZE_Y at the same time as M_PIXEL_SIZE_X or M_SIZE_X, unless M_PIXEL_ASPECT_RATIO is set to M_NULL. Note that you cannot specify a specific value for M_SIZE_Y if you have done so for M_PIXEL_SIZE_Y (and vice versa). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_NULL.	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies that M3dimCalculateMapSize() will compute the image size of the depth map in Y.	c		e	f	g	a		i		j	k	l	m	n	b

Value > 0.0	Specifies the image size of the depth map in Y, in pixel units.	c		e	f	g	a		i		j	k	l	m	n	b
Fill gaps context ID	Specifies a fill gaps 3D image processing context, allocated using M3dimAlloc() with M_FILL_GAPS_CONTEXT, and used in M3dimFillGaps() operations. Gaps are missing or invalid data in a depth or intensity map.	c		e	f	g	a		i		j	k	l	m	n	b
M_FILL_BORDER	Sets whether to propagate the boundary value across each gap that touches the image border. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_REPLICATE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to fill gaps that touch the image border.	c		e	f	g	a		i		j	k	l	m	n	b
M_REPLICATE	Specifies to propagate the unique gap boundary value across each gap that touches the image border, replacing gap pixels up to the image border.	c		e	f	g	a		i		j	k	l	m	n	b
M_FILL_MODE	Sets the mode in which to fill gaps in depth maps. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_X_THEN_Y.	c		e	f	g	a		i		j	k	l	m	n	b
M_X_THEN_Y	Specifies that each depth map row is analyzed so that gap pixels in each row are filled, and then each column is analyzed to fill the remaining gaps. The filling operation used depends on the M_FILL_SHARP_ELEVATION_DEPTH and M_FILL_SHARP_ELEVATION control types. You can specify how to fill gaps that touch the image border with M_FILL_BORDER.	c		e	f	g	a		i		j	k	l	m	n	b
M_Y_THEN_X	Specifies that each depth map column is analyzed so that gap pixels in each column are filled, and then each row is analyzed to fill the remaining gaps. The filling operation used depends on the M_FILL_SHARP_ELEVATION_DEPTH and M_FILL_SHARP_ELEVATION control types. You can specify how to fill gaps that touch the image border with M_FILL_BORDER.	c		e	f	g	a		i		j	k	l	m	n	b
M_FILL_SHARP_ELEVATION	Sets which boundary to use to fill sharp elevation gaps in depth maps. You can specify to propagate either the minimum or maximum gap border value, or not to fill sharp elevation gaps. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to fill sharp elevation gaps.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX	Specifies to set the destination pixel(s) to the maximum value of the two pixels on either side of the gap.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN	Specifies to set the destination pixel(s) to the minimum value of the two pixels on either side of the gap.	c		e	f	g	a		i		j	k	l	m	n	b

M_FILL_SHARP_ELEVATION_DEPTH	Sets the threshold used to differentiate between gradual elevation gaps and sharp elevation gaps in the depth map. If the difference between a gap's boundary values is less than the specified threshold, the gap is considered a gradual elevation gap, otherwise it is considered a sharp elevation gap. Gradual elevation gaps are filled using linear interpolation; sharp elevation gaps are either filled by propagating the value of one of the boundaries or are left unfilled, depending on the M_FILL_SHARP_ELEVATION control type. It is often better to fill gradual elevation gaps using linear interpolation since the underlying surface of such a gap is considered to be part of the same surface as its boundaries. Whereas for sharp elevation gaps, the underlying surface is considered to be discontinuous at least at one of its boundaries, so propagating the value of one of the boundaries is recommended. If a gap is greater than M_FILL_SHARP_ELEVATION_DEPTH, it is replaced using the secondary algorithm described in M_FILL_SHARP_ELEVATION. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_INFINITE.	c		e	f	g	a		i		j	k	l	m	n	b
M_INFINITE	Specifies that the threshold is infinite. When M_INFINITE is specified, all gaps are considered gradual elevation gaps, so linear interpolation is always used and M_FILL_SHARP_ELEVATION is ignored.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the threshold, in pixel or world units, depending on the units specified with M_INPUT_UNITS. When the threshold is set to 0.0, all gaps are considered sharp elevation gaps, so linear interpolation is never used and all gaps are filled according to M_FILL_SHARP_ELEVATION.	c		e	f	g	a		i		j	k	l	m	n	b
M_FILL_THRESHOLD_X	Sets the maximum X-size of gaps that will be filled. If a gap is larger than the fill threshold in the X or Y direction, it is not replaced. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_INFINITE.	c		e	f	g	a		i		j	k	l	m	n	b
M_INFINITE	Specifies no maximum in the X-direction.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the maximum X-size, in pixel or world units, depending on the units specified with M_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
M_FILL_THRESHOLD_Y	Sets the maximum Y-size of gaps that will be filled. If a gap is larger than the fill threshold in the X or Y direction, it is not replaced. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_INFINITE.	c		e	f	g	a		i		j	k	l	m	n	b
M_INFINITE	Specifies no maximum in the Y-direction.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the maximum Y-size, in pixel or world units, depending on the units specified with M_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
M_INPUT_UNITS	Sets the units with which to interpret the M_FILL_SHARP_ELEVATION_DEPTH, M_FILL_THRESHOLD_X, and M_FILL_THRESHOLD_Y control types. INQ	c		e	f	g	a		i		j	k	l	m	n	b

	M_DEFAULT	Same as M_WORLD.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PIXEL	Specifies to interpret the values in pixel units.	c		e	f	g	a		i		j	k	l	m	n	b
	M_WORLD	Specifies to interpret the values in world units.	c		e	f	g	a		i		j	k	l	m	n	b
Filter context ID		Specifies a filter 3D image processing context, allocated using M3dimAlloc() with M_FILTER_CONTEXT, and used in M3dimFilter() operations.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE_WEIGHT		Sets the amount of smoothing. A larger value applies more smoothing than a smaller value. This control type affects the calculation of the distance Gaussian weight, which is used in the smoothing computation and applies to the distance between points in a neighborhood; the exact calculation depends on the specified weight mode. If M_WEIGHT_MODE is set to M_ABSOLUTE, a point's distance Gaussian weight is set equal to M_DISTANCE_WEIGHT. If M_WEIGHT_MODE is set to M_RELATIVE, the distance Gaussian weight is set to *M_DISTANCE_WEIGHT * d* , where *d* is an internal value calculated from a point's distance to its neighbors and the number of neighborhood points. If you are unsure of exact values to use, you can set M_WEIGHT_MODE to M_RELATIVE, and Aurora Imaging Library will find a heuristic value as a function of the calculated point density, and then you can increase or decrease the multiplicative factors (M_NORMALS_WEIGHT_FACTOR and M_DISTANCE_WEIGHT) to control the degree of smoothing. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the amount of smoothing.	c		e	f	g	a		i		j	k	l	m	n	b
M_FILTER_MODE		Sets the type of smoothing operation to perform. Deciding which mode to use depends on your data and application. In general, M_SMOOTH_MLS performs a simple averaging, while M_SMOOTH_BILATERAL conserves edges. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_SMOOTH_BILATERAL.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SMOOTH_BILATERAL	Specifies to smooth the point cloud using a bilateral filter. A bilateral filter smooths a point cloud while preserving its features; it is effectively an edge-preserving filter. This operation examines a point's distance to its neighbors as well as its distance along a normal direction, to which Gaussian weights are applied. You can control the degree of smoothing and edge preservation with M_DISTANCE_WEIGHT and M_NORMALS_WEIGHT_FACTOR, respectively.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SMOOTH_MLS	Specifies to smooth the point cloud using a moving least squares (MLS) filter. MLS performs a local plane fit at each point, projecting the point onto the locally fitted tangent plane.	c		e	f	g	a		i		j	k	l	m	n	b

M_NORMALS_WEIGHT_FACTOR	Sets how well edges are preserved for an M_SMOOTH_BILATERAL operation. A smaller value results in more edge preservation. This control type sets the point normal Gaussian weight (σ_{normal}) relative to the distance Gaussian weight ($\sigma_{distance}$), according to the following equation: σ_{normal} = M_NORMALS_WEIGHT_FACTOR * $\sigma_{distance}$. This control type is ignored when M_FILTER_MODE is set to M_SMOOTH_MLS. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies how well edges are preserved for a bilateral smoothing operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_USE_SOURCE_NORMALS	Sets whether to use the source point cloud's normals information, stored in the source container's M_COMPONENT_NORMALS_AIL component. Note that point normal values are necessary for the filter operation. Setting this control type to M_FALSE means that a source normals component is not used, but that normals are freshly calculated. When M_TRUE is specified, point normals are not recalculated if they exist in the source container. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_TRUE.	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies not to use source normals information.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies to use source normals information.	c		e	f	g	a		i		j	k	l	m	n	b
M_WEIGHT_MODE	Sets how the Gaussian weights are determined in the smoothing operation set with M_FILTER_MODE. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_RELATIVE.	c		e	f	g	a		i		j	k	l	m	n	b
M_ABSOLUTE	Specifies to use the distance Gaussian weight directly, without applying a multiplying factor.	c		e	f	g	a		i		j	k	l	m	n	b
M_RELATIVE	Specifies to compute and apply a multiplying factor to the distance Gaussian weight. The relative weight (multiplying factor) is computed as a function of point distances to their neighbors and the number of points in the neighborhood.	c		e	f	g	a		i		j	k	l	m	n	b
Lattice context ID	Specifies a lattice 3D image processing context, allocated using M3dimAlloc() with M_LATTICE_CONTEXT, and used in M3dimLattice() operations.	c		e	f	g	a		i		j	k	l	m	n	b

M_FRACTION_OF_POINTS	Sets the ratio of the total number of required (non-empty) cells to the total number of valid points in the point cloud. Aurora Imaging Library will try to divide the point cloud into cells such that the required number of cells, +/- the specified tolerance (M_FRACTION_OF_POINTS_TOLERANCE), contain at least 1 valid point each. Depending on the spacing between points, the lattice might have extra cells that are empty or that contain only invalid points. If your point cloud contains 100 valid points and you specify a fraction of 0.25, M3dimLattice() will attempt to divide the valid points into approximately 25 cells. You can specify the tolerance for this amount using M_FRACTION_OF_POINTS_TOLERANCE. Given the example above, if you specify a tolerance of 0.1, M3dimLattice() will attempt to calculate a lattice such that the number of cells with valid points is between 15 and 35. M_FRACTION_OF_POINTS determines the number of valid points that will be left in the point cloud after subsampling and the number of valid depth map pixels after projecting. When using the calculated lattice to subsample the point cloud and the resulting point cloud is organized (M_ORGANIZATION_TYPE is set to M_ORGANIZED), M3dimSample() selects 1 point per cell, where empty cells and cells containing only invalid points result in points with 0 confidence. When the resulting point cloud is unorganized (M_ORGANIZATION_TYPE is set to M_UNORGANIZED), M3dimSample() ignores empty cells and cells containing only invalid points. When using the calculated lattice to project the point cloud, M3dimProject() projects all the points in a cell to the same depth map pixel. Empty cells and cells containing only invalid points will result in invalid pixels. This control type is only available when M_LATTICE_MODE is set to M_FRACTION_OF_POINTS. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.5.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 < *Value* <= 1.0	Specifies the ratio of required (non-empty) cells to the total number of valid points.	c		e	f	g	a		i		j	k	l	m	n	b
M_FRACTION_OF_POINTS_TOLERANCE	Sets the tolerance to use for M_FRACTION_OF_POINTS when calculating the lattice. M3dimLattice() will attempt to calculate a lattice where the minimum number of cells containing valid points is the total number of valid points in the point cloud * (M_FRACTION_OF_POINTS - M_FRACTION_OF_POINTS_TOLERANCE) and the maximum is the total number of valid points in the point cloud * (M_FRACTION_OF_POINTS + M_FRACTION_OF_POINTS_TOLERANCE). Note that the lattice is bound by the structure of the points. Some fractions are not always possible. Note also that the lower the tolerance, the longer M3dimLattice() will take to complete. This control type is only available when M_LATTICE_MODE is set to M_FRACTION_OF_POINTS. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.05.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 < *Value* < 1.0	Specifies the tolerance for the number of required (non-empty) cells.	c		e	f	g	a		i		j	k	l	m	n	b
M_LATTICE_MODE	Sets the mode with which to calculate the lattice. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_FRACTION_OF_POINTS.	c		e	f	g	a		i		j	k	l	m	n	b
M_FRACTION_OF_POINTS	Specifies to calculate a lattice that divides points into cells based on the required ratio of non-empty cells to the number of valid points in the point cloud. Specify the required ratio using M_FRACTION_OF_POINTS and M_FRACTION_OF_POINTS_TOLERANCE.	c		e	f	g	a		i		j	k	l	m	n	b

M_SPARSITY_CHANGE	Specifies to calculate a lattice that divides points into cells based on a plateau of sparsity changes as the grid size increases. If there are many grid sizes in which the ratio of non-empty cells to empty cells becomes stable, use M_GRID_SIZE_INDEX to specify which grid size to use.	c		e	f	g	a		i		j	k	l	m	n	b
M_RELATIVE_CELL_SIZE_X	Sets the proportion of the cell size along the X dimension relative to the cell sizes along the Y and Z dimensions. For example, if you set M_RELATIVE_CELL_SIZE_X to 1.0, M_RELATIVE_CELL_SIZE_Y to M_INFINITE, and M_RELATIVE_CELL_SIZE_Z to 2.0, the cells will be twice as long in the Z direction compared to the X direction. Note that when subsampling using the calculated lattice, calling M3dimSample() generates an error if more than 1 of M_RELATIVE_CELL_SIZE_X, M_RELATIVE_CELL_SIZE_Y, or M_RELATIVE_CELL_SIZE_Z is set to M_INFINITE. Note that when projecting using the calculated lattice, M_RELATIVE_CELL_SIZE_Z must be set to M_INFINITE and M_RELATIVE_CELL_SIZE_X and M_RELATIVE_CELL_SIZE_Y must be set to non-infinite values. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_INFINITE	Specifies an infinite cell size along X.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the relative cell size along X.	c		e	f	g	a		i		j	k	l	m	n	b
M_RELATIVE_CELL_SIZE_Y	Sets the proportion of the cell size along the Y dimension relative to the cell sizes along the X and Z dimensions. For example, if you set M_RELATIVE_CELL_SIZE_X to 1.0, M_RELATIVE_CELL_SIZE_Y to M_INFINITE, and M_RELATIVE_CELL_SIZE_Z to 2.0, the cells will be twice as long in the Z direction compared to the X direction. Note that when subsampling using the calculated lattice, calling M3dimSample() generates an error if more than 1 of M_RELATIVE_CELL_SIZE_X, M_RELATIVE_CELL_SIZE_Y, or M_RELATIVE_CELL_SIZE_Z is set to M_INFINITE. Note that when projecting using the calculated lattice, M_RELATIVE_CELL_SIZE_Z must be set to M_INFINITE and M_RELATIVE_CELL_SIZE_X and M_RELATIVE_CELL_SIZE_Y must be set to non-infinite values. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_INFINITE	Specifies an infinite cell size along Y.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the relative cell size along Y.	c		e	f	g	a		i		j	k	l	m	n	b

M_RELATIVE_CELL_SIZE_Z	Sets the proportion of the cell size along the Z dimension relative to the cell sizes along the X and Y dimensions. For example, if you set M_RELATIVE_CELL_SIZE_X to 1.0, M_RELATIVE_CELL_SIZE_Y to M_INFINITE, and M_RELATIVE_CELL_SIZE_Z to 2.0, the cells will be twice as long in the Z direction compared to the X direction. Note that when subsampling using the calculated lattice, calling M3dimSample() generates an error if more than 1 of M_RELATIVE_CELL_SIZE_X, M_RELATIVE_CELL_SIZE_Y, or M_RELATIVE_CELL_SIZE_Z is set to M_INFINITE. Note that when projecting using the calculated lattice, M_RELATIVE_CELL_SIZE_Z must be set to M_INFINITE and M_RELATIVE_CELL_SIZE_X and M_RELATIVE_CELL_SIZE_Y must be set to non-infinite values. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_INFINITE	Specifies an infinite cell size along Z.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the relative cell size along Z.	c		e	f	g	a		i		j	k	l	m	n	b
M_SPARSITY_DIMENSION_TOLERANCE	Sets the tolerance to use for identifying sparsity changes in grid sizes when calculating the lattice. M3dimLattice() will attempt to calculate a lattice where the minimum difference in dimensionality between consecutive grid sizes is at least M_SPARSITY_DIMENSION_TOLERANCE. This ensures that sparsity changes are significant enough to distinguish meaningful changes. Note that the lattice is bound by the density and distribution of the points. If the tolerance is too low, the calculation might include insignificant sparsity changes, potentially increasing processing time and noise. Conversely, setting a higher tolerance will result in fewer but more distinct sparsity changes. This control type is only available when M_LATTICE_MODE is set to M_SPARSITY_CHANGE. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.9.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the minimum allowable difference in dimensionality between consecutive grid sizes used in the lattice calculation.	c		e	f	g	a		i		j	k	l	m	n	b
Lattice result buffer ID	Specifies a lattice 3D image processing result buffer, allocated using M3dimAllocResult() with M_LATTICE_RESULT, and used to store M3dimLattice() results.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRID_SIZE_INDEX	Sets the grid size for which to return all lattice results. You can retrieve the number of grid sizes using M3dimGetResult() with M_NUMBER_OF_GRID_SIZES. Note that as the grid size increases the number of points within each cell grows according to the power of the number of dimensions that are merged. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_FIRST.	c		e	f	g	a		i		j	k	l	m	n	b
M_FIRST	Specifies to use the first grid size.	c		e	f	g	a		i		j	k	l	m	n	b
M_LAST	Specifies to use the last grid size.	c		e	f	g	a		i		j	k	l	m	n	b

Value >= 0	Specifies the index of the grid size to use. This value must be less than the number of grid sizes (M3dimGetResult() with M_NUMBER_OF_GRID_SIZES). If the index is invalid, an error will occur when calling M3dimGetResult() or M3dimCopyResult(). Note, the smallest grid size is at index 0.	c		e	f	g	a		i		j	k	l	m	n	b
Mesh context ID	Specifies a mesh 3D image processing context, allocated using M3dimAlloc() with M_MESH_CONTEXT, and used in M3dimMesh() operations. When setting mesh control types, first use M_MESH_MODE to set the surface reconstruction mode, and then specify appropriate settings as per the specified mode. Not all control types are available for each mode.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX_DISTANCE	Sets the maximum distance at which to connect 2 neighboring points. This control type is only available when M_MESH_MODE is set to M_MESH_ORGANIZED. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_INFINITE.	c		e	f	g	a		i		j	k	l	m	n	b
M_INFINITE	Specifies no distance constraint.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the maximum distance. Any 2 points separated by a distance greater than the specified value will not be connected.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAXIMUM_NUMBER_NEIGHBORS	Sets the maximum number of neighbors to consider when computing a local plane at each point. This control type is available only when M_MESH_MODE is set to M_MESH_LOCAL_PLANE. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 100.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the maximum number of neighbor points.	c		e	f	g	a		i		j	k	l	m	n	b
M_MESH_MODE	Sets the surface reconstruction mode to use for mesh generation. When you set the mode to M_MESH_SMOOTHED, you can trim the mesh to discard outlier points with M_MESH_TRIMMING_LEVEL. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_MESH_SMOOTHED.	c		e	f	g	a		i		j	k	l	m	n	b
M_MESH_LOCAL_PLANE	Specifies to generate the mesh from local planes that are fit to neighboring points. For each point, neighborhood points are projected to the local plane and the most suitable points in the source point cloud are chosen to form a mesh triangle with the point in question. With this mode, the mesh can have holes. Normal vectors at each point are required. To calculate normal vectors, use M3dimNormals().	c		e	f	g	a		i		j	k	l	m	n	b
M_MESH_ORGANIZED	Specifies to generate the mesh from an organized point cloud or depth map. This mode connects neighboring points and can result in a mesh with holes due to invalid points in the point cloud or depth map.	c		e	f	g	a		i		j	k	l	m	n	b

	M_MESH_SMOOTHED	Specifies to generate a mesh with a smoothed surface. With this mode, Aurora Imaging Library creates a new point cloud that has been smoothed to reduce noise; missing data is approximated when possible, resulting in significantly fewer holes. The operation uses a Poisson surface reconstruction algorithm. Normal vectors at each point are required. To calculate normal vectors, use M3dimNormals().	c		e	f	g	a		i		j	k	l	m	n	b
M_MESH_STEP_X		Sets the step interval between points, along the X-axis of the range component, in pixels. The M3dimMesh() function connects points that are separated by the specified interval, when creating triangular planes for surface reconstruction. M_MESH_STEP_X specifies an interval in X for each existing band. Note that, if an invalid point is located at the specified interval (that is, the point's confidence score is 0), no vertex of a triangular plane is formed at this point, and a gap is possible in the resulting mesh. This control type is only available when M_MESH_MODE is set to M_MESH_ORGANIZED. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 1.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the step interval in the X direction, in pixel units.	c		e	f	g	a		i		j	k	l	m	n	b
M_MESH_STEP_Y		Sets the step interval between points, along the Y-axis of the range component, in pixels. The M3dimMesh() function connects points that are separated by the specified interval, when creating triangular planes for surface reconstruction. M_MESH_STEP_Y specifies an interval in Y for each existing band. Note that, if an invalid point is located at the specified interval (that is, the point's confidence score is 0), no vertex of a triangular plane is formed at this point, and a gap is possible in the resulting mesh. This control type is only available when M_MESH_MODE is set to M_MESH_ORGANIZED. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 1.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the step interval in the Y direction, in pixel units.	c		e	f	g	a		i		j	k	l	m	n	b
M_MESH_TRIMMING_LEVEL		Sets the trimming level for a mesh, which is applied as a threshold to discard outlier points. Setting a higher value will trim more from the mesh. Note that Aurora Imaging Library applies the threshold to an internal measurement that is related to the density of the vertices in the mesh. A typical value for this control type is 7.0. To disable mesh trimming, set M_MESH_TRIMMING_LEVEL to 0.0. This control type is available only when M_MESH_MODE is set to M_MESH_SMOOTHED. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
	M_AUTO	Specifies to automatically calculate the trimming level.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0.0	Specifies the trimming level.	c		e	f	g	a		i		j	k	l	m	n	b

M_NEIGHBOR_SEARCH_MODE	Sets the search mode for finding the neighbors of a point. This control type is available only when M_MESH_MODE is set to M_MESH_LOCAL_PLANE. Note that M_TREE is typically the most precise because it chooses points that are closest in 3D space, using the Euclidean distance. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_TREE.	c		e	f	g	a		i		j	k	l	m	n	b
M_ORGANIZED	Specifies to use the point cloud's organizational structure to determine the neighbors of a point. This option is supported only for an organized point cloud.	c		e	f	g	a		i		j	k	l	m	n	b
M_TREE	Specifies to use a KD tree search mode to determine a point's neighbors.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEIGHBORHOOD_DISTANCE	Sets the distance that defines the neighborhood of a point; only points within this distance are considered when calculating the local plane of the point. This control type is available only when M_MESH_MODE is set to M_MESH_LOCAL_PLANE. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_INFINITE.	c		e	f	g	a		i		j	k	l	m	n	b
M_INFINITE	Specifies no distance constraint.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the neighborhood distance limit, in world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEIGHBORHOOD_ORGANIZED_SIZE	Sets the neighborhood size when finding the neighbors of a point in an organized point cloud. The size determines the number of elements along a row or column of the 2D buffer of the range and confidence components. For example, if you set M_NEIGHBORHOOD_ORGANIZED_SIZE to 7, the neighborhood is a 7-element by 7-element region in the source container's 2D grid organization (specifically, in the organized point cloud's M_COMPONENT_RANGE and M_COMPONENT_CONFIDENCE components). This control type is supported only when using an organized point cloud and only if M_NEIGHBOR_SEARCH_MODE is set to M_ORGANIZED. This control type is available only when M_MESH_MODE is set to M_MESH_LOCAL_PLANE. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 9.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 3	Specifies the organized neighborhood size, in number of pixels; the specified value must be an odd number.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_POINTS_PER_CELL	Sets the number of points within a single octree cell at which the cell will not be split further. This control type is only available when M_MESH_MODE is set to M_MESH_SMOOTHED. For example, if M_NUMBER_POINTS_PER_CELL is set to 3, any octree cell with 3 or fewer points is not divided further. A larger M_NUMBER_POINTS_PER_CELL can minimize effects due to noise in the source cloud data. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the number of points, within a single octree cell, at which the cell will not be split further.	c		e	f	g	a		i		j	k	l	m	n	b

M_SURFACE_ANGLE_MAX	Sets the maximum acceptable angle between the normal of a point and that of a neighboring point for the neighbor to be considered for triangulation. If the angle is within this limit, the neighboring point could be part of a triangle for the mesh. This control type is available only when M_MESH_MODE is set to M_MESH_LOCAL_PLANE. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 45.0.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 < *Value* < 180.0	Specifies the maximum angle, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_TREE_MAX_DEPTH	Sets the maximum allowed depth of the octree. The octree partitions the 3D space for the mesh calculation. This control type is only available when M_MESH_MODE is set to M_MESH_SMOOTHED. The maximum tree depth determines the resolution of the 3D mesh. For example, an M_TREE_MAX_DEPTH set to 8 gives a mesh with a resolution of $2^8 \times 2^8 \times 2^8$. Note that a higher octree value gives a higher resolution, but a slower calculation time. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 8.	c		e	f	g	a		i		j	k	l	m	n	b
5 <= *Value* <= 12	Specifies the maximum octree depth.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRIANGLE_ANGLE_MAX	Sets the maximum interior angle for each mesh triangle. The specified angle must be greater than or equal to the minimum interior angle (M_TRIANGLE_ANGLE_MIN). Note that this maximum is guaranteed; larger angles will not occur. This control type is available only when M_MESH_MODE is set to M_MESH_LOCAL_PLANE. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 120.0.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 < *Value* < 180.0	Specifies the maximum angle, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRIANGLE_ANGLE_MIN	Sets the minimum interior angle for each mesh triangle. The specified angle must be less than or equal to the maximum interior angle (M_TRIANGLE_ANGLE_MAX). Note that this minimum is not guaranteed; smaller angles might occur, due to the greedy nature of the meshing algorithm. This control type is available only when M_MESH_MODE is set to M_MESH_LOCAL_PLANE. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 10.0.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 < *Value* < 180.0	Specifies the minimum angle, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
Normals context ID	Specifies a normals 3D image processing context, allocated using M3dimAlloc() with M_NORMALS_CONTEXT, and used in M3dimNormals() operations.	c		e	f	g	a		i		j	k	l	m	n	b

M_DIRECTION_MODE	Sets the orientation of the normal vectors, towards a reference point or vector set with M_DIRECTION_REFERENCE_..., which are either the X-, Y-, and Z-coordinates of the reference point, or the X-, Y-, and Z-components of the reference vector. A point on a 3-dimensional surface has 2 normal vectors, each perpendicular to the surface, but pointing in opposite directions. M_DIRECTION_MODE sets a preferred orientation for the normals, and therefore sets which normal vector to choose for each point. For example, you can specify a reference point inside your 3D object and use M_AWAY_FROM_POSITION to ensure that all normal vectors point outwards from the object's surface. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_TOWARDS_DIRECTION.	c		e	f	g	a		i		j	k	l	m	n	b
M_AWAY_FROM_POSITION	Specifies to choose, for each point, the normal vector that points away from the reference point specified with coordinates M_DIRECTION_REFERENCE_X, M_DIRECTION_REFERENCE_Y, and M_DIRECTION_REFERENCE_Z.	c		e	f	g	a		i		j	k	l	m	n	b
M_TOWARDS_DIRECTION	Specifies to choose, for each point, the normal vector that most closely aligns with the reference vector specified with vector components M_DIRECTION_REFERENCE_X, M_DIRECTION_REFERENCE_Y, and M_DIRECTION_REFERENCE_Z.	c		e	f	g	a		i		j	k	l	m	n	b
M_TOWARDS_POSITION	Specifies to choose, for each point, the normal vector that points towards the reference point specified with coordinates M_DIRECTION_REFERENCE_X, M_DIRECTION_REFERENCE_Y, and M_DIRECTION_REFERENCE_Z.	c		e	f	g	a		i		j	k	l	m	n	b
M_DIRECTION_REFERENCE_X	Sets the X-coordinate or vector component along X for the reference point or vector (respectively) that determines normal vector orientation. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the X-coordinate or vector component along X.	c		e	f	g	a		i		j	k	l	m	n	b
M_DIRECTION_REFERENCE_Y	Sets the Y-coordinate or vector component along Y for the reference point or vector (respectively) that determines normal vector orientation. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the Y-coordinate or vector component along Y.	c		e	f	g	a		i		j	k	l	m	n	b
M_DIRECTION_REFERENCE_Z	Sets the Z-coordinate or vector component along Z for the reference point or vector (respectively) that determines normal vector orientation. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the Z-coordinate or vector component along Z.	c		e	f	g	a		i		j	k	l	m	n	b

M_ENFORCE_CONTINUITY	Sets whether to maintain the general direction of the normals, which prevents large, discontinuous jumps between normal vectors. For example, if 2 neighboring points have normal vectors (0,0,1) and (0,0,-0.9), 1 of the vectors will be flipped so that both point roughly in the same direction. This means that some normals will not respect the specified M_DIRECTION_MODE, but, on average, most normals will respect the specified orientation of the normal vectors. This option is supported only when M_NEIGHBOR_SEARCH_MODE is set to M_TREE. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to not enforce continuity of normal vectors.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enforce continuity of normal vectors.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAXIMUM_NUMBER_NEIGHBORS	Sets the maximum number of neighbors to consider when computing a point's normal vector. You must specify M_MAXIMUM_NUMBER_NEIGHBORS when using an M_TREE neighbor search mode. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 49.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the maximum number of neighbor points.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEIGHBOR_SEARCH_MODE	Sets the search mode for finding the neighbors of a point, when calculating a point's surface normal vector. Note that M_ORGANIZED is usually faster, but is less robust to noise. M_TREE is typically the most precise because it chooses points that are closest in 3D space, using the Euclidean distance. You can optionally specify M_NEIGHBORHOOD_DISTANCE to reduce the effect of outlier neighborhood points. Invalid points in a neighborhood are not considered when calculating normal vectors. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_TREE.	c		e	f	g	a		i		j	k	l	m	n	b
M_MESH	Specifies to use the point cloud's mesh component to determine the neighbors of a point. A mesh's triangular faces are composed of connected neighboring points. > This option is available only when the source container has an M_COMPONENT_MESH_AIL component.	c		e	f	g	a		i		j	k	l	m	n	b
M_ORGANIZED	Specifies to use the point cloud's organizational structure to determine the neighbors of a point. This option is supported only for an organized point cloud. When M_ORGANIZED is specified, you must set either M_MAXIMUM_NUMBER_NEIGHBORS or M_NEIGHBORHOOD_ORGANIZED_SIZE to determine the extent of the neighborhood.	c		e	f	g	a		i		j	k	l	m	n	b
M_TREE	Specifies to use a KD tree search mode to determine a point's neighbors. When M_TREE is specified, you must also set M_MAXIMUM_NUMBER_NEIGHBORS.	c		e	f	g	a		i		j	k	l	m	n	b

M_NEIGHBORHOOD_DISTANCE	Sets the distance that defines the neighborhood of a point; only points within this distance are considered for normal vector calculations. This option is not supported when M_NEIGHBOR_SEARCH_MODE is set to M_MESH INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_INFINITE.	c		e	f	g	a		i		j	k	l	m	n	b
M_INFINITE	Specifies no distance constraint.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the neighborhood distance limit, in world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEIGHBORHOOD_ORGANIZED_SIZE	Sets the neighborhood size when finding the neighbors of a point in an organized point cloud. The size determines the number of elements along a row or column of the 2D buffer of the range and confidence components. For example, if you set M_NEIGHBORHOOD_ORGANIZED_SIZE to 7, the neighborhood is a 7-element by 7-element region in the source container's 2D grid organization (specifically, in the organized point cloud's M_COMPONENT_RANGE and M_COMPONENT_CONFIDENCE components). Normal calculations for points close to the edge of the 2D buffer use a clipped neighborhood. This control type is supported only when using an organized point cloud and only if M_NEIGHBOR_SEARCH_MODE is set to M_ORGANIZED. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 7.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 3	Specifies the organized neighborhood size, in number of pixels; the specified value must be an odd number.	c		e	f	g	a		i		j	k	l	m	n	b
Outliers context ID	Specifies an outliers 3D image processing context, allocated using M3dimAlloc() with M_OUTLIERS_CONTEXT, and used in M3dimOutliers() operations.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE_THRESHOLD_MODE	Sets the threshold mode for a local distance operation. The specified mode establishes how to define the cutoff point (threshold) for determining which points are outliers, from a local average distance distribution across all points in the point cloud. A local average distance is the average distance of a point to other points in its neighborhood. To affect the threshold itself, set a scale factor with M_STD_DEVIATION_FACTOR. This control type is supported only when M_OUTLIER_MODE is set to M_LOCAL_DISTANCE. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ROBUST_STD_DEVIATION.	c		e	f	g	a		i		j	k	l	m	n	b
M_ROBUST_STD_DEVIATION	Specifies to determine outliers using a threshold based on the median and the mean absolute deviation (MAD) of the local average distance distribution of points in the point cloud. This mode is more robust to extreme outliers. Since such outliers can skew the mean, using the median is often a more accurate middle value.	c		e	f	g	a		i		j	k	l	m	n	b
M_STD_DEVIATION	Specifies to determine outliers using a threshold based on the mean and the standard deviation of the local average distance distribution.	c		e	f	g	a		i		j	k	l	m	n	b

M_MINIMUM_NUMBER_NEIGHBORS	Sets the minimum number of neighbors that a point must have to be considered an inlier, within the distance that defines the neighborhood. The neighborhood distance is either user-defined or automatically set, depending on the M_NEIGHBORHOOD_DISTANCE_MODE setting. This control type is supported only when M_OUTLIER_MODE is set to M_NUMBER_WITHIN_DISTANCE. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 7.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the minimum number of neighbors.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEIGHBOR_SEARCH_MODE	Sets the search mode for finding the neighbors of a point. Note that M_TREE is typically the most precise because it chooses points that are closest in 3D space, using the Euclidean distance. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_TREE.	c		e	f	g	a		i		j	k	l	m	n	b
M_ORGANIZED	Specifies to use the point cloud's organizational structure to determine the neighbors of a point. > This option is supported only for an organized point cloud.	c		e	f	g	a		i		j	k	l	m	n	b
M_TREE	Specifies to use a KD tree search mode to determine a point's neighbors.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEIGHBORHOOD_DISTANCE	Sets the distance that defines the neighborhood of a point; only points within this distance are considered for outlier calculations. This control type is supported only when M_OUTLIER_MODE is set to M_NUMBER_WITHIN_DISTANCE and M_NEIGHBORHOOD_DISTANCE_MODE is set to M_USER_DEFINED. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the distance.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEIGHBORHOOD_DISTANCE_METRIC	Sets how to establish the neighborhood of a point, either as a spherical volume around the point (M_EUCLIDEAN), or as a distance in the Z-direction (M_DIRECTION_Z). To set the distance limit of the neighborhood, use M_NEIGHBORHOOD_DISTANCE. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_EUCLIDEAN.	c		e	f	g	a		i		j	k	l	m	n	b
M_DIRECTION_Z	Specifies to use a distance in the Z-direction. > This option is supported only when M_NEIGHBOR_SEARCH_MODE is set to M_ORGANIZED and M_OUTLIER_MODE is set to M_NUMBER_WITHIN_DISTANCE.	c		e	f	g	a		i		j	k	l	m	n	b
M_EUCLIDEAN	Specifies to use a Euclidean distance.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEIGHBORHOOD_DISTANCE_MODE	Sets the mode for computing a point's neighborhood distance. This control type is supported only when M_OUTLIER_MODE is set to M_NUMBER_WITHIN_DISTANCE. INQ	c		e	f	g	a		i		j	k	l	m	n	b

	M_DEFAULT	Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
	M_AUTO	Specifies to automatically compute the neighborhood distance.	c		e	f	g	a		i		j	k	l	m	n	b
	M_USER_DEFINED	Specifies to use the distance set with M_NEIGHBORHOOD_DISTANCE.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEIGHBORHOOD_ORGANIZED_SIZE		Sets the neighborhood size when finding the neighbors of a point in an organized point cloud. The size determines the number of elements along a row or column of the 2D buffer of the range and confidence components. For example, if you set M_NEIGHBORHOOD_ORGANIZED_SIZE to 7, the neighborhood is a 7-element by 7-element region in the source container's 2D grid organization (specifically, in the organized point cloud's M_COMPONENT_RANGE and M_COMPONENT_CONFIDENCE components). This control type is supported only when using an organized point cloud and only if M_NEIGHBOR_SEARCH_MODE is set to M_ORGANIZED. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 7.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 3	Specifies the organized neighborhood size, in number of pixels; the specified value must be an odd number.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_NEIGHBORS		Sets the number of neighboring points used to calculate a point's local average distance distribution. This control type is supported only when M_OUTLIER_MODE is set to M_LOCAL_DISTANCE or M_LOCAL_DENSITY_PROBABILITY. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 30.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the number of neighbors.	c		e	f	g	a		i		j	k	l	m	n	b
M_OUTLIER_MODE		Sets the mode for determining whether a point is an outlier. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_LOCAL_DISTANCE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_LOCAL_DENSITY_PROBABILITY	Specifies to use the density of neighboring points and a probability calculation to determine whether a point is an outlier. With this mode, an outlier probability ($P_{outlier_i}$) is assigned to each point, based on the local density of its neighborhood. This probability is then compared to a local outlier threshold (below). If a point's local outlier probability is larger than the calculated local outlier threshold, it is considered an outlier. $P_{outlier_i}$ > *scale_factor* * [*local_avg_distance_i* / *mean(local_avg_distance)*]. The local outlier threshold calculation incorporates a point's local average distance to its neighbors (*local_avg_distance_i*) divided by the mean local average distance calculated from all points (*mean(local_avg_distance)*), all multiplied by a scale factor. Set the scale factor with M_PROBABILITY_THRESHOLD_FACTOR. A smaller scale factor rejects more outliers. This mode is useful when point cloud density is variable. For example, if your point cloud has a sparsely populated region (compared to the whole point cloud), other outlier modes might reject points in that region. Using M_LOCAL_DENSITY_PROBABILITY can reduce this risk.	c		e	f	g	a		i		j	k	l	m	n	b

	M_LOCAL_DISTANCE	Specifies to use a statistical calculation that considers the average distance of a point to other points in its neighborhood to determine whether the point is an outlier. With this mode, a point is deemed an outlier if its average distance to neighboring points is greater than a calculated threshold. The threshold is a function of the mean or median, the standard deviation or the MAD, and the scale factor. Set the number of neighboring points to consider with M_NUMBER_NEIGHBORS. Set the threshold mode and a scale factor for the calculation with M_DISTANCE_THRESHOLD_MODE and M_STD_DEVIATION_FACTOR, respectively.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NUMBER_WITHIN_DISTANCE	Specifies to use the number of points within a defined neighborhood to determine whether a point is an outlier. With this mode, if the number of neighboring points is less than M_MINIMUM_NUMBER_NEIGHBORS, it is considered an outlier. Use M_NEIGHBORHOOD_DISTANCE_METRIC to set either a spherical neighborhood (M_EUCLIDEAN) or one defined along Z (M_DIRECTION_Z). Define the limits of the neighborhood with M_NEIGHBORHOOD_DISTANCE. This is typically the fastest outlier mode.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROBABILITY_THRESHOLD_FACTOR		Sets the scale factor to use in the probability calculation that determines whether a point is an outlier, when working in the M_LOCAL_DENSITY_PROBABILITY outlier mode. A smaller scale factor rejects more outliers. This control type is supported only when M_OUTLIER_MODE is set to M_LOCAL_DENSITY_PROBABILITY. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 1.4.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the scale factor.	c		e	f	g	a		i		j	k	l	m	n	b
M_STD_DEVIATION_FACTOR		Sets the scale factor to use when calculating the local outlier threshold, when working in the M_LOCAL_DISTANCE outlier mode. When M_DISTANCE_THRESHOLD_MODE is set to M_STD_DEVIATION, M_STD_DEVIATION_FACTOR is the factor multiplying the standard deviation of the local average distance distribution. To illustrate, if, in the M_STD_DEVIATION mode, the mean local average distance is 1.0, the standard deviation is 0.3, and M_STD_DEVIATION_FACTOR is set to 3.0, then Aurora Imaging Library calculates the local outlier threshold as follows: *mean + std_deviation_factor * standard_deviation* = 1.0 + 3.0 * 0.3 = 1.9. For this example, points with local average distances larger than 1.9 are considered outliers. When M_DISTANCE_THRESHOLD_MODE is set to M_ROBUST_STD_DEVIATION, M_STD_DEVIATION_FACTOR is the factor multiplying the mean absolute deviation (MAD) of the local average distance distribution. For either mode, a smaller factor defines a smaller threshold, thereby more aggressively rejecting outliers. This control type is supported only when M_OUTLIER_MODE is set to M_LOCAL_DISTANCE. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 3.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the scale factor.	c		e	f	g	a		i		j	k	l	m	n	b
Profile context ID		Specifies a profile 3D image processing context, allocated using M3dimAlloc() with M_PROFILE_CONTEXT, and used in M3dimProfileEx() operations.	c		e	f	g	a		i		j	k	l	m	n	b

M_PROFILE_ACCUMULATE_TYPE	Sets how points in a given sample are accumulated to create a single profile point when there are multiple points within the thickness of the profile line. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_MEAN.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX	Specifies to use the largest value of all the points in the sample. > This option is not supported when M_PROFILE_INTERPOLATION_MODE is set to M_AVERAGE.	c		e	f	g	a		i		j	k	l	m	n	b
M_MEAN	Specifies to use the mean value of all the points in the sample.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN	Specifies to use the smallest value of all the points in the sample. > This option is not supported when M_PROFILE_INTERPOLATION_MODE is set to M_AVERAGE.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROFILE_INTERPOLATION_AVERAGE_FRACTION	Sets the fraction of the sampling distance to use when taking a weighted average of the area surrounding the profile point. For example, a value of 0.5 indicates that 50 percent of the pixel area that surrounds the profile point (defined by 50% of the sampling distance along the line) is used to calculate a weighted average for the profile point. This control type is only available when M_PROFILE_INTERPOLATION_MODE is set to M_AVERAGE. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 < *Value* <= 1.0	Specifies the fraction.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROFILE_INTERPOLATION_MODE	Sets the interpolation mode used to calculate the value for each point in the sample that will be used to create the profile point. Note that when averaging interpolation is used (M_AVERAGE), the profile point is calculated without calculating the value for each point in the sample. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_BILINEAR.	c		e	f	g	a		i		j	k	l	m	n	b
M_AVERAGE	Specifies averaging interpolation. The value of the profile point is determined by taking a weighted average of the pixel area that surrounds the profile point. You can use M_PROFILE_INTERPOLATION_AVERAGE_FRACTION to specify how much of the pixel area to consider.	c		e	f	g	a		i		j	k	l	m	n	b
M_BILINEAR	Specifies bilinear interpolation. The new value is determined by taking a weighted average of the 4 values (2x2) that surround the point in the sample.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEAREST_NEIGHBOR	Specifies nearest neighbor interpolation. The new value is that of the pixel closest to the point in the sample.	c		e	f	g	a		i		j	k	l	m	n	b

M_PROFILE_MIN_VALID_PERCENTAGE	Sets the minimum percentage of points in a given sample that must be valid to set the corresponding profile point to a valid value. When there are multiple points within the thickness for the sample that will be used to create the profile point, the profile point will be invalid if the minimum valid percentage is not met. Note that if results are stored in a profile 3D image processing result buffer, invalid points are not included in the profile. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the percentage of points in the sample. Note that a value of 0.0% means there are no constraints on the number of valid points, but if all points contributing to the calculation of a profile point are invalid, the profile point will be invalid in the destination.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROFILE_SAMPLE_SIZE	Sets the sampling distance between two consecutive points along the profile line. When the pixel size in X is not equal to the pixel size in Y in the source depth map, this ensures the extracted points are at a uniform distance. Note that this will be the pixel size of the profile. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the distance. Use M_PROFILE_SAMPLE_SIZE_MODE to specify how to interpret the specified sampling distance.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROFILE_SAMPLE_SIZE_MODE	Sets how to interpret the specified sampling distance (M_PROFILE_SAMPLE_SIZE). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_RELATIVE_TO_PIXEL_SIZE_MIN.	c		e	f	g	a		i		j	k	l	m	n	b
M_RELATIVE_TO_PIXEL_SIZE_MAX	Specifies to multiply the sampling distance by the pixel size in X or in Y, depending on which is bigger.	c		e	f	g	a		i		j	k	l	m	n	b
M_RELATIVE_TO_PIXEL_SIZE_MIN	Specifies to multiply the sampling distance by the pixel size in X or in Y, depending on which is smaller.	c		e	f	g	a		i		j	k	l	m	n	b
M_RELATIVE_TO_PIXEL_SIZE_X	Specifies to multiply the sampling distance by the pixel size in X. For example, if you set M_PROFILE_SAMPLE_SIZE to 2.0 and the pixel size in X is 0.5 mm, the sampling distance will be 1.0 mm.	c		e	f	g	a		i		j	k	l	m	n	b
M_RELATIVE_TO_PIXEL_SIZE_Y	Specifies to multiply the sampling distance by the pixel size in Y. For example, if you set M_PROFILE_SAMPLE_SIZE to 2.0 and the pixel size in Y is 0.6 mm, the sampling distance will be 1.2 mm.	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD	Specifies that the sampling distance be interpreted in world units. For example, if you set M_PROFILE_SAMPLE_SIZE to 2.0, the sampling distance will be 2.0 world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROFILE_THICKNESS	Sets the thickness of the profile line (slicing plane). Pixels along the line, within the specified thickness, are sampled and corresponding points are considered for the profile. Note that a thick profile line typically results in multiple points contributing to the same profile point. You can use M_PROFILE_ACCUMULATE_TYPE to determine how multiple point values are combined to set the value of the corresponding profile point. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b

Value > 0.0	Specifies the thickness. Use M_PROFILE_THICKNESS_MODE to specify how to interpret the specified thickness.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROFILE_THICKNESS_MODE	Sets how to interpret the specified thickness value (M_PROFILE_THICKNESS). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_RELATIVE_TO_PIXEL_SIZE_MIN.	c		e	f	g	a		i		j	k	l	m	n	b
M_RELATIVE_TO_PIXEL_SIZE_MAX	Specifies to multiply the thickness value by the pixel size in X or in Y, depending on which is bigger.	c		e	f	g	a		i		j	k	l	m	n	b
M_RELATIVE_TO_PIXEL_SIZE_MIN	Specifies to multiply the thickness value by the pixel size in X or in Y, depending on which is smaller.	c		e	f	g	a		i		j	k	l	m	n	b
M_RELATIVE_TO_PIXEL_SIZE_X	Specifies to multiply the thickness value by the pixel size in X. For example, if you set M_PROFILE_THICKNESS to 2.0 and the pixel size in X is 0.5 mm, the profile line will be 1.0 mm thick.	c		e	f	g	a		i		j	k	l	m	n	b
M_RELATIVE_TO_PIXEL_SIZE_Y	Specifies to multiply the thickness value by the pixel size in Y. For example, if you set M_PROFILE_THICKNESS to 2.0 and the pixel size in Y is 0.6 mm, the profile line will be 1.2 mm thick.	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD	Specifies that the thickness value be interpreted in world units. For example, if you set M_PROFILE_THICKNESS to 2.0, the profile line will be 2.0 world units thick.	c		e	f	g	a		i		j	k	l	m	n	b
Project context ID	Specifies a project 3D image processing context, allocated using M3dimAlloc() with M_PROJECT_CONTEXT, and used in M3dimProjectEx() operations.	c		e	f	g	a		i		j	k	l	m	n	b
M_CALCULATE_SIZE_MODE	Sets whether the source point cloud container is organized. M3dimProjectEx() calculations are faster with organized point clouds. This control type is ignored when the source is a 3D geometry object. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_UNORGANIZED.	c		e	f	g	a		i		j	k	l	m	n	b
M_ORGANIZED	Specifies an organized point cloud. Note that using this value with an unorganized point cloud will cause an error.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNORGANIZED	Specifies an unorganized point cloud.	c		e	f	g	a		i		j	k	l	m	n	b
M_CALIBRATION_ALIGNMENT	Sets where to place valid values in the depth map. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_TOP_LEFT.	c		e	f	g	a		i		j	k	l	m	n	b
M_CENTER	Specifies to project the data such that it is centered horizontally or vertically in the buffer. Pixels in unused rows or columns are set to the invalid pixel value. The source data maintains its aspect ratio.	c		e	f	g	a		i		j	k	l	m	n	b
M_TOP_LEFT	Specifies to project the data such that it aligns to the top left of the buffer, and the data extends at least the breadth of one dimension, leaving space at the bottom or right. Pixels in unused rows or columns are set to the invalid pixel value. The source data maintains its aspect ratio.	c		e	f	g	a		i		j	k	l	m	n	b

M_DEPTH_MAP_TYPE	Sets the data type and depth required for the destination container's range component. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_UNSIGNED + 16.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNSIGNED + 8	Specifies 8-bit unsigned data.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNSIGNED + 16	Specifies 16-bit unsigned data.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNSIGNED + 32	Specifies 32-bit unsigned data.	c		e	f	g	a		i		j	k	l	m	n	b
M_OVERLAP_MODE	Sets the overlap mode, which determines a pixel's gray level when multiple points correspond to the same destination depth map pixel. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_MAX_Z.	c		e	f	g	a		i		j	k	l	m	n	b
M_AVERAGE	Specifies to use the average gray level of all the 3D points projected on a single depth map pixel for custom components. Note that for range components, the average Z-coordinate is used. For reflectance and intensity components, the average value is used. For normals components, the average vector is renormalized and flipped towards the most common direction. Note that no other components are projected. > Note that this value is not available when the source is a 3D geometry object.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX_INTENSITY	Specifies to use the gray level of the 3D point with the highest intensity. The source container must have a reflectance or intensity component. If both reflectance and intensity components exist, only the reflectance component is considered. > Note that this value is not available when the source is a 3D geometry object.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX_Z	Specifies to use the gray level of the 3D point with the largest Z-value, of all the 3D points projected on a single depth map pixel. Note that this value can be bright or dark, depending on whether the Z-scale is positive or negative (M_SIGN_Z set to M_POSITIVE or M_NEGATIVE).	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_Z	Specifies to use the gray level of the 3D point with the smallest Z-value, of all the 3D points projected on a single depth map pixel. Note that this value can be dark or bright, depending on whether the Z-scale is positive or negative (M_SIGN_Z set to M_POSITIVE or M_NEGATIVE).	c		e	f	g	a		i		j	k	l	m	n	b
M_MOST_FREQUENT	Specifies to use the most frequent gray level for custom components of type M_UNSIGNED or M_SIGNED, or the average gray level for custom components of type M_FLOAT, of all the 3D points projected on a single depth map pixel. Note that for range components, the average Z-coordinate is used. For reflectance and intensity components, the average value is used. For normals components, the average vector is renormalized and flipped towards the most common direction. Note that no other components are projected.	c		e	f	g	a		i		j	k	l	m	n	b

	M_OVERWRITE	Specifies to use the last gray level of all the 3D points projected on a single depth map pixel. This mode continually overwrites the previous gray value for the pixel. M_OVERWRITE can be marginally faster than the other overlap modes.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL_ASPECT_RATIO		Sets the pixel aspect ratio of the depth map. The pixel aspect ratio is used to calculate the depth map buffer size when the Options parameter of M3dimProjectEx() is set to M_USE_BOX_WORLD_SIZE or M_USE_SRC_DATA, and to apply the calibration when not using the M_ACCUMULATE option. This control type must be set to M_NULL if you have already specified a size in X with M_PIXEL_SIZE_X or M_SIZE_X and a size in Y with M_PIXEL_SIZE_Y or M_SIZE_Y. Otherwise, an error will occur. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NULL	Specifies that other data determines the pixel aspect ratio, such as the values set with M_PIXEL_SIZE_X and M_PIXEL_SIZE_Y.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies an aspect ratio for the depth map's pixels. The pixel size is computed such that the pixel's length in X divided by its length in Y equals the specified M_PIXEL_ASPECT_RATIO value.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL_SIZE_X		Sets the length in X of one pixel in the depth map. The pixel size is used to calculate the depth map buffer size when the Options parameter of M3dimProjectEx() is set to M_USE_BOX_WORLD_SIZE or M_USE_SRC_DATA, and to apply the calibration when not using the M_ACCUMULATE option. Note that you cannot set M_PIXEL_SIZE_X at the same time as M_PIXEL_SIZE_Y or M_SIZE_Y, unless M_PIXEL_ASPECT_RATIO is set to M_NULL. Note that you cannot specify a specific value for M_PIXEL_SIZE_X if you have done so for M_SIZE_X (and vice versa). INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_NULL.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NULL	Specifies that M3dimProjectEx() will compute the pixel size in X.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the length in X of one pixel in the depth map, in world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL_SIZE_Y		Sets the length in Y of one pixel in the depth map. The pixel size is used to calculate the depth map buffer size when the Options parameter of M3dimProjectEx() is set to M_USE_BOX_WORLD_SIZE or M_USE_SRC_DATA, and to apply the calibration when not using the M_ACCUMULATE option. Note that you cannot set M_PIXEL_SIZE_Y at the same time as M_PIXEL_SIZE_X or M_SIZE_X, unless M_PIXEL_ASPECT_RATIO is set to M_NULL. Note that you cannot specify a specific value for M_PIXEL_SIZE_Y if you have done so for M_SIZE_Y (and vice versa). INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_NULL.	c		e	f	g	a		i		j	k	l	m	n	b

M_NULL	Specifies that M3dimProjectEx() will compute the pixel size in Y.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the length in Y of one pixel in the depth map, in world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROJECTED_COMPONENTS	Sets which components to project into the destination depth map. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_COMPONENT_RANGE.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_ALL	Specifies to project all components that have the same dimensions as the source container's range component. > Note that using this value with a destination image buffer will cause an error.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_RANGE	Specifies to project the range component.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_RANGE_AND_REFLECTANCE	Specifies to project the range component and the reflectance or intensity component. If both reflectance and intensity components exist, the intensity component is not projected. > Note that using this value with a destination image buffer will cause an error.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROJECTION_MODE	Sets the projection mode, which determines how points are projected. This control type is ignored when the source is a 3D geometry object. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_POINT_BASED.	c		e	f	g	a		i		j	k	l	m	n	b
M_MESH_BASED	Specifies to consider the source container's mesh component when extracting 3D points for projection into the depth map. For a meshed point cloud, each triangular face has 3 vertices, and a vertex's Z-value determines the gray level of its corresponding pixel in the depth map. The remaining pixels (in the depth map) are assigned interpolated gray levels, since they correspond to the triangular faces between the vertices (in the point cloud's mesh). The container must have an M_COMPONENT_MESH_AIL component before using the M_MESH_BASED mode.	c		e	f	g	a		i		j	k	l	m	n	b
M_POINT_BASED	Specifies to project the 3D points directly into the depth map.	c		e	f	g	a		i		j	k	l	m	n	b
M_SATURATION	Sets whether to saturate pixels. Note that saturation can occur, depending on the Z-scale and offset of the destination depth map buffer. If, for a given pixel of the destination buffer, the corresponding gray level of the 3D geometry or point of the point cloud falls outside the buffer's grayscale range, the pixel will be saturated to either 0 or the buffer's maximum value minus 1 (the maximum value is reserved to represent missing data). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to saturate pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to saturate pixels.	c		e	f	g	a		i		j	k	l	m	n	b

M_SIGN_Z	Sets the sign of the Z-scale in the destination depth map. The sign applies to M_3D_SCALE_Z, which sets the length, in world units, of one gray level in a depth map container. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_POSITIVE.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEGATIVE	Specifies a negative value for the Z-scale. A negative value indicates that larger Z-values are represented by darker depth map pixels, and smaller Z-values are represented by brighter depth map pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITIVE	Specifies a positive value for the Z-scale. A positive value indicates that larger Z-values are represented by brighter depth map pixels, and smaller Z-values are represented by darker depth map pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIZE_X	Sets the size of the depth map along the X dimension. Note that you cannot set M_SIZE_X at the same time as M_PIXEL_SIZE_Y or M_SIZE_Y, unless M_PIXEL_ASPECT_RATIO is set to M_NULL. Note that you cannot specify a specific value for M_SIZE_X if you have done so for M_PIXEL_SIZE_X (and vice versa). This control type is only used when the Options parameter of M3dimProjectEx() is set to M_USE_BOX_WORLD_SIZE or M_USE_SRC_DATA. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_NULL.	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies that M3dimProjectEx() will compute the size of the depth map in X.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the size of the depth map in X, in pixel units.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIZE_Y	Sets the size of the depth map along the Y dimension. Note that you cannot set M_SIZE_Y at the same time as M_PIXEL_SIZE_X or M_SIZE_X, unless M_PIXEL_ASPECT_RATIO is set to M_NULL. Note that you cannot specify a specific value for M_SIZE_Y if you have done so for M_PIXEL_SIZE_Y (and vice versa). This control type is only used when the Options parameter of M3dimProjectEx() is set to M_USE_BOX_WORLD_SIZE or M_USE_SRC_DATA. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_NULL.	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies that M3dimProjectEx() will compute the size of the depth map in Y.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the size of the depth map in Y, in pixel units.	c		e	f	g	a		i		j	k	l	m	n	b
Remap context ID	Specifies a remap 3D image processing context, allocated using M3dimAlloc() with M_REMAP_CONTEXT, and used in M3dimRemapDepthMap() operations.	c		e	f	g	a		i		j	k	l	m	n	b
M_DIRECTION_Z	Sets the sign of the Z-scale in the destination depth map. The sign applies to M_GRAY_LEVEL_SIZE_Z, which sets the length, in world units, of one gray level in a fully-corrected depth map. Note that when M_REMAP_MODE is set to M_USE_DESTINATION_CALIBRATION, this control type has no effect. INQ	c		e	f	g	a		i		j	k	l	m	n	b

M_DEFAULT	Same as M_SAME.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEGATIVE	Specifies a negative value for the Z-scale. A negative value indicates that larger Z-values are represented by darker depth map pixels, and smaller Z-values are represented by brighter depth map pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITIVE	Specifies a positive value for the Z-scale. A positive value indicates that larger Z-values are represented by brighter depth map pixels, and smaller Z-values are represented by darker depth map pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_REVERSE	Specifies that the sign of the Z-scale is opposite that of the source depth map.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAME	Specifies that the sign of the Z-scale is the same as that of the source depth map.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX_Z	Sets the maximum Z-value for the destination depth map when M_REMAP_MODE is set to M_USER_DEFINED. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the maximum Z-value.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_Z	Sets the minimum Z-value for the destination depth map when M_REMAP_MODE is set to M_USER_DEFINED. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the minimum Z-value.	c		e	f	g	a		i		j	k	l	m	n	b
M_REMAP_MODE	Sets how to determine the Z-range of the destination depth map. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_BUFFER_LIMITS.	c		e	f	g	a		i		j	k	l	m	n	b
M_BUFFER_LIMITS	Specifies to remap the source depth map using its M_MIN and M_MAX settings as the values to define the destination's Z-range.	c		e	f	g	a		i		j	k	l	m	n	b
M_DATA_EXTREMES	Specifies to remap the source depth map using its maximum and minimum pixel values to define the destination's Z-range.	c		e	f	g	a		i		j	k	l	m	n	b
M_GAUSSIAN	Specifies to remap the source depth map using a specified number of standard deviations from the mean Z-value to define the destination's Z-range. Set the number of standard deviations with M_STANDARD_DEVIATION_FACTOR, which is applied using the following formula: *Source mean +/- Source standard deviation* * M_STANDARD_DEVIATION_FACTOR.	c		e	f	g	a		i		j	k	l	m	n	b
M_USE_DESTINATION_CALIBRATION	Specifies to remap the source depth map using the destination image buffer's calibration information to define the destination's Z-range.	c		e	f	g	a		i		j	k	l	m	n	b
M_USER_DEFINED	Specifies to remap the source depth map using custom values to define the destination's Z range; use M_MAX_Z and M_MIN_Z to set the range.	c		e	f	g	a		i		j	k	l	m	n	b

M_STANDARD_DEVIATION_FACTOR	Sets the standard deviation factor, for when M_REMAP_MODE is set to M_GAUSSIAN. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 2.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the standard deviation factor.	c		e	f	g	a		i		j	k	l	m	n	b
Statistics context ID	Specifies a statistics 3D image processing context, allocated using M3dimAlloc() with M_STATISTICS_CONTEXT, and used in M3dimStat() operations.	c		e	f	g	a		i		j	k	l	m	n	b
M_BOUNDING_BOX	Sets whether to enable bounding box statistics calculations. The bounding box is an axis-aligned or semi-oriented box that contains all or most of the points in the 3D scene, or it encloses a 3D geometry object. To compute a bounding box (for a point cloud or depth map) that contains most of the valid points instead of all of them, set M_BOUNDING_BOX_ALGORITHM to M_ROBUST. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable bounding box calculations.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable bounding box calculations.	c		e	f	g	a		i		j	k	l	m	n	b
M_BOUNDING_BOX_ALGORITHM	Sets the algorithm to use when bounding box calculations have been enabled (M_BOUNDING_BOX). The default setting (M_ALL_POINTS) includes all valid points; no valid points exist outside the bounding box. Invalid points can exist inside or outside the bounding box. This setting is not available for 3D geometries. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ALL_POINTS.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL_POINTS	Specifies to compute the axis-aligned bounding box that contains all the valid points.	c		e	f	g	a		i		j	k	l	m	n	b
M_ROBUST	Specifies to compute an axis-aligned box that contains most of the points, but rejects outliers.	c		e	f	g	a		i		j	k	l	m	n	b
M_BOUNDING_BOX_OUTLIER_RATIO_X	Sets the fraction of points to exclude from the bounding box, based on X-coordinates. A point is excluded when its X-coordinate places the point in an outlier position, and therefore outside the bounding box. The specified value is a decimal fraction between 0.0 and 1.0. For example, if set to 0.01, the furthest 1% of points along X are excluded. Set this control type when bounding box calculations have been enabled (M_BOUNDING_BOX) and M_BOUNDING_BOX_ALGORITHM is set to M_ROBUST. This setting is not available for 3D geometries. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.002.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* < 1.0	Specifies the fraction.	c		e	f	g	a		i		j	k	l	m	n	b

M_BOUNDING_BOX_OUTLIER_RATIO_Y	Sets the fraction of points to exclude from the bounding box, based on Y-coordinates. A point is excluded when its Y-coordinate places the point in an outlier position, and therefore outside the bounding box. The specified value is a decimal fraction between 0.0 and 1.0. For example, if set to 0.01, the furthest 1% of points along Y are excluded. Set this control type when bounding box calculations have been enabled (M_BOUNDING_BOX) and M_BOUNDING_BOX_ALGORITHM is set to M_ROBUST. This setting is not available for 3D geometries. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.002.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* < 1.0	Specifies the fraction.	c		e	f	g	a		i		j	k	l	m	n	b
M_BOUNDING_BOX_OUTLIER_RATIO_Z	Sets the fraction of points to exclude from the bounding box, based on Z-coordinates. A point is excluded when its Z-coordinate places the point in an outlier position, and therefore outside the bounding box. The specified value is a decimal fraction between 0.0 and 1.0. For example, if set to 0.01, the furthest 1% of points along Z are excluded. Set this option when bounding box calculations have been enabled (M_BOUNDING_BOX) and when M_BOUNDING_BOX_ALGORITHM is set to M_ROBUST. This setting is not available for 3D geometries. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.010.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* < 1.0	Specifies the fraction.	c		e	f	g	a		i		j	k	l	m	n	b
M_BOX_ORIENTATION	Sets whether to compute an axis-aligned or semi-oriented bounding box. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_AXIS_ALIGNED.	c		e	f	g	a		i		j	k	l	m	n	b
M_AXIS_ALIGNED	Specifies to compute the minimum volume axis-aligned bounding box.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEMI_ORIENTED	Specifies to compute the minimum volume bounding box that is, by default, axis-aligned in Z but not in X and Y. You can change the axis in which the semi-oriented bounding box is aligned, using M_BOX_SEMI_ORIENTED_ROTATION_AXIS.	c		e	f	g	a		i		j	k	l	m	n	b
M_BOX_SEMI_ORIENTED_ROTATION_AXIS	Sets the axis with which to align the semi-oriented bounding box, when M_BOX_ORIENTATION is set to M_SEMI_ORIENTED. The minimum volume bounding box is computed such that it is aligned with the specified axis and not constrained by the other two axes. This affects the plane to which the semi-oriented bounding box is parallel. For example, if you set this control type to M_AXIS_X, the semi-oriented bounding box will be axis-aligned in X and parallel to the YZ-plane. This control type is only used when M_BOX_ORIENTATION is set to M_SEMI_ORIENTED. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_AXIS_Z.	c		e	f	g	a		i		j	k	l	m	n	b
M_AXIS_X	Specifies to align the semi-oriented bounding box with the X-axis.	c		e	f	g	a		i		j	k	l	m	n	b

M_AXIS_Y	Specifies to align the semi-oriented bounding box with the Y-axis.	c		e	f	g	a		i		j	k	l	m	n	b
M_AXIS_Z	Specifies to align the semi-oriented bounding box with the Z-axis.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRINCIPAL_AXIS_1	Specifies to align the semi-oriented bounding box with the first principal axis. The first principal axis is the direction along which there is the most variance.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRINCIPAL_AXIS_2	Specifies to align the semi-oriented bounding box with the second principal axis. The second principal axis is the direction along which there is the second most variance, perpendicular to the first principal axis.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRINCIPAL_AXIS_3	Specifies to align the semi-oriented bounding box with the third principal axis. The third principal axis is perpendicular to the first and second principal axes.	c		e	f	g	a		i		j	k	l	m	n	b
M_CALCULATE_AVERAGE	Sets whether to calculate the average value for distance-to-nearest-neighbor, surface variation, aspect ratio, and number-of-valid-neighbors calculations. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to calculate the average value.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to calculate the average value.	c		e	f	g	a		i		j	k	l	m	n	b
M_CALCULATE_MAX	Sets whether to calculate the maximum value for distance-to-nearest-neighbor, surface variation, aspect ratio, and number-of-valid-neighbors calculations. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to calculate the maximum value.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to calculate the maximum value.	c		e	f	g	a		i		j	k	l	m	n	b
M_CALCULATE_MEDIAN	Sets whether to calculate the median value for distance-to-nearest-neighbor, surface variation, aspect ratio, and number-of-valid-neighbors calculations. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to calculate the median.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to calculate the median.	c		e	f	g	a		i		j	k	l	m	n	b
M_CALCULATE_MIN	Sets whether to calculate the minimum value for distance-to-nearest-neighbor, surface variation, aspect ratio, and number-of-valid-neighbors calculations. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to calculate the minimum value.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to calculate the minimum value.	c		e	f	g	a		i		j	k	l	m	n	b

M_CALCULATE_ROBUST_STDEV	Sets whether to calculate the robust standard deviation for distance-to-nearest-neighbor, surface variation, aspect ratio, and number-of-valid-neighbors calculations. This is computed using the median absolute deviation (MAD) and a scale factor of 1.4826. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to calculate the robust standard deviation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to calculate the robust standard deviation.	c		e	f	g	a		i		j	k	l	m	n	b
M_CALCULATE_STDEV	Sets whether to calculate the standard deviation for distance-to-nearest-neighbor, surface variation, aspect ratio, and number-of-valid-neighbors calculations. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to calculate the standard deviation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to calculate the standard deviation.	c		e	f	g	a		i		j	k	l	m	n	b
M_CENTROID	Sets whether to enable centroid statistics calculations. The centroid is the center of mass of a point cloud, depth map, or 3D geometry. Centroid statistics include the X-, Y-, and Z-coordinates of the centroid. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable centroid calculations.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable centroid calculations.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_OF_INTEREST	Sets the component for which to calculate statistics, when computing statistics on a point cloud. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_COMPONENT_RANGE.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_NORMALS_AIL	Specifies to calculate the statistics on the normals component. > Note that surface variation, aspect ratio, number-of-valid-neighbors, bounding box, and nearest-neighbor statistics are not supported on the normals component, and will generate an error.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_RANGE	Specifies to calculate the statistics on the range component.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE_TO_NEAREST_NEIGHBOR	Sets whether to enable distance-to-nearest-neighbor statistics calculations. Distance-to-nearest-neighbor statistics include the average distance, the maximum distance, and the minimum distance of 3D points or real world coordinates to their nearest neighbor. Note that distance-to-nearest-neighbor statistics are not available for 3D geometries. INQ	c		e	f	g	a		i		j	k	l	m	n	b

M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable distance-to-nearest-neighbor calculations.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable distance-to-nearest-neighbor calculations.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX_DISTANCE	Sets the maximum distance from a point within which other points are considered its neighbors when calculating distance-to-nearest-neighbor, surface variation, aspect ratio, and number-of-valid-neighbors statistics. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_INFINITE.	c		e	f	g	a		i		j	k	l	m	n	b
M_INFINITE	Specifies that there is no maximum distance.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the maximum distance.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAXIMUM_NUMBER_NEIGHBORS	Sets the maximum number of neighbors to consider when calculating distance-to-nearest-neighbor, surface variation, aspect ratio, and number-of-valid-neighbors statistics. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 49.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the maximum number of neighbors.	c		e	f	g	a		i		j	k	l	m	n	b
M_MOMENT_ORDER	Sets the order up to which moments are calculated. For example, setting this to 3 will calculate third order moments, but also second and first order moments. Note that there are $(N+1)(N+2)/2$ moments for order N. For example, if M_MOMENT_ORDER is set to 3, the number of computed moments are 10, 6, and 3 for third, second, and first order moments, respectively. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 2.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* <= 15	Specifies the moment order.	c		e	f	g	a		i		j	k	l	m	n	b
M_MOMENTS	Sets whether to enable moments statistics calculations. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable moments calculations.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable moments calculations.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEIGHBOR_SEARCH_MODE	Sets the search mode for finding the neighbors of a point when calculating distance-to-nearest-neighbor, surface variation, aspect ratio, and number-of-valid-neighbors statistics. Note that M_TREE is typically the most precise because it chooses points that are closest in 3D space, using the Euclidean distance. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_TREE.	c		e	f	g	a		i		j	k	l	m	n	b

	M_AUTO	Specifies to automatically set the search mode based on the point cloud's organization, either M_ORGANIZED if organized or M_TREE if unorganized.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ORGANIZED	Specifies to use the point cloud's organizational structure to determine the neighbors of a point. This option is supported only for an organized point cloud.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TREE	Specifies to use a KD tree search mode to determine a point's neighbors.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEIGHBORHOOD_ASPECT_RATIO		Sets whether to enable local aspect ratio statistics calculations. For each point, Aurora Imaging Library fits a local plane to neighboring points and determines that plane's aspect ratio. The aspect ratio is calculated by dividing the central point's distance to its closest neighbor along one axis by the distance to its closest neighbor along the other axis. Local aspect ratio statistics describe how skewed the point density is in one direction compared to the other. You typically do not want an aspect ratio higher than 5. Neighborhood-based functions do not benefit from the point cloud being denser in only one direction. In the worst case, a neighborhood with only a single row of points will produce faulty results. You can use M3dimSample() to subsample the point cloud to help correct aspect ratios that are too high. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies to disable local aspect ratio calculations.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to enable local aspect ratio calculations.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEIGHBORHOOD_ASPECT_RATIO_CLIPPING_VALUE		Sets the maximum value of the neighborhood aspect ratio, to avoid skewed statistics. For example, a neighborhood that looks like a line might have an infinite aspect ratio, which will disproportionately affect statistics such as M_NEIGHBORHOOD_ASPECT_RATIO_AVERAGE and M_NEIGHBORHOOD_ASPECT_RATIO_STDEV. To avoid this, it can be clipped to a maximum value using this control. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 100.0. This means that when the aspect ratio exceeds 100:1 (that is, 100 times more points in one direction), the aspect ratio will be clipped to 100 in statistic calculations.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 1.0	Specifies the maximum value.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEIGHBORHOOD_HEALTHY_THRESHOLD		Sets the minimum number of neighbors that constitute a healthy neighborhood for the M_NEIGHBORHOOD_HEALTHY_PERCENTAGE result type (M3dimGetResult()). Most neighborhood-based functions (for example, M3dimNormals()) will work poorly if the percentage of points with a healthy neighborhood is too low. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 7.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the number of neighbors.	c		e	f	g	a		i		j	k	l	m	n	b

M_NEIGHBORHOOD_ORGANIZED_SIZE	Sets the neighborhood size when finding the neighbors of a point in an organized point cloud for distance-to-nearest-neighbor, surface variation, aspect ratio, and number-of-valid-neighbors calculations. The size determines the number of elements along a row or column of the 2D buffer of the range and confidence components. For example, if you set M_NEIGHBORHOOD_ORGANIZED_SIZE to 7, the neighborhood is a 7-element by 7-element region in the source container's 2D grid organization (specifically, in the organized point cloud's M_COMPONENT_RANGE and M_COMPONENT_CONFIDENCE components). This control type is supported only when using an organized point cloud and only if M_NEIGHBOR_SEARCH_MODE is set to M_ORGANIZED. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 7.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 3	Specifies the organized neighborhood size, in number of pixels; the specified value must be an odd number.	c		e	f	g	a		i		j	k	l	m	n	b
M_NORMALIZATION_MODE	Sets whether the normalization matrix should transform the point cloud or 3D geometry to fit a signed (M_NORMALIZE_SIGNED) or unsigned (M_NORMALIZE_UNSIGNED) unit box. A signed unit box has a maximum length of 2 (stretches from -1 to +1 along at least one dimension) and is centered at (0,0,0). An unsigned unit box has a maximum length of 1 (stretches from 0 to +1 along at least one dimension) and is centered at (0.5,0.5,0.5). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_NORMALIZE_UNSIGNED.	c		e	f	g	a		i		j	k	l	m	n	b
M_NORMALIZE_SIGNED	Specifies to transform the point cloud or 3D geometry to fit a signed unit box.	c		e	f	g	a		i		j	k	l	m	n	b
M_NORMALIZE_UNSIGNED	Specifies to transform the point cloud or 3D geometry to fit an unsigned unit box.	c		e	f	g	a		i		j	k	l	m	n	b
M_NORMALIZATION_SCALE	Sets how to apply scale values when transforming the point cloud or 3D geometry to fit a unit box. If set to M_UNIFORM, each dimension is scaled using the same value such that the largest dimension fits the unit box, and the remaining dimensions will have the same or smaller lengths. If set to M_NON_UNIFORM, scale values are applied such that all dimensions have the same length and fit a unit cube. A normalization matrix computed using non-uniform scaling cannot be applied to 3D geometries of type M_SPHERE, M_BOX, or M_CYLINDER. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_UNIFORM.	c		e	f	g	a		i		j	k	l	m	n	b
M_NON_UNIFORM	Applies scale values so that the transformed point cloud or 3D geometry occupies a unit cube.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNIFORM	Applies the same scale value to all dimensions.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_POINTS	Sets whether to enable number-of-points statistics calculations. Number-of-points statistics include the total number of points, the number of points that are missing data, and the number of valid points. Note that number-of-points statistics are not available for 3D geometries. INQ	c		e	f	g	a		i		j	k	l	m	n	b

	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies to disable number-of-points calculations.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to enable number-of-points calculations.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_SAMPLES		Sets the number of sample points to use when calculating distance-to-nearest-neighbor, surface variation, aspect ratio, and number-of-valid-neighbors statistics. By default, Aurora Imaging Library uses all points for the calculation. However, if speed is an issue, you can try specifying a fraction of the total points, which could yield a satisfactory approximate calculation. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_INFINITE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_INFINITE	Specifies to use all points when calculating distance-to-nearest-neighbor and surface variation statistics.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the number of sample points to use when calculating distance-to-nearest-neighbor and surface variation statistics. If the specified value is greater than the number of valid points, all points are used.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_VALID_NEIGHBORS		Sets whether to enable number-of-valid-neighbors statistics calculations. Number-of-valid-neighbors statistics allow you to check for degenerate neighborhoods, which negatively affect neighborhood-based functions (for example, M3dimNormals()). INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies to disable number-of-valid-neighbors calculations.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to enable number-of-valid-neighbors calculations.	c		e	f	g	a		i		j	k	l	m	n	b
M_PCA		Sets whether to enable principal component analysis (PCA) statistics calculations. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies to disable PCA calculations.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to enable PCA calculations.	c		e	f	g	a		i		j	k	l	m	n	b
M_PCA_MODE		Sets whether to calculate the principal component analysis (PCA) around the origin (M_ORDINARY) or around the centroid (M_CENTRAL). Ordinary mode is usually only used on data that is already centralized, such as normals. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_CENTRAL.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CENTRAL	Specifies to calculate the PCA around the centroid.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ORDINARY	Specifies to calculate the PCA around the origin.	c		e	f	g	a		i		j	k	l	m	n	b

M_SURFACE_AREA	Sets whether to calculate the surface area of the mesh. The surface area is the sum of the area of all the mesh triangles in the container. This setting is only available for 3D-processable point cloud and depth map containers that have an M_COMPONENT_MESH_AIL component. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable surface area statistics calculations.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable surface area statistics calculations.	c		e	f	g	a		i		j	k	l	m	n	b
M_SURFACE_VARIATION	Sets whether to enable surface variation statistics calculations. For each point, Aurora Imaging Library fits a local plane to neighboring points and measures the perpendicular distance of the point to the plane. Surface variation statistics are a measure of a point cloud's noise level. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable surface variation statistics calculations.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable surface variation statistics calculations.	c		e	f	g	a		i		j	k	l	m	n	b
Stitch context ID	Specifies a stitch 3D image processing context, allocated using M3dimAlloc() with M_STITCH_CONTEXT, and used in M3dimMerge() operations.	c		e	f	g	a		i		j	k	l	m	n	b
M_CALIBRATION_MODE	Sets the calibration mode with which to map depth map pixel coordinates to real-world coordinates. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO	Specifies to automatically calibrate the depth maps so that the destination depth map includes all the source depth maps.	c		e	f	g	a		i		j	k	l	m	n	b
M_USER_DEFINED	Specifies to calibrate the depth maps using the values set with M_OFFSET_X and M_OFFSET_Y.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERPOLATION_FALLBACK_MODE	Sets the type of interpolation to use when stitching the depth maps and there are invalid pixels in the neighborhood of a point. Note that bilinear interpolation is used when all pixels in the neighborhood of a point are valid. Interpolation is only done when M_STITCH_MODE is set to M_PRECISE. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_INTERPOLATION.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERPOLATION	Specifies interpolated resizing.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEAREST_NEIGHBOR	Specifies nearest neighbor interpolation. The new value is that of the pixel closest to the source point.	c		e	f	g	a		i		j	k	l	m	n	b

M_OFFSET_X	Sets the X-coordinate of the top-left corner of the destination depth map. This control type is only used when M_CALIBRATION_MODE is set to M_USER_DEFINED. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the X-coordinate, in world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_OFFSET_Y	Sets the Y-coordinate of the top-left corner of the destination depth map. This control type is only used when M_CALIBRATION_MODE is set to M_USER_DEFINED. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the Y-coordinate, in world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_OVERLAP_MODE	Sets how to determine the pixel value when two or more depth maps overlap. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_AVERAGE.	c		e	f	g	a		i		j	k	l	m	n	b
M_AVERAGE	Specifies to use the average value of the pixels in the overlapping region.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX	Specifies to use the maximum value of the pixels in the overlapping region.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN	Specifies to use the minimum value of the pixels in the overlapping region.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIZE_X	Sets the size of the stitched destination depth map along the X dimension. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the size of the depth map in X, in world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIZE_Y	Sets the size of the stitched destination depth map along the Y dimension. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the size of the depth map in Y, in world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_STATIC_INDEX	Sets the source depth map to which all other depth map pixels will be stitched. For example, if depth maps 1 and 2's pixel size=1, depth map 1 starts at world x=3, and depth map 2 starts at world x=5.25, depth map 2 will be stitched to the nearest depth map 1 pixel, such that the intensity values of the destination pixels at world x=5 and world x=6 will be those of the source pixels at world x=5.25 and world x=6.25, or their intensity values will be interpolated (depending on the value of M_STITCH_MODE). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b

M_DISABLE	Specifies to use the first depth map in the array.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the index of the depth map to use. This value must be less than the value passed to the NumContainers parameter of M3dimMerge().	c		e	f	g	a		i		j	k	l	m	n	b
M_STITCH_DIRECTION	Sets the direction in which to stitch the point clouds' component values. Note that stitching does not depend on the coordinates of points. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DIRECTION_Y.	c		e	f	g	a		i		j	k	l	m	n	b
M_DIRECTION_X	Specifies to stitch the component values of each successive point cloud to the right of the previous point cloud's component values.	c		e	f	g	a		i		j	k	l	m	n	b
M_DIRECTION_Y	Specifies to stitch the component values of each successive point cloud below the previous point cloud's component values.	c		e	f	g	a		i		j	k	l	m	n	b
M_STITCH_MODE	Sets the algorithm to use for stitching depth maps. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_FAST.	c		e	f	g	a		i		j	k	l	m	n	b
M_FAST	Specifies that the depth maps will be stitched quickly, but less accurately. The depth maps are stitched to the nearest pixel. No interpolation is done, even if the source depth maps are misaligned.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRECISE	Specifies that the depth maps will be accurately stitched. Interpolation is done to accurately align the depth maps when stitching them.	c		e	f	g	a		i		j	k	l	m	n	b
Subsample context ID	Specifies a subsample 3D image processing context, allocated using M3dimAlloc() with M_SUBSAMPLE_CONTEXT, and used in M3dimSample() subsampling operations. You can set the type of subsampling operation to perform with M_SUBSAMPLE_MODE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTINCT_ANGLE_DIFFERENCE	Sets the minimum angle difference between the normal vector of a point and that of its neighbors, for the point to be kept in the subsample. If a point's normal vector angle is distinct from that of all of its neighbors, the point is kept. Use M_NEIGHBORHOOD_DISTANCE to define the neighborhood of a point. This control type only applies when M_SUBSAMPLE_MODE is set to M_SUBSAMPLE_NORMAL. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 10.0.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 180.0	Specifies the minimum angle difference, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_FRACTION_OF_POINTS	Sets the fraction of all valid points to select from the source point cloud. Non-valid points are not considered in the calculation. This control type only applies when M_SUBSAMPLE_MODE is set to M_SUBSAMPLE_RANDOM or M_SUBSAMPLE_GEOMETRIC. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.1.	c		e	f	g	a		i		j	k	l	m	n	b

0.0 < *Value* < 1.0	Specifies the fraction of all valid points to select randomly from the source point cloud.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRID_SIZE_X	Sets the cell size in the X-direction. Note that, when M_ORGANIZATION_TYPE is set to M_ORGANIZED, exactly 1 of M_GRID_SIZE_X, M_GRID_SIZE_Y, or M_GRID_SIZE_Z must be set to M_INFINITE. This control type only applies when M_SUBSAMPLE_MODE is set to M_SUBSAMPLE_GRID. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_INFINITE	Specifies an infinite cell size along X.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the cell size along X, in world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRID_SIZE_Y	Sets the cell size in the Y-direction. Note that, when M_ORGANIZATION_TYPE is set to M_ORGANIZED, exactly 1 of M_GRID_SIZE_X, M_GRID_SIZE_Y, or M_GRID_SIZE_Z must be set to M_INFINITE. This control type only applies when M_SUBSAMPLE_MODE is set to M_SUBSAMPLE_GRID. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_INFINITE	Specifies an infinite cell size along Y.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the cell size along Y, in world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRID_SIZE_Z	Sets the cell size in the Z-direction. Note that, when M_ORGANIZATION_TYPE is set to M_ORGANIZED, exactly 1 of M_GRID_SIZE_X, M_GRID_SIZE_Y, or M_GRID_SIZE_Z must be set to M_INFINITE. This control type only applies when M_SUBSAMPLE_MODE is set to M_SUBSAMPLE_GRID. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_INFINITE	Specifies an infinite cell size along Z.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the cell size along Z, in world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEIGHBORHOOD_DISTANCE	Sets the distance that defines the neighborhood of a point; for a point to be kept in the subsample, it must be distinct in this neighborhood. Use M_DISTINCT_ANGLE_DIFFERENCE to set whether a point is distinct from other points in its neighborhood. This control type only applies when M_SUBSAMPLE_MODE is set to M_SUBSAMPLE_NORMAL. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the neighborhood distance, in world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_ORGANIZATION_TYPE	Sets the resulting subsampled point cloud's organizational type. INQ	c		e	f	g	a		i		j	k	l	m	n	b

	M_DEFAULT	Same as M_UNORGANIZED.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ORGANIZED	Specifies an organized resulting subsampled point cloud. This control value only applies when M_SUBSAMPLE_MODE is set to M_SUBSAMPLE_DECIMATE or M_SUBSAMPLE_GRID. Note also that, when M_SUBSAMPLE_MODE is set to M_SUBSAMPLE_DECIMATE, M_ORGANIZED is supported only when the source point cloud is organized.	c		e	f	g	a		i		j	k	l	m	n	b
	M_UNORGANIZED	Specifies an unorganized resulting subsampled point cloud.	c		e	f	g	a		i		j	k	l	m	n	b
M_POINT_SELECTED		Sets which point to choose when multiple points occupy the same cell, and when 1 of M_GRID_SIZE_... is set to M_INFINITE. Note that this control type is ignored when none of M_GRID_SIZE_... is set to M_INFINITE. This control type only applies when M_SUBSAMPLE_MODE is set to M_SUBSAMPLE_GRID.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default point selection mode. Same as M_MAX.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CENTER	Specifies to select the point closest to the center of each 2D cell, as if the point cloud is collapsed onto the plane formed by the 2 non-infinite dimensions.	c		e	f	g	a		i		j	k	l	m	n	b
	M_MAX	Specifies to select the highest point along the M_INFINITE axis, for each cell.	c		e	f	g	a		i		j	k	l	m	n	b
	M_MIN	Specifies to select the lowest point along the M_INFINITE axis, for each cell.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEED_VALUE		Sets the seed for the random selection of points, for an M_SUBSAMPLE_RANDOM operation. When set to **M_DEFAULT**, the same sequence of random samples is generated with each call to M3dimSample() (for the same source point cloud). INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 1.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the seed for the random selection of points.	c		e	f	g	a		i		j	k	l	m	n	b
M_STEP_SIZE_X		Sets the step interval between points, along the X dimension. Points are selected from the source point cloud's M_COMPONENT_RANGE and M_COMPONENT_CONFIDENCE components, where the point cloud's X-, Y-, and Z-coordinates are stored in separate bands of a 2D (organized) or 1D (unorganized) buffer. M_STEP_SIZE_X specifies an interval in X for each band. If the resulting point cloud is organized, invalid points are included; otherwise they are ignored. This control type only applies when M_SUBSAMPLE_MODE is set to M_SUBSAMPLE_DECIMATE. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 1.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 1	Specifies the step interval value.	c		e	f	g	a		i		j	k	l	m	n	b

M_STEP_SIZE_Y	Sets the step interval between points, along the Y dimension. Points are selected from the source point cloud's M_COMPONENT_RANGE and M_COMPONENT_CONFIDENCE components, where the point cloud's X-, Y-, and Z-coordinates are stored in separate bands of a 2D (organized) or 1D (unorganized) buffer. Note that, if the source point cloud is unorganized, M_STEP_SIZE_Y is ignored. M_STEP_SIZE_Y specifies an interval in Y for each band in the range and confidence components. If the resulting point cloud is organized, invalid points are included; otherwise they are ignored. This control type only applies when M_SUBSAMPLE_MODE is set to M_SUBSAMPLE_DECIMATE. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 1	Specifies the step interval value.	c		e	f	g	a		i		j	k	l	m	n	b
M_SUBSAMPLE_MODE	Sets the type of subsampling operation to perform. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_SUBSAMPLE_DECIMATE.	c		e	f	g	a		i		j	k	l	m	n	b
M_SUBSAMPLE_DECIMATE	Specifies to subsample using a decimation operation, which selects points at regular intervals. Set the interval size with M_STEP_SIZE_X and M_STEP_SIZE_Y. You can set the resulting subsampled point cloud's organizational type with M_ORGANIZATION_TYPE. > Note that M_SUBSAMPLE_DECIMATE can sometimes generate undesirable regular patterns. To avoid such patterns, use random mode (M_SUBSAMPLE_RANDOM) instead.	c		e	f	g	a		i		j	k	l	m	n	b
M_SUBSAMPLE_GEOMETRIC	Specifies to perform a geometrically stable subsampling, which selects points that help 3D registration converge faster and avoid local minima. The underlying algorithm identifies and rejects points that are likely to cause registration to slide away from convergence, while retaining points that constrain rotational and translational transforms during registration. You can set the fraction of points that are retained from the source point cloud with M_FRACTION_OF_POINTS.	c		e	f	g	a		i		j	k	l	m	n	b

M_SUBSAMPLE_GRID	Specifies to subsample using a grid operation, which divides the 3D space into cells and, for each subgroup of points, selects a single point per cell. If you want the grid operation to select a particular fraction of points from the source point cloud, you can use M3dimLattice() to calculate the cell size required. Storing or copying the lattice results into a subsample 3D image processing context automatically sets M_SUBSAMPLE_MODE to M_SUBSAMPLE_GRID, M_GRID_SIZE_... to the calculated lattice's cell sizes, and, if 1 of the cell dimensions is infinite, M_ORGANIZATION_TYPE to M_ORGANIZED. Otherwise, set the cell size with M_GRID_SIZE_... and the resulting subsampled point cloud's organizational type with M_ORGANIZATION_TYPE. > Note that, when M_ORGANIZATION_TYPE is set to M_ORGANIZED, exactly 1 of M_GRID_SIZE_X, M_GRID_SIZE_Y, or M_GRID_SIZE_Z must be set to M_INFINITE. When M_ORGANIZATION_TYPE is set to M_UNORGANIZED, you can set at most 1 of M_GRID_SIZE_... to M_INFINITE. > When one of M_GRID_SIZE_... is set to M_INFINITE, the point selected for a cell depends on the mode specified with M_POINT_SELECTED. When none of M_GRID_SIZE_... is set to M_INFINITE, the point selected for a cell is the point closest to the cell's volumetric center.	c	e	f	g	a	i	j	k	l	m	n	b
M_SUBSAMPLE_NORMAL	Specifies to subsample based on normal vector angles and a neighborhood distance. A neighborhood point is selected only if its surface normal vector is distinct from that of all of its neighbors (M_DISTINCT_ANGLE_DIFFERENCE) within the neighborhood defined with M_NEIGHBORHOOD_DISTANCE.	c	e	f	g	a	i	j	k	l	m	n	b
M_SUBSAMPLE_RANDOM	Specifies to subsample using an operation that randomly selects a specified fraction of points from the source point cloud. Set the seed for the random number generation with M_SEED_VALUE; set the fraction with M_FRACTION_OF_POINTS.	c	e	f	g	a	i	j	k	l	m	n	b
M_SUBSAMPLE_NORMAL_MODE	Sets the algorithm to use for subsampling based on normals. This control type only applies when M_SUBSAMPLE_MODE is set to M_SUBSAMPLE_NORMAL. INQ	c	e	f	g	a	i	j	k	l	m	n	b
M_DEFAULT	Same as M_FAST.	c	e	f	g	a	i	j	k	l	m	n	b
M_FAST	Specifies faster but approximate subsampling based on normals. Some extra, non-distinct points might be selected in the same neighborhood.	c	e	f	g	a	i	j	k	l	m	n	b
M_PRECISE	Specifies exact subsampling based on normals. All selected points will be distinct in their neighborhood.	c	e	f	g	a	i	j	k	l	m	n	b
Surface sample context ID	Specifies a surface sample 3D image processing context, allocated using M3dimAlloc() with M_SURFACE_SAMPLE_CONTEXT, and used in M3dimSample() surface sampling operations.	c	e	f	g	a	i	j	k	l	m	n	b
M_RESOLUTION	Sets the resolution at which to sample the mesh or 3D geometry. This represents the distance between neighboring points on the surface; the smaller the resolution, the more points generated. INQ	c	e	f	g	a	i	j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.0.	c	e	f	g	a	i	j	k	l	m	n	b

	Value > 0.0	Specifies the distance between newly added points on the surface, in world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_MODE		Sets the mode of the surface sampling operation. The mode establishes whether the destination point cloud will have a mesh structure. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_SURFACE_GRID.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SURFACE_GRID	Specifies that surface sampling invalidates the mesh component and the destination container will contain a denser non-meshed point cloud. If an M_COMPONENT_MESH_AIL component exists in the destination container, it is deleted.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TESSELLATION	Specifies that surface sampling preserves the mesh component and the destination container will contain a denser meshed point cloud.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dim11.lib.
DLL	Requires ail11.dll; ail3dim11.dll.

M3dimConvertDepthMap

Synopsis

Convert data from a depth map container to a depth map image, or vice versa.

Syntax

```
void M3dimConvertDepthMap(
    AIL_ID SrcDepthMapBufId,          //in
    AIL_ID SrcIntensityMapImageBufId, //in
    AIL_ID DstDepthMapBufId,          //out
    AIL_ID DstIntensityMapImageBufId, //out
    AIL_INT64 Operation,              //in
    AIL_INT64 Options,                //in
    AIL_INT64 ControlFlag             //in
)
```

Description

This function converts data from a specified depth map container or depth map image buffer into an equivalent depth map image buffer or depth map container. This function can optionally copy only the calibration information between depth map containers, from a depth map container to an image buffer, or from a depth map image buffer to a container. **M3dimConvertDepthMap**() can also copy intensity values from the source depth map container's M_COMPONENT_REFLECTANCE or M_COMPONENT_INTENSITY component (if one exists) to a destination intensity map image buffer. If a source intensity map image buffer is provided, intensity values are copied to the destination container's M_COMPONENT_REFLECTANCE component, which is created if not previously existing in the destination container. You can optionally specify M_PROPAGATE_REGION to propagate a region of interest (ROI) from the source depth map container's M_COMPONENT_REGION_AIL component (if one exists) to an ROI in the destination depth map image buffer, or vice versa.

Parameters

SrcDepthMapBufId

Specifies the identifier of the source depth map image buffer or depth map container.

See the Parameter associations section for possible values that can be specified.

SrcIntensityMapImageBufId

Specifies the identifier of the source intensity map image buffer.

See the Parameter associations section for possible values that can be specified.

DstDepthMapBufId

Specifies the identifier of the destination image buffer or container in which to store the converted data or calibration information for a depth map.

See the Parameter associations section for possible values that can be returned.

DstIntensityMapImageBufId

Specifies the identifier of the destination image buffer in which to store the converted data or calibration information for an intensity map.

See the Parameter associations section for possible values that can be returned.

Operation

Specifies whether to convert the data or copy the calibration information.

See the Parameter associations section for possible values that can be specified.

Options

Specifies whether to propagate the ROI.

See the Parameter associations section for possible values that can be specified.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Parameter associations

The table below lists possible values for the **SrcDepthMapBufId**, **SrcIntensityMapImageBufId**, **Operation**, and **Options** parameters and

possible values returned to the **DstDepthMapBufId** and **DstIntensityMapImageBufId** parameters.

- For specifying the conversion

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the conversion																	
Operation	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
SrcDepthMapBufId																
SrcIntensityMapImageBufId																
DstDepthMapBufId - Possible values returned																
DstIntensityMapImageBufId - Possible values returned																
Options																
M_CALIBRATE	Specifies to copy the calibration information from the source depth map container or image buffer to the destination.															
Source depth map container ID	Specifies the identifier of the depth map container from which to copy the calibration information. The container must have been previously allocated using MbufAllocContainer() with M_PROC, and must store data in a 3D-processable depth map format (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE_DEPTH_MAP, the function returns M_TRUE).	c		e	f	g	a		i		j	k	l	m	n	b
SrcIntensityMapImageBufId	This parameter must be set to M_NULL.	c		e	f	g	a		i		j	k	l	m	n	b
DstDepthMapBufId	Specifies the identifier of the image buffer or container in which to copy the calibration information for a depth map. The image buffer must be a 1-band, 8-bit, 16-bit or 32-bit unsigned image buffer. The container must have been previously allocated using MbufAllocContainer() with M_PROC, and must not be a child container.	c		e	f	g	a		i		j	k	l	m	n	b
DstIntensityMapImageBufId	Specifies the identifier of the image buffer in which to copy the calibration information for an intensity map. The image buffer must be an 8-bit or 16-bit unsigned, 1-band or 3-band buffer. If you pass a container to the DstDepthMapBufId parameter, you must set this parameter to M_NULL. If an intensity map is not required, set this parameter to M_NULL.	c		e	f	g	a		i		j	k	l	m	n	b
Options	This parameter must be set to M_DEFAULT.	c		e	f	g	a		i		j	k	l	m	n	b
Source depth map image buffer ID	Specifies the identifier of the depth map image buffer from which to copy the calibration information. The image buffer must be a 1-band, 8-bit, 16-bit, or 32-bit unsigned buffer, and must contain a fully corrected depth map (that is, if you call McalInquire() with M_DEPTH_MAP, the function returns M_TRUE).	c		e	f	g	a		i		j	k	l	m	n	b
SrcIntensityMapImageBufId	This parameter must be set to M_NULL.	c		e	f	g	a		i		j	k	l	m	n	b
DstDepthMapBufId	Specifies the identifier of the container in which to copy the calibration information for a depth map. The container must have been previously allocated using MbufAllocContainer() with M_PROC, and must not be a child container.	c		e	f	g	a		i		j	k	l	m	n	b
DstIntensityMapImageBufId	This parameter must be set to M_NULL.	c		e	f	g	a		i		j	k	l	m	n	b

Options	This parameter must be set to M_DEFAULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_CONVERT	Specifies to convert the data from the depth map container into the equivalent depth map and intensity map images or vice versa.	c		e	f	g	a		i		j	k	l	m	n	b
Source depth map container ID	Specifies the identifier of the depth map container from which to convert the data. The container must have been previously allocated using MbufAllocContainer() with M_PROC, and must store data in a 3D-processable depth map format (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE_DEPTH_MAP, the function returns M_TRUE).	c		e	f	g	a		i		j	k	l	m	n	b
SrcIntensityMapImageBufId	This parameter must be set to M_NULL.	c		e	f	g	a		i		j	k	l	m	n	b
DstDepthMapBufId	Specifies the identifier of the image buffer in which to store the depth map. The image buffer must be a 1-band, 8-bit, 16-bit or 32-bit unsigned image buffer.	c		e	f	g	a		i		j	k	l	m	n	b
DstIntensityMapImageBufId	Specifies the identifier of the image buffer in which to store the intensity map. The image buffer must be an 8-bit or 16-bit unsigned, 1-band or 3-band buffer. If an intensity map is not required, set this parameter to M_NULL.	c		e	f	g	a		i		j	k	l	m	n	b
Options	Specifies whether to propagate the ROI from the source depth map container's M_COMPONENT_REGION_AIL component, if one exists.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies not to propagate the ROI. If an M_COMPONENT_REGION_AIL component exists in the source, each pixel outside the ROI will be set to the invalid value in the destination depth map image buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROPAGATE_REGION	Specifies to propagate the ROI to an M_RASTER ROI in the destination depth map image buffer.	c		e	f	g	a		i		j	k	l	m	n	b
Source depth map image buffer ID	Specifies the identifier of the depth map image buffer from which to convert the data. The image buffer must be a 1-band, 8-bit, 16-bit, or 32-bit unsigned buffer, and must contain a fully corrected depth map (that is, if you call McalInquire() with M_DEPTH_MAP, the function returns M_TRUE).	c		e	f	g	a		i		j	k	l	m	n	b
SrcIntensityMapImageBufId	Specifies the identifier of the intensity map image buffer from which to convert the data. The image buffer must be an 8-bit or 16-bit unsigned, 1-band or 3-band buffer. If an intensity map is not required, set this parameter to M_NULL.	c		e	f	g	a		i		j	k	l	m	n	b
DstDepthMapBufId	Specifies the identifier of the container in which to store the depth map. The container must have been previously allocated using MbufAllocContainer() with M_PROC, and must not be a child container.	c		e	f	g	a		i		j	k	l	m	n	b
DstIntensityMapImageBufId	This parameter must be set to M_NULL.	c		e	f	g	a		i		j	k	l	m	n	b
Options	Specifies whether to propagate the ROI from the source depth map image buffer, if one exists.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies not to propagate the ROI. If an ROI is associated with the source, each pixel outside the ROI will be given a confidence value of 0 in the destination depth map container.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROPAGATE_REGION	Specifies to propagate the ROI to an M_COMPONENT_REGION_AIL component in the destination depth map container.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dim11.lib.
DLL	Requires ail11.dll; ail3dim11.dll.

M3dimCopy

Synopsis

Copy the extent box of a depth map into a 3D geometry.

Syntax

```
void M3dimCopy(
    AIL_ID SrcDepthMapImageBufId, //in
    AIL_ID DstGeometry3dgeoId,    //out
    AIL_INT64 CopyType,           //in
    AIL_INT64 ControlFlag         //in
)
```

Description

This function copies the specified depth map's extent box, calculated from the depth map buffer's size and calibration information, into the specified 3D geometry object.

Parameters

SrcDepthMapImageBufId

Specifies the identifier of the source depth map container or depth map image buffer.

See the Parameter associations section for possible values that can be specified.

DstGeometry3dgeoId

Specifies the identifier of the destination 3D geometry object.

See the Parameter associations section for possible values that can be returned.

CopyType

Specifies how to copy the depth map's extent box into the 3D geometry object.

See the Parameter associations section for possible values that can be specified.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Parameter associations

The table below lists possible values for the **SrcDepthMapImageBufId** and **CopyType** parameters and possible values returned to the **DstGeometry3dgeoId** parameter.

- For specifying the copy type

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the copy type																
CopyType SrcDepthMapImageBufId DstGeometry3dgeoId - Possible values returned	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_EXTENT_BOX	Specifies to copy the depth map's extent box into the 3D geometry object. The extent box represents the maximum box that a depth map can span, irrespective of its data. The box defines the minimum/maximum X-, Y-, and Z-coordinates that would be representable in the depth map buffer, based on the buffer's size and calibration information.															

	Depth map container ID from which to copy	Specifies the identifier of the depth map container from which to copy the extent box. The container must have been previously allocated using MbufAllocContainer() with M_PROC, and must store data in a 3D-processable depth map format (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE_DEPTH_MAP, the function returns M_TRUE).	c		e	f	g	a		i		j	k	l	m	n	b
	3D geometry object ID in which to copy	Specifies the identifier of the destination 3D geometry object in which to copy the depth map's extent box. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY. Note that, after calling **M3dimCopy**(), the destination 3D geometry object will return type M_BOX if inquired using M3dgeoInquire() with M_GEOMETRY_TYPE.	c		e	f	g	a		i		j	k	l	m	n	b
	Depth map image buffer ID from which to copy	Specifies the identifier of the depth map image buffer from which to copy the extent box. The image buffer must be a 1-band, 8-bit, 16-bit, or 32-bit unsigned buffer, and must contain a fully corrected depth map (that is, if you call McalInquire() with M_DEPTH_MAP, the function returns M_TRUE). If the image buffer has a region of interest (ROI) associated with it, the ROI is ignored.	c		e	f	g	a		i		j	k	l	m	n	b
	3D geometry object ID in which to copy	Specifies the identifier of the destination 3D geometry object in which to copy the depth map's extent box. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY. Note that, after calling **M3dimCopy**(), the destination 3D geometry object will return type M_BOX if inquired using M3dgeoInquire() with M_GEOMETRY_TYPE.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dim11.lib.
DLL	Requires ail11.dll; ail3dim11.dll.

M3dimCopyResult

Synopsis

Copy a group of results into a 3D geometry object, transformation matrix object, subsample 3D image processing context, or container.

Syntax

```
void M3dimCopyResult(
    AIL_ID SrcResult3dimId, //in
    AIL_ID DstAilObjectId,  //out
    AIL_INT64 CopyType,     //in
    AIL_INT64 ControlFlag   //in
)
```

Description

This function copies a group of 3D image processing results into a 3D geometry object, transformation matrix object, subsample 3D image processing context, or container, according to the specified copy operation. You can copy statistics results, profile results, lattice results, or find transformation results (calculated using M3dimStat(), M3dimProfile(), M3dimLattice(), or M3dimFindTransformation(), respectively).

Parameters

SrcResult3dimId

Specifies the identifier of a 3D image processing result buffer.

See the Parameter associations section for possible values that can be specified.

DstAilObjectId

Specifies the identifier of a 3D geometry object, transformation matrix object, subsample 3D image processing context, or container.

See the Parameter associations section for possible values that can be returned.

CopyType

Specifies the type of copy operation to perform.

See the Parameter associations section for possible values that can be specified.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Parameter associations

The table below lists possible values for the **SrcResult3dimId** and **CopyType** parameters and possible values returned to the **DstAilObjectId** parameter.

- For specifying the copy type and destination object for a 3D image processing result buffer

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the copy type and destination object for a 3D image processing result buffer																
SrcResult3dimId	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
CopyType																
DstAilObjectId - Possible values returned																
Find transformation 3D image processing result buffer ID from which to copy	Specifies the identifier of a find transformation 3D image processing result buffer from which to copy. The result buffer must have been previously allocated using M3dimAllocResult() with M_FIND_TRANSFORMATION_RESULT. The result buffer must contain the results of a call to M3dimFindTransformation(), which finds the transformation between a source set of points and a target set of points.															
M_TRANSFORMATION_MATRIX	Specifies to copy, into the transformation matrix, find transformation results that transform source coordinate values into target coordinate values as best as possible, when used with M3dimMatrixTransform().	c		e	f	g	a		i		j	k	l	m	n	b

Transformation matrix object ID in which to copy	Specifies the identifier of the transformation matrix object in which to copy the find transformation results. The transformation matrix object must have been previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.	c		e	f	g	a		i		j	k	l	m	n	b
Lattice 3D image processing result buffer ID from which to copy	Specifies the identifier of a lattice 3D image processing result buffer from which to copy. The result buffer must have been previously allocated using M3dimAllocResult() with M_LATTICE_RESULT. The result buffer must contain the results of a call to M3dimLattice(), which calculates the lattice for a point cloud.	c		e	f	g	a		i		j	k	l	m	n	b
M_BOUNDING_BOX	Specifies to copy the lattice's bounding box into the specified 3D geometry object, establishing a 3D box geometry.	c		e	f	g	a		i		j	k	l	m	n	b
3D geometry object ID in which to copy	Specifies the identifier of a 3D geometry object in which to copy the lattice results. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY. Note that, after calling **M3dimCopyResult**(), the destination 3D geometry object will resturn type M_BOX if inquired using M3dgeoInquire() with M_GEOMETRY_TYPE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEPTH_MAP_CONTAINER + *Depth value*	Specifies to copy lattice results into a container, establishing a 3D-processable depth map container with the same size and calibration of the lattice. This sets the size of the container's range component to M3dimGetResult() with M_START_POINT_... and the scales to M_CELL_SIZE_.... Note that you must also specify the bit depth (8, 16, or 32).	c		e	f	g	a		i		j	k	l	m	n	b
Container ID in which to copy	Specifies the container in which to copy the lattice results. The container must have been previously allocated using MbufAllocContainer() with M_PROC, and must not be a child container.	c		e	f	g	a		i		j	k	l	m	n	b
M_SUBSAMPLE_CONTEXT	Specifies to copy lattice results into the subsample 3D image processing context. This sets the M_SUBSAMPLE_MODE control type of the subsample 3D image processing context to M_SUBSAMPLE_GRID and its M_GRID_SIZE_... control types to the cell sizes of the lattice results (M3dimGetResult() with M_CELL_SIZE_...). If 1 of the cell dimensions is infinite, M_ORGANIZATION_TYPE is set to M_ORGANIZED.	c		e	f	g	a		i		j	k	l	m	n	b
Subsample 3D image processing context ID in which to copy	Specifies the identifier of the subsample 3D image processing context in which to copy the lattice results. The context must have been previously allocated using M3dimAlloc() with M_SUBSAMPLE_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
Profile 3D image processing result buffer ID from which to copy	Specifies the identifier of a profile 3D image processing result buffer from which to copy. The result buffer must have been previously allocated using M3dimAllocResult() with M_PROFILE_RESULT. The result buffer must contain the results of a call to M3dimProfile(), which takes the profile of a depth map, 3D geometry, mesh, or point cloud.	c		e	f	g	a		i		j	k	l	m	n	b
M_MATRIX_PROFILE_PLANE_TO_WORLD	Specifies to copy, into the transformation matrix, profile results that transform M_PROFILE_PLANE_... coordinate values into M_WORLD_... coordinate values, when used with M3dimMatrixTransformList(). Note that M_PROFILE_PLANE_... coordinates are expressed in the slicing plane's coordinate system, while M_WORLD_... coordinates are expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
Transformation matrix object ID in which to copy	Specifies the identifier of the transformation matrix object in which to copy the profile results. The transformation matrix object must have been previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.	c		e	f	g	a		i		j	k	l	m	n	b

M_MATRIX_WORLD_TO_PROFILE_PLANE	Specifies to copy, into the transformation matrix, profile results that transform M_WORLD_... coordinate values into M_PROFILE_PLANE_... coordinate values, when used with M3dimMatrixTransformList(). Note that M_WORLD_... coordinates are expressed in the working coordinate system, while M_PROFILE_PLANE_... coordinates are expressed in the slicing plane's coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
Transformation matrix object ID in which to copy	Specifies the identifier of the transformation matrix object in which to copy the profile results. The transformation matrix object must have been previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.	c		e	f	g	a		i		j	k	l	m	n	b
Statistics 3D image processing result buffer ID from which to copy	Specifies the identifier of a statistics 3D image processing result buffer from which to copy. The result buffer must have been previously allocated using M3dimAllocResult() with M_STATISTICS_RESULT. The result buffer must contain the results returned by a call to M3dimStat(), which computes statistics on a point cloud, depth map, or 3D geometry.	c		e	f	g	a		i		j	k	l	m	n	b
M_BOUNDING_BOX	Specifies to copy bounding box statistics results into the specified 3D geometry object, establishing a 3D box geometry.	c		e	f	g	a		i		j	k	l	m	n	b
3D geometry object ID in which to copy	Specifies the identifier of a 3D geometry object in which to copy the statistics results. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY. Note that, after calling **M3dimCopyResult**(), the destination 3D geometry object will return type M_BOX if inquired using M3dgeoInquire() with M_GEOMETRY_TYPE.	c		e	f	g	a		i		j	k	l	m	n	b
M_BOX_CENTER	Specifies to copy bounding box center coordinate statistics results into the specified 3D geometry object, establishing a 3D point geometry.	c		e	f	g	a		i		j	k	l	m	n	b
3D geometry object ID in which to copy	Specifies the identifier of a 3D geometry object in which to copy the statistics results. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY. Note that, after calling **M3dimCopyResult**(), the destination 3D geometry object will return type M_POINT if inquired using M3dgeoInquire() with M_GEOMETRY_TYPE.	c		e	f	g	a		i		j	k	l	m	n	b
M_CENTROID	Specifies to copy centroid statistics results into the specified 3D geometry object, establishing a 3D point geometry.	c		e	f	g	a		i		j	k	l	m	n	b
3D geometry object ID in which to copy	Specifies the identifier of a 3D geometry object in which to copy the statistics results. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY. Note that, after calling **M3dimCopyResult**(), the destination 3D geometry object will return type M_POINT if inquired using M3dgeoInquire() with M_GEOMETRY_TYPE.	c		e	f	g	a		i		j	k	l	m	n	b
M_FIXTURING_MATRIX	Specifies to copy, into the transformation matrix, principal component analysis (PCA) statistics results that can bring points along the object's principal axes to the X-, Y-, and Z-axes. This matrix is the inverse of the PCA matrix (M_PCA_MATRIX).	c		e	f	g	a		i		j	k	l	m	n	b
Transformation matrix object ID in which to copy	Specifies the identifier of the transformation matrix object in which to copy the principal component analysis (PCA) statistics results. The transformation matrix object must have been previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.	c		e	f	g	a		i		j	k	l	m	n	b

M_NORMALIZATION_MATRIX	Specifies to copy, into the transformation matrix, bounding box statistics results that can transform the bounding box into a unit box. The unit box's maximum length is 1, except in the case of a signed unit box, whose maximum length is 2. Use M3dimControl() with M_NORMALIZATION_MODE to specify whether the transformed point cloud or 3D geometry should fit a signed or unsigned unit box. When M_NORMALIZATION_MODE is set to M_NORMALIZE_SIGNED, the unit box of the transformed cloud is signed, centered at (0,0,0), and its unit length is at most 2 (stretches from -1 to +1). When M_NORMALIZATION_MODE is set to M_NORMALIZE_UNSIGNED, the unit box of the transformed cloud is unsigned, centered at (0.5,0.5,0.5), and its unit length is at most 1 (stretches from 0 to +1). When M_NORMALIZATION_SCALE is set to M_UNIFORM, the same scale factor is applied to all dimensions. This means that the largest dimension is scaled to fit the unit box, and the remaining dimensions will have the same or smaller lengths. When M_NORMALIZATION_SCALE is set to M_NON_UNIFORM, a different scale factor is applied along each dimension and the resulting box is a unit cube. This copy operation is not available for a semi-oriented bounding box.	c		e	f	g	a		i		j	k	l	m	n	b
Transformation matrix object ID in which to copy	Specifies the identifier of the transformation matrix object in which to copy the bounding box statistics results. The transformation matrix object must have been previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.	c		e	f	g	a		i		j	k	l	m	n	b
M_PCA_MATRIX	Specifies to copy, into the transformation matrix, PCA statistics results that can bring points on the X-, Y-, and Z-axes to the object's principal axes. If M_PCA_MODE is set to M_CENTRAL, the matrix contains a translation and rotation to the centroid; otherwise, it is a pure rotation. The centroid is that of the original depth map or point cloud from which the statistics were calculated. If the source was a point cloud, it is the centroid of the points or the points' unit normal vectors, depending on whether the statistics were calculated on the range or normals component, respectively.	c		e	f	g	a		i		j	k	l	m	n	b
Transformation matrix object ID in which to copy	Specifies the identifier of the transformation matrix object in which to copy the principal component analysis (PCA) statistics results. The transformation matrix object must have been previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRINCIPAL_AXIS_1	Specifies to copy PCA statistics results into the specified 3D geometry object, establishing a 3D line geometry; the line marks the first principal axis and has a length of 1 unit. If M_PCA_MODE is set to M_CENTRAL, the line starts at the centroid; otherwise, it starts at the origin. The centroid is that of the original depth map or point cloud from which the statistics were calculated. If the source was a point cloud, it is the centroid of the points or the points' unit normal vectors, depending on whether the statistics were calculated on the range or normals component, respectively.	c		e	f	g	a		i		j	k	l	m	n	b
3D geometry object ID in which to copy	Specifies the identifier of a 3D geometry object in which to copy the statistics results. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY. Note that, after calling **M3dimCopyResult**(), the destination 3D geometry object will return type M_LINE if inquired using M3dgeoInquire() with M_GEOMETRY_TYPE.	c		e	f	g	a		i		j	k	l	m	n	b

M_PRINCIPAL_AXIS_2	Specifies to copy PCA statistics results into the specified 3D geometry object, establishing a 3D line geometry; the line marks the second principal axis and has a length of 1 unit. If M_PCA_MODE is set to M_CENTRAL, the line starts at the centroid; otherwise, it starts at the origin. The centroid is that of the original depth map or point cloud from which the statistics were calculated. If the source was a point cloud, it is the centroid of the points or the points' unit normal vectors, depending on whether the statistics were calculated on the range or normals component, respectively.	c		e	f	g	a		i		j	k	l	m	n	b
3D geometry object ID in which to copy	Specifies the identifier of a 3D geometry object in which to copy the statistics results. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY. Note that, after calling **M3dimCopyResult**(), the destination 3D geometry object will return type M_LINE if inquired using M3dgeoInquire() with M_GEOMETRY_TYPE.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRINCIPAL_AXIS_3	Specifies to copy PCA statistics results into the specified 3D geometry object, establishing a 3D line geometry; the line marks the third principal axis and has a length of 1 unit. If M_PCA_MODE is set to M_CENTRAL, the line starts at the centroid; otherwise, it starts at the origin. The centroid is that of the original depth map or point cloud from which the statistics were calculated. If the source was a point cloud, it is the centroid of the points or the points' unit normal vectors, depending on whether the statistics were calculated on the range or normals component, respectively.	c		e	f	g	a		i		j	k	l	m	n	b
3D geometry object ID in which to copy	Specifies the identifier of a 3D geometry object in which to copy the statistics results. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY. Note that, after calling **M3dimCopyResult**(), the destination 3D geometry object will return type M_LINE if inquired using M3dgeoInquire() with M_GEOMETRY_TYPE.	c		e	f	g	a		i		j	k	l	m	n	b
M_STANDARDIZATION_MATRIX	Specifies to copy, into the transformation matrix, moments statistics results that can scale the point cloud to dimensions at or near one unit. This means that the centroid is positioned at (0,0,0) and the point cloud is transformed so that it has unit variance along each axis. Therefore, on average, each point's X-coordinate will be one unit away from the origin, and similarly for each Y- and Z-coordinate. Note that the moments results must be of order 2 or greater. To transform the point cloud to fit inside a unit box, copy bounding box statistic results using M_NORMALIZATION_MATRIX.	c		e	f	g	a		i		j	k	l	m	n	b
Transformation matrix object ID in which to copy	Specifies the identifier of the transformation matrix object in which to copy the moments statistics results. The transformation matrix object must have been previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dim11.lib.
DLL	Requires ail11.dll; ail3dim11.dll.

M3dimCrop

Synopsis

Crop points in a point cloud based on a specified 3D geometry or mask image buffer.

Syntax

```
void M3dimCrop(
    AIL_ID SrcContainerBufId,         //in
    AIL_ID DstContainerBufId,         //out
    AIL_ID ImageBufOrGeometry3dgeoId, //in
    AIL_ID Matrix3dgeoId,             //in
    AIL_INT64 OrganizationType,       //in
    AIL_INT64 ControlFlag             //in
)
```

Description

This function crops 3D points in the source container's point cloud, based on a specified 3D geometry mask or image buffer. You can optionally specify a transformation matrix to fixture the source point cloud before the cropping operation.

Source components are copied into the destination container where the destination's confidence component determines a point's viability after the crop operation. Specifically, rejected points (those cropped out) are assigned a 0 confidence value in the destination confidence component. Corresponding points in the destination range component are considered cropped out.

Note that normals and reflectance (or intensity) components are not required, but the crop operation is applied to these components if they are present in the source container.

If the crop is based on a 3D geometry, the default operation keeps points located inside the specified geometry, rejecting all others.

If the crop is based on an image buffer, the mask is applied differently based on the type of image buffer:

- When using an uncalibrated image buffer, points are mapped to the same location in the mask image buffer as the point's location in the range component of its point cloud container.
- When using a uniformly calibrated image buffer, points are projected orthogonally onto the mask.
- When using a 3D-based calibrated image buffer, points are projected onto the mask following the frustum of the camera's view, as represented by the associated camera calibration information.

Points are kept only if they correspond to non-zero pixels in the mask image buffer; any points corresponding to zero pixels are rejected. Note that you can specify to invert the crop operation (for example, to keep points located outside, instead of inside, the specified geometry). For more information on how different image buffers can affect how the crop is performed, see the Cropping using a mask image subsection of the Cropping or masking points section of Chapter 35: 3D image processing.

You can specify how the destination container is organized (for example, as an unorganized point cloud).

If a mesh component exists in the source point cloud container, it is also affected by the crop operation. Any cropped point that correspond to the vertex of a triangular face results in that face's removal. This can result in a hole in the mesh.

Parameters

SrcContainerBufId

Specifies the identifier of the source container containing a 3D-processable point cloud. The container must have been previously allocated using MbufAllocContainer() with M_PROC, and must be 3D-processable (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE, the function returns M_PROCESSABLE).

DstContainerBufId

Specifies the identifier of the destination container in which to store the resulting point cloud. The container must have been previously allocated using MbufAllocContainer() with M_PROC, and must not be a child container.

> Note that, if the OrganizationType parameter is set to M_UNORGANIZED and there are no valid points remaining after the crop, the destination container's range component's dimensions are reduced to 1x1 and its corresponding confidence equals 0.

ImageBufOrGeometry3dgeoId

Specifies the identifier of the 3D geometry object or image buffer used to crop the points.

For specifying the mask																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_XY_PLANE	Specifies the XY (Z=0) plane. Points are kept above this plane, unless M_INVERSE is specified; see the ControlFlag parameter.	c		e	f	g	a		i		j	k	l	m	n	b
3D geometry object identifier	Specifies the 3D geometry object identifier. The 3D geometry object must have been previously allocated on the required system using M3dgeoAlloc() with M_GEOMETRY, and must have been successfully defined. Supported 3D geometries include box, cylinder, plane, and sphere.	c		e	f	g	a		i		j	k	l	m	n	b
Image buffer identifier	Specifies the mask image buffer. The mask must be a 1-band, 8-bit, 16-bit or 32-bit unsigned image buffer, and must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error. You can use a calibrated or uncalibrated image buffer. When cropping using an uncalibrated image buffer, the point cloud's points are mapped to the same location in the mask image buffer as the point's location in the range component of its point cloud container. A point is not kept if it corresponds to a zero pixel in the mask. The image buffer's size in X and Y must be the same as the source container's range component. To inquire the range component's size, use MbufInquireContainer() with M_COMPONENT_RANGE, M_SIZE_X, and M_SIZE_Y. When cropping using a calibrated mask, the point cloud's points are projected onto the mask image. A point is not kept if it is projected onto a zero pixel in the mask. The projection is orthogonal for a uniformly calibrated mask. That is, world coordinate values in the point cloud correspond linearly to pixel positions in the 2D mask image onto which the point cloud is projected. If a calibrated mask image is of type M_3D_CALIBRATION, point projection is not orthogonal, but instead follows the frustum of the camera's view, as represented by the associated camera calibration information. To keep points that correspond to zero pixels and crop out the others, set the ControlFlag parameter to M_INVERSE.	c		e	f	g	a		i		j	k	l	m	n	b

Matrix3dgeoId

Specifies the identifier of the optional transformation matrix used to fixture the source container before applying the cropping criteria.

> Note that passing a transformation matrix to **M3dimCrop**() is the equivalent to first calling M3dimMatrixTransform(), and then calling **M3dimCrop**() on the resulting transformed point cloud.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the identifier of a 3D geometry transformation matrix																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies to ignore this parameter.	c		e	f	g	a		i		j	k	l	m	n	b
M_IDENTITY_MATRIX	Specifies to use an identity matrix. This means that the source point cloud will not be transformed or rotated in any way before the cropping operation.	c		e	f	g	a		i		j	k	l	m	n	b
Transformation matrix object identifier	Specifies the identifier of a transformation matrix object, previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX. The transformation matrix must be any valid affine transformation matrix, where the last row is (0,0,0,1). That is, if you call M3dgeoInquire() with M_AFFINE, the function returns M_TRUE.	c		e	f	g	a		i		j	k	l	m	n	b

OrganizationType

Specifies how to organize the resulting point cloud.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the organization type																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_DEFAULT	Same as M_SAME.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAME	Specifies to copy all the source container's components into the destination container; points set to 0 in the destination confidence component indicate cropped points.	c		e	f	g	a		i		j	k	l	m	n	b
M_SHRINK	Specifies to copy, from source to destination, the smallest region that holds all valid points (remaining after the crop operation) in all components; points set to 0 in the destination confidence component indicate cropped points. Note that the region is a rectangular region within the bands of the 2D data buffers that hold point cloud data.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNORGANIZED	Specifies to store results in an unorganized point cloud. Components in the destination container will store results of the crop operation in 1D, unorganized buffers.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Specifies which points to keep, relative to the cropping object set with ImageBufOrGeometry3dgeoId.

By default, the operation only affects the core components of a 3D-processable container. These core components are listed in the description of M_3D_PROCESSABLE. You can use M_APPLY_TO_ALL_COMPONENTS to apply the operation to all other components in the container, including custom components. The components must have the same dimensions as M_COMPONENT_RANGE; otherwise, they will be copied to the destination unmodified.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying which points to keep																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies to keep points that are inside the specified 3D geometry object, above the specified plane, or not set to 0 in the specified mask, depending on the cropping object set with ImageBufOrGeometry3dgeoId. > Note that, for a plane geometry, above is in the direction of the plane normal vector. To inquire this vector, use M3dgeoInquire() with M_NORMAL_....	c		e	f	g	a		i		j	k	l	m	n	b
M_INVERSE +	Specifies to keep points that are outside the specified 3D geometry object, below the specified plane, or set to 0 in the specified mask, depending on the cropping object set with ImageBufOrGeometry3dgeoId. > Note that, for a plane geometry, below is in the opposite direction of the plane normal vector. To inquire this vector, use M3dgeoInquire() with M_NORMAL_....	c		e	f	g	a		i		j	k	l	m	n	b

Combination value that can be used alone or as combination value for M_INVERSE.

You can use the following value on its own, or add it to the above-mentioned value, to specify to apply the cropping operation to all components.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying to apply the cropping operation to all components																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_APPLY_TO_ALL_COMPONENTS	Specifies that the operation will affect the core components of a 3D-processable container, as well as any other component (including custom components) that has the same dimension as M_COMPONENT_RANGE. These components must not have a region of interest (ROI) associated with them. For example, using a component that is an image buffer with an ROI will cause an error. > Note that if a custom component has an associated calibration context, the component's information is cropped, but its calibration context is ignored and discarded. If the resulting operation yields an empty container with 0 valid points, the components with the same dimensions are not created and will not exist in the destination container.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dim11.lib.
DLL	Requires ail11.dll; ail3dim11.dll.

M3dimDraw3d

Synopsis

Draw the result of a profile operation into a 3D graphics list.

Syntax

```
AIL_INT64 M3dimDraw3d(
    AIL_ID OperationDraw3dContext3dimId, //in
    AIL_ID SrcProfileResult3dimId,       //in
    AIL_ID DstList3dgraId,               //out
    AIL_INT64 DstParentLabel,            //in
    AIL_INT64 ControlFlag                //in
)
```

Description

This function draws the profile points, stored in the specified 3D profile result buffer, into a 3D graphics list. To take a profile of a depth map, use M3dimProfile() or M3dimProfileEx(). To take a profile of a 3D geometry, mesh, or point cloud, use M3dimProfile().

Parameters

OperationDraw3dContext3dimId

Reserved for future expansion and must be set to **M_DEFAULT**.

SrcProfileResult3dimId

Specifies the identifier of the profile 3D image processing result buffer, previously allocated using M3dimAllocResult() with M_PROFILE_RESULT.

DstList3dgraId

Specifies the identifier of the 3D graphics list in which to draw. You can specify a 3D graphics list that you have previously allocated using M3dgraAlloc(), or you can specify the identifier of the 3D display's internal graphics list (inquired using M3ddispInquire() with M_3D_GRAPHIC_LIST_ID).

DstParentLabel

Specifies the label of the 3D graphic in the 3D graphics list to use as the profile annotation's parent.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the parent label																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_ROOT_NODE.	c		e	f	g	a		i		j	k	l	m	n	b
M_ROOT_NODE	Specifies the top-most node of the 3D graphics list.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the label of the 3D graphic in the 3D graphics list. Label 0 is the 3D graphics list's root node.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Return value

Returns the parent label of all 3D graphics that the function added to the 3D graphics list. If the specified profile 3D image processing result buffer has 0 points, a node 3D graphic is added to the graphics list and the node's label is returned.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dim11.lib.
DLL	Requires ail11.dll; ail3dim11.dll.

M3dimFillGaps

Synopsis

Fill gaps in a depth map and, optionally, an intensity map, according to gap filling context controls.

Syntax

```
void M3dimFillGaps(
    AIL_ID GapFillingContext3dimId, //in
    AIL_ID DepthMapImageBufId,      //out
    AIL_ID IntensityMapImageBufId,  //out
    AIL_INT64 ControlFlag           //in
)
```

Description

This function replaces missing or invalid data (gaps) in a depth map and/or intensity map. You can choose to fill gaps with interpolated values, or propagate a boundary value across the gap. The gap filling operation is performed according to the context specified with GapFillingContext3dimId.

For some applications, you can reasonably approximate gap values with an interpolation of neighboring values. In other situations, such as when the gap spans an abrupt change in depth, or the gap is very wide, you can choose to propagate one of the gap's boundary values across the gap. To specify how wide a gap to fill and other gap filling settings, use M3dimControl().

Gaps are filled in a 2-step process: first along X, and then along Y (or vice versa). For example, if you choose to fill first along the X-direction, all gaps along rows in the depth or intensity map are evaluated and filled before any columns are examined. Therefore, gaps are examined not as an X-by-Y area to be filled, but on a per-row (or column) basis. To set the mode in which to fill gaps, use M3dimControl() with M_FILL_MODE and M_X_THEN_Y or M_Y_THEN_X.

Parameters

GapFillingContext3dimId

Specifies a fill gaps 3D image processing context.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the fill gaps 3D image processing context identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default fill gaps 3D image processing context of the current Aurora Imaging Library application. The fill gaps operation will use default values for all gap filling control types listed in M3dimControl().	c		e	f	g	a		i		j	k	l	m	n	b
Fill gaps 3D image processing context identifier	Specifies the identifier of a fill gaps 3D image processing context, previously allocated using M3dimAlloc() with M_FILL_GAPS_CONTEXT. If a previously allocated context is specified, the function applies the fill gaps control settings specified using M3dimControl().	c		e	f	g	a		i		j	k	l	m	n	b

DepthMapImageBufId

Specifies the depth map container or image buffer identifier of the depth map on which to fill gaps.

The container must have been previously allocated using MbufAllocContainer() with M_PROC, must store data in a 3D-processable depth map format (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE_DEPTH_MAP, the function returns M_TRUE), and must not be a child container.

The image buffer must be an 8-bit, 16-bit, or 32-bit unsigned, 1-band buffer, and must be 3D-corrected (that is, if you call McalInquire() with M_DEPTH_MAP, the function returns M_TRUE).

IntensityMapImageBufId

Specifies the image buffer identifier of the intensity map on which to fill gaps. The image buffer must be an 8-bit or 16-bit unsigned, 1-band or 3-band buffer. The image buffer must have the same dimensions as the buffer specified for DepthMapImageBufId (or, if DepthMapImageBufId is a depth map container, the same dimensions as the range component).

If the intensity map is not required, set this parameter to **M_NULL**.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dim11.lib.
DLL	Requires ail11.dll; ail3dim11.dll.

M3dimFilter

Synopsis

Apply a smoothing filter to a point cloud.

Syntax

```
void M3dimFilter(
    AIL_ID FilterContext3dimId, //in
    AIL_ID SrcContainerBufId,   //in
    AIL_ID DstContainerBufId,   //out
    AIL_INT64 ControlFlag       //in
)
```

Description

This function applies a smoothing filter to a point cloud. Filtering reduces noise by changing the positions of points, but does not eliminate them. If you want to invalidate outlier points, use M3dimOutliers().

You can choose the type of smoothing operation to perform, either bilateral or moving least squares (MLS). Use M3dimControl() with M_FILTER_MODE to set the mode. The MLS filter applies smoothing equally to surface and edge points. The bilateral filter, on the other hand, smooths points on faces, leaving edge points much less affected.

The bilateral filter (M_SMOOTH_BILATERAL) uses the source point cloud's normal values. If a normals component does not already exist in the source point cloud, **M3dimFilter**() will calculate point normals and create a normals component, adding it to the destination container. In this case, use M3dimControl() with M_USE_SOURCE_NORMALS set to M_FALSE. Note that this is more efficient than separately calculating a normals component using M3dimNormals(). Nevertheless, the method used for computing normals could improve filter results. For more information, see the Smoothing a point cloud subsection of the Smoothing a point cloud and invalidating outlier points section of Chapter 35: 3D image processing.

Parameters

FilterContext3dimId

Specifies a filter 3D image processing context.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the filter 3D image processing context identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Filter 3D image processing context identifier	Specifies the identifier of a filter 3D image processing context, previously allocated using M3dimAlloc() with M_FILTER_CONTEXT. > The function applies the filter control settings specified using M3dimControl().	c		e	f	g	a		i		j	k	l	m	n	b

SrcContainerBufId

Specifies the identifier of the 3D-processable point cloud container to use as the source. The container must be 3D-processable (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE, the function returns M_PROCESSABLE).

The container must have been previously allocated using MbufAllocContainer() with M_PROC.

DstContainerBufId

Specifies the identifier of the destination container, previously allocated using MbufAllocContainer() with M_PROC. The destination container must not be a child container.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dim11.lib.
DLL	Requires ail11.dll; ail3dim11.dll.

M3dimFindTransformation

Synopsis

Find the transformation between 2 sets of points.

Syntax

```
void M3dimFindTransformation(
    AIL_ID FindTransformationContext3dimId, //in
    AIL_ID SourceContainerOrArrayBufId,     //in
    AIL_ID TargetContainerOrArrayBufId,     //in
    AIL_ID Result3dimOrMatrix3dgeoId,       //out
    AIL_INT64 ControlFlag                   //in
)
```

Description

This function finds the rigid or affine transformation between the source and target sets of points. Aurora Imaging Library finds the transformation matrix A such that A * source (as per M3dimMatrixTransform()) best approximates the target. Specifically, the transformation minimizes the squared error between pairs of points. Source[x, y] is compared with Target[x, y]. Accordingly, this function works only when you know a specific pairing between the source and target points (for example, through feature extraction). For a function that finds an alignment but does not require an exact pairing, see M3dregCalculate(). Results are stored in the specified result buffer or transformation matrix object. To retrieve the matrix from a result buffer, use M3dimCopyResult() with M_TRANSFORMATION_MATRIX.

The source and target can be both containers or both Aurora Imaging Library array buffers (M_ARRAY), or any combination of the two. Note that only pairs with 2 valid points are considered. For an Aurora Imaging Library array buffer, all points are valid; for a container, points are valid if their corresponding confidence score is non-zero.

For this function to succeed, at least 3 non-collinear point pairs are required for a rigid transformation, and at least 4 non-coplanar point pairs are required for an affine transformation. If the operation fails, M3dimGetResult() with M_STATUS returns M_NOT_ENOUGH_POINT_PAIRS, M_ALL_POINTS_COPLANAR, or M_NOT_INITIALIZED, and the zero matrix is returned. Note that attempting to use the zero matrix in functions such as M3dimMatrixTransform() will cause an error. To verify that the matrix is a valid rigid or affine transformation and not the zero matrix, call M3dgeoInquire() with M_RIGID or M_AFFINE (respectively) and ensure that the function returns M_TRUE.

Parameters

FindTransformationContext3dimId

Specifies the identifier of a find transformation 3D image processing context.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the find transformation 3D image processing context identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FIND_TRANSFORMATION_CONTEXT_AFFINE	Specifies to find an affine transformation between the source and target sets of points. You should use this mode when one set of points (either source or target) originates from an uncalibrated source, which might contain a distortion such as a shear. Note that this mode requires at least 4 non-coplanar point pairs.	c		e	f	g	a		i		j	k	l	m	n	b
M_FIND_TRANSFORMATION_CONTEXT_RIGID	Specifies to find a rigid transformation between the source and target sets of points. You should use this mode when the two sets of points (source and target) differ by a physical displacement in three dimensions (for example, a rotation and/or translation). Note that this mode requires at least 3 non-collinear point pairs.	c		e	f	g	a		i		j	k	l	m	n	b

SourceContainerOrArrayBufId

Specifies the identifier of the source container containing a 3D-processable point cloud, or specifies the identifier of the source Aurora Imaging Library array buffer.

The container must have been previously allocated using MbufAllocContainer() with M_PROC, and must be 3D-processable (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE, the function returns M_PROCESSABLE).

The source Aurora Imaging Library array buffer must be a 3-band floating-point buffer, allocated using MbufAllocColor() with M_ARRAY, and must have the same size as the target. You can load values into the buffer using, for example, MbufPutColor().

TargetContainerOrArrayBufId

Specifies the identifier of the target container containing a 3D-processable point cloud, or specifies the identifier of the target Aurora Imaging Library array buffer.

The container must have been previously allocated using MbufAllocContainer() with M_PROC, and must be 3D-processable (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE, the function returns M_PROCESSABLE).

The target Aurora Imaging Library array buffer must be a 3-band floating-point buffer, allocated using MbufAllocColor() with M_ARRAY, and must have the same size as the source. You can load values into the buffer using, for example, MbufPutColor().

Result3dimOrMatrix3dgeoId

Specifies the identifier of the find transformation 3D image processing result buffer or transformation matrix object in which to store the calculated transformation matrix.

The result buffer must have been allocated using M3dimAllocResult() with M_FIND_TRANSFORMATION_RESULT; the result buffer holds the transformation matrix and the status of the operation.

The transformation matrix object must have been allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dim11.lib.
DLL	Requires ail11.dll; ail3dim11.dll.

M3dimFix

Synopsis

Determine if a container satisfies certain conditions and optionally fixes them if they are not met.

Syntax

```
void M3dimFix(
    AIL_ID SrcContainerBufId, //in
    AIL_ID DstContainerBufId, //out
    AIL_INT64 Conditions,     //in
    AIL_INT64 ControlFlag,    //in
    AIL_INT64 *ResultPtr      //in
)
```

Description

This function allows you to validate whether a container meets a certain condition. You can use **M3dimFix**() to verify that a container is suitable for use with another 3D function. You can optionally specify a destination container to obtain a modified copy of the source container that fixes the specified conditions.

Parameters

SrcContainerBufId

Specifies the identifier of the source point cloud container, previously allocated using MbufAllocContainer() with M_PROC. The container must be 3D processable (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE, the function returns M_PROCESSABLE).

DstContainerBufId

Specifies the identifier of the destination container in which to place the modified point cloud. The container must have been previously allocated using MbufAllocContainer() with M_PROC, and must not be a child container.

If you do not want to retrieve a modified point cloud that satisfies the given conditions, you must set this parameter to **M_NULL**.

Conditions

Specifies which conditions you would like to be validated and optionally fixed.

The values in the tables below can be added together to specify multiple conditions. For example, to verify that a mesh contains only valid points and that there are no degenerate triangles, specify M_MESH_VALID_POINTS + M_MESH_VALID_NEIGHBORS.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the conditions to validate																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ALL	Specifies to verify that all the conditions listed in this table are met. Conditions are only validated if the specified source point cloud container has the component required to validate the condition. If you specify a DstContainerBufId, you will retrieve a modified version of the source point cloud container that satisfies all the valid conditions for it.	c		e	f	g	a		i		j	k	l	m	n	b
M_MESH_VALID_NEIGHBORS	Specifies to check each edge in the mesh to verify that the edge appears in at most two triangular faces. Triangular faces are also verified to ensure that they are constructed from three different vertices to ensure that there are no degenerate triangular faces in the mesh. This option is available when the source container has an M_COMPONENT_MESH_AIL component. M_MESH_VALID_NEIGHBORS must be satisfied for some functions which use a mesh component, such as M3dmetVolume(). If you specify a destination container and more than two triangular faces are found sharing a common edge, the first two faces are kept, while the subsequent triangles are removed.	c		e	f	g	a		i		j	k	l	m	n	b

M_MESH_VALID_POINTS	Specifies to check that each point in the mesh is valid; a valid point has a non-zero confidence score. This option is available when the source container has an M_COMPONENT_MESH_AIL component. M_MESH_VALID_POINTS must be satisfied for all functions that use a mesh component. If you specify a destination container, any invalid point in the mesh that corresponds to the vertex of a triangular face results in that face's removal.	c		e	f	g	a		i		j	k	l	m	n	b
M_NORMALS_FINITE	Specifies to check that each point in the range component has a finite normal vector. This option is available when the source container has an M_COMPONENT_NORMALS_AIL component. M_NORMALS_FINITE must be satisfied for all functions that use a normals component. If you specify a DstContainerBufId and points with infinite normal vectors are found, the point's confidence is set to 0 and the normals length is set to the range's invalid value. You can set this value using M_3D_INVALID_DATA_VALUE with MbufControl().	c		e	f	g	a		i		j	k	l	m	n	b
M_NORMALS_NORMALIZED	Specifies to check that each point with a non-zero confidence has a normal with a norm of exactly one or zero. This option is available when the source container has an M_COMPONENT_NORMALS_AIL component. M_NORMALS_NORMALIZED must be satisfied for all functions that use a normals component. If you specify a DstContainerBufId and normals are found without a norm of one or zero, they will be normalized to have a norm of one.	c		e	f	g	a		i		j	k	l	m	n	b
M_RANGE_FINITE	Specifies to check that each point in the range component is finite. M_RANGE_FINITE is required for all 3D functions. If you specify a DstContainerBufId and invalid points are found, the point's confidence is set to 0 and the point is set to the range's invalid value. You can set this value using M_3D_INVALID_DATA_VALUE with MbufControl().	c		e	f	g	a		i		j	k	l	m	n	b
M_UNREPLICATED_POINTS	Specifies to check that each point in the range component with a non-zero confidence is unreplicated. If you specify a DstContainerBufId and replicated points are found, the point's confidence is set to 0. Note that when two points are replicates, either of the points could be removed.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion. This parameter must be set to **M_DEFAULT**.

ResultPtr

Specifies the address of the variable in which to write the conditions that were detected. Alternatively, if you do not want to return the conditions, you can set this parameter to **M_NULL**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dim11.lib.
DLL	Requires ail11.dll; ail3dim11.dll.

M3dimFree

Synopsis

Free a 3D image processing context or result buffer.

Syntax

```
void M3dimFree(
    AIL_ID ContextOrResult3dimId //in
)
```

Description

This function deletes the specified 3D image processing context or result buffer identifier, and releases any memory associated with it.

All 3D image processing contexts or result buffers allocated on a particular system must be freed before the system can be freed, unless M_UNIQUE_ID was specified during allocation.

Parameter

ContextOrResult3dimId

Specifies the identifier of the 3D image processing context or result buffer to free. These must have been successfully allocated (using M3dimAlloc() or M3dimAllocResult()) prior to calling this function.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dim11.lib.
DLL	Requires ail11.dll; ail3dim11.dll.

M3dimGet

Synopsis

Get valid points or other component values from a container's point cloud or depth map, or get values from a depth map image.

Syntax

```
AIL_INT M3dimGet(
    AIL_ID ContainerOrImageBufId,          //in
    AIL_INT64 Component,                   //in
    AIL_INT ArraySize,                     //in
    AIL_INT64 Options,                     //in
    AIL_DOUBLE *DstCoord1OrPackedArrayPtr, //out
    AIL_DOUBLE *DstCoord2ArrayPtr,         //out
    AIL_DOUBLE *DstCoord3ArrayPtr          //out
)
```

Description

This function returns a component's values that correspond to valid points in the source point cloud or depth map container, or it returns valid values in a depth map image, excluding invalid points and points outside a region of interest (if the depth map image buffer is associated with an ROI).

When copying from a floating-point buffer to an integer array, the values are truncated. If the source buffer depth is greater than that of the destination array, the most significant bits are truncated when the data is copied into the destination.

For a depth map image, you can limit this function's results to a region of the depth map image buffer using a region of interest (ROI) set using MbufSetRegion(). The ROI must be defined in raster format (M_RASTER or M_VECTOR_AND_RASTER). An error is generated if the ROI is only in vector format (M_VECTOR).

To get all the coordinates stored in a component, regardless of the corresponding confidence score, use MbufGet().

> Note that presented above is an example prototype of this function. You can call M3dimGet() with the same prototype as any of the type-specific functions below, without using the function's type-specific name; that is, use M3dimGet() and not, for example, M3dimGetFloat(). The data is automatically cast into the appropriate type. This is true except when using a C compiler. When using a C compiler, you must always call this function with its type-specific name (you cannot use the standard name of this function).

Parameters

ContainerOrImageBufId

Specifies the identifier of the source point cloud container, depth map container, or depth map image buffer.

The point cloud or depth map container must have been previously allocated using MbufAllocContainer() with M_PROC.

The point cloud container must be 3D-processable (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE, the function returns M_PROCESSABLE).

The depth map container must store data in a 3D-processable depth map format (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE_DEPTH_MAP, the function returns M_TRUE).

The image buffer must be a 1-band, 8-bit, 16-bit, or 32-bit unsigned buffer, and must contain a fully corrected depth map (that is, if you call McalInquire() with M_DEPTH_MAP, the function returns M_TRUE). If you specify an image buffer that has an ROI associated with it, the ROI must be in raster format; otherwise, you will get an error.

Component

Specifies the component from which to get values. The specified component must be of the same size as that of M_COMPONENT_RANGE, and must be an 8-bit, 16-bit or 32-bit image buffer. If the required component is not the same size, use MbufGet() instead.

When ContainerOrImageBufId is a depth map image buffer, you must set Component to **M_DEFAULT**.

Use the following settings to specify a component using its identifier or index. An error is generated if there is no component in the container matching the specified identifier or index.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a component by its identifier or index																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_COMPONENT_BY_ID(AIL_INT ID)	Specifies the component, in the container, that has the specified buffer identifier.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	ID Specifies the Aurora Imaging Library identifier of the component.															
M_COMPONENT_BY_INDEX(AIL_INT ComponentIndex)	Specifies the component, in the container, at the specified index.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	ComponentIndex Specifies the index of the component.															

Use the following settings to specify a component by its component type, which identifies what kind of information the component stores.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a component by its component type																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_COMPONENT_CONFIDENCE	Specifies that the component stores confidence information for the M_COMPONENT_RANGE component of the container. Coordinates associated with the confidence value 0 are considered invalid data and will not be used by 3D image processing functions.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_COORDINATE_MAP_A_AIL	Specifies that the component stores the A coordinate map provided by the camera. When using MbufConvert3d() with a projective map, this component can be used instead of the column index of each corresponding pixel and will result in a better representation of the camera's lens model when generating X-coordinates. A coordinate map component is associated with a range component in the same container when there is only one range component and only one type of each coordinate map component in that container. > Note that this component is only available if it is provided by the camera. For more information refer to your cameras manual.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_COORDINATE_MAP_B_AIL	Specifies that the component stores the B coordinate map provided by the camera. When using MbufConvert3d() with a projective map, this component can be used instead of the row index of each corresponding pixel and will result in a better representation of the camera's lens model when generating Y-coordinates. A coordinate map component is associated with a range component in the same container when there is only one range component and only one type of each coordinate map component in that container. > Note that this component is only available if it is provided by the camera. For more information refer to your cameras manual.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_CUSTOM + *n*	Specifies that the component has a custom component type, identified by *n*, where n can be a value between 0 and 254. You can use custom component types to identify components which store information that does not fit one of the standard component types. In addition, some cameras transmit components with custom component types. Refer to your camera manual to determine what type of information is stored in transmitted components.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_DISPARITY	Specifies that the component stores a disparity map. Each pixel of a disparity map indicates the apparent distance (typically measured in pixel units) between where an object appears in the left and right images captured by a stereoscopic camera.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_INFRARED	Specifies that the component stores an intensity image of infrared light.	c		e	f	g	a		i		j	k	l	m	n	b

M_COMPONENT_INTENSITY	Specifies that the component stores an intensity image of visible light.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_METADATA	Specifies that the component stores metadata information. Metadata components are used by Aurora Imaging Library internally. Typically, you should ignore metadata components in your application.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_MULTISPECTRAL	Specifies that the component stores an intensity image where each band represents the intensity of a specific wavelength of light. Unlike an intensity component, a multispectral component might include information about non-visible light.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_NORMALS_AIL	Specifies that the component stores normals information for each point in the M_COMPONENT_RANGE component of the container.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_RANGE	Specifies that the component stores 3D position information. This is a 3-band buffer that stores coordinates of 3D points.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_REFLECTANCE	Specifies that the component stores a reflectance map. Each pixel of a reflectance map indicates how much of the light hitting an object at that location is reflected back. Typically, this is an intensity image of the light spectrum used by a 3D sensor to detect 3D distance/position information. Typically, if the map was generated by a laser profiler, each row indicates the detected intensity of the laser for a single scan.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_REGION_AIL	Specifies that the component stores a region of interest (ROI) for the M_COMPONENT_RANGE component of the container. 3D image processing functions will only operate on points inside the ROI. Only depth map containers can have a region component. A region component is associated with a range component in the same container when there is only one range component and no other region components in that container.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_SCATTER	Specifies that the component stores a scatter map. Each pixel of a scatter map indicates how much of the light hitting an object at that location is detected scattering beneath the object's surface.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_ULTRAVIOLET	Specifies that the component stores an intensity image of ultraviolet light.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_UNDEFINED	Specifies that the component stores information of an unknown type.	c		e	f	g	a		i		j	k	l	m	n	b

ArraySize

Specifies the maximum number of elements that the arrays can hold.

If the specified ArraySize is larger than the number of points written, the rest of the array is left unchanged; if ArraySize is less than the number of points available, the points are written up to the specified ArraySize.

When determining the required size of the arrays for subsequent calls to **M3dimGet**(), set this parameter to **M_NULL**.

> When using a standard vector (std::vector) overload function in C++, you can pass **M_DEFAULT** to this parameter; the size will be determined automatically based on the number of items in the vector passed to the DstCoord1OrPackedArrayPtr, DstCoord2ArrayPtr, or DstCoord3ArrayPtr parameter.

Options

Specifies additional options.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the storage format																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_PLANAR.	c		e	f	g	a		i		j	k	l	m	n	b

M_PACKED	Specifies to store the component data in a packed format.	c		e	f	g	a		i		j	k	l	m	n	b
M_PLANAR	Specifies to store the component data in a planar format.	c		e	f	g	a		i		j	k	l	m	n	b

DstCoord1OrPackedArrayPtr
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>] (and a maximum array rank for C# of 1) Required array size:(ArraySize)
- array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>] (and a maximum array rank for C# of 1) Required array size:(ArraySize)
- array of type AIL_INT16 [optionally, in C++: a reference to a std::vector<AIL_INT16>] (and a maximum array rank for C# of 1) Required array size:(ArraySize)
- array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>] (and a maximum array rank for C# of 1) Required array size:(ArraySize)
- array of type AIL_INT8 [optionally, in C++: a reference to a std::vector<AIL_INT8>] (and a maximum array rank for C# of 1) Required array size:(ArraySize)
- array of type AIL_UINT16 [optionally, in C++: a reference to a std::vector<AIL_UINT16>] (and a maximum array rank for C# of 1) Required array size:(ArraySize)
- array of type AIL_UINT32 [optionally, in C++: a reference to a std::vector<AIL_UINT32>] (and a maximum array rank for C# of 1) Required array size:(ArraySize)
- array of type AIL_UINT8 [optionally, in C++: a reference to a std::vector<AIL_UINT8>] (and a maximum array rank for C# of 1) Required array size:(ArraySize)

Specifies the address of the user array in which to store the retrieved data. This array can hold the values of the specified point cloud or depth map's component, or a packed array of all the data. For instance, a depth map's 3 bands of coordinate data, if packed, is stored in the array in an interleaved manner (for example, XYZ XYZ XYZ...).

When determining the required size of this array for subsequent calls to **M3dimGet**(), set this parameter to **M_NULL**.

DstCoord2ArrayPtr
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>] (and a maximum array rank for C# of 1) Required array size:(ArraySize)
- array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>] (and a maximum array rank for C# of 1) Required array size:(ArraySize)
- array of type AIL_INT16 [optionally, in C++: a reference to a std::vector<AIL_INT16>] (and a maximum array rank for C# of 1) Required array size:(ArraySize)
- array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>] (and a maximum array rank for C# of 1) Required array size:(ArraySize)
- array of type AIL_INT8 [optionally, in C++: a reference to a std::vector<AIL_INT8>] (and a maximum array rank for C# of 1) Required array size:(ArraySize)
- array of type AIL_UINT16 [optionally, in C++: a reference to a std::vector<AIL_UINT16>] (and a maximum array rank for C# of 1) Required array size:(ArraySize)
- array of type AIL_UINT32 [optionally, in C++: a reference to a std::vector<AIL_UINT32>] (and a maximum array rank for C# of 1) Required array size:(ArraySize)
- array of type AIL_UINT8 [optionally, in C++: a reference to a std::vector<AIL_UINT8>] (and a maximum array rank for C# of 1) Required array size:(ArraySize)

Specifies the address of the user array in which to store the retrieved data. If the component is 3-band and Options is set to M_PLANAR, the pointer passed to DstCoord2ArrayPtr must be valid. If the component is 1-band or Options is set to M_PACKED, set DstCoord2ArrayPtr to **M_NULL**.

When determining the required size of this array for subsequent calls to **M3dimGet**(), set this parameter to **M_NULL**.

DstCoord3ArrayPtr
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>] (and a maximum array rank for C# of 1) Required array size:(ArraySize)
- array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>] (and a maximum array rank for C# of 1) Required array size:(ArraySize)
- array of type AIL_INT16 [optionally, in C++: a reference to a std::vector<AIL_INT16>] (and a maximum array rank for C# of 1) Required array size:(ArraySize)
- array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>] (and a maximum array rank for C# of 1) Required array size:(ArraySize)
- array of type AIL_INT8 [optionally, in C++: a reference to a std::vector<AIL_INT8>] (and a maximum array rank for C# of 1)

Required array size:(ArraySize)

- array of type AIL_UINT16 [optionally, in C++: a reference to a std::vector<AIL_UINT16>] (and a maximum array rank for C# of 1)
  Required array size:(ArraySize)
- array of type AIL_UINT32 [optionally, in C++: a reference to a std::vector<AIL_UINT32>] (and a maximum array rank for C# of 1)
  Required array size:(ArraySize)
- array of type AIL_UINT8 [optionally, in C++: a reference to a std::vector<AIL_UINT8>] (and a maximum array rank for C# of 1)
  Required array size:(ArraySize)

Specifies the address of the user array in which to store the retrieved data. If the component is 3-band and Options is set to M_PLANAR, the pointer passed to DstCoord3ArrayPtr must be valid. If the component is 1-band or Options is set to M_PACKED, set DstCoord3ArrayPtr to **M_NULL**.

When determining the required size of this array for subsequent calls to **M3dimGet**(), set this parameter to **M_NULL**.

Return value

The returned value is the number of entries written into the array(s). If DstCoord1OrPackedArrayPtr, DstCoord2ArrayPtr and DstCoord3ArrayPtr are set to **M_NULL**, **M3dimGet**() returns the required size of the array(s) for subsequent calls to the function.

Type-specific versions of the function when using a C compiler

```
AIL_INT M3dimGetDouble (AIL_ID ContainerOrImageBufId, AIL_INT64 Component, AIL_INT ArraySize, AIL_INT64 Options, AIL_DOUBLE
*DstCoord1OrPackedArrayPtr, AIL_DOUBLE *DstCoord2ArrayPtr, AIL_DOUBLE *DstCoord3ArrayPtr)
```

Details

Parameters

ContainerOrImageBufId

See ContainerOrImageBufId of the main function for a description.

Component

See Component of the main function for a description.

ArraySize

See ArraySize of the main function for a description.

Options

See Options of the main function for a description.

DstCoord1OrPackedArrayPtr

See DstCoord1OrPackedArrayPtr of the main function for a description.

DstCoord2ArrayPtr

See DstCoord2ArrayPtr of the main function for a description.

DstCoord3ArrayPtr

See DstCoord3ArrayPtr of the main function for a description.

```
AIL_INT M3dimGetFloat (AIL_ID ContainerOrImageBufId, AIL_INT64 Component, AIL_INT ArraySize, AIL_INT64 Options, AIL_FLOAT
*DstCoord1OrPackedArrayPtr, AIL_FLOAT *DstCoord2ArrayPtr, AIL_FLOAT *DstCoord3ArrayPtr)
```

Details

Parameters

ContainerOrImageBufId

See ContainerOrImageBufId of the main function for a description.

Component

See Component of the main function for a description.

ArraySize

See ArraySize of the main function for a description.

Options

See Options of the main function for a description.

DstCoord1OrPackedArrayPtr

See DstCoord1OrPackedArrayPtr of the main function for a description.

DstCoord2ArrayPtr

See DstCoord2ArrayPtr of the main function for a description.

DstCoord3ArrayPtr

See DstCoord3ArrayPtr of the main function for a description.

```
AIL_INT M3dimGetUint32 (AIL_ID ContainerOrImageBufId, AIL_INT64 Component, AIL_INT ArraySize, AIL_INT64 Options, AIL_UINT32
*DstCoord1OrPackedArrayPtr, AIL_UINT32 *DstCoord2ArrayPtr, AIL_UINT32 *DstCoord3ArrayPtr)
```

Details

Parameters

ContainerOrImageBufId

See ContainerOrImageBufId of the main function for a description.

Component

See Component of the main function for a description.

ArraySize

See ArraySize of the main function for a description.

Options

See Options of the main function for a description.

DstCoord1OrPackedArrayPtr

See DstCoord1OrPackedArrayPtr of the main function for a description.

DstCoord2ArrayPtr

See DstCoord2ArrayPtr of the main function for a description.

DstCoord3ArrayPtr

See DstCoord3ArrayPtr of the main function for a description.

```
AIL_INT M3dimGetUint16 (AIL_ID ContainerOrImageBufId, AIL_INT64 Component, AIL_INT ArraySize, AIL_INT64 Options, AIL_UINT16
*DstCoord1OrPackedArrayPtr, AIL_UINT16 *DstCoord2ArrayPtr, AIL_UINT16 *DstCoord3ArrayPtr)
```

Details

Parameters

ContainerOrImageBufId

See ContainerOrImageBufId of the main function for a description.

Component

See Component of the main function for a description.

ArraySize

See ArraySize of the main function for a description.

Options

See Options of the main function for a description.

DstCoord1OrPackedArrayPtr

See DstCoord1OrPackedArrayPtr of the main function for a description.

DstCoord2ArrayPtr

See DstCoord2ArrayPtr of the main function for a description.

DstCoord3ArrayPtr

See DstCoord3ArrayPtr of the main function for a description.

```
AIL_INT M3dimGetUint8 (AIL_ID ContainerOrImageBufId, AIL_INT64 Component, AIL_INT ArraySize, AIL_INT64 Options, AIL_UINT8
*DstCoord1OrPackedArrayPtr, AIL_UINT8 *DstCoord2ArrayPtr, AIL_UINT8 *DstCoord3ArrayPtr)
```

Details

Parameters

ContainerOrImageBufId

See ContainerOrImageBufId of the main function for a description.

Component

See Component of the main function for a description.

ArraySize

See ArraySize of the main function for a description.

Options

See Options of the main function for a description.

DstCoord1OrPackedArrayPtr

See DstCoord1OrPackedArrayPtr of the main function for a description.

DstCoord2ArrayPtr

See DstCoord2ArrayPtr of the main function for a description.

DstCoord3ArrayPtr

See DstCoord3ArrayPtr of the main function for a description.

```
AIL_INT M3dimGetInt32 (AIL_ID ContainerOrImageBufId, AIL_INT64 Component, AIL_INT ArraySize, AIL_INT64 Options, AIL_INT32
*DstCoord1OrPackedArrayPtr, AIL_INT32 *DstCoord2ArrayPtr, AIL_INT32 *DstCoord3ArrayPtr)
```

Details

Parameters

ContainerOrImageBufId

See ContainerOrImageBufId of the main function for a description.

Component

See Component of the main function for a description.

ArraySize

See ArraySize of the main function for a description.

Options

See Options of the main function for a description.

DstCoord1OrPackedArrayPtr

See DstCoord1OrPackedArrayPtr of the main function for a description.

DstCoord2ArrayPtr

See DstCoord2ArrayPtr of the main function for a description.

DstCoord3ArrayPtr

See DstCoord3ArrayPtr of the main function for a description.

```
AIL_INT M3dimGetInt16 (AIL_ID ContainerOrImageBufId, AIL_INT64 Component, AIL_INT ArraySize, AIL_INT64 Options, AIL_INT16
*DstCoord1OrPackedArrayPtr, AIL_INT16 *DstCoord2ArrayPtr, AIL_INT16 *DstCoord3ArrayPtr)
```

Details

Parameters

ContainerOrImageBufId

See ContainerOrImageBufId of the main function for a description.

Component

See Component of the main function for a description.

ArraySize

See ArraySize of the main function for a description.

Options

See Options of the main function for a description.

DstCoord1OrPackedArrayPtr

See DstCoord1OrPackedArrayPtr of the main function for a description.

DstCoord2ArrayPtr

See DstCoord2ArrayPtr of the main function for a description.

DstCoord3ArrayPtr

See DstCoord3ArrayPtr of the main function for a description.

```
AIL_INT M3dimGetInt8 (AIL_ID ContainerOrImageBufId, AIL_INT64 Component, AIL_INT ArraySize, AIL_INT64 Options, AIL_INT8
*DstCoord1OrPackedArrayPtr, AIL_INT8 *DstCoord2ArrayPtr, AIL_INT8 *DstCoord3ArrayPtr)
```

Details

Parameters

ContainerOrImageBufId

See ContainerOrImageBufId of the main function for a description.

Component

See Component of the main function for a description.

ArraySize

See ArraySize of the main function for a description.

Options

See Options of the main function for a description.

DstCoord1OrPackedArrayPtr

See DstCoord1OrPackedArrayPtr of the main function for a description.

DstCoord2ArrayPtr

See DstCoord2ArrayPtr of the main function for a description.

DstCoord3ArrayPtr

See DstCoord3ArrayPtr of the main function for a description.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dim11.lib.
DLL	Requires ail11.dll; ail3dim11.dll.

M3dimGetList

Synopsis

Read the values in a source container or depth map image, at the specified positions, and place the values in the specified destination arrays.

Syntax

```
AIL_INT M3dimGetList(
    AIL_ID ContainerOrImageBufId,          //in
    AIL_INT64 Component,                   //in
    AIL_INT SrcArraySize,                  //in
    const AIL_DOUBLE *SrcPixelXArrayPtr,   //in
    const AIL_DOUBLE *SrcPixelYArrayPtr,   //in
    AIL_INT64 Options,                     //in
    AIL_DOUBLE *DstCoord1OrPackedArrayPtr, //out
    AIL_DOUBLE *DstCoord2ArrayPtr,         //out
    AIL_DOUBLE *DstCoord3ArrayPtr,         //out
    AIL_INT32 *IndexArrayPtr               //out
)
```

Description

This function reads the values in a source container or depth map image buffer, at the specified positions, and places the values in the specified destination arrays. If the source is a container, you must specify from which component to copy the data. Each X/Y pair listed in the source arrays specifies the position of a point in its container's 2D grid organization (or pixel location in a depth map). Unlike MbufGetList(), this function has options for casting, storing the data in packed or planar format, and handling invalid points. It also returns the number of points written to the arrays.

Note that the source array(s) can specify positions with subpixel precision. In this case, the source buffer is typically an organized point cloud. To establish values for the destination array(s), you can specify to interpolate between neighboring values that surround each subpixel position in the container's specified component.

To determine the size of the arrays required to store the requested values, call this function and set DstCoord1OrPackedArrayPtr, DstCoord2ArrayPtr and DstCoord3ArrayPtr to **M_NULL**. The function will return the required array size(s).

> Note that presented above is an example prototype of this function. You can call M3dimGetList() with the same prototype as any of the type-specific functions below, without using the function's type-specific name; that is, use M3dimGetList() and not, for example, M3dimGetListDoubleInt16(). The data is automatically cast into the appropriate type. This is true except when using a C compiler. When using a C compiler, you must always call this function with its type-specific name (you cannot use the standard name of this function).

Parameters

ContainerOrImageBufId

Specifies the identifier of the source point cloud container, depth map container, or depth map image buffer.

The point cloud or depth map container must have been previously allocated using MbufAllocContainer() with M_PROC.

The point cloud container must be 3D-processable (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE, the function returns M_PROCESSABLE).

The depth map container must store data in a 3D-processable depth map format (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE_DEPTH_MAP, the function returns M_TRUE).

The image buffer must be a 1-band, 8-bit, 16-bit, or 32-bit unsigned buffer, and must contain a fully corrected depth map (that is, if you call McalInquire() with M_DEPTH_MAP, the function returns M_TRUE). If you specify an image buffer that has an ROI associated with it, the ROI must be in raster format; otherwise, you will get an error.

Component

Specifies the component from which to get point information. The specified component must be the same size as that of M_COMPONENT_RANGE, and must be an 8-bit, 16-bit or 32-bit image buffer. If the required component is not the same size, use MbufGetList() instead.

When ContainerOrImageBufId is a depth map image buffer, you must set Component to **M_DEFAULT**.

Use the following settings to specify a component using its identifier or index. An error is generated if there is no component in the container matching the specified identifier or index.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a component by its identifier or index																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_COMPONENT_BY_ID(AIL_INT ID)	Specifies the component, in the container, that has the specified buffer identifier.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	ID Specifies the Aurora Imaging Library identifier of the component.															
M_COMPONENT_BY_INDEX(AIL_INT ComponentIndex)	Specifies the component, in the container, at the specified index.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	ComponentIndex Specifies the index of the component.															

Use the following settings to specify a component by its component type, which identifies what kind of information the component stores.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a component by its component type																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_COMPONENT_CONFIDENCE	Specifies that the component stores confidence information for the M_COMPONENT_RANGE component of the container. Coordinates associated with the confidence value 0 are considered invalid data and will not be used by 3D image processing functions.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_COORDINATE_MAP_A_AIL	Specifies that the component stores the A coordinate map provided by the camera. When using MbufConvert3d() with a projective map, this component can be used instead of the column index of each corresponding pixel and will result in a better representation of the camera's lens model when generating X-coordinates. A coordinate map component is associated with a range component in the same container when there is only one range component and only one type of each coordinate map component in that container. > Note that this component is only available if it is provided by the camera. For more information refer to your cameras manual.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_COORDINATE_MAP_B_AIL	Specifies that the component stores the B coordinate map provided by the camera. When using MbufConvert3d() with a projective map, this component can be used instead of the row index of each corresponding pixel and will result in a better representation of the camera's lens model when generating Y-coordinates. A coordinate map component is associated with a range component in the same container when there is only one range component and only one type of each coordinate map component in that container. > Note that this component is only available if it is provided by the camera. For more information refer to your cameras manual.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_CUSTOM + *n*	Specifies that the component has a custom component type, identified by *n*, where n can be a value between 0 and 254. You can use custom component types to identify components which store information that does not fit one of the standard component types. In addition, some cameras transmit components with custom component types. Refer to your camera manual to determine what type of information is stored in transmitted components.	c		e	f	g	a		i		j	k	l	m	n	b

M_COMPONENT_DISPARITY	Specifies that the component stores a disparity map. Each pixel of a disparity map indicates the apparent distance (typically measured in pixel units) between where an object appears in the left and right images captured by a stereoscopic camera.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_INFRARED	Specifies that the component stores an intensity image of infrared light.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_INTENSITY	Specifies that the component stores an intensity image of visible light.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_METADATA	Specifies that the component stores metadata information. Metadata components are used by Aurora Imaging Library internally. Typically, you should ignore metadata components in your application.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_MULTISPECTRAL	Specifies that the component stores an intensity image where each band represents the intensity of a specific wavelength of light. Unlike an intensity component, a multispectral component might include information about non-visible light.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_NORMALS_AIL	Specifies that the component stores normals information for each point in the M_COMPONENT_RANGE component of the container.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_RANGE	Specifies that the component stores 3D distance/position information. This is a 3-band buffer that stores coordinates of 3D points.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_REFLECTANCE	Specifies that the component stores a reflectance map. Each pixel of a reflectance map indicates how much of the light hitting an object at that location is reflected back. Typically, this is an intensity image of the light spectrum used by a 3D sensor to detect 3D distance/position information. Typically, if the map was generated by a laser profiler, each row indicates the detected intensity of the laser for a single scan.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_REGION_AIL	Specifies that the component stores a region of interest (ROI) for the M_COMPONENT_RANGE component of the container. 3D image processing functions will only operate on points inside the ROI. Only depth map containers can have a region component. A region component is associated with a range component in the same container when there is only one range component and no other region components in that container.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_SCATTER	Specifies that the component stores a scatter map. Each pixel of a scatter map indicates how much of the light hitting an object at that location is detected scattering beneath the object's surface.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_ULTRAVIOLET	Specifies that the component stores an intensity image of ultraviolet light.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_UNDEFINED	Specifies that the component stores information of an unknown type.	c		e	f	g	a		i		j	k	l	m	n	b

SrcArraySize

Specifies the number of values in the source array(s).

> When using a standard vector (std::vector) overload function in C++, you can pass **M_DEFAULT** to this parameter; the size will be determined automatically based on the number of items in the vector passed to the SrcPixelXArrayPtr or SrcPixelYArrayPtr parameter.

SrcPixelXArrayPtr
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE>] (and a maximum array rank for C# of 1)
  Required array size:(SrcArraySize)
- array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>] (and a maximum array rank for C# of 1)
  Required array size:Call **M3dimGetList**() with DstCoord1OrPackedArrayPtr set to **M_NULL**, DstCoord2ArrayPtr set to **M_NULL**, DstCoord3ArrayPtr set to **M_NULL**, and IndexArrayPtr set to **M_NULL**. The array size is stored in the return value of the function.
- array of type AIL_INT16 [optionally, in C++: a reference to a std::vector<AIL_INT16>] (and a maximum array rank for C# of 1)

Required array size:Call **M3dimGetList**() with DstCoord1OrPackedArrayPtr set to **M_NULL**, DstCoord2ArrayPtr set to **M_NULL**, DstCoord3ArrayPtr set to **M_NULL**, and IndexArrayPtr set to **M_NULL**. The array size is stored in the return value of the function.

- array of type AIL_INT32 [optionally, in C++: a reference to a constant std::vector<AIL_INT32>] (and a maximum array rank for C# of 1)
  Required array size:(SrcArraySize)
- array of type AIL_INT8 [optionally, in C++: a reference to a std::vector<AIL_INT8>] (and a maximum array rank for C# of 1)
  Required array size:Call **M3dimGetList**() with DstCoord1OrPackedArrayPtr set to **M_NULL**, DstCoord2ArrayPtr set to **M_NULL**, DstCoord3ArrayPtr set to **M_NULL**, and IndexArrayPtr set to **M_NULL**. The array size is stored in the return value of the function.
- array of type AIL_UINT16 [optionally, in C++: a reference to a std::vector<AIL_UINT16>] (and a maximum array rank for C# of 1)
  Required array size:Call **M3dimGetList**() with DstCoord1OrPackedArrayPtr set to **M_NULL**, DstCoord2ArrayPtr set to **M_NULL**, DstCoord3ArrayPtr set to **M_NULL**, and IndexArrayPtr set to **M_NULL**. The array size is stored in the return value of the function.
- array of type AIL_UINT32 [optionally, in C++: a reference to a std::vector<AIL_UINT32>] (and a maximum array rank for C# of 1)
  Required array size:Call **M3dimGetList**() with DstCoord1OrPackedArrayPtr set to **M_NULL**, DstCoord2ArrayPtr set to **M_NULL**, DstCoord3ArrayPtr set to **M_NULL**, and IndexArrayPtr set to **M_NULL**. The array size is stored in the return value of the function.
- array of type AIL_UINT8 [optionally, in C++: a reference to a std::vector<AIL_UINT8>] (and a maximum array rank for C# of 1)
  Required array size:Call **M3dimGetList**() with DstCoord1OrPackedArrayPtr set to **M_NULL**, DstCoord2ArrayPtr set to **M_NULL**, DstCoord3ArrayPtr set to **M_NULL**, and IndexArrayPtr set to **M_NULL**. The array size is stored in the return value of the function.

Specifies the address of an array containing the X-positions of values to get. Each position is that of a point in its container's 2D grid organization (or pixel location in a depth map). Positions can be specified with subpixel precision.

SrcPixelYArrayPtr
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE>] (and a maximum array rank for C# of 1)
  Required array size:(SrcArraySize)
- array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>] (and a maximum array rank for C# of 1)
  Required array size:Call **M3dimGetList**() with DstCoord1OrPackedArrayPtr set to **M_NULL**, DstCoord2ArrayPtr set to **M_NULL**, DstCoord3ArrayPtr set to **M_NULL**, and IndexArrayPtr set to **M_NULL**. The array size is stored in the return value of the function.
- array of type AIL_INT16 [optionally, in C++: a reference to a std::vector<AIL_INT16>] (and a maximum array rank for C# of 1)
  Required array size:Call **M3dimGetList**() with DstCoord1OrPackedArrayPtr set to **M_NULL**, DstCoord2ArrayPtr set to **M_NULL**, DstCoord3ArrayPtr set to **M_NULL**, and IndexArrayPtr set to **M_NULL**. The array size is stored in the return value of the function.
- array of type AIL_INT32 [optionally, in C++: a reference to a constant std::vector<AIL_INT32>] (and a maximum array rank for C# of 1)
  Required array size:(SrcArraySize)
- array of type AIL_INT8 [optionally, in C++: a reference to a std::vector<AIL_INT8>] (and a maximum array rank for C# of 1)
  Required array size:Call **M3dimGetList**() with DstCoord1OrPackedArrayPtr set to **M_NULL**, DstCoord2ArrayPtr set to **M_NULL**, DstCoord3ArrayPtr set to **M_NULL**, and IndexArrayPtr set to **M_NULL**. The array size is stored in the return value of the function.
- array of type AIL_UINT16 [optionally, in C++: a reference to a std::vector<AIL_UINT16>] (and a maximum array rank for C# of 1)
  Required array size:Call **M3dimGetList**() with DstCoord1OrPackedArrayPtr set to **M_NULL**, DstCoord2ArrayPtr set to **M_NULL**, DstCoord3ArrayPtr set to **M_NULL**, and IndexArrayPtr set to **M_NULL**. The array size is stored in the return value of the function.
- array of type AIL_UINT32 [optionally, in C++: a reference to a std::vector<AIL_UINT32>] (and a maximum array rank for C# of 1)
  Required array size:Call **M3dimGetList**() with DstCoord1OrPackedArrayPtr set to **M_NULL**, DstCoord2ArrayPtr set to **M_NULL**, DstCoord3ArrayPtr set to **M_NULL**, and IndexArrayPtr set to **M_NULL**. The array size is stored in the return value of the function.
- array of type AIL_UINT8 [optionally, in C++: a reference to a std::vector<AIL_UINT8>] (and a maximum array rank for C# of 1)
  Required array size:Call **M3dimGetList**() with DstCoord1OrPackedArrayPtr set to **M_NULL**, DstCoord2ArrayPtr set to **M_NULL**, DstCoord3ArrayPtr set to **M_NULL**, and IndexArrayPtr set to **M_NULL**. The array size is stored in the return value of the function.

Specifies the address of an array containing the Y-positions of values to get. Each position is that of a point in its container's 2D grid organization (or pixel location in a depth map). Positions can be specified with subpixel precision.

If the source container is unorganized, you can set this parameter to **M_NULL**.

Options

Specifies additional options.

> Note that you can combine settings (for example, M_PACKED + M_BILINEAR + M_INCLUDE_INVALID_POINTS). Any option not specified uses the default value. For instance, setting this parameter to M_PACKED also sets M_NEAREST_NEIGHBOR and M_SKIP_INVALID_POINTS, even though M_PACKED is the only explicitly-set value.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying default options																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_PLANAR + M_NEAREST_NEIGHBOR + M_SKIP_INVALID_POINTS.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values that can be used alone or as combination values for any of the possible values of the Options parameter.

You can use one of the following values on its own, or add it to the above-mentioned value, to specify the storage format in the destination array(s).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the storage format																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_PACKED	Specifies to store the data in a packed format.	c		e	f	g	a		i		j	k	l	m	n	b
M_PLANAR	Specifies to store the data in a planar format. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values that can be used alone or as combination values for any of the possible values of the Options parameter.

You can use one of the following values on its own, or add it to the above-mentioned value, to specify how to interpret subpixel positions in the source array(s).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying how to interpret subpixel positions in the source array(s)																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_BILINEAR	Specifies bilinear interpolation. The value placed in the user-defined array is determined by taking a weighted average of the 4 values (2x2) that surround the subpixel position. > If any of the 4 surrounding values is invalid, M_NEAREST_NEIGHBOR is used instead.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEAREST_NEIGHBOR	Specifies nearest neighbor interpolation. The value placed in the user-defined array is that of the value of the pixel closest to the subpixel position. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values that can be used alone or as combination values for any of the possible values of the Options parameter.

You can use one of the following values on its own, or add it to the above-mentioned value, to specify how to deal with invalid points.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying how to deal with invalid points																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_INCLUDE_INVALID_POINTS	Specifies to include invalid points. If a point is invalid, the function writes an unspecified value in the data output array(s) and -1 in the source-index array (IndexArrayPtr). For example, if the input confidence = [1, 0, 1, 0, 1], and the input component = [3, 1, 4, 1, 5], then the data output array = [3, ?, 4, ?, 5], with indices = [0, -1, 2, -1, 4].	c		e	f	g	a		i		j	k	l	m	n	b
M_SKIP_INVALID_POINTS	Specifies to skip invalid points so that the output arrays contain only valid points. For example, if the input confidence = [1, 0, 1, 0, 1], and the input component = [3, 1, 4, 1, 5], then the data output array = [3, 4, 5], with indices = [0, 2, 4]. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b

DstCoord1OrPackedArrayPtr
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE>] (and a maximum array rank for C# of 1)
  Required array size:(SrcArraySize)
- array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>] (and a maximum array rank for C# of 1)
  Required array size:Call **M3dimGetList**() with DstCoord1OrPackedArrayPtr set to **M_NULL**, DstCoord2ArrayPtr set to **M_NULL**, DstCoord3ArrayPtr set to **M_NULL**, and IndexArrayPtr set to **M_NULL**. The array size is stored in the return value of the function.
- array of type AIL_INT16 [optionally, in C++: a reference to a std::vector<AIL_INT16>] (and a maximum array rank for C# of 1)
  Required array size:Call **M3dimGetList**() with DstCoord1OrPackedArrayPtr set to **M_NULL**, DstCoord2ArrayPtr set to **M_NULL**, DstCoord3ArrayPtr set to **M_NULL**, and IndexArrayPtr set to **M_NULL**. The array size is stored in the return value of the function.
- array of type AIL_INT32 [optionally, in C++: a reference to a constant std::vector<AIL_INT32>] (and a maximum array rank for C# of 1)

Required array size:(SrcArraySize)
- array of type AIL_INT8 [optionally, in C++: a reference to a std::vector<AIL_INT8>] (and a maximum array rank for C# of 1)
  Required array size:Call **M3dimGetList**() with DstCoord1OrPackedArrayPtr set to **M_NULL**, DstCoord2ArrayPtr set to **M_NULL**, DstCoord3ArrayPtr set to **M_NULL**, and IndexArrayPtr set to **M_NULL**. The array size is stored in the return value of the function.
- array of type AIL_UINT16 [optionally, in C++: a reference to a std::vector<AIL_UINT16>] (and a maximum array rank for C# of 1)
  Required array size:Call **M3dimGetList**() with DstCoord1OrPackedArrayPtr set to **M_NULL**, DstCoord2ArrayPtr set to **M_NULL**, DstCoord3ArrayPtr set to **M_NULL**, and IndexArrayPtr set to **M_NULL**. The array size is stored in the return value of the function.
- array of type AIL_UINT32 [optionally, in C++: a reference to a std::vector<AIL_UINT32>] (and a maximum array rank for C# of 1)
  Required array size:Call **M3dimGetList**() with DstCoord1OrPackedArrayPtr set to **M_NULL**, DstCoord2ArrayPtr set to **M_NULL**, DstCoord3ArrayPtr set to **M_NULL**, and IndexArrayPtr set to **M_NULL**. The array size is stored in the return value of the function.
- array of type AIL_UINT8 [optionally, in C++: a reference to a std::vector<AIL_UINT8>] (and a maximum array rank for C# of 1)
  Required array size:Call **M3dimGetList**() with DstCoord1OrPackedArrayPtr set to **M_NULL**, DstCoord2ArrayPtr set to **M_NULL**, DstCoord3ArrayPtr set to **M_NULL**, and IndexArrayPtr set to **M_NULL**. The array size is stored in the return value of the function.

Specifies the address of a user-defined array in which to store the retrieved data. This array can hold the values of the specified point cloud or depth map's component, or a packed array of all the data. For instance, the range component's 3 bands of coordinate data, if packed, is stored in the array in an interleaved manner (for example, XYZ XYZ XYZ...).

> Note that you can specify an array of the required type; Aurora Imaging Library automatically casts the data into the appropriate type.

When determining the required size of this array for subsequent calls to **M3dimGetList**(), set this parameter to **M_NULL**.

DstCoord2ArrayPtr
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE>] (and a maximum array rank for C# of 1)
  Required array size:(SrcArraySize)
- array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>] (and a maximum array rank for C# of 1)
  Required array size:Call **M3dimGetList**() with DstCoord1OrPackedArrayPtr set to **M_NULL**, DstCoord2ArrayPtr set to **M_NULL**, DstCoord3ArrayPtr set to **M_NULL**, and IndexArrayPtr set to **M_NULL**. The array size is stored in the return value of the function.
- array of type AIL_INT16 [optionally, in C++: a reference to a std::vector<AIL_INT16>] (and a maximum array rank for C# of 1)
  Required array size:Call **M3dimGetList**() with DstCoord1OrPackedArrayPtr set to **M_NULL**, DstCoord2ArrayPtr set to **M_NULL**, DstCoord3ArrayPtr set to **M_NULL**, and IndexArrayPtr set to **M_NULL**. The array size is stored in the return value of the function.
- array of type AIL_INT32 [optionally, in C++: a reference to a constant std::vector<AIL_INT32>] (and a maximum array rank for C# of 1)
  Required array size:(SrcArraySize)
- array of type AIL_INT8 [optionally, in C++: a reference to a std::vector<AIL_INT8>] (and a maximum array rank for C# of 1)
  Required array size:Call **M3dimGetList**() with DstCoord1OrPackedArrayPtr set to **M_NULL**, DstCoord2ArrayPtr set to **M_NULL**, DstCoord3ArrayPtr set to **M_NULL**, and IndexArrayPtr set to **M_NULL**. The array size is stored in the return value of the function.
- array of type AIL_UINT16 [optionally, in C++: a reference to a std::vector<AIL_UINT16>] (and a maximum array rank for C# of 1)
  Required array size:Call **M3dimGetList**() with DstCoord1OrPackedArrayPtr set to **M_NULL**, DstCoord2ArrayPtr set to **M_NULL**, DstCoord3ArrayPtr set to **M_NULL**, and IndexArrayPtr set to **M_NULL**. The array size is stored in the return value of the function.
- array of type AIL_UINT32 [optionally, in C++: a reference to a std::vector<AIL_UINT32>] (and a maximum array rank for C# of 1)
  Required array size:Call **M3dimGetList**() with DstCoord1OrPackedArrayPtr set to **M_NULL**, DstCoord2ArrayPtr set to **M_NULL**, DstCoord3ArrayPtr set to **M_NULL**, and IndexArrayPtr set to **M_NULL**. The array size is stored in the return value of the function.
- array of type AIL_UINT8 [optionally, in C++: a reference to a std::vector<AIL_UINT8>] (and a maximum array rank for C# of 1)
  Required array size:Call **M3dimGetList**() with DstCoord1OrPackedArrayPtr set to **M_NULL**, DstCoord2ArrayPtr set to **M_NULL**, DstCoord3ArrayPtr set to **M_NULL**, and IndexArrayPtr set to **M_NULL**. The array size is stored in the return value of the function.

Specifies the address of a user-defined array in which to store the retrieved data. If the component is 3-band and Options is set to M_PLANAR, the pointer passed to DstCoord2ArrayPtr must be valid. If the component is 1-band or Options is set to M_PACKED, set DstCoord2ArrayPtr to **M_NULL**.

> Note that you can specify an array of the required type; Aurora Imaging Library automatically casts the data into the appropriate type.

When determining the required size of this array for subsequent calls to **M3dimGetList**(), set this parameter to **M_NULL**.

DstCoord3ArrayPtr
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE>] (and a maximum array rank for C# of 1)
  Required array size:(SrcArraySize)
- array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>] (and a maximum array rank for C# of 1)
  Required array size:Call **M3dimGetList**() with DstCoord1OrPackedArrayPtr set to **M_NULL**, DstCoord2ArrayPtr set to **M_NULL**, DstCoord3ArrayPtr set to **M_NULL**, and IndexArrayPtr set to **M_NULL**. The array size is stored in the return value of the function.
- array of type AIL_INT16 [optionally, in C++: a reference to a std::vector<AIL_INT16>] (and a maximum array rank for C# of 1)
  Required array size:Call **M3dimGetList**() with DstCoord1OrPackedArrayPtr set to **M_NULL**, DstCoord2ArrayPtr set to **M_NULL**, DstCoord3ArrayPtr set to **M_NULL**, and IndexArrayPtr set to **M_NULL**. The array size is stored in the return value of the function.

- array of type AIL_INT32 [optionally, in C++: a reference to a constant std::vector<AIL_INT32>] (and a maximum array rank for C# of 1)
  Required array size:(SrcArraySize)
- array of type AIL_INT8 [optionally, in C++: a reference to a std::vector<AIL_INT8>] (and a maximum array rank for C# of 1)
  Required array size:Call **M3dimGetList**() with DstCoord1OrPackedArrayPtr set to **M_NULL**, DstCoord2ArrayPtr set to **M_NULL**, DstCoord3ArrayPtr set to **M_NULL**, and IndexArrayPtr set to **M_NULL**. The array size is stored in the return value of the function.
- array of type AIL_UINT16 [optionally, in C++: a reference to a std::vector<AIL_UINT16>] (and a maximum array rank for C# of 1)
  Required array size:Call **M3dimGetList**() with DstCoord1OrPackedArrayPtr set to **M_NULL**, DstCoord2ArrayPtr set to **M_NULL**, DstCoord3ArrayPtr set to **M_NULL**, and IndexArrayPtr set to **M_NULL**. The array size is stored in the return value of the function.
- array of type AIL_UINT32 [optionally, in C++: a reference to a std::vector<AIL_UINT32>] (and a maximum array rank for C# of 1)
  Required array size:Call **M3dimGetList**() with DstCoord1OrPackedArrayPtr set to **M_NULL**, DstCoord2ArrayPtr set to **M_NULL**, DstCoord3ArrayPtr set to **M_NULL**, and IndexArrayPtr set to **M_NULL**. The array size is stored in the return value of the function.
- array of type AIL_UINT8 [optionally, in C++: a reference to a std::vector<AIL_UINT8>] (and a maximum array rank for C# of 1)
  Required array size:Call **M3dimGetList**() with DstCoord1OrPackedArrayPtr set to **M_NULL**, DstCoord2ArrayPtr set to **M_NULL**, DstCoord3ArrayPtr set to **M_NULL**, and IndexArrayPtr set to **M_NULL**. The array size is stored in the return value of the function.

Specifies the address of a user-defined array in which to store the retrieved data. If the component is 3-band and Options is set to M_PLANAR, the pointer passed to DstCoord3ArrayPtr must be valid. If the component is 1-band or Options is set to M_PACKED, set DstCoord3ArrayPtr to **M_NULL**.

> Note that you can specify an array of the required type; Aurora Imaging Library automatically casts the data into the appropriate type.

When determining the required size of this array for subsequent calls to **M3dimGetList**(), set this parameter to **M_NULL**.

IndexArrayPtr
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE>] (and a maximum array rank for C# of 1)
  Required array size:(SrcArraySize)
- array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>] (and a maximum array rank for C# of 1)
  Required array size:Call **M3dimGetList**() with DstCoord1OrPackedArrayPtr set to **M_NULL**, DstCoord2ArrayPtr set to **M_NULL**, DstCoord3ArrayPtr set to **M_NULL**, and IndexArrayPtr set to **M_NULL**. The array size is stored in the return value of the function.
- array of type AIL_INT16 [optionally, in C++: a reference to a std::vector<AIL_INT16>] (and a maximum array rank for C# of 1)
  Required array size:Call **M3dimGetList**() with DstCoord1OrPackedArrayPtr set to **M_NULL**, DstCoord2ArrayPtr set to **M_NULL**, DstCoord3ArrayPtr set to **M_NULL**, and IndexArrayPtr set to **M_NULL**. The array size is stored in the return value of the function.
- array of type AIL_INT32 [optionally, in C++: a reference to a constant std::vector<AIL_INT32>] (and a maximum array rank for C# of 1)
  Required array size:(SrcArraySize)
- array of type AIL_INT8 [optionally, in C++: a reference to a std::vector<AIL_INT8>] (and a maximum array rank for C# of 1)
  Required array size:Call **M3dimGetList**() with DstCoord1OrPackedArrayPtr set to **M_NULL**, DstCoord2ArrayPtr set to **M_NULL**, DstCoord3ArrayPtr set to **M_NULL**, and IndexArrayPtr set to **M_NULL**. The array size is stored in the return value of the function.
- array of type AIL_UINT16 [optionally, in C++: a reference to a std::vector<AIL_UINT16>] (and a maximum array rank for C# of 1)
  Required array size:Call **M3dimGetList**() with DstCoord1OrPackedArrayPtr set to **M_NULL**, DstCoord2ArrayPtr set to **M_NULL**, DstCoord3ArrayPtr set to **M_NULL**, and IndexArrayPtr set to **M_NULL**. The array size is stored in the return value of the function.
- array of type AIL_UINT32 [optionally, in C++: a reference to a std::vector<AIL_UINT32>] (and a maximum array rank for C# of 1)
  Required array size:Call **M3dimGetList**() with DstCoord1OrPackedArrayPtr set to **M_NULL**, DstCoord2ArrayPtr set to **M_NULL**, DstCoord3ArrayPtr set to **M_NULL**, and IndexArrayPtr set to **M_NULL**. The array size is stored in the return value of the function.
- array of type AIL_UINT8 [optionally, in C++: a reference to a std::vector<AIL_UINT8>] (and a maximum array rank for C# of 1)
  Required array size:Call **M3dimGetList**() with DstCoord1OrPackedArrayPtr set to **M_NULL**, DstCoord2ArrayPtr set to **M_NULL**, DstCoord3ArrayPtr set to **M_NULL**, and IndexArrayPtr set to **M_NULL**. The array size is stored in the return value of the function.

Specifies the address of a user-defined array in which to store either the index of the element in the input array(s), or -1 for elements that correspond to invalid points.

If not required, set this parameter to **M_NULL**.

Return value

The returned value is the number of entries written into the destination array(s). If DstCoord1OrPackedArrayPtr, DstCoord2ArrayPtr and DstCoord3ArrayPtr are set to **M_NULL**, **M3dimGetList**() returns the required size of the array(s) for subsequent calls to the function.

Type-specific versions of the function when using a C compiler

```
AIL_INT M3dimGetListDoubleInt8 (AIL_ID ContainerOrImageBufId, AIL_INT64 Component, AIL_INT SrcArraySize, const AIL_DOUBLE
*SrcPixelXArrayPtr, const AIL_DOUBLE *SrcPixelYArrayPtr, AIL_INT64 Options, AIL_INT8 *DstCoord1OrPackedArrayPtr, AIL_INT8
*DstCoord2ArrayPtr, AIL_INT8 *DstCoord3ArrayPtr, AIL_INT32 *IndexArrayPtr)
```

Details

Parameters

ContainerOrImageBufId

See ContainerOrImageBufId of the main function for a description.

Component

See Component of the main function for a description.

SrcArraySize

See SrcArraySize of the main function for a description.

SrcPixelXArrayPtr

See SrcPixelXArrayPtr of the main function for a description.

SrcPixelYArrayPtr

See SrcPixelYArrayPtr of the main function for a description.

Options

See Options of the main function for a description.

DstCoord1OrPackedArrayPtr

See DstCoord1OrPackedArrayPtr of the main function for a description.

DstCoord2ArrayPtr

See DstCoord2ArrayPtr of the main function for a description.

DstCoord3ArrayPtr

See DstCoord3ArrayPtr of the main function for a description.

IndexArrayPtr

See IndexArrayPtr of the main function for a description.

```
AIL_INT M3dimGetListDoubleInt16 (AIL_ID ContainerOrImageBufId, AIL_INT64 Component, AIL_INT SrcArraySize, const AIL_DOUBLE
*SrcPixelXArrayPtr, const AIL_DOUBLE *SrcPixelYArrayPtr, AIL_INT64 Options, AIL_INT16 *DstCoord1OrPackedArrayPtr, AIL_INT16
*DstCoord2ArrayPtr, AIL_INT16 *DstCoord3ArrayPtr, AIL_INT32 *IndexArrayPtr)
```

Details

Parameters

ContainerOrImageBufId

See ContainerOrImageBufId of the main function for a description.

Component

See Component of the main function for a description.

SrcArraySize

See SrcArraySize of the main function for a description.

SrcPixelXArrayPtr

See SrcPixelXArrayPtr of the main function for a description.

SrcPixelYArrayPtr

See SrcPixelYArrayPtr of the main function for a description.

Options

See Options of the main function for a description.

DstCoord1OrPackedArrayPtr

See DstCoord1OrPackedArrayPtr of the main function for a description.

DstCoord2ArrayPtr

See DstCoord2ArrayPtr of the main function for a description.

DstCoord3ArrayPtr

See DstCoord3ArrayPtr of the main function for a description.

IndexArrayPtr

See IndexArrayPtr of the main function for a description.

```
AIL_INT M3dimGetListDoubleInt32 (AIL_ID ContainerOrImageBufId, AIL_INT64 Component, AIL_INT SrcArraySize, const AIL_DOUBLE
*SrcPixelXArrayPtr, const AIL_DOUBLE *SrcPixelYArrayPtr, AIL_INT64 Options, AIL_INT32 *DstCoord1OrPackedArrayPtr, AIL_INT32
*DstCoord2ArrayPtr, AIL_INT32 *DstCoord3ArrayPtr, AIL_INT32 *IndexArrayPtr)
```

Details

Parameters

ContainerOrImageBufId

See ContainerOrImageBufId of the main function for a description.

Component

See Component of the main function for a description.

SrcArraySize

See SrcArraySize of the main function for a description.

SrcPixelXArrayPtr

See SrcPixelXArrayPtr of the main function for a description.

SrcPixelYArrayPtr

See SrcPixelYArrayPtr of the main function for a description.

Options

See Options of the main function for a description.

DstCoord1OrPackedArrayPtr

See DstCoord1OrPackedArrayPtr of the main function for a description.

DstCoord2ArrayPtr

See DstCoord2ArrayPtr of the main function for a description.

DstCoord3ArrayPtr

See DstCoord3ArrayPtr of the main function for a description.

IndexArrayPtr

See IndexArrayPtr of the main function for a description.

```
AIL_INT M3dimGetListDoubleUint8 (AIL_ID ContainerOrImageBufId, AIL_INT64 Component, AIL_INT SrcArraySize, const AIL_DOUBLE
*SrcPixelXArrayPtr, const AIL_DOUBLE *SrcPixelYArrayPtr, AIL_INT64 Options, AIL_UINT8 *DstCoord1OrPackedArrayPtr, AIL_UINT8
*DstCoord2ArrayPtr, AIL_UINT8 *DstCoord3ArrayPtr, AIL_INT32 *IndexArrayPtr)
```

Details

Parameters

ContainerOrImageBufId

See ContainerOrImageBufId of the main function for a description.

Component

See Component of the main function for a description.

SrcArraySize

See SrcArraySize of the main function for a description.

SrcPixelXArrayPtr

See SrcPixelXArrayPtr of the main function for a description.

SrcPixelYArrayPtr

See SrcPixelYArrayPtr of the main function for a description.

Options

See Options of the main function for a description.

DstCoord1OrPackedArrayPtr

See DstCoord1OrPackedArrayPtr of the main function for a description.

DstCoord2ArrayPtr

See DstCoord2ArrayPtr of the main function for a description.

DstCoord3ArrayPtr

See DstCoord3ArrayPtr of the main function for a description.

IndexArrayPtr

See IndexArrayPtr of the main function for a description.

```
AIL_INT M3dimGetListDoubleUint16 (AIL_ID ContainerOrImageBufId, AIL_INT64 Component, AIL_INT SrcArraySize, const AIL_DOUBLE
*SrcPixelXArrayPtr, const AIL_DOUBLE *SrcPixelYArrayPtr, AIL_INT64 Options, AIL_UINT16 *DstCoord1OrPackedArrayPtr, AIL_UINT16
*DstCoord2ArrayPtr, AIL_UINT16 *DstCoord3ArrayPtr, AIL_INT32 *IndexArrayPtr)
```

Details

Parameters

ContainerOrImageBufId

See ContainerOrImageBufId of the main function for a description.

Component

See Component of the main function for a description.

SrcArraySize

See SrcArraySize of the main function for a description.

SrcPixelXArrayPtr

See SrcPixelXArrayPtr of the main function for a description.

SrcPixelYArrayPtr

See SrcPixelYArrayPtr of the main function for a description.

Options

See Options of the main function for a description.

DstCoord1OrPackedArrayPtr

See DstCoord1OrPackedArrayPtr of the main function for a description.

DstCoord2ArrayPtr

See DstCoord2ArrayPtr of the main function for a description.

DstCoord3ArrayPtr

See DstCoord3ArrayPtr of the main function for a description.

IndexArrayPtr

See IndexArrayPtr of the main function for a description.

```
AIL_INT M3dimGetListDoubleUint32 (AIL_ID ContainerOrImageBufId, AIL_INT64 Component, AIL_INT SrcArraySize, const AIL_DOUBLE
*SrcPixelXArrayPtr, const AIL_DOUBLE *SrcPixelYArrayPtr, AIL_INT64 Options, AIL_UINT32 *DstCoord1OrPackedArrayPtr, AIL_UINT32
*DstCoord2ArrayPtr, AIL_UINT32 *DstCoord3ArrayPtr, AIL_INT32 *IndexArrayPtr)
```

Details

Parameters

ContainerOrImageBufId

See ContainerOrImageBufId of the main function for a description.

Component

See Component of the main function for a description.

SrcArraySize

See SrcArraySize of the main function for a description.

SrcPixelXArrayPtr

See SrcPixelXArrayPtr of the main function for a description.

SrcPixelYArrayPtr

See SrcPixelYArrayPtr of the main function for a description.

Options

See Options of the main function for a description.

DstCoord1OrPackedArrayPtr

See DstCoord1OrPackedArrayPtr of the main function for a description.

DstCoord2ArrayPtr

See DstCoord2ArrayPtr of the main function for a description.

DstCoord3ArrayPtr

See DstCoord3ArrayPtr of the main function for a description.

IndexArrayPtr

See IndexArrayPtr of the main function for a description.

```
AIL_INT M3dimGetListDoubleFloat (AIL_ID ContainerOrImageBufId, AIL_INT64 Component, AIL_INT SrcArraySize, const AIL_DOUBLE
*SrcPixelXArrayPtr, const AIL_DOUBLE *SrcPixelYArrayPtr, AIL_INT64 Options, AIL_FLOAT *DstCoord1OrPackedArrayPtr, AIL_FLOAT
*DstCoord2ArrayPtr, AIL_FLOAT *DstCoord3ArrayPtr, AIL_INT32 *IndexArrayPtr)
```

Details

Parameters

ContainerOrImageBufId

See ContainerOrImageBufId of the main function for a description.

Component

See Component of the main function for a description.

SrcArraySize

See SrcArraySize of the main function for a description.

SrcPixelXArrayPtr

See SrcPixelXArrayPtr of the main function for a description.

SrcPixelYArrayPtr

See SrcPixelYArrayPtr of the main function for a description.

Options

See Options of the main function for a description.

DstCoord1OrPackedArrayPtr

See DstCoord1OrPackedArrayPtr of the main function for a description.

DstCoord2ArrayPtr

See DstCoord2ArrayPtr of the main function for a description.

DstCoord3ArrayPtr

See DstCoord3ArrayPtr of the main function for a description.

IndexArrayPtr

See IndexArrayPtr of the main function for a description.

```
AIL_INT M3dimGetListDoubleDouble (AIL_ID ContainerOrImageBufId, AIL_INT64 Component, AIL_INT SrcArraySize, const AIL_DOUBLE
*SrcPixelXArrayPtr, const AIL_DOUBLE *SrcPixelYArrayPtr, AIL_INT64 Options, AIL_DOUBLE *DstCoord1OrPackedArrayPtr, AIL_DOUBLE
*DstCoord2ArrayPtr, AIL_DOUBLE *DstCoord3ArrayPtr, AIL_INT32 *IndexArrayPtr)
```

Details

Parameters

ContainerOrImageBufId

See ContainerOrImageBufId of the main function for a description.

Component

See Component of the main function for a description.

SrcArraySize

See SrcArraySize of the main function for a description.

SrcPixelXArrayPtr

See SrcPixelXArrayPtr of the main function for a description.

SrcPixelYArrayPtr

See SrcPixelYArrayPtr of the main function for a description.

Options

See Options of the main function for a description.

DstCoord1OrPackedArrayPtr

See DstCoord1OrPackedArrayPtr of the main function for a description.

DstCoord2ArrayPtr

See DstCoord2ArrayPtr of the main function for a description.

DstCoord3ArrayPtr

See DstCoord3ArrayPtr of the main function for a description.

IndexArrayPtr

See IndexArrayPtr of the main function for a description.

```
AIL_INT M3dimGetListInt32Int8 (AIL_ID ContainerOrImageBufId, AIL_INT64 Component, AIL_INT SrcArraySize, const AIL_INT32 *SrcPixelXArrayPtr, const AIL_INT32 *SrcPixelYArrayPtr, AIL_INT64 Options, AIL_INT8 *DstCoord1OrPackedArrayPtr, AIL_INT8 *DstCoord2ArrayPtr, AIL_INT8 *DstCoord3ArrayPtr, AIL_INT32 *IndexArrayPtr)
```

Details

Parameters

ContainerOrImageBufId

See ContainerOrImageBufId of the main function for a description.

Component

See Component of the main function for a description.

SrcArraySize

See SrcArraySize of the main function for a description.

SrcPixelXArrayPtr

See SrcPixelXArrayPtr of the main function for a description.

SrcPixelYArrayPtr

See SrcPixelYArrayPtr of the main function for a description.

Options

See Options of the main function for a description.

DstCoord1OrPackedArrayPtr

See DstCoord1OrPackedArrayPtr of the main function for a description.

DstCoord2ArrayPtr

See DstCoord2ArrayPtr of the main function for a description.

DstCoord3ArrayPtr

See DstCoord3ArrayPtr of the main function for a description.

IndexArrayPtr

See IndexArrayPtr of the main function for a description.

```
AIL_INT M3dimGetListInt32Int16 (AIL_ID ContainerOrImageBufId, AIL_INT64 Component, AIL_INT SrcArraySize, const AIL_INT32 *SrcPixelXArrayPtr, const AIL_INT32 *SrcPixelYArrayPtr, AIL_INT64 Options, AIL_INT16 *DstCoord1OrPackedArrayPtr, AIL_INT16 *DstCoord2ArrayPtr, AIL_INT16 *DstCoord3ArrayPtr, AIL_INT32 *IndexArrayPtr)
```

Details

Parameters

ContainerOrImageBufId

See ContainerOrImageBufId of the main function for a description.

Component

See Component of the main function for a description.

SrcArraySize

See SrcArraySize of the main function for a description.

SrcPixelXArrayPtr

See SrcPixelXArrayPtr of the main function for a description.

SrcPixelYArrayPtr

See SrcPixelYArrayPtr of the main function for a description.

Options

See Options of the main function for a description.

DstCoord1OrPackedArrayPtr

See DstCoord1OrPackedArrayPtr of the main function for a description.

DstCoord2ArrayPtr

See DstCoord2ArrayPtr of the main function for a description.

DstCoord3ArrayPtr

See DstCoord3ArrayPtr of the main function for a description.

IndexArrayPtr

See IndexArrayPtr of the main function for a description.

```
AIL_INT M3dimGetListInt32Int32 (AIL_ID ContainerOrImageBufId, AIL_INT64 Component, AIL_INT SrcArraySize, const AIL_INT32 *SrcPixelXArrayPtr, const AIL_INT32 *SrcPixelYArrayPtr, AIL_INT64 Options, AIL_INT32 *DstCoord1OrPackedArrayPtr, AIL_INT32 *DstCoord2ArrayPtr, AIL_INT32 *DstCoord3ArrayPtr, AIL_INT32 *IndexArrayPtr)
```

Details

Parameters

ContainerOrImageBufId

See ContainerOrImageBufId of the main function for a description.

Component

See Component of the main function for a description.

SrcArraySize

See SrcArraySize of the main function for a description.

SrcPixelXArrayPtr

See SrcPixelXArrayPtr of the main function for a description.

SrcPixelYArrayPtr

See SrcPixelYArrayPtr of the main function for a description.

Options

See Options of the main function for a description.

DstCoord1OrPackedArrayPtr

See DstCoord1OrPackedArrayPtr of the main function for a description.

DstCoord2ArrayPtr

See DstCoord2ArrayPtr of the main function for a description.

DstCoord3ArrayPtr

See DstCoord3ArrayPtr of the main function for a description.

IndexArrayPtr

See IndexArrayPtr of the main function for a description.

```
AIL_INT M3dimGetListInt32Uint8 (AIL_ID ContainerOrImageBufId, AIL_INT64 Component, AIL_INT SrcArraySize, const AIL_INT32 *SrcPixelXArrayPtr, const AIL_INT32 *SrcPixelYArrayPtr, AIL_INT64 Options, AIL_UINT8 *DstCoord1OrPackedArrayPtr, AIL_UINT8 *DstCoord2ArrayPtr, AIL_UINT8 *DstCoord3ArrayPtr, AIL_INT32 *IndexArrayPtr)
```

Details

Parameters

ContainerOrImageBufId

See ContainerOrImageBufId of the main function for a description.

Component

See Component of the main function for a description.

SrcArraySize

See SrcArraySize of the main function for a description.

SrcPixelXArrayPtr

See SrcPixelXArrayPtr of the main function for a description.

SrcPixelYArrayPtr

See SrcPixelYArrayPtr of the main function for a description.

Options

See Options of the main function for a description.

DstCoord1OrPackedArrayPtr

See DstCoord1OrPackedArrayPtr of the main function for a description.

DstCoord2ArrayPtr

See DstCoord2ArrayPtr of the main function for a description.

DstCoord3ArrayPtr

See DstCoord3ArrayPtr of the main function for a description.

IndexArrayPtr

See IndexArrayPtr of the main function for a description.

```
AIL_INT M3dimGetListInt32Uint16 (AIL_ID ContainerOrImageBufId, AIL_INT64 Component, AIL_INT SrcArraySize, const AIL_INT32
*SrcPixelXArrayPtr, const AIL_INT32 *SrcPixelYArrayPtr, AIL_INT64 Options, AIL_UINT16 *DstCoord1OrPackedArrayPtr, AIL_UINT16
*DstCoord2ArrayPtr, AIL_UINT16 *DstCoord3ArrayPtr, AIL_INT32 *IndexArrayPtr)
```

Details

Parameters

ContainerOrImageBufId

See ContainerOrImageBufId of the main function for a description.

Component

See Component of the main function for a description.

SrcArraySize

See SrcArraySize of the main function for a description.

SrcPixelXArrayPtr

See SrcPixelXArrayPtr of the main function for a description.

SrcPixelYArrayPtr

See SrcPixelYArrayPtr of the main function for a description.

Options

See Options of the main function for a description.

DstCoord1OrPackedArrayPtr

See DstCoord1OrPackedArrayPtr of the main function for a description.

DstCoord2ArrayPtr

See DstCoord2ArrayPtr of the main function for a description.

DstCoord3ArrayPtr

See DstCoord3ArrayPtr of the main function for a description.

IndexArrayPtr

See IndexArrayPtr of the main function for a description.

```
AIL_INT M3dimGetListInt32Uint32 (AIL_ID ContainerOrImageBufId, AIL_INT64 Component, AIL_INT SrcArraySize, const AIL_INT32
*SrcPixelXArrayPtr, const AIL_INT32 *SrcPixelYArrayPtr, AIL_INT64 Options, AIL_UINT32 *DstCoord1OrPackedArrayPtr, AIL_UINT32
*DstCoord2ArrayPtr, AIL_UINT32 *DstCoord3ArrayPtr, AIL_INT32 *IndexArrayPtr)
```

Details

Parameters

ContainerOrImageBufId

See ContainerOrImageBufId of the main function for a description.

Component

See Component of the main function for a description.

SrcArraySize

See SrcArraySize of the main function for a description.

SrcPixelXArrayPtr

See SrcPixelXArrayPtr of the main function for a description.

SrcPixelYArrayPtr

See SrcPixelYArrayPtr of the main function for a description.

Options

See Options of the main function for a description.

DstCoord1OrPackedArrayPtr

See DstCoord1OrPackedArrayPtr of the main function for a description.

DstCoord2ArrayPtr

See DstCoord2ArrayPtr of the main function for a description.

DstCoord3ArrayPtr

See DstCoord3ArrayPtr of the main function for a description.

IndexArrayPtr

See IndexArrayPtr of the main function for a description.

```
AIL_INT M3dimGetListInt32Float (AIL_ID ContainerOrImageBufId, AIL_INT64 Component, AIL_INT SrcArraySize, const AIL_INT32 *SrcPixelXArrayPtr, const AIL_INT32 *SrcPixelYArrayPtr, AIL_INT64 Options, AIL_FLOAT *DstCoord1OrPackedArrayPtr, AIL_FLOAT *DstCoord2ArrayPtr, AIL_FLOAT *DstCoord3ArrayPtr, AIL_INT32 *IndexArrayPtr)
```

Details

Parameters

ContainerOrImageBufId

See ContainerOrImageBufId of the main function for a description.

Component

See Component of the main function for a description.

SrcArraySize

See SrcArraySize of the main function for a description.

SrcPixelXArrayPtr

See SrcPixelXArrayPtr of the main function for a description.

SrcPixelYArrayPtr

See SrcPixelYArrayPtr of the main function for a description.

Options

See Options of the main function for a description.

DstCoord1OrPackedArrayPtr

See DstCoord1OrPackedArrayPtr of the main function for a description.

DstCoord2ArrayPtr

See DstCoord2ArrayPtr of the main function for a description.

DstCoord3ArrayPtr

See DstCoord3ArrayPtr of the main function for a description.

IndexArrayPtr

See IndexArrayPtr of the main function for a description.

```
AIL_INT M3dimGetListInt32Double (AIL_ID ContainerOrImageBufId, AIL_INT64 Component, AIL_INT SrcArraySize, const AIL_INT32 *SrcPixelXArrayPtr, const AIL_INT32 *SrcPixelYArrayPtr, AIL_INT64 Options, AIL_DOUBLE *DstCoord1OrPackedArrayPtr, AIL_DOUBLE *DstCoord2ArrayPtr, AIL_DOUBLE *DstCoord3ArrayPtr, AIL_INT32 *IndexArrayPtr)
```

Details

Parameters

ContainerOrImageBufId

See ContainerOrImageBufId of the main function for a description.

Component

See Component of the main function for a description.

SrcArraySize

See SrcArraySize of the main function for a description.

SrcPixelXArrayPtr

See SrcPixelXArrayPtr of the main function for a description.

SrcPixelYArrayPtr

See SrcPixelYArrayPtr of the main function for a description.

Options

See Options of the main function for a description.

DstCoord1OrPackedArrayPtr

See DstCoord1OrPackedArrayPtr of the main function for a description.

DstCoord2ArrayPtr

See DstCoord2ArrayPtr of the main function for a description.

DstCoord3ArrayPtr

See DstCoord3ArrayPtr of the main function for a description.

IndexArrayPtr

See IndexArrayPtr of the main function for a description.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dim11.lib.
DLL	Requires ail11.dll; ail3dim11.dll.

M3dimGetResult

Synopsis

Get the specified type of result from a 3D image processing result buffer.

Syntax

```
AIL_DOUBLE M3dimGetResult(
    AIL_ID Result3dimId,  //in
    AIL_INT64 ResultType, //in
    void *ResultArrayPtr  //out
)
```

Description

This function retrieves the result(s) of the specified type from a result buffer.

Parameters

Result3dimId

Specifies the identifier of the 3D image processing result buffer from which to retrieve results. The result buffer must have been previously allocated on the system using M3dimAllocResult() with M_FIND_TRANSFORMATION_RESULT, M_LATTICE_RESULT, M_PROFILE_RESULT, or M_STATISTICS_RESULT.

See the Parameter associations section for possible values that can be specified.

ResultType

Specifies the type of result to retrieve.

See the Parameter associations section for possible values that can be specified.

ResultArrayPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]
- array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>]
- array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]
- array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]
- array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]
- AIL_DOUBLE
- AIL_FLOAT
- AIL_INT
- AIL_INT32
- AIL_INT64

Specifies the address of the array in which to write the requested information.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The table below lists possible values for the **Result3dimId** and **ResultType** parameters and possible values returned to the **ResultArrayPtr** parameter.

- For specifying the type of result to retrieve from a find transformation, profile, or statistics 3D image processing result buffer

The following Result3dimId, ResultType, and ResultArrayPtr parameter settings can be specified for different types of 3D image processing result buffers.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of result to retrieve from a find transformation, profile, or statistics 3D image processing result buffer																		
Result3dimId			Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ResultType																	
		ResultArrayPtr - Possible values returned																

Find transformation 3D image processing result ID	Specifies the identifier of a find transformation 3D image processing result buffer. For an M_FIND_TRANSFORMATION_RESULT result buffer, results are available after using M3dimFindTransformation() to fill the result buffer with the rigid or affine transformation matrix.														
M_NUMBER_OF_POINT_PAIRS +	Retrieves the number of valid point pairs used when finding the transformation.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_RMS_ERROR +	Retrieves the root-mean-square (RMS) error between the point pairs if the matrix was applied. Aurora Imaging Library calculates the RMS error using the following formula: $RMS\ error = \sqrt{\frac{1}{N} \times \sum \lVert Matrix * Source - Target \rVert^2}$	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_STATUS +	Retrieves the status of the find transformation operation.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL_POINTS_COPLANAR	Specifies that all point pairs are in the same plane and that the operation to find an affine transformation could not proceed.	c		e	f	g	a		i		j	k	l	m	n	b
M_NOT_ENOUGH_POINT_PAIRS	Specifies that there are either less than 3 point pairs (for a rigid transformation) or 4 point pairs (for an affine transformation) and that the operation could not proceed.	c		e	f	g	a		i		j	k	l	m	n	b
M_NOT_INITIALIZED	Specifies that the operation was not initialized.	c		e	f	g	a		i		j	k	l	m	n	b
M_SUCCESS	Specifies that the transformation matrix was successfully found.	c		e	f	g	a		i		j	k	l	m	n	b
Lattice 3D image processing result ID	Specifies the identifier of a lattice 3D image processing result buffer. For an M_LATTICE_RESULT result buffer, results are available after using M3dimLattice() to fill the result buffer with a lattice.	c		e	f	g	a		i		j	k	l	m	n	b
M_CELL_SIZE_X +	Retrieves the cell size in the X-direction. Note that if M_NUMBER_OF_GRID_SIZES is greater than 1, this result depends on the grid size specified using M3dimControl() with M_GRID_SIZE_INDEX.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_INFINITE	Specifies an infinite cell size along X.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the cell size along X, in world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_CELL_SIZE_Y +	Retrieves the cell size in the Y-direction. Note that if M_NUMBER_OF_GRID_SIZES is greater than 1, this result depends on the grid size specified using M3dimControl() with M_GRID_SIZE_INDEX.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

	M_INFINITE	Specifies an infinite cell size along Y.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the cell size along Y, in world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_CELL_SIZE_Z +		Retrieves the cell size in the Z-direction. Note that if M_NUMBER_OF_GRID_SIZES is greater than 1, this result depends on the grid size specified using M3dimControl() with M_GRID_SIZE_INDEX.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_INFINITE	Specifies an infinite cell size along Z.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the cell size along Z, in world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_END_POINT_X +		Retrieves the X-coordinate of the lattice's end point, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_END_POINT_Y +		Retrieves the Y-coordinate of the lattice's end point, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_END_POINT_Z +		Retrieves the Z-coordinate of the lattice's end point, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_CELLS_X +		Retrieves the number of cells in the X-direction. Note that if M_NUMBER_OF_GRID_SIZES is greater than 1, this result depends on the grid size specified using M3dimControl() with M_GRID_SIZE_INDEX.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the number of cells along X.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_CELLS_Y +		Retrieves the number of cells in the Y-direction. Note that if M_NUMBER_OF_GRID_SIZES is greater than 1, this result depends on the grid size specified using M3dimControl() with M_GRID_SIZE_INDEX.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the number of cells along Y.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_CELLS_Z +		Retrieves the number of cells in the Z-direction. Note that if M_NUMBER_OF_GRID_SIZES is greater than 1, this result depends on the grid size specified using M3dimControl() with M_GRID_SIZE_INDEX.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the number of cells along Z.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_GRID_SIZES +		Retrieves the number of possible grid sizes for the lattice. You can specify the index of the grid size to use, using M3dimControl() with M_GRID_SIZE_INDEX.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b

	Value > 0	Specifies the number of grid sizes.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIZE_X +		Retrieves the size of the lattice along the X-direction, in world units.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_SIZE_Y +		Retrieves the size of the lattice along the Y-direction, in world units.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_SIZE_Z +		Retrieves the size of the lattice along the Z-direction, in world units.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_SPARSITY_DIMENSION +		Retrieves the approximate number of dimensions in which points from the source point cloud are merged into each cell of the lattice. For example, if the source point cloud is much denser in X than in Y and Z, merging might only occur along the X-dimension when the grid size is small. As the grid size increases, the cells merge points along more dimensions. Note that if M_NUMBER_OF_GRID_SIZES is greater than 1, this result depends on the grid size specified using M3dimControl() with M_GRID_SIZE_INDEX. This result type is only available when M3dimControl() with M_LATTICE_MODE is set to M_SPARSITY_CHANGE.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_INVALID	Specifies that the dimensions could not be calculated because a bounding box containing a single point was supplied to M3dimLattice().	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the approximate number of dimensions.	c		e	f	g	a		i		j	k	l	m	n	b
M_START_POINT_X +		Retrieves the X-coordinate of the lattice's start point, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_START_POINT_Y +		Retrieves the Y-coordinate of the lattice's start point, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_START_POINT_Z +		Retrieves the Z-coordinate of the lattice's start point, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_STATUS +		Retrieves the status of the lattice operation.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_NOT_ENOUGH_VALID_DATA	Specifies that the lattice operation failed because a bounding box was not provided and could not be calculated from the point cloud because there were not enough valid points (less than 2).	c		e	f	g	a		i		j	k	l	m	n	b
	M_NOT_INITIALIZED	Specifies that the lattice 3D image processing result buffer was not used in a call to M3dimLattice(), and contains no results.	c		e	f	g	a		i		j	k	l	m	n	b

	M_SUCCESS	Specifies that the lattice operation completed successfully.	c		e	f	g	a		i		j	k	l	m	n	b
Profile 3D image processing result ID		Specifies the identifier of a profile 3D image processing result buffer. For an M_PROFILE_RESULT result buffer, results are available after using M3dimProfile() or M3dimProfileEx() to fill the result buffer with a profile. Note that you can transform coordinates from M_PROFILE_PLANE_... to M_WORLD_... (and vice versa). To do so, copy profile results into a transformation matrix, using M3dimCopyResult() with M_MATRIX_PROFILE_PLANE_TO_WORLD or M_MATRIX_WORLD_TO_PROFILE_PLANE, and then pass the resulting transformation matrix to M3dimMatrixTransformList(). For depth maps, transforming M_WORLD_... point coordinate values using McalTransformCoordinate3dList() will yield values close to those obtained with M_PROFILE_PLANE_..., but exactly on the profile line.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NUMBER_OF_POINTS_MISSING_DATA +	Retrieves the number of missing data points (gap pixels) encountered when creating the depth map profile; for other profiles, this result type always returns 0. This result type is available only after an M3dimProfile() operation.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_NUMBER_OF_POINTS_VALID +	Retrieves the number of valid points along the profile. Note that all extracted profile points are valid; therefore, this number equals the total number of points in the profile. This result type is available only after an M3dimProfile() operation.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_PIXEL_X +	Retrieves the pixel X-coordinates for valid points included in the profile. Each 3D point has a corresponding pixel in the depth map image buffer or point cloud range component. M_PIXEL_X returns an integer value, for each profile point, which corresponds to the pixel's position along X in the depth map or range component. Note that M_PIXEL_X is unavailable for profiles of type M_PROFILE_MESH and M_PROFILE_GEOMETRY. This result type is available only after an M3dimProfile() operation.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_PIXEL_Y +	Retrieves the pixel Y-coordinates for valid points included in the profile. Each 3D point has a corresponding pixel in the depth map image buffer or point cloud range component. M_PIXEL_Y returns an integer value, for each profile point, which corresponds to the pixel's position along Y in the depth map or range component. Note that M_PIXEL_Y is unavailable for profiles of type M_PROFILE_MESH. This result type is available only after an M3dimProfile() operation.	c		e	f	g	a		i		j	k	l	m	n	b

		ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_PROFILE_PLANE_X +		Retrieves the X-coordinates from the resulting profile, expressed in the coordinate system of the slicing plane itself, in world units. For a depth map profile, points are extracted from pixels along the specified line. M_PROFILE_PLANE_X returns the real world distance along that line, for each extracted point. Note that the line's start point (set using M3dimProfile() with M_PROFILE_DEPTH_MAP and Param2 and Param3, or using M3dimProfileEx() with Param2 and Param3) represents a distance of 0. The line is effectively the X-axis for a plot of M_PROFILE_PLANE_X and M_PROFILE_PLANE_Y values, which sit on the slicing plane. For other profile types, the returned values correspond to the X-coordinates of the profile's points, expressed in the specified slicing plane's coordinate system. You can draw the M_PROFILE_PLANE_X and M_PROFILE_PLANE_Y values in a 2D display using MgraDots(). Alternatively, you can draw the profile points using M3dimDraw3d().	c		e	f	g	a		i		j	k	l	m	n	b
		ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_PROFILE_PLANE_Y +		Retrieves the Y-coordinates from the resulting profile, expressed in the coordinate system of the slicing plane itself, in world units. For a depth map profile, the returned values correspond to the depth at each pixel underneath the specified line. For other profile types, the returned values correspond to the Y-coordinates of the profile's points, expressed in the specified slicing plane's coordinate system. You can draw the M_PROFILE_PLANE_X and M_PROFILE_PLANE_Y values in a 2D display using MgraDots(). Alternatively, you can draw the profile points using M3dimDraw3d().	c		e	f	g	a		i		j	k	l	m	n	b
		ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_PROFILE_TYPE +		Retrieves the profile type.	c		e	f	g	a		i		j	k	l	m	n	b
		ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
		M_NOT_INITIALIZED	Specifies that a profile type has not been defined for the profile result.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROFILE_DEPTH_MAP	Specifies a depth map profile. This result is available only after an M3dimProfile() operation.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROFILE_DEPTH_MAP_UNIFORM	Specifies a uniform depth map profile, where the points are at equal distances from each other along the profile line defined by the start and end points. This result is available only after an M3dimProfileEx() operation.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROFILE_GEOMETRY	Specifies a 3D geometry profile. This result is available only after an M3dimProfile() operation.	c		e	f	g	a		i		j	k	l	m	n	b

		M_PROFILE_MESH	Specifies a mesh profile. This result is available only after an M3dimProfile() operation.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROFILE_POINT_CLOUD	Specifies a point cloud profile. This result is available only after an M3dimProfile() operation.	c		e	f	g	a		i		j	k	l	m	n	b
	M_STATUS +		Retrieves the status of the profile operation.	c		e	f	g	a		i		j	k	l	m	n	b
		ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
		M_COMPLETE	Specifies that the profile operation completed successfully.	c		e	f	g	a		i		j	k	l	m	n	b
		M_INFINITE_PROFILE	Specifies that the profile operation failed and yielded an infinite result. This result is available only after an M3dimProfile() operation.	c		e	f	g	a		i		j	k	l	m	n	b
		M_MISSING_COMPONENT_MESH_AIL	Specifies that the profile operation failed because the source point cloud container does not have an M_COMPONENT_MESH_AIL component. This result is available only after an M3dimProfile() operation.	c		e	f	g	a		i		j	k	l	m	n	b
		M_NO_PROFILE	Specifies that no profile was found; the slicing plane did not intersect the source.	c		e	f	g	a		i		j	k	l	m	n	b
		M_NOT_INITIALIZED	Specifies that the profile operation was not initialized.	c		e	f	g	a		i		j	k	l	m	n	b
	M_WORLD_X +		Retrieves the world X-coordinates for the extracted points. For a depth map, the returned values correspond to each extracted point's pixel position along X, in world units. For a point cloud, mesh, or 3D geometry profile, the returned values correspond to each extracted point's position in the point cloud along X, in world units. You can draw the M_WORLD_X, M_WORLD_Y, and M_WORLD_Z values in a 3D display using M3dgraDots().	c		e	f	g	a		i		j	k	l	m	n	b
		ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_WORLD_Y +		Retrieves the world Y-coordinates for the extracted points. For a depth map, the returned values correspond to each extracted point's pixel position along Y, in world units. For a point cloud, mesh, or 3D geometry profile, the returned values correspond to each extracted point's position in the point cloud along Y, in world units. You can draw the M_WORLD_X, M_WORLD_Y, and M_WORLD_Z values in a 3D display using M3dgraDots().	c		e	f	g	a		i		j	k	l	m	n	b
		ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

		c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD_Z +	Retrieves the world Z-coordinates for the extracted points. For a depth map, the returned values correspond to each extracted point's depth value (distance along Z), in world units. For a point cloud, mesh, or 3D geometery profile, the returned values correspond to each extracted point's position in the point cloud along Z, in world units. You can draw the M_WORLD_X, M_WORLD_Y, and M_WORLD_Z values in a 3D display using M3dgraDots().															
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
Statistics 3D image processing result ID	Specifies the identifier of a statistics 3D image processing result buffer. For an M_STATISTICS_RESULT result buffer, results are available after using M3dimStat() to fill the result buffer with statistics on a container's point cloud, on a depth map, or on a 3D geometry.	c		e	f	g	a		i		j	k	l	m	n	b
M_MOMENT_CENTRAL_XYZ(AIL_INT PowerX, AIL_INT PowerY, AIL_INT PowerZ) +	Retrieves the central moment corresponding to the specified powers of X, Y, and Z. For point clouds and depth maps, this is equivalent to the following formula, where μ is the centroid and (x_i, y_i, z_i) is the ith point: $\sum_i (x_i - \mu_x)^{PowerX} (y_i - \mu_y)^{PowerY} (z_i - \mu_z)^{PowerZ}$ For 3D geometries, this is equivalent to the following surface integral formula, where μ is the centroid. $\iint_S (x - \mu_x)^{PowerX} (y - \mu_y)^{PowerY} (z - \mu_z)^{PowerZ} dS$ For line geometries, Aurora Imaging Library uses a line integral instead. This result type is available for centroid, moments, number-of-points, or PCA statistics results. To retrieve the respective central moment results, you must ensure that the sum of PowerX, PowerY, and PowerZ complies with the following: Statistics type: (PowerX + PowerY + PowerZ) must be: centroid: <= 1 moments: <= M_MOMENT_ORDER (set using M3dimControl()) number-of-points: 0 PCA: <= 2	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	PowerX Specifies the power in X.															

| | | |

| | | |

M_CENTROID_Z +	Retrieves the Z-coordinate of the centroid. This result type is available for centroid, moments, or PCA statistics results; moments results must be of order 1 or greater.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_OF_INTEREST +	Retrieves the component for which the statistics were calculated. This result type is available for centroid, moments, or PCA statistics results; moments results must be of order 1 or greater.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_NORMALS_AIL	Specifies the normals component.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_RANGE	Specifies the range component.	c		e	f	g	a		i		j	k	l	m	n	b
M_COVARIANCE_MATRIX +	Retrieves the 9 entries that make up the covariance matrix, in row-major order. The covariance matrix is defined as a 3x3 matrix of second order moments: xx xy xz xy yy yz xz yz zz The moments are central or ordinary, according to M3dimControl() with M_PCA_MODE. This result type is available for moments or PCA statistics results; moments results must be of order 2 or greater.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE_TO_NEAREST_NEIGHBOR_AVERAGE +	Retrieves the average distance to the nearest neighbor. This result is available only when you have enabled the M3dimControl() M_DISTANCE_TO_NEAREST_NEIGHBOR and M_CALCULATE_AVERAGE control types.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE_TO_NEAREST_NEIGHBOR_MAX +	Retrieves the maximum distance to the nearest neighbor. This result is available only when you have enabled the M3dimControl() M_DISTANCE_TO_NEAREST_NEIGHBOR and M_CALCULATE_MAX control types.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE_TO_NEAREST_NEIGHBOR_MEDIAN +	Retrieves the median distance to the nearest neighbor. This result is available only when you have enabled the M3dimControl() M_DISTANCE_TO_NEAREST_NEIGHBOR and M_CALCULATE_MEDIAN control types.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

	M_DISTANCE_TO_NEAREST_NEIGHBOR_MIN +	Retrieves the minimum distance to the nearest neighbor. This result is available only when you have enabled the M3dimControl() M_DISTANCE_TO_NEAREST_NEIGHBOR and M_CALCULATE_MIN control types.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISTANCE_TO_NEAREST_NEIGHBOR_ROBUST_STDEV +	Retrieves the robust standard deviation of distances between each point and its nearest neighbor. M_DISTANCE_TO_NEAREST_NEIGHBOR_ROBUST_STDEV is computed using the median absolute deviation (MAD) and a scale factor of 1.4826. This result is available only when you have enabled the M3dimControl() M_DISTANCE_TO_NEAREST_NEIGHBOR and M_CALCULATE_ROBUST_STDEV control types.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISTANCE_TO_NEAREST_NEIGHBOR_STDEV +	Retrieves the standard deviation of distances between each point and its nearest neighbor. This result is available only when you have enabled the M3dimControl() M_DISTANCE_TO_NEAREST_NEIGHBOR and M_CALCULATE_STDEV control types.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_EIGENVALUE_1 +	Retrieves the eigenvalue corresponding to the first principal axis. The first principal axis is the direction along which there is the most variance. This result type is available for moments or PCA statistics results; moments results must be of order 2 or greater.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_EIGENVALUE_2 +	Retrieves the eigenvalue corresponding to the second principal axis. The second principal axis is the direction along which there is the second most variance, perpendicular to the first principal axis. This result type is available for moments or PCA statistics results; moments results must be of order 2 or greater.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_EIGENVALUE_3 +	Retrieves the eigenvalue corresponding to the third principal axis. The third principal axis is perpendicular to the first and second principal axes. This result type is available for moments or PCA statistics results; moments results must be of order 2 or greater.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_MAX_X +	Retrieves the maximum X-coordinate of the bounding box. This result type is available for bounding box statistics results. The bounding box is an axis-aligned or semi-oriented box that contains all or most of the points in the 3D scene, or it encloses a 3D geometry object. This result type is not available for a semi-oriented bounding box, unless M3dimControl() with M_BOX_SEMI_ORIENTED_ROTATION_AXIS is set to M_AXIS_X.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

	M_MAX_Y +	Retrieves the maximum Y-coordinate of the bounding box. This result type is available for bounding box statistics results. The bounding box is an axis-aligned or semi-oriented box that contains all or most of the points in the 3D scene, or it encloses a 3D geometry object. This result type is not available for a semi-oriented bounding box, unless M3dimControl() with M_BOX_SEMI_ORIENTED_ROTATION_AXIS is set to M_AXIS_Y.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_MAX_Z +	Retrieves the maximum Z-coordinate of the bounding box. This result type is available for bounding box statistics results. The bounding box is an axis-aligned or semi-oriented box that contains all or most of the points in the 3D scene, or it encloses a 3D geometry object. This result type is not available for a semi-oriented bounding box, unless M3dimControl() with M_BOX_SEMI_ORIENTED_ROTATION_AXIS is set to M_AXIS_Z.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_MIN_X +	Retrieves the minimum X-coordinate of the bounding box. This result type is available for bounding box statistics results. The bounding box is an axis-aligned or semi-oriented box that contains all or most of the points in the 3D scene, or it encloses a 3D geometry object. This result type is not available for a semi-oriented bounding box, unless M3dimControl() with M_BOX_SEMI_ORIENTED_ROTATION_AXIS is set to M_AXIS_X.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_MIN_Y +	Retrieves the minimum Y-coordinate of the bounding box. This result type is available for bounding box statistics results. The bounding box is an axis-aligned or semi-oriented box that contains all or most of the points in the 3D scene, or it encloses a 3D geometry object. This result type is not available for a semi-oriented bounding box, unless M3dimControl() with M_BOX_SEMI_ORIENTED_ROTATION_AXIS is set to M_AXIS_Y.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_MIN_Z +	Retrieves the minimum Z-coordinate of the bounding box. This result type is available for bounding box statistics results. The bounding box is an axis-aligned or semi-oriented box that contains all or most of the points in the 3D scene, or it encloses a 3D geometry object. This result type is not available for a semi-oriented bounding box, unless M3dimControl() with M_BOX_SEMI_ORIENTED_ROTATION_AXIS is set to M_AXIS_Z.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

	M_NEIGHBORHOOD_ASPECT_RATIO_AVERAGE +	Retrieves the average local aspect ratio, across all points. For each point, Aurora Imaging Library fits a local plane to neighboring points and determines the plane's aspect ratio. This result is available only when you have enabled the M3dimControl() M_NEIGHBORHOOD_ASPECT_RATIO and M_CALCULATE_AVERAGE control types.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_NEIGHBORHOOD_ASPECT_RATIO_MAX +	Retrieves the maximum local aspect ratio, across all points. Note that this value is limited by M_NEIGHBORHOOD_ASPECT_RATIO_CLIPPING_VALUE. For each point, Aurora Imaging Library fits a local plane to neighboring points and determines the plane's aspect ratio. This result is available only when you have enabled the M3dimControl() M_NEIGHBORHOOD_ASPECT_RATIO and M_CALCULATE_MAX control types.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_NEIGHBORHOOD_ASPECT_RATIO_MEDIAN +	Retrieves the median local aspect ratio, across all points. For each point, Aurora Imaging Library fits a local plane to neighboring points and determines the plane's aspect ratio. This result is available only when you have enabled the M3dimControl() M_NEIGHBORHOOD_ASPECT_RATIO and M_CALCULATE_MEDIAN control types.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_NEIGHBORHOOD_ASPECT_RATIO_MIN +	Retrieves the minimum local aspect ratio, across all points. For each point, Aurora Imaging Library fits a local plane to neighboring points and determines the plane's aspect ratio. This result is available only when you have enabled the M3dimControl() M_NEIGHBORHOOD_ASPECT_RATIO and M_CALCULATE_MIN control types.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_NEIGHBORHOOD_ASPECT_RATIO_ROBUST_STDEV +	Retrieves the robust standard deviation of local aspect ratios, across all points. M_NEIGHBORHOOD_ASPECT_RATIO_ROBUST_STDEV is computed using the median absolute deviation (MAD) and a scale factor of 1.4826. For each point, Aurora Imaging Library fits a local plane to neighboring points, determines the plane's aspect ratio, and incorporates the local aspect ratio into the robust standard deviation calculation. This result is available only when you have enabled the M3dimControl() M_NEIGHBORHOOD_ASPECT_RATIO and M_CALCULATE_ROBUST_STDEV control types.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

M_NEIGHBORHOOD_ASPECT_RATIO_STDEV +	Retrieves the standard deviation of local aspect ratios, across all points. For each point, Aurora Imaging Library fits a local plane to neighboring points, determines the plane's aspect ratio, and incorporates the local aspect ratio into the standard deviation calculation. This result is available only when you have enabled the M3dimControl() M_NEIGHBORHOOD_ASPECT_RATIO and M_CALCULATE_STDEV control types.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_NEIGHBORHOOD_HEALTHY_PERCENTAGE +	Retrieves the percentage of points that have a healthy neighborhood. This is the percentage of points with at least M_NEIGHBORHOOD_HEALTHY_THRESHOLD valid neighbors. Most neighborhood-based functions (for example, M3dimNormals()) will work poorly if this value is too low. This result is available only when you have enabled the M3dimControl() M_NUMBER_OF_VALID_NEIGHBORS control type.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_NORMALIZATION_MODE +	Retrieves the mode used to compute the normalization matrix. This result type is available for bounding box statistics results; the bounding box must be axis-aligned (M_BOX_ORIENTATION set to M_AXIS_ALIGNED).	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_NORMALIZE_SIGNED	Specifies that the point cloud fits a signed unit box.	c		e	f	g	a		i		j	k	l	m	n	b
M_NORMALIZE_UNSIGNED	Specifies that the point cloud fits an unsigned unit box.	c		e	f	g	a		i		j	k	l	m	n	b
M_NORMALIZATION_SCALE +	Retrieves the scale mode used in the statistics context to compute the normalization matrix. This result type is available for bounding box statistics results; the bounding box must be axis-aligned (M_BOX_ORIENTATION set to M_AXIS_ALIGNED).	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_NON_UNIFORM	Specifies that each dimension was scaled differently.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNIFORM	Specifies that the same scale value was used for all dimensions. This means that the largest dimension was scaled to fit the unit box, and the remaining dimensions have the same or smaller lengths.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_POINTS_ISOLATED +	Retrieves the number of isolated points found during the distance-to-nearest-neighbor, surface variation, aspect ratio, and number-of-valid-neighbors statistics calculations. Isolated points are points with no neighbors except themselves (as defined by the neighborhood controls, M_MAXIMUM_NUMBER_NEIGHBORS, M_MAX_DISTANCE, M_NEIGHBOR_SEARCH_MODE, and M_NEIGHBORHOOD_ORGANIZED_SIZE). This result is available only when you have enabled the M3dimControl() M_DISTANCE_TO_NEAREST_NEIGHBOR, M_SURFACE_VARIATION, M_NEIGHBORHOOD_ASPECT_RATIO, or M_NUMBER_OF_VALID_NEIGHBORS control types.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b

M_NUMBER_OF_POINTS_MISSING_DATA +	Retrieves the number of points that are missing data. This result type is available for centroid, moments, number-of-points, or PCA statistics results.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_POINTS_REPLICATED +	Retrieves the number of replicated points found during the distance-to-nearest-neighbor, surface variation, aspect ratio, and number-of-valid-neighbors statistics calculations. Replicated points are distinct points with the same XYZ-coordinates. Note that replicated points are automatically ignored when calculating distance-to-nearest-neighbor statistics; however, replicated points can have a significant effect on surface variation, aspect ratio, and number-of-valid-neighbor results. This result is available only when you have enabled the M3dimControl() M_DISTANCE_TO_NEAREST_NEIGHBOR, M_SURFACE_VARIATION, M_NEIGHBORHOOD_ASPECT_RATIO, or M_NUMBER_OF_VALID_NEIGHBORS control types.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_POINTS_TOTAL +	Retrieves the total number of points. This result type is available for centroid, moments, number-of-points, or PCA statistics results.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_POINTS_VALID +	Retrieves the number of valid points. Valid points are those with non-zero confidence. This result type is available for centroid, moments, number-of-points, or PCA statistics results.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_VALID_NEIGHBORS_AVERAGE +	Retrieves the average number of valid neighbors, across all points. Valid neighbors are those with non-zero confidence. This result is available only when you have enabled the M3dimControl() M_NUMBER_OF_VALID_NEIGHBORS and M_CALCULATE_AVERAGE control types.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_VALID_NEIGHBORS_MAX +	Retrieves the maximum number of valid neighbors, across all points. Valid neighbors are those with non-zero confidence. This result is available only when you have enabled the M3dimControl() M_NUMBER_OF_VALID_NEIGHBORS and M_CALCULATE_MAX control types.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_VALID_NEIGHBORS_MEDIAN +	Retrieves the median number of valid neighbors, across all points. Valid neighbors are those with non-zero confidence. This result is available only when you have enabled the M3dimControl() M_NUMBER_OF_VALID_NEIGHBORS and M_CALCULATE_MEDIAN control types.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

M_NUMBER_OF_VALID_NEIGHBORS_MIN +	Retrieves the minimum number of valid neighbors, across all points. Valid neighbors are those with non-zero confidence. This result is available only when you have enabled the M3dimControl() M_NUMBER_OF_VALID_NEIGHBORS and M_CALCULATE_MIN control types.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_VALID_NEIGHBORS_ROBUST_STDEV +	Retrieves the robust standard deviation of each point's number of valid neighbors. Valid neighbors are those with non-zero confidence. M_NUMBER_OF_VALID_NEIGHBORS_ROBUST_STDEV is computed using the median absolute deviation (MAD) and a scale factor of 1.4826. This result is available only when you have enabled the M3dimControl() M_NUMBER_OF_VALID_NEIGHBORS and M_CALCULATE_ROBUST_STDEV control types.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_VALID_NEIGHBORS_STDEV +	Retrieves the standard deviation of each point's number of valid neighbors. Valid neighbors are those with non-zero confidence. This result is available only when you have enabled the M3dimControl() M_NUMBER_OF_VALID_NEIGHBORS and M_CALCULATE_STDEV control types.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_PCA_MODE +	Retrieves whether the results for principal component analysis (PCA) or moments statistics were measured relative to the object's centroid (M_CENTRAL, default) or the origin (M_ORDINARY). This result type is available for moments or PCA statistics results; moments results must be of order 2 or greater.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_CENTRAL	Specifies that the PCA was calculated around the centroid.	c		e	f	g	a		i		j	k	l	m	n	b
M_ORDINARY	Specifies that the PCA was calculated around the origin.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRINCIPAL_AXIS_1_X +	Retrieves the X-component of the normalized vector along the first principal axis, which is the axis with the largest eigenvalue. This result type is available for moments or PCA statistics results; moments results must be of order 2 or greater.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_PRINCIPAL_AXIS_1_Y +	Retrieves the Y-component of the normalized vector along the first principal axis, which is the axis with the largest eigenvalue. This result type is available for moments or PCA statistics results; moments results must be of order 2 or greater.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

M_PRINCIPAL_AXIS_1_Z +	Retrieves the Z-component of the normalized vector along the first principal axis, which is the axis with the largest eigenvalue. This result type is available for moments or PCA statistics results; moments results must be of order 2 or greater.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_PRINCIPAL_AXIS_2_X +	Retrieves the X-component of the normalized vector along the second principal axis, which is the axis with the second largest eigenvalue. The second principal axis is perpendicular to the first principal axis. This result type is available for moments or PCA statistics results; moments results must be of order 2 or greater.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_PRINCIPAL_AXIS_2_Y +	Retrieves the Y-component of the normalized vector along the second principal axis, which is the axis with the second largest eigenvalue. The second principal axis is perpendicular to the first principal axis. This result type is available for moments or PCA statistics results; moments results must be of order 2 or greater.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_PRINCIPAL_AXIS_2_Z +	Retrieves the Z-component of the normalized vector along the second principal axis, which is the axis with the second largest eigenvalue. The second principal axis is perpendicular to the first principal axis. This result type is available for moments or PCA statistics results; moments results must be of order 2 or greater.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_PRINCIPAL_AXIS_3_X +	Retrieves the X-component of the normalized vector along the third principal axis, which is the axis perpendicular to the first and second principal axes. This result type is available for moments or PCA statistics results; moments results must be of order 2 or greater.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_PRINCIPAL_AXIS_3_Y +	Retrieves the Y-component of the normalized vector along the third principal axis, which is the axis perpendicular to the first and second principal axes. This result type is available for moments or PCA statistics results; moments results must be of order 2 or greater.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_PRINCIPAL_AXIS_3_Z +	Retrieves the Z-component of the normalized vector along the third principal axis, which is the axis perpendicular to the first and second principal axes. This result type is available for moments or PCA statistics results; moments results must be of order 2 or greater.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

	M_SEMI_ORIENTED_BOX_ANGLE +	Retrieves the angle of the semi-oriented bounding box's rotation around the axis specified using M3dimControl() with M_BOX_SEMI_ORIENTED_ROTATION_AXIS, respecting the right-hand rule. This result type is available for bounding box statistics results.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_SIZE_X +	Retrieves the length along X of the 3D scene, in world units. This result type is available for bounding box statistics results.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_SIZE_Y +	Retrieves the length along Y of the 3D scene, in world units. This result type is available for bounding box statistics results.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_SIZE_Z +	Retrieves the length along Z of the 3D scene, in world units. This result type is available for bounding box statistics results.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_STANDARD_DEVIATION_X +	Retrieves the standard deviation in X. This result type is available for moments statistics results of order 2 or greater.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_STANDARD_DEVIATION_Y +	Retrieves the standard deviation in Y. This result type is available for moments statistics results of order 2 or greater.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_STANDARD_DEVIATION_Z +	Retrieves the standard deviation in Z. This result type is available for moments statistics results of order 2 or greater.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

M_STATUS_NEIGHBOR_STATS +	Retrieves whether replicated points were found while calculating neighborhood statistics, as such these statistics might be skewed. Replicated points are distinct points with the same XYZ-coordinates. Note that replicated points are automatically ignored when calculating distance-to-nearest-neighbor statistics; however, replicated points can have a significant effect on surface variation, aspect ratio, and number-of-valid-neighbors results. If you have enabled the M_SURFACE_VARIATION, M_NEIGHBORHOOD_ASPECT_RATIO, or M_NUMBER_OF_VALID_NEIGHBORS control type and replicated points were found, you can use M3dimFix() with M_UNREPLICATED_POINTS to remove the replicated points, and then recalculate the statistics. You can use M_NUMBER_OF_POINTS_REPLICATED to check the number of replicated points that were found. This result is available only when you have enabled the M3dimControl() M_DISTANCE_TO_NEAREST_NEIGHBOR, M_SURFACE_VARIATION, M_NEIGHBORHOOD_ASPECT_RATIO, or M_NUMBER_OF_VALID_NEIGHBORS control types.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_REPLICATED_POINTS	Specifies that replicated points were found; the calculated statistics might be skewed.	c		e	f	g	a		i		j	k	l	m	n	b
M_SUCCESS	Specifies that no replicated points were found.	c		e	f	g	a		i		j	k	l	m	n	b
M_SURFACE_AREA +	Retrieves the surface area of the mesh. This result is available only when you have enabled the M3dimControl() M_SURFACE_AREA control type.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_SURFACE_VARIATION_AVERAGE +	Retrieves the average distance to the local plane, across all points. This is the average of each point's calculated distance to its local plane, not an average of neighborhood point distances to a single plane. For each point, Aurora Imaging Library fits a local plane to neighboring points and measures the perpendicular distance of the point to the plane. This result is available only when you have enabled the M3dimControl() M_SURFACE_VARIATION and M_CALCULATE_AVERAGE control types.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_SURFACE_VARIATION_MAX +	Retrieves the maximum distance to the local plane, across all points. For each point, Aurora Imaging Library fits a local plane to neighboring points and measures the perpendicular distance of the point to the plane. This result is available only when you have enabled the M3dimControl() M_SURFACE_VARIATION and M_CALCULATE_MAX control types.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

M_SURFACE_VARIATION_MEDIAN +	Retrieves the median distance to the local plane, across all points. For each point, Aurora Imaging Library fits a local plane to neighboring points and measures the perpendicular distance of the point to the plane. This result is available only when you have enabled the M3dimControl() M_SURFACE_VARIATION and M_CALCULATE_MEDIAN control types.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_SURFACE_VARIATION_MIN +	Retrieves the minimum distance to the local plane, across all points. For each point, Aurora Imaging Library fits a local plane to neighboring points and measures the perpendicular distance of the point to the plane. This result is available only when you have enabled the M3dimControl() M_SURFACE_VARIATION and M_CALCULATE_MIN control types.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_SURFACE_VARIATION_ROBUST_STDEV +	Retrieves the robust standard deviation of point distances to their local plane. For each point, Aurora Imaging Library fits a local plane to neighboring points, measures the perpendicular distance of the point to the plane, and incorporates the distance into the robust standard deviation calculation. M_SURFACE_VARIATION_ROBUST_STDEV is computed using the median absolute deviation (MAD) and a scale factor of 1.4826. This result is available only when you have enabled the M3dimControl() M_SURFACE_VARIATION and M_CALCULATE_ROBUST_STDEV control types.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_SURFACE_VARIATION_STDEV +	Retrieves the standard deviation of point distances to their local plane. For each point, Aurora Imaging Library fits a local plane to neighboring points, measures the perpendicular distance of the point to the plane, and incorporates the distance into the standard deviation calculation. This result is available only when you have enabled the M3dimControl() M_SURFACE_VARIATION and M_CALCULATE_STDEV control types.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for the values of the ResultType parameter listed in For specifying the type of result to retrieve from a find transformation, profile, or statistics 3D image processing result buffer.

You can add the following value to the above-mentioned values to determine whether a result is available.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For determining whether results are available																
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																
M_AVAILABLE	Retrieves whether the requested result type is available for retrieval.															
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the requested result type is not available.	c		e	f	g	a		i		j	k	l	m	n	b

	M_TRUE	Specifies that the requested result type is available.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values of the ResultType parameter listed in For specifying the type of result to retrieve from a find transformation, profile, or statistics 3D image processing result buffer.

You can add one of the following values to the above-mentioned values to cast the requested information to the required data type.

> Note: In C++, when using a standard vector (std::vector) instead of an array, the values listed in this combination table must not be used. If you need to use a specific data type, instantiate the vector with the necessary data type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the data type																	
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																	
M_TYPE_AIL_DOUBLE	Casts the requested information to an *AIL_DOUBLE*.																
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_FLOAT	Casts the requested information to an *AIL_FLOAT*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_FLOAT* **|** *array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT	Casts the requested information to an *AIL_INT*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT* **|** *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT32	Casts the requested information to an *AIL_INT32*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT32* **|** *array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT64	Casts the requested information to an *AIL_INT64*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT64* **|** *array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]*	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the requested information, cast to an *AIL_DOUBLE*. If the requested information does not fit into an *AIL_DOUBLE*, this function will return **M_NULL** or truncate the information.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dim11.lib.
DLL	Requires ail11.dll; ail3dim11.dll.

M3dimInquire

Synopsis

Inquire about a 3D image processing context or result buffer.

Syntax

```
AIL_INT64 M3dimInquire(
    AIL_ID Context3dimId,  //in
    AIL_INT64 InquireType, //in
    void *UserVarPtr       //out
)
```

Description

This function inquires about a specified setting of a 3D image processing context or lattice 3D image processing result buffer.

Parameters

Context3dimId

Specifies the identifier of the 3D image processing context or result buffer about which to inquire information.

See the Parameter associations section for possible values that can be specified.

InquireType

Specifies the setting to inquire.

See the Parameter associations section for possible values that can be specified.

UserVarPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- AIL_DOUBLE
- AIL_FLOAT
- AIL_ID
- AIL_INT
- AIL_INT32
- AIL_INT64

Specifies the address in which to write the requested information. Since the **M3dimInquire**() function also returns the requested information, you can set this parameter to **M_NULL** when the return type is *AIL_INT*.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The table below lists possible values for the **Context3dimId** and **InquireType** parameters and possible values returned to the **UserVarPtr** parameter.

- For inquiring about a 3D image processing context or result buffer

To inquire about a 3D image processing context or lattice 3D image processing result buffer, set the Context3dimId and InquireType parameters to one of the following.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about a 3D image processing context or result buffer																
Context3dimId	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
InquireType																
UserVarPtr - Possible values returned																
Calculate map size context ID	Specifies a calculate map size 3D image processing context, allocated using M3dimAlloc() with M_CALCULATE_MAP_SIZE_CONTEXT, and used in M3dimCalculateMapSize() operations.															

M_PIXEL_ASPECT_RATIO +	Inquires the pixel aspect ratio of the depth map. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies that other data determines the pixel aspect ratio, such as the values set with M_PIXEL_SIZE_X and M_PIXEL_SIZE_Y.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies an aspect ratio for the depth map's pixels. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL_SIZE_CONSTRAINT +	Inquires how to calculate the depth map pixel sizes to fit the required pixel aspect ratio (M_PIXEL_ASPECT_RATIO). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_INCREASE_PIXEL_SIZE	Specifies to calculate the pixel sizes such that the pixel size in one dimension is the same as in the source container and the pixel size in the other dimension is increased to fit the required pixel aspect ratio. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_PRESERVE_WORLD_AREA	Specifies to calculate the pixel sizes such that the area of a pixel is the same as in the source container. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_REDUCE_PIXEL_SIZE	Specifies to calculate the pixel sizes such that the pixel size in one dimension is the same as in the source container and the pixel size in the other dimension is decreased to fit the required pixel aspect ratio. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL_SIZE_X +	Inquires the length in X of one pixel in the depth map. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies that M3dimCalculateMapSize() will compute the pixel size in X.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the length in X of one pixel in the depth map, in world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL_SIZE_Y +	Inquires the length in Y of one pixel in the depth map. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies that M3dimCalculateMapSize() will compute the pixel size in Y.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the length in Y of one pixel in the depth map, in world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIZE_X +	Inquires the image size of the depth map along the X dimension. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies that M3dimCalculateMapSize() will compute the image size of the depth map in X.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the image size of the depth map in X, in pixel units.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIZE_Y +	Inquires the image size of the depth map along the Y dimension. SET	c		e	f	g	a		i		j	k	l	m	n	b

UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies that M3dimCalculateMapSize() will compute the image size of the depth map in Y.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the image size of the depth map in Y, in pixel units.	c		e	f	g	a		i		j	k	l	m	n	b
Fill gaps context ID	Specifies a fill gaps 3D image processing context, allocated using M3dimAlloc() with M_FILL_GAPS_CONTEXT, and used in M3dimFillGaps() operations.	c		e	f	g	a		i		j	k	l	m	n	b
M_FILL_BORDER +	Inquires whether to propagate the boundary value across each gap that touches the image border. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to fill gaps that touch the image border.	c		e	f	g	a		i		j	k	l	m	n	b
M_REPLICATE	Specifies to propagate the unique gap boundary value across each gap that touches the image border, replacing gap pixels up to the image border.	c		e	f	g	a		i		j	k	l	m	n	b
M_FILL_MODE +	Inquires the mode in which to fill gaps in depth maps. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_X_THEN_Y	Specifies that each depth map row is analyzed so that gap pixels in each row are filled, and then each column is analyzed to fill the remaining gaps. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_Y_THEN_X	Specifies that each depth map column is analyzed so that gap pixels in each column are filled, and then each row is analyzed to fill the remaining gaps. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_FILL_SHARP_ELEVATION +	Inquires which boundary to use to fill sharp elevation gaps in depth maps. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to fill sharp elevation gaps.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX	Specifies to set the destination pixel(s) to the maximum value of the two pixels on either side of the gap.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN	Specifies to set the destination pixel(s) to the minimum value of the two pixels on either side of the gap.	c		e	f	g	a		i		j	k	l	m	n	b
M_FILL_SHARP_ELEVATION_DEPTH +	Inquires the threshold used to differentiate between gradual elevation gaps and sharp elevation gaps in the depth map. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_INFINITE	Specifies that the threshold is infinite. INFO	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the threshold, in pixel or world units, depending on the units specified with M_INPUT_UNITS. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_FILL_THRESHOLD_X +	Inquires the maximum X-size of gaps that will be filled. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

M_INFINITE	Specifies no maximum in the X-direction.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the maximum X-size, in pixel or world units, depending on the units specified with M_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
M_FILL_THRESHOLD_Y +	Inquires the maximum Y-size of gaps that will be filled. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_INFINITE	Specifies no maximum in the Y-direction.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the maximum Y-size, in pixel or world units, depending on the units specified with M_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
M_INPUT_UNITS +	Inquires the units with which to interpret M_FILL_SHARP_ELEVATION_DEPTH, M_FILL_THRESHOLD_X, and M_FILL_THRESHOLD_Y. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL	Specifies to interpret the values in pixel units.	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD	Specifies to interpret the values in world units.	c		e	f	g	a		i		j	k	l	m	n	b
Filter context ID	Specifies a filter 3D image processing context, allocated using M3dimAlloc() with M_FILTER_CONTEXT, and used in M3dimFilter() operations.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE_WEIGHT +	Inquires the amount of smoothing. A larger value applies more smoothing than a smaller value. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the amount of smoothing.	c		e	f	g	a		i		j	k	l	m	n	b
M_FILTER_MODE +	Inquires the type of smoothing operation to perform. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_SMOOTH_BILATERAL	Specifies to smooth the point cloud using a bilateral filter. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SMOOTH_MLS	Specifies to smooth the point cloud using a moving least squares (MLS) filter. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_NORMALS_CONTEXT_ID +	Inquires the identifier of the internal normals context. The internal context is used to set the neighborhood of a point, and, if normals are computed, it specifies the normals' orientation.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_ID*	c		e	f	g	a		i		j	k	l	m	n	b
M_NORMALS_WEIGHT_FACTOR +	Inquires how well edges are preserved for an M_SMOOTH_BILATERAL operation. A smaller value results in more edge preservation. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies how well edges are preserved for a bilateral smoothing operation.	c		e	f	g	a		i		j	k	l	m	n	b

M_USE_SOURCE_NORMALS +	Inquires whether to use the source point cloud's normals information, stored in the source container's M_COMPONENT_NORMALS_AIL component. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies not to use source normals information.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies to use source normals information.	c		e	f	g	a		i		j	k	l	m	n	b
M_WEIGHT_MODE +	Inquires how the Gaussian weights are determined in the smoothing calculation set with M_FILTER_MODE. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_ABSOLUTE	Specifies to use the distance Gaussian weight directly, without applying a multiplying factor.	c		e	f	g	a		i		j	k	l	m	n	b
M_RELATIVE	Specifies to compute and apply a multiplying factor to the distance Gaussian weight. INFO	c		e	f	g	a		i		j	k	l	m	n	b
Lattice context ID	Specifies a lattice 3D image processing context, allocated using M3dimAlloc() with M_LATTICE_CONTEXT, and used in M3dimLattice() operations.	c		e	f	g	a		i		j	k	l	m	n	b
M_FRACTION_OF_POINTS +	Inquires the ratio of the total number of required (non-empty) cells to the total number of valid points in the point cloud. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 < *Value* <= 1.0	Specifies the ratio of required (non-empty) cells to the total number of valid points.	c		e	f	g	a		i		j	k	l	m	n	b
M_FRACTION_OF_POINTS_TOLERANCE +	Inquires the tolerance to use for M_FRACTION_OF_POINTS when calculating the lattice. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 < *Value* < 1.0	Specifies the tolerance for the number of required (non-empty) cells.	c		e	f	g	a		i		j	k	l	m	n	b
M_LATTICE_MODE +	Inquires the mode with which to calculate the lattice. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_FRACTION_OF_POINTS	Specifies to calculate a lattice that divides points into cells based on the required ratio of non-empty cells to the number of valid points in the point cloud. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SPARSITY_CHANGE	Specifies to calculate a lattice that divides points into cells based on a plateau of sparsity changes as the grid size increases. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_RELATIVE_CELL_SIZE_X +	Inquires the proportion of the cell size along the X dimension relative to the cell sizes along the Y and Z dimensions. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_INFINITE	Specifies an infinite cell size along X.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the relative cell size along X.	c		e	f	g	a		i		j	k	l	m	n	b

M_RELATIVE_CELL_SIZE_Y +	Inquires the proportion of the cell size along the Y dimension relative to the cell sizes along the X and Z dimensions. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_INFINITE	Specifies an infinite cell size along Y.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the relative cell size along Y.	c		e	f	g	a		i		j	k	l	m	n	b
M_RELATIVE_CELL_SIZE_Z +	Inquires the proportion of the cell size along the Z dimension relative to the cell sizes along the X and Y dimensions. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_INFINITE	Specifies an infinite cell size along Z.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the relative cell size along Z.	c		e	f	g	a		i		j	k	l	m	n	b
M_SPARSITY_DIMENSION_TOLERANCE +	Inquires the minimum difference in dimensionality between consecutive grid sizes when calculating the lattice. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the minimum allowable difference in dimensionality between consecutive grid sizes used in the lattice calculation.	c		e	f	g	a		i		j	k	l	m	n	b
Lattice result buffer ID	Specifies a lattice 3D image processing result buffer, allocated using M3dimAllocResult() with M_LATTICE_RESULT, and used to store M3dimLattice() results.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRID_SIZE_INDEX +	Inquires the grid size for which to return all lattice results. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_FIRST	Specifies to use the first grid size.	c		e	f	g	a		i		j	k	l	m	n	b
M_LAST	Specifies to use the last grid size.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the index of the grid size to use. INFO	c		e	f	g	a		i		j	k	l	m	n	b
Mesh context ID	Specifies a mesh 3D image processing context, allocated using M3dimAlloc() with M_MESH_CONTEXT, and used in M3dimMesh() operations.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX_DISTANCE +	Inquires the maximum distance at which to connect 2 neighboring points. This inquire type is only available when M_MESH_MODE is set to M_MESH_ORGANIZED. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_INFINITE	Specifies no distance constraint.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the maximum distance. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_MAXIMUM_NUMBER_NEIGHBORS +	Inquires the maximum number of neighbors to consider when computing a local plane at each point. This inquire type is available only when M_MESH_MODE is set to M_MESH_LOCAL_PLANE. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the maximum number of neighbor points.	c		e	f	g	a		i		j	k	l	m	n	b
M_MESH_MODE +	Inquires the surface reconstruction mode to use for mesh generation. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_MESH_LOCAL_PLANE	Specifies to generate the mesh from local planes that are fit to neighboring points. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_MESH_ORGANIZED	Specifies to generate the mesh from an organized point cloud or depth map. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_MESH_SMOOTHED	Specifies to generate a mesh with a smoothed surface. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_MESH_STEP_X +	Inquires the step interval between points, along the X-axis of the range component, in pixels. This inquire type is only available when M_MESH_MODE is set to M_MESH_ORGANIZED. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the step interval in the X direction, in pixel units.	c		e	f	g	a		i		j	k	l	m	n	b
M_MESH_STEP_Y +	Inquires the step interval between points, along the Y-axis of the range component, in pixels. This inquire type is only available when M_MESH_MODE is set to M_MESH_ORGANIZED. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the step interval in the Y direction, in pixel units.	c		e	f	g	a		i		j	k	l	m	n	b
M_MESH_TRIMMING_LEVEL +	Inquires the trimming level for a mesh. This inquire type is only available when M_MESH_MODE is set to M_MESH_SMOOTHED. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO	Specifies to automatically calculate the trimming level.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the trimming level.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEIGHBOR_SEARCH_MODE +	Inquires the search mode for finding the neighbors of a point. This inquire type is available only when M_MESH_MODE is set to M_MESH_LOCAL_PLANE. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_ORGANIZED	Specifies to use the point cloud's organizational structure to determine the neighbors of a point. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_TREE	Specifies to use a KD tree search mode to determine a point's neighbors.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEIGHBORHOOD_DISTANCE +	Inquires the distance that defines the neighborhood of a point; only points within this distance are considered when calculating the local plane of the point. This inquire type is available only when M_MESH_MODE is set to M_MESH_LOCAL_PLANE. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_INFINITE	Specifies no distance constraint.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the neighborhood distance limit, in world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEIGHBORHOOD_ORGANIZED_SIZE +	Inquires the neighborhood size when finding the neighbors of a point in an organized point cloud. This inquire type is available only when M_MESH_MODE is set to M_MESH_LOCAL_PLANE. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 3	Specifies the organized neighborhood size, in number of pixels; the specified value must be an odd number.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_POINTS_PER_CELL +	Inquires the number of points within a single octree cell at which the cell will not be split further. This inquire type is only available when M_MESH_MODE is set to M_MESH_SMOOTHED. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the number of points, within a single octree cell, at which the cell will not be split further.	c		e	f	g	a		i		j	k	l	m	n	b
M_SURFACE_ANGLE_MAX +	Inquires the maximum acceptable angle, in degrees, between the normal of a point and that of a neighboring point for the neighbor to be considered for triangulation. This inquire type is available only when M_MESH_MODE is set to M_MESH_LOCAL_PLANE. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 < *Value* < 180.0	Specifies the maximum angle, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_TREE_MAX_DEPTH +	Inquires the maximum allowed depth of the octree. This inquire type is only available when M_MESH_MODE is set to M_MESH_SMOOTHED. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
5 <= *Value* <= 12	Specifies the maximum octree depth.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRIANGLE_ANGLE_MAX +	Inquires the maximum interior angle, in degrees, for each mesh triangle. This inquire type is available only when M_MESH_MODE is set to M_MESH_LOCAL_PLANE. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 < *Value* < 180.0	Specifies the maximum angle, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b

M_TRIANGLE_ANGLE_MIN +	Inquires the minimum interior angle, in degrees, for each mesh triangle. This inquire type is available only when M_MESH_MODE is set to M_MESH_LOCAL_PLANE. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 < *Value* < 180.0	Specifies the minimum angle, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
Normals context ID	Specifies a normals 3D image processing context, allocated using M3dimAlloc() with M_NORMALS_CONTEXT, and used in M3dimNormals() operations.	c		e	f	g	a		i		j	k	l	m	n	b
M_DIRECTION_MODE +	Inquires the orientation of the normal vectors. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_AWAY_FROM_POSITION	Specifies to choose, for each point, the normal vector that points away from the reference point specified with coordinates M_DIRECTION_REFERENCE_X, M_DIRECTION_REFERENCE_Y, and M_DIRECTION_REFERENCE_Z.	c		e	f	g	a		i		j	k	l	m	n	b
M_TOWARDS_DIRECTION	Specifies to choose, for each point, the normal vector that most closely aligns with the reference vector specified with vector components M_DIRECTION_REFERENCE_X, M_DIRECTION_REFERENCE_Y, and M_DIRECTION_REFERENCE_Z.	c		e	f	g	a		i		j	k	l	m	n	b
M_TOWARDS_POSITION	Specifies to choose, for each point, the normal vector that points towards the reference point specified with coordinates M_DIRECTION_REFERENCE_X, M_DIRECTION_REFERENCE_Y, and M_DIRECTION_REFERENCE_Z.	c		e	f	g	a		i		j	k	l	m	n	b
M_DIRECTION_REFERENCE_X +	Inquires the X-coordinate or vector component along X for the reference point or vector (respectively) that determines normal vector orientation. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the X-coordinate or vector component along X.	c		e	f	g	a		i		j	k	l	m	n	b
M_DIRECTION_REFERENCE_Y +	Inquires the Y-coordinate or vector component along Y for the reference point or vector (respectively) that determines normal vector orientation. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the Y-coordinate or vector component along Y.	c		e	f	g	a		i		j	k	l	m	n	b
M_DIRECTION_REFERENCE_Z +	Inquires the Z-coordinate or vector component along Z for the reference point or vector (respectively) that determines normal vector orientation. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the Z-coordinate or vector component along Z.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENFORCE_CONTINUITY +	Inquires whether to maintain the general direction of the normals, which prevents large, discontinuous jumps between normal vectors. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b

M_DISABLE	Specifies to not enforce continuity of normal vectors.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enforce continuity of normal vectors.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAXIMUM_NUMBER_NEIGHBORS +	Inquires the maximum number of neighbors to consider when computing a point's normal vector. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the maximum number of neighbor points.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEIGHBOR_SEARCH_MODE +	Inquires the search mode for finding the neighbors of a point, when calculating a point's surface normal vector. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_MESH	Specifies to use the point cloud's mesh component to determine the neighbors of a point. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ORGANIZED	Specifies to use the point cloud's organizational structure to determine the neighbors of a point. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_TREE	Specifies to use a KD tree search mode to determine a point's neighbors. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_NEIGHBORHOOD_DISTANCE +	Inquires the distance that defines the neighborhood of a point; only points within this distance are considered for normal vector calculations. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_INFINITE	Specifies no distance constraint.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the neighborhood distance limit, in world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEIGHBORHOOD_ORGANIZED_SIZE +	Inquires the neighborhood size when finding the neighbors of a point in an organized point cloud. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 3	Specifies the organized neighborhood size, in number of pixels; the specified value must be an odd number.	c		e	f	g	a		i		j	k	l	m	n	b
Outliers context ID	Specifies an outliers 3D image processing context, allocated using M3dimAlloc() with M_OUTLIERS_CONTEXT, and used in M3dimOutliers() operations.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE_THRESHOLD_MODE +	Inquires the threshold mode for a local distance operation. This inquire type is supported only when M_OUTLIER_MODE is set to M_LOCAL_DISTANCE. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_ROBUST_STD_DEVIATION	Specifies to determine outliers using a threshold based on the median and the mean absolute deviation (MAD) of the local average distance distribution of points in the point cloud. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_STD_DEVIATION	Specifies to determine outliers using a threshold based on the mean and the standard deviation of the local average distance distribution.	c		e	f	g	a		i		j	k	l	m	n	b
M_MINIMUM_NUMBER_NEIGHBORS +		Inquires the minimum number of neighbors that a point must have to be considered an inlier, within the distance that defines the neighborhood. This inquire type is supported only when M_OUTLIER_MODE is set to M_NUMBER_WITHIN_DISTANCE. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the minimum number of neighbors.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEIGHBOR_SEARCH_MODE +		Inquires the search mode for finding the neighbors of a point. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_ORGANIZED	Specifies to use the point cloud's organizational structure to determine the neighbors of a point. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_TREE	Specifies to use a KD tree search mode to determine a point's neighbors.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEIGHBORHOOD_DISTANCE +		Inquires the distance from a point within which other points are considered its neighbors. This inquire type is supported only when M_OUTLIER_MODE is set to M_NUMBER_WITHIN_DISTANCE and M_NEIGHBORHOOD_DISTANCE_MODE is set to M_USER_DEFINED. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the distance.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEIGHBORHOOD_DISTANCE_METRIC +		Inquires how to establish the neighborhood of a point, either as a spherical volume around the point (M_EUCLIDEAN), or as a distance in the Z-direction (M_DIRECTION_Z). SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DIRECTION_Z	Specifies to use a distance in the Z-direction. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_EUCLIDEAN	Specifies to use a Euclidean distance.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEIGHBORHOOD_DISTANCE_MODE +		Inquires the mode for computing a point's neighborhood distance. This inquire type is supported only when M_OUTLIER_MODE is set to M_NUMBER_WITHIN_DISTANCE. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_AUTO	Specifies to automatically compute the neighborhood distance. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
	M_USER_DEFINED	Specifies to use the distance set with M_NEIGHBORHOOD_DISTANCE.	c		e	f	g	a		i		j	k	l	m	n	b

M_NEIGHBORHOOD_ORGANIZED_SIZE +	Inquires the neighborhood size when finding the neighbors of a point in an organized point cloud. This inquire type is supported only when using an organized point cloud and only if M_NEIGHBOR_SEARCH_MODE is set to M_ORGANIZED. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 3	Specifies the organized neighborhood size, in number of pixels; the specified value must be an odd number. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_NEIGHBORS +	Inquires the number of neighboring points used to calculate a point's local average distance distribution. This inquire type is supported only when M_OUTLIER_MODE is set to M_LOCAL_DENSITY_PROBABILITY. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the number of neighbors.	c		e	f	g	a		i		j	k	l	m	n	b
M_OUTLIER_MODE +	Inquires the mode for determining whether a point is an outlier. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_LOCAL_DENSITY_PROBABILITY	Specifies to use the density of neighboring points and a probability calculation to determine whether a point is an outlier. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_LOCAL_DISTANCE	Specifies to use a statistical calculation that considers the average distance of a point to other points in its neighborhood to determine whether the point is an outlier. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_WITHIN_DISTANCE	Specifies to use the number of points within a defined neighborhood to determine whether a point is an outlier. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_PROBABILITY_THRESHOLD_FACTOR +	Inquires the scale factor to use in the probability calculation that determines whether a point is an outlier, when working in the M_LOCAL_DENSITY_PROBABILITY outlier mode. This inquire type is supported only when M_OUTLIER_MODE is set to M_LOCAL_DENSITY_PROBABILITY. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the scale factor.	c		e	f	g	a		i		j	k	l	m	n	b
M_STD_DEVIATION_FACTOR +	Inquires the scale factor to use when calculating the local outlier threshold, when working in the M_LOCAL_DISTANCE outlier mode. When M_DISTANCE_THRESHOLD_MODE is set to M_STD_DEVIATION, it is the factor multiplying the standard deviation of the local average distance distribution. When M_DISTANCE_THRESHOLD_MODE is set to M_ROBUST_STD_DEVIATION, it is the factor multiplying the mean absolute deviation (MAD) of the local average distance distribution. This inquire type is supported only when M_OUTLIER_MODE is set to M_LOCAL_DISTANCE. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

Value > 0.0	Specifies the scale factor.	c		e	f	g	a		i		j	k	l	m	n	b
Profile context ID	Specifies a profile 3D image processing context, allocated using M3dimAlloc() with M_PROFILE_CONTEXT, and used in M3dimProfileEx() operations.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROFILE_ACCUMULATE_TYPE +	Inquires how points in a given sample are accumulated to create a single profile point when there are multiple points within the thickness of the profile line. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX	Specifies to use the largest value of all the points in the sample. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_MEAN	Specifies to use the mean value of all the points in the sample.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN	Specifies to use the smallest value of all the points in the sample. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_PROFILE_INTERPOLATION_AVERAGE_FRACTION +	Inquires the fraction of the sampling distance to use when taking a weighted average of the area surrounding the profile point. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 < *Value* <= 1.0	Specifies the fraction.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROFILE_INTERPOLATION_MODE +	Inquires the interpolation mode used to calculate the value for each point in the sample that will be used to create the profile point. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_AVERAGE	Specifies averaging interpolation. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_BILINEAR	Specifies bilinear interpolation. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_NEAREST_NEIGHBOR	Specifies nearest neighbor interpolation. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_PROFILE_MIN_VALID_PERCENTAGE +	Inquires the minimum percentage of points in a given sample that must be valid to set the corresponding profile point to a valid value. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the percentage of points in the sample. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_PROFILE_SAMPLE_SIZE +	Inquires the sampling distance between two consecutive points along the profile line. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the distance. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_PROFILE_SAMPLE_SIZE_MODE +	Inquires how to interpret the specified sampling distance (M_PROFILE_SAMPLE_SIZE). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_RELATIVE_TO_PIXEL_SIZE_MAX	Specifies to multiply the sampling distance by the pixel size in X or in Y, depending on which is bigger.	c		e	f	g	a		i		j	k	l	m	n	b

M_RELATIVE_TO_PIXEL_SIZE_MIN	Specifies to multiply the sampling distance by the pixel size in X or in Y, depending on which is smaller.	c		e	f	g	a		i		j	k	l	m	n	b
M_RELATIVE_TO_PIXEL_SIZE_X	Specifies to multiply the sampling distance by the pixel size in X. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_RELATIVE_TO_PIXEL_SIZE_Y	Specifies to multiply the sampling distance by the pixel size in Y. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD	Specifies that the sampling distance be interpreted in world units. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_PROFILE_THICKNESS +	Inquires the thickness of the profile line (slicing plane). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the thickness. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_PROFILE_THICKNESS_MODE +	Inquires how to interpret the specified thickness value (M_PROFILE_THICKNESS). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_RELATIVE_TO_PIXEL_SIZE_MAX	Specifies to multiply the thickness value by the pixel size in X or in Y, depending on which is bigger.	c		e	f	g	a		i		j	k	l	m	n	b
M_RELATIVE_TO_PIXEL_SIZE_MIN	Specifies to multiply the thickness value by the pixel size in X or in Y, depending on which is smaller.	c		e	f	g	a		i		j	k	l	m	n	b
M_RELATIVE_TO_PIXEL_SIZE_X	Specifies to multiply the thickness value by the pixel size in X. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_RELATIVE_TO_PIXEL_SIZE_Y	Specifies to multiply the thickness value by the pixel size in Y. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD	Specifies that the thickness value be interpreted in world units. INFO	c		e	f	g	a		i		j	k	l	m	n	b
Project context ID	Specifies a project 3D image processing context, allocated using M3dimAlloc() with M_PROJECT_CONTEXT, and used in M3dimProjectEx() operations.	c		e	f	g	a		i		j	k	l	m	n	b
M_CALCULATE_SIZE_MODE +	Inquires whether the source point cloud container is organized. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_ORGANIZED	Specifies an organized point cloud. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_UNORGANIZED	Specifies an unorganized point cloud.	c		e	f	g	a		i		j	k	l	m	n	b
M_CALIBRATION_ALIGNMENT +	Inquires where to place valid values in the depth map. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_CENTER	Specifies to project the data such that it is centered horizontally or vertically in the buffer. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_TOP_LEFT	Specifies to project the data such that it aligns to the top left of the buffer, and the data extends at least the breadth of one dimension, leaving space at the bottom or right. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_DEPTH_MAP_TYPE +	Inquires the data type and depth required for the destination container's range component. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_UNSIGNED + 8	Specifies 8-bit unsigned data.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNSIGNED + 16	Specifies 16-bit unsigned data.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNSIGNED + 32	Specifies 32-bit unsigned data.	c		e	f	g	a		i		j	k	l	m	n	b
M_OVERLAP_MODE +	Inquires the overlap mode, which determines a pixel's gray level when multiple points correspond to the same destination depth map pixel. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_AVERAGE	Specifies to use the average gray level of all the 3D points projected on a single depth map pixel for custom components. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX_INTENSITY	Specifies to use the gray level of the 3D point with the highest intensity. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX_Z	Specifies to use the gray level of the 3D point with the largest Z-value, of all the 3D points projected on a single depth map pixel. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_Z	Specifies to use the gray level of the 3D point with the smallest Z-value, of all the 3D points projected on a single depth map pixel. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_MOST_FREQUENT	Specifies to use the most frequent gray level for custom components of type M_UNSIGNED or M_SIGNED, or the average gray level for custom components of type M_FLOAT, of all the 3D points projected on a single depth map pixel. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_OVERWRITE	Specifies to use the last gray level of all the 3D points projected on a single depth map pixel. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL_ASPECT_RATIO +	Inquires the pixel aspect ratio of the depth map. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies that other data determines the pixel aspect ratio, such as the values set with M_PIXEL_SIZE_X and M_PIXEL_SIZE_Y.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies an aspect ratio for the depth map's pixels. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL_SIZE_X +	Inquires the length in X of one pixel in the depth map. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies that M3dimProjectEx() will compute the pixel size in X.	c		e	f	g	a		i		j	k	l	m	n	b

	Value > 0.0	Specifies the length in X of one pixel in the depth map, in world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL_SIZE_Y +		Inquires the length in Y of one pixel in the depth map. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_NULL	Specifies that M3dimProjectEx() will compute the pixel size in Y.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the length in Y of one pixel in the depth map, in world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROJECTED_COMPONENTS +		Inquires which components to project into the destination depth map. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_ALL	Specifies to project all components that have the same dimensions as the source container's range component. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_RANGE	Specifies to project the range component.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_RANGE_AND_REFLECTANCE	Specifies to project the range component and the reflectance or intensity component. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_PROJECTION_MODE +		Inquires the projection mode. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_MESH_BASED	Specifies to consider the source container's mesh component when extracting 3D points for projection into the depth map. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_POINT_BASED	Specifies to project the 3D points directly into the depth map.	c		e	f	g	a		i		j	k	l	m	n	b
M_SATURATION +		Inquires whether to saturate pixels. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies not to saturate pixels.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to saturate pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIGN_Z +		Inquires the sign of the Z-scale in the destination depth map. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_NEGATIVE	Specifies a negative value for the Z-scale. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_POSITIVE	Specifies a positive value for the Z-scale. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SIZE_X +		Inquires the size of the depth map along the X dimension. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b

M_NULL	Specifies that M3dimProjectEx() will compute the size of the depth map in X.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the size of the depth map in X, in pixel units.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIZE_Y +	Inquires the size of the depth map along the Y dimension. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies that M3dimProjectEx() will compute the size of the depth map in Y.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the size of the depth map in Y, in pixel units.	c		e	f	g	a		i		j	k	l	m	n	b
Remap context ID	Specifies a remap 3D image processing context, allocated using M3dimAlloc() with M_REMAP_CONTEXT, and used in M3dimRemapDepthMap() operations.	c		e	f	g	a		i		j	k	l	m	n	b
M_DIRECTION_Z +	Inquires the sign of the Z-scale in the destination depth map. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_NEGATIVE	Specifies a negative value for the Z-scale. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITIVE	Specifies a positive value for the Z-scale. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_REVERSE	Specifies that the sign of the Z-scale is opposite that of the source depth map.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAME	Specifies that the sign of the Z-scale is the same as that of the source depth map.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX_Z +	Inquires the maximum Z-value for the destination depth map when M_REMAP_MODE is set to M_USER_DEFINED. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the maximum Z-value.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_Z +	Inquires the minimum Z-value for the destination depth map when M_REMAP_MODE is set to M_USER_DEFINED. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the minimum Z-value.	c		e	f	g	a		i		j	k	l	m	n	b
M_REMAP_MODE +	Inquires how to determine the Z-range of the destination depth map. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_BUFFER_LIMITS	Specifies to remap the source depth map using its M_MIN and M_MAX settings as the values to define the destination's Z-range.	c		e	f	g	a		i		j	k	l	m	n	b
M_DATA_EXTREMES	Specifies to remap the source depth map using its maximum and minimum pixel values to define the destination's Z-range.	c		e	f	g	a		i		j	k	l	m	n	b

	M_GAUSSIAN	Specifies to remap the source depth map using a specified number of standard deviations from the mean Z-value to define the destination's Z-range. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_USE_DESTINATION_CALIBRATION	Specifies to remap the source depth map using the destination image buffer's calibration information to define the destination's Z-range.	c		e	f	g	a		i		j	k	l	m	n	b
	M_USER_DEFINED	Specifies to remap the source depth map using custom values to define the destination's Z range; use M_MAX_Z and M_MIN_Z to set the range.	c		e	f	g	a		i		j	k	l	m	n	b
M_STANDARD_DEVIATION_FACTOR +		Inquires the standard deviation factor, for when M_REMAP_MODE is set to M_GAUSSIAN. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the standard deviation factor.	c		e	f	g	a		i		j	k	l	m	n	b
Statistics context ID		Specifies a statistics 3D image processing context, allocated using M3dimAlloc() with M_STATISTICS_CONTEXT, and used in M3dimStat() operations.	c		e	f	g	a		i		j	k	l	m	n	b
M_BOUNDING_BOX +		Inquires whether to enable bounding box statistics calculations. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies to disable bounding box calculations.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to enable bounding box calculations.	c		e	f	g	a		i		j	k	l	m	n	b
M_BOUNDING_BOX_ALGORITHM +		Inquires the algorithm to use when bounding box calculations have been enabled (M_BOUNDING_BOX). SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_ALL_POINTS	Specifies to compute the axis-aligned bounding box that contains all the valid points.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ROBUST	Specifies to compute an axis-aligned box that contains most of the points, but rejects outliers.	c		e	f	g	a		i		j	k	l	m	n	b
M_BOUNDING_BOX_OUTLIER_RATIO_X +		Inquires the fraction of points to exclude from the bounding box, based on X-coordinates. A point is excluded when its X-coordinate places the point in an outlier position, and therefore outside the bounding box. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* < 1.0	Specifies the fraction.	c		e	f	g	a		i		j	k	l	m	n	b
M_BOUNDING_BOX_OUTLIER_RATIO_Y +		Inquires the fraction of points to exclude from the bounding box, based on Y-coordinates. A point is excluded when its Y-coordinate places the point in an outlier position, and therefore outside the bounding box. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* < 1.0	Specifies the fraction. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_BOUNDING_BOX_OUTLIER_RATIO_Z +	Inquires the fraction of points to exclude from the bounding box, based on Z-coordinates. A point is excluded when its Z-coordinate places the point in an outlier position, and therefore outside the bounding box. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* < 1.0	Specifies the fraction.	c		e	f	g	a		i		j	k	l	m	n	b
M_BOX_ORIENTATION +	Inquires whether to compute an axis-aligned or semi-oriented bounding box. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_AXIS_ALIGNED	Specifies to compute the minimum volume axis-aligned bounding box.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEMI_ORIENTED	Specifies to compute the minimum volume bounding box that is, by default, axis-aligned in Z but not in X and Y. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_BOX_SEMI_ORIENTED_ROTATION_AXIS +	Inquires the axis with which to align the semi-oriented bounding box, when M_BOX_ORIENTATION is set to M_SEMI_ORIENTED. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_AXIS_X	Specifies to align the semi-oriented bounding box with the X-axis.	c		e	f	g	a		i		j	k	l	m	n	b
M_AXIS_Y	Specifies to align the semi-oriented bounding box with the Y-axis.	c		e	f	g	a		i		j	k	l	m	n	b
M_AXIS_Z	Specifies to align the semi-oriented bounding box with the Z-axis.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRINCIPAL_AXIS_1	Specifies to align the semi-oriented bounding box with the first principal axis. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_PRINCIPAL_AXIS_2	Specifies to align the semi-oriented bounding box with the second principal axis. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_PRINCIPAL_AXIS_3	Specifies to align the semi-oriented bounding box with the third principal axis. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_CALCULATE_AVERAGE +	Inquires whether to calculate the average value for distance-to-nearest-neighbor, surface variation, aspect ratio, and number-of-valid-neighbors calculations. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to calculate the average value.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to calculate the average value.	c		e	f	g	a		i		j	k	l	m	n	b
M_CALCULATE_MAX +	Inquires whether to calculate the maximum value for distance-to-nearest-neighbor, surface variation, aspect ratio, and number-of-valid-neighbors calculations. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to calculate the maximum value.	c		e	f	g	a		i		j	k	l	m	n	b

	M_ENABLE	Specifies to calculate the maximum value.	c		e	f	g	a		i		j	k	l	m	n	b
M_CALCULATE_MEDIAN +		Inquires whether to calculate the median value for distance-to-nearest-neighbor, surface variation, aspect ratio, and number-of-valid-neighbors calculations. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies not to calculate the median.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to calculate the median.	c		e	f	g	a		i		j	k	l	m	n	b
M_CALCULATE_MIN +		Inquires whether to calculate the minimum value for distance-to-nearest-neighbor, surface variation, aspect ratio, and number-of-valid-neighbors calculations. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies not to calculate the minimum value.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to calculate the minimum value.	c		e	f	g	a		i		j	k	l	m	n	b
M_CALCULATE_ROBUST_STDEV +		Inquires whether to calculate the robust standard deviation for distance-to-nearest-neighbor, surface variation, aspect ratio, and number-of-valid-neighbors calculations. This is computed using the median absolute deviation (MAD) and a scale factor of 1.4826. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies not to calculate the robust standard deviation.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to calculate the robust standard deviation.	c		e	f	g	a		i		j	k	l	m	n	b
M_CALCULATE_STDEV +		Inquires whether to calculate the standard deviation for distance-to-nearest-neighbor, surface variation, aspect ratio, and number-of-valid-neighbors calculations. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies not to calculate the standard deviation.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to calculate the standard deviation.	c		e	f	g	a		i		j	k	l	m	n	b
M_CENTROID +		Inquires whether to enable centroid statistics calculations. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies to disable centroid calculations.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to enable centroid calculations.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_OF_INTEREST +		Inquires the component for which to calculate statistics. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_NORMALS_AIL	Specifies to calculate the statistics on the normals component. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_COMPONENT_RANGE	Specifies to calculate the statistics on the range component.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE_TO_NEAREST_NEIGHBOR +		Inquires whether to enable distance-to-nearest-neighbor statistics calculations. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies to disable distance-to-nearest-neighbor calculations.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to enable distance-to-nearest-neighbor calculations.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX_DISTANCE +		Inquires the maximum distance used to find neighbors when calculating distance-to-nearest-neighbor, surface variation, aspect ratio, and number-of-valid-neighbors statistics. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_INFINITE	Specifies that there is no maximum distance.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the maximum distance.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAXIMUM_NUMBER_NEIGHBORS +		Inquires the maximum number of neighbors to consider when calculating distance-to-nearest-neighbor, surface variation, aspect ratio, and number-of-valid-neighbors statistics. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the maximum number of neighbors.	c		e	f	g	a		i		j	k	l	m	n	b
M_MOMENT_ORDER +		Inquires the order up to which moments are calculated. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	0 <= *Value* <= 15	Specifies the moment order.	c		e	f	g	a		i		j	k	l	m	n	b
M_MOMENTS +		Inquires whether to enable moments statistics calculations. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies to disable moments calculations.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to enable moments calculations.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEIGHBOR_SEARCH_MODE +		Inquires the search mode for finding the neighbors of a point when calculating distance-to-nearest-neighbor, surface variation, aspect ratio, and number-of-valid-neighbors statistics. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_AUTO	Specifies to automatically set the search mode based on the point cloud's organization, either M_ORGANIZED if organized or M_TREE if unorganized.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ORGANIZED	Specifies to use the point cloud's organizational structure to determine the neighbors of a point. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_TREE	Specifies to use a KD tree search mode to determine a point's neighbors.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEIGHBORHOOD_ASPECT_RATIO +	Inquires whether to enable local aspect ratio statistics calculations. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable local aspect ratio calculations.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable local aspect ratio calculations.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEIGHBORHOOD_ASPECT_RATIO_CLIPPING_VALUE +	Inquires the maximum value of the neighborhood aspect ratio, to avoid skewed statistics. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value > 1.0	Specifies the maximum value.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEIGHBORHOOD_HEALTHY_THRESHOLD +	Inquires the minimum number of neighbors that constitute a healthy neighborhood for the M_NEIGHBORHOOD_HEALTHY_PERCENTAGE result type (M3dimGetResult()). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the number of neighbors.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEIGHBORHOOD_ORGANIZED_SIZE +	Inquires the kernel size to use when using organized search mode for distance-to-nearest-neighbor, surface variation, aspect ratio, and number-of-valid-neighbors calculations. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 3	Specifies the organized neighborhood size, in number of pixels; the specified value must be an odd number. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_NORMALIZATION_MODE +	Inquires whether the normalization matrix should transform the point cloud to fit a signed or unsigned unit box. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_NORMALIZE_SIGNED	Specifies to transform the point cloud or 3D geometry to fit a signed unit box.	c		e	f	g	a		i		j	k	l	m	n	b
M_NORMALIZE_UNSIGNED	Specifies to transform the point cloud or 3D geometry to fit an unsigned unit box.	c		e	f	g	a		i		j	k	l	m	n	b
M_NORMALIZATION_SCALE +	Inquires how to apply scale values when transforming the point cloud or 3D geometry to fit a unit box. If set to M_UNIFORM, each dimension is scaled using the same value such that the largest dimension fits a unit box, and the remaining dimensions will have the same or smaller lengths. If set to M_NON_UNIFORM, scale values are applied such that all dimensions have the same length and fit a unit cube. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_NON_UNIFORM	Applies scale values so that the transformed point cloud or 3D geometry occupies a unit cube.	c		e	f	g	a		i		j	k	l	m	n	b

M_UNIFORM	Applies the same scale value to all dimensions.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_POINTS +	Inquires whether to enable number-of-points statistics calculations. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable number-of-points calculations.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable number-of-points calculations.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_SAMPLES +	Inquires the number of sample points to use when calculating distance-to-nearest-neighbor, surface variation, aspect ratio, and number-of-valid-neighbors statistics. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_INFINITE	Specifies to use all points when calculating distance-to-nearest-neighbor and surface variation statistics.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the number of sample points to use when calculating distance-to-nearest-neighbor and surface variation statistics. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_VALID_NEIGHBORS +	Inquires whether to enable number-of-valid-neighbors statistics calculations. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable number-of-valid-neighbors calculations.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable number-of-valid-neighbors calculations.	c		e	f	g	a		i		j	k	l	m	n	b
M_PCA +	Inquires whether to enable principal component analysis (PCA) statistics calculations. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable PCA calculations.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable PCA calculations.	c		e	f	g	a		i		j	k	l	m	n	b
M_PCA_MODE +	Inquires whether to calculate the principal component analysis (PCA) around the origin (M_ORDINARY) or around the centroid (M_CENTRAL). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_CENTRAL	Specifies to calculate the PCA around the centroid.	c		e	f	g	a		i		j	k	l	m	n	b
M_ORDINARY	Specifies to calculate the PCA around the origin.	c		e	f	g	a		i		j	k	l	m	n	b
M_SURFACE_AREA +	Inquires whether to calculate the surface area of the mesh. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable surface area statistics calculations.	c		e	f	g	a		i		j	k	l	m	n	b

M_ENABLE	Specifies to enable surface area statistics calculations.	c		e	f	g	a		i		j	k	l	m	n	b
M_SURFACE_VARIATION +	Inquires whether to enable surface variation statistics calculations. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable surface variation statistics calculations.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable surface variation statistics calculations.	c		e	f	g	a		i		j	k	l	m	n	b
Stitch context ID	Specifies a stitch 3D image processing context, allocated using M3dimAlloc() with M_STITCH_CONTEXT, and used in M3dimMerge() operations.	c		e	f	g	a		i		j	k	l	m	n	b
M_CALIBRATION_MODE +	Inquires the calibration mode with which to map depth map pixel coordinates to real-world coordinates. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO	Specifies to automatically calibrate the depth maps so that the destination depth map includes all the source depth maps.	c		e	f	g	a		i		j	k	l	m	n	b
M_USER_DEFINED	Specifies to calibrate the depth maps using the values set with M_OFFSET_X and M_OFFSET_Y.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERPOLATION_FALLBACK_MODE +	Inquires the type of interpolation to use when stitching the depth maps and there are invalid pixels in the neighborhood of a point. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERPOLATION	Specifies interpolated resizing.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEAREST_NEIGHBOR	Specifies nearest neighbor interpolation. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_OFFSET_X +	Inquires the X-coordinate of the top-left corner of the destination depth map. This inquire type is only used when M_CALIBRATION_MODE is set to M_USER_DEFINED. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the X-coordinate, in world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_OFFSET_Y +	Inquires the Y-coordinate of the top-left corner of the destination depth map. This inquire type is only used when M_CALIBRATION_MODE is set to M_USER_DEFINED. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the Y-coordinate, in world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_OVERLAP_MODE +	Inquires how to determine the pixel value when two or more depth maps overlap. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_AVERAGE	Specifies to use the average value of the pixels in the overlapping region.	c		e	f	g	a		i		j	k	l	m	n	b

M_MAX	Specifies to use the maximum value of the pixels in the overlapping region.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN	Specifies to use the minimum value of the pixels in the overlapping region.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIZE_X +	Inquires the size of the stitched destination depth map along the X dimension. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the size of the depth map in X, in world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIZE_Y +	Inquires the size of the stitched destination depth map along the Y dimension. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the size of the depth map in Y, in world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_STATIC_INDEX +	Inquires the source depth map to which all other depth map pixels will be stitched. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to use the first depth map in the array.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the index of the depth map to use. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_STITCH_DIRECTION +	Inquires the direction in which to stitch the point clouds' component values. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DIRECTION_X	Specifies to stitch the component values of each successive point cloud to the right of the previous point cloud's component values.	c		e	f	g	a		i		j	k	l	m	n	b
M_DIRECTION_Y	Specifies to stitch the component values of each successive point cloud below the previous point cloud's component values.	c		e	f	g	a		i		j	k	l	m	n	b
M_STITCH_MODE +	Inquires the algorithm to use for stitching depth maps. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_FAST	Specifies that the depth maps will be stitched quickly, but less accurately. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_PRECISE	Specifies that the depth maps will be accurately stitched. INFO	c		e	f	g	a		i		j	k	l	m	n	b
Subsample context ID	Specifies a subsample 3D image processing context, allocated using M3dimAlloc() with M_SUBSAMPLE_CONTEXT, and used in M3dimSample() subsampling operations.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTINCT_ANGLE_DIFFERENCE +	Inquires the minimum angle difference between the normal vector of a point and that of its neighbors, for the point to be kept in the subsample. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

0.0 <= *Value* <= 180.0	Specifies the minimum angle difference, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_FRACTION_OF_POINTS +	Inquires the fraction of all valid points to select from the source point cloud. This only applies when M_SUBSAMPLE_MODE is set to M_SUBSAMPLE_RANDOM or M_SUBSAMPLE_GEOMETRIC. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 < *Value* < 1.0	Specifies the fraction of all valid points to select randomly from the source point cloud.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRID_SIZE_X +	Inquires the cell size in the X-direction. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_INFINITE	Specifies an infinite cell size along X.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the cell size along X, in world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRID_SIZE_Y +	Inquires the cell size in the Y-direction. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_INFINITE	Specifies an infinite cell size along Y.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the cell size along Y, in world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRID_SIZE_Z +	Inquires the cell size in the Z-direction. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_INFINITE	Specifies an infinite cell size along Z.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the cell size along Z, in world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEIGHBORHOOD_DISTANCE +	Inquires the distance that defines the neighborhood of a point; for a point to be kept in the subsample, it must be distinct in this neighborhood. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the neighborhood distance, in world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_ORGANIZATION_TYPE +	Inquires the resulting subsampled point cloud's organizational type. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_ORGANIZED	Specifies an organized resulting subsampled point cloud. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_UNORGANIZED	Specifies an unorganized resulting subsampled point cloud.	c		e	f	g	a		i		j	k	l	m	n	b
M_POINT_SELECTED +	Inquires which point to choose when multiple points occupy the same cell.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b

M_CENTER	Specifies to select the point closest to the center of each 2D cell, as if the point cloud is collapsed onto the plane formed by the 2 non-infinite dimensions.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX	Specifies to select the highest point along the M_INFINITE axis, for each cell. This is the default setting for an M_ORGANIZED destination point cloud.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN	Specifies to select the lowest point along the M_INFINITE axis, for each cell.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEED_VALUE +	Inquires the seed for the random selection of points, for an M_SUBSAMPLE_RANDOM operation. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the seed for the random selection of points.	c		e	f	g	a		i		j	k	l	m	n	b
M_STEP_SIZE_X +	Inquires the step interval between points, along the X dimension. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 1	Specifies the step interval value.	c		e	f	g	a		i		j	k	l	m	n	b
M_STEP_SIZE_Y +	Inquires the step interval between points, along the Y dimension. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 1	Specifies the step interval value.	c		e	f	g	a		i		j	k	l	m	n	b
M_SUBSAMPLE_MODE +	Inquires the type of subsampling operation to perform. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_SUBSAMPLE_DECIMATE	Specifies to subsample using a decimation operation, which selects points at regular intervals. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SUBSAMPLE_GEOMETRIC	Specifies to perform a geometrically stable subsampling, which selects points that help 3D registration converge faster and avoid local minima. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SUBSAMPLE_GRID	Specifies to subsample using a grid operation, which divides the 3D space into cells and, for each subgroup of points, selects a single point per cell. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SUBSAMPLE_NORMAL	Specifies to subsample based on normal vector angles and a neighborhood distance. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SUBSAMPLE_RANDOM	Specifies to subsample using an operation that randomly selects a specified fraction of points from the source point cloud. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SUBSAMPLE_NORMAL_MODE +	Inquires the algorithm to use for subsampling based on normals. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_FAST	Specifies faster but approximate subsampling based on normals. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_PRECISE	Specifies exact subsampling based on normals. INFO	c		e	f	g	a		i		j	k	l	m	n	b
Surface sample context ID	Specifies a surface sample 3D image processing context, allocated using M3dimAlloc() with M_SURFACE_SAMPLE_CONTEXT, and used in M3dimSample() surface sampling operations.	c		e	f	g	a		i		j	k	l	m	n	b
M_RESOLUTION +	Inquires the resolution at which to sample the mesh or 3D geometry. This represents the distance between neighboring points on the surface; the smaller the resolution, the more points generated. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the distance between newly added points on the surface, in world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_MODE +	Inquires the mode of the surface sampling operation. The mode establishes whether the destination point cloud will have a mesh structure. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_SURFACE_GRID	Specifies that surface sampling invalidates the mesh component and the destination container will contain a denser non-meshed point cloud. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_TESSELLATION	Specifies that surface sampling preserves the mesh component and the destination container will contain a denser meshed point cloud.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values of the InquireType parameter listed in For inquiring about a 3D image processing context or result buffer. (exceptions might exist)

You can add one of the following values to the above-mentioned values to get information about the default value of an inquire type, regardless of the current value of the inquire type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about the default value															
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_DEFAULT	Inquires the default value of the specified inquire type.															
UserVarPtr extra info	Data type info: Data type: *defined elsewhere*	c		e	f	g	a		i		j	k	l	m	n	b
M_IS_SET_TO_DEFAULT	Inquires whether the specified inquire type is set to its default value.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the inquire type is not set to its default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the inquire type is set to its default value.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values of the InquireType parameter listed in For inquiring about a 3D image processing context or result buffer.

You can add one of the following values to the above-mentioned values to determine whether an inquire type is supported.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring whether an inquire type is supported															
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

UserVarPtr - Possible values returned															
M_HAS_DEFAULT	Inquires whether the specified inquire type has a default value.														
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the inquire type does not have a default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the inquire type has a default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_SUPPORTED	Inquires whether the specified inquire type is supported.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the inquire type is not supported.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the inquire type is supported.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values of the InquireType parameter listed in For inquiring about a 3D image processing context or result buffer.

You can add one of the following values to the above-mentioned values to cast the requested information to the required data type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the data type																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_TYPE_AIL_DOUBLE	Casts the requested information to an *AIL_DOUBLE*.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_FLOAT	Casts the requested information to an *AIL_FLOAT*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_FLOAT*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT	Casts the requested information to an *AIL_INT*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT32	Casts the requested information to an *AIL_INT32*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT32*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT64	Casts the requested information to an *AIL_INT64*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the requested information, cast to an *AIL_INT64*. If the requested information does not fit into an *AIL_INT64*, this function will return **M_NULL** or truncate the information.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dim11.lib.
DLL	Requires ail11.dll; ail3dim11.dll.

M3dimLattice

Synopsis

Calculate a lattice structure for a container's point cloud.

Syntax

```
void M3dimLattice(
    AIL_ID LatticeContext3dimId,         //in
    AIL_ID ContainerBufId,               //in
    AIL_ID Box3dgeoId,                   //in
    AIL_ID LatticeResultOrContext3dimId, //out
    AIL_INT64 ControlFlag                //in
)
```

Description

This function calculates a lattice that divides points into cells. You can use the calculated lattice to subsample or project the point cloud, where one point is selected per cell, or all points in a cell are projected into the same depth map pixel, respectively.

Use M3dimControl() to set constraints on the lattice before calling this function. For example, you can specify the relative cell dimensions (that is, the sizes of X, Y, and Z relative to each other) using M_RELATIVE_CELL_SIZE_....

Results are stored in the specified lattice 3D image processing result buffer or subsample 3D image processing context. You can retrieve results from the lattice 3D image processing result buffer, using M3dimGetResult().

To use the lattice to subsample the point cloud, specify a subsample 3D image processing context as the destination for results. Alternatively, you can store results in a lattice 3D image processing result buffer, and then copy them into a subsample 3D image processing context, using M3dimCopyResult() with M_SUBSAMPLE_CONTEXT. When you call M3dimSample() with a subsample 3D image processing context containing lattice results, a grid operation is performed where one point is selected per cell in the lattice. Empty cells or cells containing only invalid points are ignored when the resulting point cloud is unorganized, whereas they result in points with 0 confidence when the resulting point cloud is organized.

You can use this function instead of M3dimCalculateMapSize() to calculate a recommended depth map buffer size for projection. M3dimCalculateMapSize() recommends a size such that most points map to their own pixel. You should use **M3dimLattice**() if you want multiple points to map to the same pixel. Use M3dimControl() with M_LATTICE_MODE to specify the mode with which to calculate the lattice. If using M_FRACTION_OF_POINTS, you can specify the required ratio of valid pixels in the depth map to valid points in the point cloud, using M_FRACTION_OF_POINTS and M_FRACTION_OF_POINTS_TOLERANCE. If using M_SPARSITY_CHANGE, you can specify the tolerance to use for identifying sparsity changes in grid sizes, using M_SPARSITY_DIMENSION_TOLERANCE. In this case, when calling **M3dimLattice**(), you must store results in a lattice 3D image processing result buffer, so that you can then retrieve the results to set up an appropriate depth map image buffer or container for M3dimProject(); for more information, see the Generating fully corrected depth and intensity maps section of Chapter 35: 3D image processing. When you call M3dimProject(), all the points in a cell are projected into the same pixel of the depth map image buffer or depth map container. Empty cells or cells containing only invalid points will result in invalid pixels.

Parameters

LatticeContext3dimId

Specifies the identifier of the lattice 3D image processing context. The context must have been previously allocated using M3dimAlloc() with M_LATTICE_CONTEXT.

ContainerBufId

Specifies the identifier of the source container containing a 3D-processable point cloud. The container must have been previously allocated using MbufAllocContainer() with M_PROC, and must be 3D-processable (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE, the function returns M_PROCESSABLE).

Box3dgeoId

Specifies the identifier of the axis-aligned 3D box geometry object that defines the space for which to calculate the lattice. Only points inside the box are used to calculate the lattice. The 3D box geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a box.

To use the source container's axis-aligned bounding box, set this parameter to **M_NULL**. The source container's axis-aligned bounding box is the bounding box that encloses all points in the point cloud and is aligned with the working coordinate system's axes.

LatticeResultOrContext3dimId

Specifies the identifier of the lattice 3D image processing result buffer or subsample 3D image processing context in which to write the results.

For specifying the lattice 3D image processing result buffer or subsample 3D image processing context identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Lattice 3D image processing result buffer identifier	Specifies the identifier of a lattice 3D image processing result buffer, previously allocated using M3dimAllocResult() with M_LATTICE_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
Subsample 3D image processing context identifier	Specifies the identifier of a subsample 3D image processing context, previously allocated using M3dimAlloc() with M_SUBSAMPLE_CONTEXT. This sets the M_SUBSAMPLE_MODE control type of the subsample 3D image processing context to M_SUBSAMPLE_GRID and its M_GRID_SIZE_... control types to the cell sizes of the lattice results (M3dimGetResult() with M_CELL_SIZE_...). If 1 of the cell dimensions is infinite, M_ORGANIZATION_TYPE is set to M_ORGANIZED. Note that this is equivalent to specifying a lattice 3D image processing result buffer and using M3dimCopyResult() with M_SUBSAMPLE_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dim11.lib.
DLL	Requires ail11.dll; ail3dim11.dll.

M3dimMatrixTransform

Synopsis

Apply the specified transformation matrix to the 3D points of a specified point cloud or to a 3D geometry.

Syntax

```
void M3dimMatrixTransform(
    AIL_ID SrcContainerBufOrGeometry3dgeoId, //in
    AIL_ID DstContainerBufOrGeometry3dgeoId, //out
    AIL_ID Matrix3dgeoId,                    //in
    AIL_INT64 ControlFlag                    //in
)
```

Description

This function applies the specified transformation matrix to the 3D points of a specified point cloud or to a 3D geometry.

> Note that, when transforming a point cloud, this function affects the coordinates of the points and not their location in the container.

If an M_COMPONENT_NORMALS_AIL component exists in the source container, the component is recalculated then added to the destination container. All other source components are copied to the destination container. Any previously existing components in the destination container are removed.

Parameters

SrcContainerBufOrGeometry3dgeoId

Specifies the source point cloud container or 3D geometry object.

> Note that the source and destination must match so that both are containers or both are 3D geometries.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the source point cloud container or 3D geometry object																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_XY_PLANE	Specifies the XY (Z=0) plane.	c		e	f	g	a		i		j	k	l	m	n	b
Source 3D geometry object identifier	Specifies the identifier of the source 3D geometry object. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and must have been successfully defined. > Note that sphere, box, and cylinder 3D geometry objects do not accept non-uniform scaling.	c		e	f	g	a		i		j	k	l	m	n	b
Source point cloud container identifier	Specifies the identifier of the source point cloud container. The container must be 3D-processable (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE, the function returns M_PROCESSABLE). The container must have been previously allocated using MbufAllocContainer() with M_PROC.	c		e	f	g	a		i		j	k	l	m	n	b

DstContainerBufOrGeometry3dgeoId

Specifies the destination container or 3D geometry object.

> Note that the source and destination must match so that both are containers or both are 3D geometries.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the destination container or 3D geometry object																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

Destination 3D geometry object identifier	Specifies the identifier of the destination 3D geometry object. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY.	c		e	f	g	a		i		j	k	l	m	n	b
Destination container identifier	Specifies the identifier of the destination container. The container must have been previously allocated using MbufAllocContainer() with M_PROC, and must not be a child container.	c		e	f	g	a		i		j	k	l	m	n	b

Matrix3dgeoId

Specifies the identifier of the transformation matrix object. The transformation matrix object must have been previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX. The transformation matrix must be any valid affine transformation matrix, where the last row is (0,0,0,1). That is, if you call M3dgeoInquire() with M_AFFINE, the function returns M_TRUE.

> Note that, for a 3D geometry of type M_SPHERE, M_BOX, or M_CYLINDER, the transformation can include rotation, translation, and uniform scaling only. That is, if you call M3dgeoInquire() with M_SIMILARITY, the function returns M_TRUE. Other transformation types are not supported.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dim11.lib.
DLL	Requires ail11.dll; ail3dim11.dll.

M3dimMatrixTransformList

Synopsis

Transform a list of 3D coordinates using the specified transformation matrix, and store the resulting coordinates in the specified destination arrays.

Syntax

```
void M3dimMatrixTransformList(
    AIL_ID Matrix3dgeoId,                //in
    AIL_INT NumPoints,                   //in
    const AIL_DOUBLE *SrcCoordXArrayPtr, //in
    const AIL_DOUBLE *SrcCoordYArrayPtr, //in
    const AIL_DOUBLE *SrcCoordZArrayPtr, //in
    AIL_DOUBLE *DstCoordXArrayPtr,       //out
    AIL_DOUBLE *DstCoordYArrayPtr,       //out
    AIL_DOUBLE *DstCoordZArrayPtr,       //out
    AIL_INT64 ControlFlag                //in
)
```

Description

This function transforms a list of 3D coordinates using the specified transformation matrix, and stores the resulting coordinates in the specified destination arrays.

Parameters

Matrix3dgeoId

Specifies the identifier of the transformation matrix object. The transformation matrix object must have been previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX. The transformation matrix must be any valid affine transformation matrix, where the last row is (0,0,0,1). That is, if you call M3dgeoInquire() with M_AFFINE, the function returns M_TRUE.

NumPoints

Specifies the number of points in the arrays.

> When using a standard vector (std::vector) overload function in C++, you can pass **M_DEFAULT** to this parameter; the size will be determined automatically based on the number of items in the vector passed to the SrcCoordXArrayPtr, SrcCoordYArrayPtr, or SrcCoordZArrayPtr parameter.

SrcCoordXArrayPtr
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE>] (and a maximum array rank for C# of 1)
  Required array size:(NumPoints)
- array of type AIL_FLOAT [optionally, in C++: a reference to a constant std::vector<AIL_FLOAT>] (and a maximum array rank for C# of 1)
  Required array size:(NumPoints)

Specifies the address of the array that contains the source X-coordinates, or when ControlFlag is set to M_PACKED, the address of the array that contains a packed set of source X-, Y-, and Z-coordinates.

SrcCoordYArrayPtr
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE>] (and a maximum array rank for C# of 1)
  Required array size:(NumPoints)
- array of type AIL_FLOAT [optionally, in C++: a reference to a constant std::vector<AIL_FLOAT>] (and a maximum array rank for C# of 1)
  Required array size:(NumPoints)

Specifies the address of the array that contains the source Y-coordinates.

This parameter can be set to **M_NULL** when ControlFlag is set to M_PACKED. See the description of M_PACKED for more details.

SrcCoordZArrayPtr
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE>] (and a maximum array rank for C# of 1)

Required array size:(NumPoints)
- array of type AIL_FLOAT [optionally, in C++: a reference to a constant std::vector<AIL_FLOAT>] (and a maximum array rank for C# of 1)
  Required array size:(NumPoints)

Specifies the address of the array that contains the source Z-coordinates.

This parameter can be set to **M_NULL** when ControlFlag is set to M_PACKED. See the description of M_PACKED for more details.

DstCoordXArrayPtr
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE>] (and a maximum array rank for C# of 1)
  Required array size:(NumPoints)
- array of type AIL_FLOAT [optionally, in C++: a reference to a constant std::vector<AIL_FLOAT>] (and a maximum array rank for C# of 1)
  Required array size:(NumPoints)

Specifies the address of the array in which to store the destination X-coordinates, or when ControlFlag is set to M_PACKED, the address of the array in which to store a packed set of destination X-, Y-, and Z-coordinates.

DstCoordYArrayPtr
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE>] (and a maximum array rank for C# of 1)
  Required array size:(NumPoints)
- array of type AIL_FLOAT [optionally, in C++: a reference to a constant std::vector<AIL_FLOAT>] (and a maximum array rank for C# of 1)
  Required array size:(NumPoints)

Specifies the address of the array in which to store the destination Y-coordinates.

This parameter can be set to **M_NULL** when ControlFlag is set to M_PACKED. See the description of M_PACKED for more details.

DstCoordZArrayPtr
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE>] (and a maximum array rank for C# of 1)
  Required array size:(NumPoints)
- array of type AIL_FLOAT [optionally, in C++: a reference to a constant std::vector<AIL_FLOAT>] (and a maximum array rank for C# of 1)
  Required array size:(NumPoints)

Specifies the address of the array in which to store the destination Z-coordinates.

This parameter can be set to **M_NULL** when ControlFlag is set to M_PACKED. See the description of M_PACKED for more details.

ControlFlag

Specifies whether source or destination points are passed and/or returned in a packed format, or specifies to apply only rotation and scale transformations.

> Note that you can combine M_ROTATION_AND_SCALE with M_PACKED or M_PLANAR (that is, you can set the ControlFlag parameter to M_ROTATION_AND_SCALE + M_PACKED or M_ROTATION_AND_SCALE + M_PLANAR).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the control flag																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_PLANAR.	c		e	f	g	a		i		j	k	l	m	n	b

M_PACKED	Specifies that the source points are passed in a packed format and/or that the destination points should be returned in a packed format, depending on what you pass to the SrcCoordYArrayPtr, SrcCoordZArrayPtr, DstCoordYArrayPtr, and DstCoordZArrayPtr parameters. If both SrcCoordYArrayPtr and SrcCoordZArrayPtr are passed **M_NULL**, SrcCoordXArrayPtr is assumed to be a packed set of X-, Y-, and Z-coordinates. If both DstCoordYArrayPtr and DstCoordZArrayPtr are passed **M_NULL**, DstCoordXArrayPtr is filled with a packed set of X-, Y-, and Z-coordinates. If SrcCoordYArrayPtr, SrcCoordZArrayPtr, DstCoordYArrayPtr, and DstCoordZArrayPtr are passed **M_NULL**, SrcCoordXArrayPtr is assumed to be a packed set of X-, Y-, and Z-coordinates, and DstCoordXArrayPtr is filled with a packed set of X-, Y-, and Z-coordinates.	c		e	f	g	a		i		j	k	l	m	n	b
M_PLANAR	Specifies that the source and destination coordinates are stored in a planar format; that is, the X-, Y-, and Z-coordinate values are stored in 3 separate arrays. > When M_PLANAR is specified, no source or destination array pointer parameters can be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
M_ROTATION_AND_SCALE	Specifies to apply only the rotation and scale parts of the transformation to the source points.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dim11.lib.
DLL	Requires ail11.dll; ail3dim11.dll.

M3dimMerge

Synopsis

Merge multiple point clouds or depth maps into a single point cloud or depth map.

Syntax

```
void M3dimMerge(
    const AIL_ID *SrcContainerBufIdArrayPtr, //in
    AIL_ID DstContainerBufId,                //out
    AIL_INT NumContainers,                   //in
    AIL_ID Context3dimId,                    //in
    AIL_INT64 ControlFlag                    //in
)
```

Description

This function merges all the specified point clouds or depth maps into a single point cloud or depth map. Merged point clouds will either be unorganized or organized in a 3D-processable point cloud container, whereas merged depth maps will always be organized in a 3D-processable depth map container.

By default, each depth map will be stitched in fast mode (M_FAST) to the first depth map provided in the array. You can specify to stitch to a different depth map, using M3dimControl() with M_STATIC_INDEX. When M_STITCH_MODE is set to M_FAST, the depth maps are combined quickly but without interpolation, risking an inaccurate stitch. If the depth map pixels are not perfectly aligned with the chosen depth map (for example, the world X-coordinate is 1 for the first pixel in the depth map that you specify with M_STATIC_INDEX, but the world X-coordinate is 3.2 for the first pixel in a depth map to be stitched, and they both have a pixel width of 1 world unit), the stitch will not be accurate in fast mode because its pixels will be forced to shift to align with the pixels of the chosen depth map in the destination. Also, there will be no gray level correction to represent the new position. For example, the intensity value of the destination pixel at (3,0) in the world will be the intensity value of the source pixel at (3.2,0) in the world. You can set M_STITCH_MODE to M_PRECISE so that the destination depth map pixels are assigned interpolated gray levels. In the above example, Aurora Imaging Library will attempt to calculate an intensity value for the destination pixel at (3,0) in the world by taking a weighted average of the four source pixels nearest that point. Note that stitching depth maps in precise mode is slower, but more accurate.

If 2 depth maps overlap in X and/or Y, the pixel value in the resulting depth map will be the average value of the overlapping pixels, by default. You can specify to use the minimum or maximum value instead, using M3dimControl() with M_OVERLAP_MODE.

Optionally, to reduce the number of points in the resulting point cloud, you can specify a subsample 3D image processing context, to apply a subsampling operation to the merged point cloud.

Note that if you specify a stitch or subsample 3D image processing context, the stitch or subsample control settings specified using M3dimControl() are applied.

> Note that after the merge operation, a resulting point cloud is unorganized, unless a stitch context is specified or the optional subsample context specifies an organized resulting point cloud (that is, M3dimControl() with M_ORGANIZATION_TYPE is set to M_ORGANIZED). If the resulting merged point cloud is unorganized, you can still create an organized point cloud from the data. To do so, pass the unorganized point cloud to M3dimSample() along with a subsample 3D image processing context configured using M3dimControl() with M_SUBSAMPLE_MODE set to M_SUBSAMPLE_GRID and M_ORGANIZATION_TYPE set to M_ORGANIZED. Note that in this case, you cannot set M_SUBSAMPLE_MODE to M_SUBSAMPLE_DECIMATE, since this mode is supported only when the source point cloud is organized.

Parameters

SrcContainerBufIdArrayPtr
Accepts the address of one of the following:

- array of type AIL_ID [optionally, in C++: a reference to a constant std::vector<AIL_ID>] (and a maximum array rank for C# of 1)
  Required array size:(NumContainers)

Specifies the address of the array that holds the identifiers of the point cloud containers, depth map containers, or depth map image buffers to merge. You can set some entries in the array to **M_NULL** to exclude them from the merge operation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the array of containers or image buffers																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

Array of depth map container IDs	Specifies an array containing the identifiers of depth map containers. The depth map containers must have been previously allocated using MbufAllocContainer() with M_PROC, and must store data in a 3D-processable depth map format (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE_DEPTH_MAP, the function returns M_TRUE).	c		e	f	g	a		i		j	k	l	m	n	b
Array of depth map image buffer IDs	Specifies an array containing the identifiers of depth map image buffers. The image buffers must be 8-bit, 16-bit, or 32-bit unsigned, 1-band buffers, and must contain a fully corrected depth map (that is, if you call McalInquire() with M_DEPTH_MAP, the function returns M_TRUE).	c		e	f	g	a		i		j	k	l	m	n	b
Array of point cloud container IDs	Specifies an array containing the identifiers of point cloud containers. The point cloud containers must have been previously allocated using MbufAllocContainer() with M_PROC, and must be 3D-processable (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE, the function returns M_PROCESSABLE).	c		e	f	g	a		i		j	k	l	m	n	b

DstContainerBufId

Specifies the identifier of the destination container in which to store the resulting point cloud or depth map. The container must have been previously allocated using MbufAllocContainer() with M_PROC, and must not be a child container.

When merging point clouds, the resulting point cloud will be unorganized, unless the optional subsample context specifies an organized resulting point cloud or the optional stitch context is specified.

NumContainers

Specifies the size of the array passed to SrcContainerBufIdArrayPtr.

> When using a standard vector (std::vector) overload function in C++, you can pass **M_DEFAULT** to this parameter; the size will be determined automatically based on the number of items in the vector passed to the SrcContainerBufIdArrayPtr parameter.

Context3dimId

Specifies the stitch or subsample 3D image processing context identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the stitch or subsample 3D image processing context identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies to ignore this parameter. Note that the resulting merged point cloud will be unorganized. > This value is only available when merging point clouds.	c		e	f	g	a		i		j	k	l	m	n	b
M_STITCH_CONTEXT_DEFAULT	Specifies a predefined stitch 3D image processing context with the default settings. Note that the resulting merged point cloud or depth map will be organized. > This value is only available when merging organized point clouds or depth maps.	c		e	f	g	a		i		j	k	l	m	n	b
Stitch 3D image processing context identifier	Specifies the identifier of a stitch 3D image processing context, previously allocated using M3dimAlloc() with M_STITCH_CONTEXT. Note that M3dimControl() with M_STATIC_INDEX must be set to a value less than the NumContainers parameter. Note that the resulting merged point cloud or depth map will be organized. > This value is only available when merging organized point clouds or depth maps.	c		e	f	g	a		i		j	k	l	m	n	b

Subsample 3D image processing context identifier	Specifies the identifier of a subsample 3D image processing context, previously allocated using M3dimAlloc() with M_SUBSAMPLE_CONTEXT. Note that the organization of the resulting merged point clouds depends on the value of M3dimControl() with M_ORGANIZATION_TYPE. > This value is only available when merging point clouds.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Specifies which components will be affected by the operation. Note that when merging depth map image buffers, this parameter is reserved for future expansion and must be set to M_DEFAULT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying whether to merge points from all components

Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that the operation will only affect the core components of a 3D-processable point cloud or depth map container. These core components are listed in the description of MbufInquireContainer() with M_3D_PROCESSABLE or M_3D_PROCESSABLE_DEPTH_MAP. All other components are copied to the destination container unmodified.	c		e	f	g	a		i		j	k	l	m	n	b
M_APPLY_TO_ALL_COMPONENTS	Specifies that the operation will affect the core components of a 3D-processable point cloud container, as well as any other component (including custom components) that has the same dimension as M_COMPONENT_RANGE. These components must not have a region of interest (ROI) associated with them. For example, using a component that is an image buffer with an ROI will cause an error. > Note that if a custom component has an associated calibration context, the component's information is merged, but its calibration context is discarded. To avoid ambiguity, within each source point cloud container, each component must be unique such that its data type and number of bands are not identical to any other component of the same type in the container. M_APPLY_TO_ALL_COMPONENTS does not support 1-bit buffers. If the resulting operation yields an empty container with 0 valid points, the components with the same dimensions are not created and will not exist in the destination container. > This value is only available when merging point clouds.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dim11.lib.
DLL	Requires ail11.dll; ail3dim11.dll.

M3dimMesh

Synopsis

Create a 3D mesh structure for a point cloud or depth map.

Syntax

```
void M3dimMesh(
    AIL_ID MeshContext3dimId, //in
    AIL_ID SrcContainerBufId, //in
    AIL_ID DstContainerBufId, //out
    AIL_INT64 ControlFlag     //in
)
```

Description

This function generates a triangular mesh structure for the source container's point cloud or depth map, using the specified surface reconstruction mode. The mesh data is stored in an M_COMPONENT_MESH_AIL component, which is added to the destination container.

You can choose the surface reconstruction mode with which to create the mesh, either organized, local plane, or smoothed. Use M3dimControl() with M_MESH_MODE to set the mode, or use this function to specify a predefined context that uses default settings to create the mesh.

The organized surface reconstruction mode (M_MESH_ORGANIZED) requires an organized point cloud or depth map and connects neighboring points to create the mesh. With this mode, invalid data points in the point cloud or depth map result in a mesh with holes. No change is made to the source container's M_COMPONENT_RANGE and M_COMPONENT_CONFIDENCE components; they are simply copied over to the destination container. If the components M_COMPONENT_INTENSITY, M_COMPONENT_NORMALS_AIL, and M_COMPONENT_REFLECTANCE exist in the source container, these are also copied to the destination container.

The local plane reconstruction mode (M_MESH_LOCAL_PLANE) also connects neighboring points to create the mesh; however, it uses local planes computed from each point's neighborhood to establish which neighbors to connect. For each point, neighborhood points are projected to the local plane and the most suitable points are chosen to form a mesh triangle with the point in question. Note that the triangle's vertices are points from the original point cloud, which can be organized or unorganized. With this mode, the mesh can have holes. Components are handled in the same way as for the organized surface reconstruction mode. Note that normal vectors at each 3D point are required (M_COMPONENT_NORMALS_AIL component). Note also that this mode is only available for point clouds.

The smoothed surface reconstruction mode (M_MESH_SMOOTHED) also requires a point cloud and normal vectors at each 3D point (M_COMPONENT_NORMALS_AIL component), and generates a mesh by solving a Poisson equation. This approach generates a smooth surface even when the source point cloud is noisy; missing data is approximated when possible, resulting in significantly fewer holes. Before solving the Poisson equation, an octree is built, which partitions the 3D space for the mesh calculation. The octree's depth determines the mesh's resolution. Note that a higher octree depth gives a higher resolution, but a slower calculation time. Smoothed surface reconstruction generates a new point cloud; new M_COMPONENT_RANGE and M_COMPONENT_CONFIDENCE components are added to the destination container, along with the mesh component.

To remove points from the point cloud whose confidence is equal to zero, as well as points that are not part of the mesh, use M3dimRemovePoints() with M_NOT_MESH_POINTS.

> Note that, when using the smoothed surface reconstruction mode, no M_COMPONENT_NORMALS_AIL component is added to the destination container.

To determine the number of triangular faces in the mesh, inquire the M_COMPONENT_MESH_AIL component using MbufInquireContainer() with M_SIZE_X.

Inquiring the M_SIZE_Y value of the M_COMPONENT_MESH_AIL component returns 3 for an unorganized destination point cloud (3 vertices for each triangular face), and 6 for an organized destination point cloud or depth map (3 vertices with corresponding coordinates in both X and Y).

Parameters

MeshContext3dimId

Specifies a mesh 3D image processing context.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the mesh 3D image processing context identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_MESH_CONTEXT_LOCAL_PLANE	Specifies a predefined mesh 3D image processing context with all mesh context control types (M3dimControl()) set to their default, except for M_MESH_MODE which is set to M_MESH_LOCAL_PLANE. Use this predefined context to create the mesh from local planes that are fit to neighboring points. With the local plane surface reconstruction mode, local planes help determine which points in the source point cloud to connect. With this mode, the mesh can have holes. This value is not available for depth map containers.	c		e	f	g	a		i		j	k	l	m	n	b
M_MESH_CONTEXT_ORGANIZED	Specifies a predefined mesh 3D image processing context with all mesh context control types (M3dimControl()) set to their default, except for M_MESH_MODE which is set to M_MESH_ORGANIZED. Use this predefined context to create the mesh using the organized surface reconstruction mode. With surface reconstruction from an organized point cloud or depth map, neighboring points are connected. Any invalid points will result in a mesh with holes.	c		e	f	g	a		i		j	k	l	m	n	b
M_MESH_CONTEXT_SMOOTHED	Specifies a predefined mesh 3D image processing context with all mesh context control types (M3dimControl()) set to their default, except for M_MESH_MODE which is set to M_MESH_SMOOTHED. Use this predefined context to generate a mesh with a smoothed surface. With the smoothed surface reconstruction mode, the mesh structure is extrapolated from existing points and smoothing is applied to reduce the noise in the point cloud; missing data is approximated when possible, resulting in significantly fewer holes. This mode generates a new point cloud with a smooth surface. This value is not available for depth map containers.	c		e	f	g	a		i		j	k	l	m	n	b
Mesh 3D image processing context identifier	Specifies the identifier of a mesh 3D image processing context, previously allocated using M3dimAlloc() with M_MESH_CONTEXT. If a previously allocated context is specified, the function applies the mesh control settings specified using M3dimControl().	c		e	f	g	a		i		j	k	l	m	n	b

SrcContainerBufId

Specifies the identifier of the 3D-processable point cloud or depth map container to use as the source.

The container must have been previously allocated using MbufAllocContainer() with M_PROC.

The point cloud container must be 3D-processable (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE, the function returns M_PROCESSABLE).

The depth map container must store data in a 3D-processable depth map format (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE_DEPTH_MAP, the function returns M_TRUE).

DstContainerBufId

Specifies the identifier of the destination container, previously allocated using MbufAllocContainer() with M_PROC. The destination container must not be a child container.

The destination container's M_COMPONENT_MESH_AIL component stores the mesh data in an unsigned, 32-bit, 1-band, 2D M_ARRAY buffer.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dim11.lib.
DLL	Requires ail11.dll; ail3dim11.dll.

M3dimNormals

Synopsis

Compute a point cloud's unit normal vectors.

Syntax

```
void M3dimNormals(
    AIL_ID NormalsContext3dimId, //in
    AIL_ID SrcContainerBufId,    //in
    AIL_ID DstContainerBufId,    //out
    AIL_INT64 ControlFlag        //in
)
```

Description

This function computes the unit normal vector for each point in a point cloud, and stores the values in an M_COMPONENT_NORMALS_AIL component, which is added to the destination container. Any previously existing M_COMPONENT_NORMALS_AIL component in the destination container is overwritten.

All other source components are copied to the destination container, and replace any previously existing components.

For some functions that use normal values, how those values were calculated can affect results. For example, when using M3dimFilter() with a bilateral smoothing filter (M_SMOOTH_BILATERAL), the filter operation works best with normals that were calculated using the M_MESH or M_TREE neighbor search mode (M3dimControl() with M_NEIGHBOR_SEARCH_MODE). However, normals created using the M_ORGANIZED mode can give faster filtering results.

Parameters

NormalsContext3dimId

Specifies a normals 3D image processing context.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the normals 3D image processing context identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NORMALS_CONTEXT_MESH	Specifies a predefined normals 3D image processing context with all normals context control types (M3dimControl()) set to their default, except M_NEIGHBOR_SEARCH_MODE which is set to M_MESH. Use this predefined context to determine the neighbors of a point using the point cloud's M_COMPONENT_MESH_AIL component.	c		e	f	g	a		i		j	k	l	m	n	b
M_NORMALS_CONTEXT_ORGANIZED	Specifies a predefined normals 3D image processing context with all normals context control types (M3dimControl()) set to their default, except M_NEIGHBOR_SEARCH_MODE which is set to M_ORGANIZED. Use this predefined context to determine the neighbors of a point using the point cloud's organizational structure.	c		e	f	g	a		i		j	k	l	m	n	b
M_NORMALS_CONTEXT_TREE	Specifies a predefined normals 3D image processing context with all normals context control types (M3dimControl()) set to their default, including M_NEIGHBOR_SEARCH_MODE which is set to M_TREE. Use this predefined context to determine the neighbors of a point using a KD tree search mode.	c		e	f	g	a		i		j	k	l	m	n	b
Normals 3D image processing context identifier	Specifies the identifier of a normals 3D image processing context, previously allocated using M3dimAlloc() with M_NORMALS_CONTEXT. > If a previously allocated context is specified, the function applies the normals control settings specified using M3dimControl().	c		e	f	g	a		i		j	k	l	m	n	b

SrcContainerBufId

Specifies the identifier of the source container containing a 3D-processable point cloud. The container must be 3D-processable (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE, the function returns M_PROCESSABLE).

The container must have been previously allocated using MbufAllocContainer() with M_PROC.

> If M_NEIGHBOR_SEARCH_MODE is set to M_MESH, the component M_COMPONENT_MESH_AIL must exist in the source container.

DstContainerBufId

Specifies the identifier of the destination container, previously allocated using MbufAllocContainer() with M_PROC. The destination container must not be a child container.

The destination's M_COMPONENT_NORMALS_AIL component is a 32-bit, floating-point, 3-band buffer.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dim11.lib.
DLL	Requires ail11.dll; ail3dim11.dll.

M3dimOutliers

Synopsis

Invalidate outlier points in a point cloud or a depth map.

Syntax

```
void M3dimOutliers(
    AIL_ID OutliersContext3dimId,     //in
    AIL_ID SrcContainerOrImageBufId,  //in
    AIL_ID Dst1ContainerOrImageBufId, //out
    AIL_ID Dst2ImageBufId,            //out
    AIL_INT64 ControlFlag             //in
)
```

Description

This function invalidates outlier points in a point cloud or a depth map. An outlier is a point identified as anomalous (for instance, due to noise), typically because it is too distant from neighboring points. Use M3dimControl() to set the outlier mode (M_OUTLIER_MODE) and other options for the operation.

M3dimOutliers() sets the confidence of outlier points to zero; the points will still exist but are invalidated.

You can specify one or two destination objects. If specifying two, the first receives a copy of the source with outlier points invalidated; the second receives a mask image in which pixels that correspond to outlier points are set to a non-zero value, while the rest of the pixels are set to zero. If specifying only one destination object, set the other to **M_NULL**. Depending on the destination parameter that you set, the function will produce either a point cloud (or depth map) with outlier points invalidated, or an outlier mask image.

If you only create a mask image, you can examine the identified outliers before committing to invalidating them in the point cloud or depth map. For example, you can display outliers in a contrasting color and compare different masks created from the same point cloud or depth map. If the outcome is not satisfactory (for example, not enough outliers are identified or some are misidentified), you can adjust the outliers control settings and repeat the operation until the result is acceptable. Once you are satisfied, you can call **M3dimOutliers**() and specify a destination container that you are confident will have the true outlier points invalidated.

Parameters

OutliersContext3dimId

Specifies an outliers 3D image processing context.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the outliers 3D image processing context identifier																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_OUTLIERS_CONTEXT_NUMBER_WITHIN_DISTANCE	Specifies a predefined outliers 3D image processing context with all outliers context control types (M3dimControl()) set to their default, except M_OUTLIER_MODE which is set to M_NUMBER_WITHIN_DISTANCE. Use this predefined context to determine outliers based on the number of neighboring points with the automatically calculated neighborhood distance.	c		e	f	g	a		i		j	k	l	m	n	b
M_OUTLIERS_CONTEXT_PROBABILITY	Specifies a predefined outliers 3D image processing context with all outliers context control types (M3dimControl()) set to their default, except M_OUTLIER_MODE which is set to M_LOCAL_DENSITY_PROBABILITY. Use this predefined context to determine outliers based on the density of neighboring points and a probability calculation.	c		e	f	g	a		i		j	k	l	m	n	b
M_OUTLIERS_CONTEXT_ROBUST_STD_DEVIATION	Specifies a predefined outliers 3D image processing context with all outliers context control types (M3dimControl()) set to their default, including M_DISTANCE_THRESHOLD_MODE which is set to M_ROBUST_STD_DEVIATION. Use this predefined context to determine outliers based on the median and mean absolute deviation (MAD) of the local average distance distribution, which considers the average distance of all points to their closest neighbors. This context uses the default outlier mode (M_LOCAL_DISTANCE).	c		e	f	g	a		i		j	k	l	m	n	b

M_OUTLIERS_CONTEXT_STD_DEVIATION	Specifies a predefined outliers 3D image processing context with all outliers context control types (M3dimControl()) set to their default, except M_DISTANCE_THRESHOLD_MODE which is set to M_STD_DEVIATION. Use this predefined context to determine outliers based on the mean and standard deviation of the local average distance distribution, which considers the average distance of all points to their closest neighbors. This context uses the default outlier mode (M_LOCAL_DISTANCE).	c		e	f	g	a		i		j	k	l	m	n	b
Outliers 3D image processing context identifier	Specifies the identifier of an outliers 3D image processing context, previously allocated using M3dimAlloc() with M_OUTLIERS_CONTEXT. > If a previously allocated context is specified, the function applies the outliers control settings specified using M3dimControl().	c		e	f	g	a		i		j	k	l	m	n	b

SrcContainerOrImageBufId

Specifies the identifier of the source point cloud container, depth map container, or depth map image buffer.

The point cloud container must be 3D-processable (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE, the function returns M_PROCESSABLE), and must have been previously allocated using MbufAllocContainer() with M_PROC.

The depth map container must have been previously allocated using MbufAllocContainer() with M_PROC, and must store data in a 3D-processable depth map format (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE_DEPTH_MAP, the function returns M_TRUE).

The image buffer must be a 1-band, 8-bit, 16-bit, or 32-bit unsigned buffer, and must contain a fully corrected depth map (that is, if you call McalInquire() with M_DEPTH_MAP, the function returns M_TRUE).

Dst1ContainerOrImageBufId

Specifies the identifier of the destination container or depth map image buffer.

The container must have been previously allocated using MbufAllocContainer() with M_PROC, and must not be a child container. The destination will have the same components as the source container, with the confidence component updated accordingly.

The image buffer must have the same dimensions as the buffer specified for SrcContainerOrImageBufId. In addition, the image buffer must be an unsigned, 1-band buffer with the same depth as the source buffer.

If not required, set this parameter to **M_NULL**.

> Note that this parameter and Dst2ImageBufId cannot both be set to null.

> When not set to null, this parameter setting must match the source so that both are containers or both are depth map image buffers.

Dst2ImageBufId

Specifies the identifier of the destination image buffer in which pixels that correspond to outlier points will be set to a non-zero value, creating an outlier mask image. The rest of the pixels are set to zero, indicating inliers or preexisting invalid points in the source object. The image buffer must have the same dimensions as the buffer specified for SrcContainerOrImageBufId. In addition, the image buffer must be an 8-bit, unsigned, 1-band buffer.

If not required, set this parameter to **M_NULL**.

> Note that this parameter and Dst1ContainerOrImageBufId cannot both be set to null.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dim11.lib.
DLL	Requires ail11.dll; ail3dim11.dll.

M3dimProfile

Synopsis

Takes the profile of a depth map, 3D geometry, mesh, or point cloud, along a specified slicing plane.

Syntax

```
void M3dimProfile(
    AIL_ID SrcAilObjectId,      //in
    AIL_ID ProfileResult3dimId, //out
    AIL_INT64 ProfileType,      //in
    AIL_INT64 Param1,           //in
    AIL_DOUBLE Param2,          //in
    AIL_DOUBLE Param3,          //in
    AIL_DOUBLE Param4,          //in
    AIL_DOUBLE Param5,          //in
    AIL_INT64 ControlFlag       //in
)
```

Description

This function takes the profile of a depth map, 3D geometry, mesh, or point cloud, along a specified slicing plane. The specified result buffer is filled with coordinates that make up the profile.

For a depth map profile, pixels are sampled along a specified line and corresponding points are extracted; if the specified line crosses a valid depth map pixel, the corresponding point is included in the profile. The specified line effectively defines a slicing plane for the depth map, since the pixels' intensity values indicate depth (Z-values). You can use M3dimProfileEx() to consider multiple source points for each profile point so that the thickness of the line is taken into consideration. Note that M3dimProfileEx() always extracts a uniform depth map profile, where each point is at an equal distance away from the next along the profile line defined by the start and end points.

For a 3D geometry or mesh profile, 3D points are sampled and extracted along an intersection of the specified slicing plane and the geometry or mesh.

For a point cloud profile, 3D points are sampled and extracted along a specified slicing plane, which is divided into a grid of cells. For each cell, if points exist within the specified cutoff distance (perpendicular to the plane), the point closest to the plane is selected for the profile.

> Note that the profile contains valid points only. No profile points are extracted from gaps in the depth map or invalid points in the source container.

You can retrieve results using M3dimGetResult(). To draw the profile, use M3dimDraw3d(). Alternatively, you can use MgraDots() or M3dgraDots() to draw the profile points.

Parameters

SrcAilObjectId

Specifies the identifier of the source point cloud container, depth map container, 3D geometry object, or depth map image buffer.

The point cloud or depth map container must have been previously allocated using MbufAllocContainer() with M_PROC.

The point cloud container must be 3D-processable (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE, the function returns M_PROCESSABLE).

The depth map container must store data in a 3D-processable depth map format (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE_DEPTH_MAP, the function returns M_TRUE). This container must not have a region component. Using a container with a region component will cause an error.

The geometry object must be successfully defined, and must not have a type of M_LINE or M_POINT.

The image buffer must be an 8-bit, 16-bit, or 32-bit unsigned, 1-band buffer, and must contain a fully corrected depth map (that is, if you call McalInquire() with M_DEPTH_MAP, the function returns M_TRUE). This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

ProfileResult3dimId

Specifies the identifier of the profile 3D image processing result buffer. The result buffer must have been allocated using M3dimAllocResult() with M_PROFILE_RESULT.

ProfileType

Specifies the type of profile to generate.

See the Parameter associations section for possible values that can be specified.

Param1

Specifies the first parameter. Depends on ProfileType.

See the Parameter associations section for possible values that can be specified.

Param2

Specifies the second parameter. Depends on ProfileType.

See the Parameter associations section for possible values that can be specified.

Param3

Specifies the third parameter. Depends on ProfileType.

See the Parameter associations section for possible values that can be specified.

Param4

Specifies the fourth parameter. Depends on ProfileType.

See the Parameter associations section for possible values that can be specified.

Param5

Specifies the fifth parameter. Depends on ProfileType.

See the Parameter associations section for possible values that can be specified.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Parameter associations

The table below lists possible values for the **ProfileType**, **Param1**, **Param2**, **Param3**, **Param4**, and **Param5** parameters.

- For specifying the profile type

Set unused parameters to **M_DEFAULT**.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the profile type																	
ProfileType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Param1																
Param2																
Param3																
Param4																
Param5																
M_PROFILE_DEPTH_MAP	Specifies to take the profile of a depth map. Points are extracted along the specified line. If the line crosses a valid depth map pixel, the corresponding point is included in the profile. Note that invalid pixels (gaps) are ignored. The source object (SrcAilObjectId) must be a depth map container or depth map image buffer.															
Param1	Sets the units with which to interpret the Param2, Param3, Param4, and Param5 parameters, when taking the profile of a depth map.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL	Specifies to interpret the values in pixel units.	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD	Specifies to interpret the values in world units.	c		e	f	g	a		i		j	k	l	m	n	b
Param2	Specifies the X-coordinate of the first point that defines the line.	c		e	f	g	a		i		j	k	l	m	n	b

Param3	Specifies the Y-coordinate of the first point that defines the line.	c		e	f	g	a		i		j	k	l	m	n	b
Param4	Specifies the X-coordinate of the second point that defines the line.	c		e	f	g	a		i		j	k	l	m	n	b
Param5	Specifies the Y-coordinate of the second point that defines the line.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROFILE_GEOMETRY	Specifies to take the profile of a 3D geometry. 3D points are extracted at the intersection of the specified slicing plane and the geometry. Note that the source object (SrcAilObjectId) must be a 3D geometry object, previously allocated using M3dgeoAlloc() and successfully defined as a box, cylinder, plane, or sphere.	c		e	f	g	a		i		j	k	l	m	n	b
Param1	Specifies the identifier of a transformation matrix object that defines the slicing plane along which to sample points for the profile. The transformation matrix object must have been previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.	c		e	f	g	a		i		j	k	l	m	n	b
Param2	Specifies the sampling distance, in world units, between two consecutive points along the profile. The specified distance is either angular or Euclidean, depending on the specified sampling mode (Param4).	c		e	f	g	a		i		j	k	l	m	n	b
Param3	Specifies a limit along the profile's X-dimension.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies no limit along X. Note that, when M_DEFAULT is specified, an included point's X-coordinate can have any value, positive or negative. > This option is not available when taking the profile of a plane.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the length along the X-axis beyond which points are not included for the profile. This means that included points are those whose X-coordinate is between 0 and the specified Param3 value.	c		e	f	g	a		i		j	k	l	m	n	b
Param4	Specifies the sampling mode.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies either M_ANGULAR for 3D geometries of type M_CYLINDER or M_SPHERE, or M_EUCLIDEAN for other 3D geometry object types.	c		e	f	g	a		i		j	k	l	m	n	b
M_ANGULAR	Specifies that the sampling distance is an angular distance, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_EUCLIDEAN	Specifies that the sampling distance is a Euclidean distance.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROFILE_MESH	Specifies to take the profile of a mesh. 3D points are extracted at the intersection of the specified slicing plane and the mesh. Note that the source object (SrcAilObjectId) must be a container with an M_COMPONENT_MESH_AIL component.	c		e	f	g	a		i		j	k	l	m	n	b
Param1	Specifies the identifier of a transformation matrix object that defines the slicing plane along which to sample points for the profile. The transformation matrix object must have been previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX, and the source container of the mesh must have an M_COMPONENT_MESH_AIL component.	c		e	f	g	a		i		j	k	l	m	n	b
Param2	Specifies the sampling distance, in world units, along the specified slicing plane, in the X-direction of the plane coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
Param3	Specifies a limit along the profile's X-dimension.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies no limit along X. Note that, when M_DEFAULT is specified, an included point's X-coordinate can have any value, positive or negative.	c		e	f	g	a		i		j	k	l	m	n	b

		Value > 0	Specifies the length along the X-axis beyond which points are not included for the profile. This means that included points are those whose X-coordinate is between 0 and the specified Param3 value.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROFILE_POINT_CLOUD			Specifies to take the profile of a point cloud. 3D points are extracted along the specified slicing plane, and within a specified distance from the plane. Invalid points in the point cloud are ignored. The source object (SrcAilObjectId) must be a container.	c		e	f	g	a		i		j	k	l	m	n	b
	Param1		Specifies the identifier of a transformation matrix object that defines the slicing plane along which to sample points for the profile. The transformation matrix object must have been previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX. > You can use M3dgeoMatrixSetWithAxes() with M_COORDINATE_SYSTEM_TRANSFORMATION to generate the required transformation matrix.	c		e	f	g	a		i		j	k	l	m	n	b
	Param2		Specifies the sampling distance, in world units, along the specified slicing plane, in the X-direction of the plane coordinate system. Param2 and Param3 define a grid of cells along the slicing plane. For each cell, if points exist within the cutoff distance (Param4), the point closest to the slicing plane is selected for the profile.	c		e	f	g	a		i		j	k	l	m	n	b
	Param3		Specifies the sampling distance, in world units, along the specified slicing plane, in the Y-direction of the plane coordinate system. Param2 and Param3 define a grid of cells along the slicing plane. For each cell, if points exist within the cutoff distance (Param4), the point closest to the slicing plane is selected for the profile.	c		e	f	g	a		i		j	k	l	m	n	b
	Param4		Specifies the cutoff distance (perpendicular to, and on either side of the specified slicing plane), in world units, beyond which points will not be considered for the profile. For example, if 5 is specified, only points within 5 units above and 5 units below the slicing plane are considered.	c		e	f	g	a		i		j	k	l	m	n	b
	Param5		Specifies a limit along the profile's X-dimension.	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Specifies no limit along X. Note that, when M_DEFAULT is specified, an included point's X-coordinate can have any value, positive or negative.	c		e	f	g	a		i		j	k	l	m	n	b
		Value > 0	Specifies the length along the X-axis beyond which points are not included for the profile. This means that included points are those whose X-coordinate is between 0 and the specified Param5 value.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dim11.lib.
DLL	Requires ail11.dll; ail3dim11.dll.

M3dimProfileEx

Synopsis

Takes the uniform profile of a depth map along a specified slicing plane according to the settings of the profile 3D image processing context.

Syntax

```
void M3dimProfileEx(
    AIL_ID ProfileContext3dimId,          //in
    AIL_ID SrcDepthBufId,                 //in
    AIL_ID DstContainerBufOrResult3dimId, //out
    AIL_INT64 Param1,                     //in
    AIL_DOUBLE Param2,                    //in
    AIL_DOUBLE Param3,                    //in
    AIL_DOUBLE Param4,                    //in
    AIL_DOUBLE Param5,                    //in
    AIL_INT64 ControlFlag                 //in
)
```

Description

This function extracts a uniform depth map profile along a specified slicing plane. The specified destination container or result buffer is filled with coordinates that make up the profile. You can use M3dimControl() to specify settings, such as the thickness of the profile and the interpolation mode to use.

Pixels are sampled along a specified line and corresponding points are considered for the profile. The specified line effectively defines a slicing plane for the depth map, since the pixels' intensity values indicate depth (Z-values). In M3dimProfile(), if the specified line crosses a valid depth map pixel, the corresponding point is included in the profile. Whereas, **M3dimProfileEx**() can sample multiple source pixels for each profile point so that the thickness of the line is taken into consideration; you can use M3dimControl() with M_PROFILE_THICKNESS to specify a thickness perpendicular to the profile line, and all points corresponding to pixels within the specified thickness of the line will be considered when determining the value of the profile point.

This function always uniformly extracts points along the profile line defined by the start and end points; pixels are sampled along the line using a specified sampling distance (M3dimControl() with M_PROFILE_SAMPLE_SIZE) so that each point in the profile is at an equal distance away from the next.

> Note that when the destination is a result buffer, the profile contains valid points only; no profile points are extracted from gaps in the depth map. Invalid points can exist in the profile only when the destination is a container.

You can retrieve results using M3dimGetResult(). To draw the profile, use M3dimDraw3d(). Alternatively, you can use MgraDots() or M3dgraDots() to draw the profile points.

Parameters

ProfileContext3dimId

Specifies the identifier of the profile 3D image processing context. The context must have been allocated using M3dimAlloc() with M_PROFILE_CONTEXT.

SrcDepthBufId

Specifies the identifier of the source depth map container or depth map image buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the source depth map container or depth map image buffer identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Depth map container identifier	Specifies the identifier of a depth map container. The container must have been previously allocated using MbufAllocContainer() with M_PROC, and must store data in a 3D-processable depth map format (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE_DEPTH_MAP, the function returns M_TRUE). The container must not have a region component. Using a container with a region component will cause an error.	c		e	f	g	a		i		j	k	l	m	n	b

Depth map image buffer identifier	Specifies the identifier of a depth map image buffer. The image buffer must be an 8-bit, 16-bit, or 32-bit unsigned, 1-band buffer, and must contain a fully corrected depth map (that is, if you call McalInquire() with M_DEPTH_MAP, the function returns M_TRUE). The image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.	c		e	f	g	a		i		j	k	l	m	n	b

DstContainerBufOrResult3dimId

Specifies the identifier of the destination container or profile 3D image processing result buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the destination container or profile result buffer identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Container identifier	Specifies the identifier of a container. The container must have been previously allocated using MbufAllocContainer() with M_PROC, and must not be a child container.	c		e	f	g	a		i		j	k	l	m	n	b
Profile 3D image processing result buffer identifier	Specifies the identifier of a profile 3D image processing result buffer. The result buffer must have been previously allocated using M3dimAllocResult() with M_PROFILE_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b

Param1

Specifies the first parameter, which determines the units with which to interpret the Param2, Param3, Param4, and Param5 parameters.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the input units																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_PIXEL	Specifies to interpret the values in pixel units.	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD	Specifies to interpret the values in world units.	c		e	f	g	a		i		j	k	l	m	n	b

Param2

Specifies the X-coordinate of the first point that defines the line along which to extract points.

Param3

Specifies the Y-coordinate of the first point that defines the line along which to extract points.

Param4

Specifies the X-coordinate of the second point that defines the line along which to extract points.

Param5

Specifies the Y-coordinate of the second point that defines the line along which to extract points.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dim11.lib.
DLL	Requires ail11.dll; ail3dim11.dll.

M3dimProject

Synopsis

Project a point cloud or 3D geometry onto the XY (Z=0) plane to generate a fully corrected depth map.

Syntax

```
void M3dimProject(
    AIL_ID SrcContainerBufOrGeometry3dgeoId, //in
    AIL_ID DepthMapImageBufId,               //out
    AIL_ID IntensityMapImageBufId,           //out
    AIL_INT64 ProjectionMode,                //in
    AIL_INT64 OverlapMode,                   //in
    AIL_INT64 Options,                       //in
    AIL_INT64 ControlFlag                    //in
)
```

Description

This function projects a point cloud or a 3D geometry onto the XY (Z=0) plane, generating a fully corrected depth map. A depth map pixel's gray level indicates depth. **M3dimProject**() can also generate an intensity map image from the source container, if an appropriate component is available (M_COMPONENT_REFLECTANCE and/or M_COMPONENT_INTENSITY). An intensity map pixel's gray level indicates the luminosity of the original scene at the location represented by the corresponding pixel in the depth map.

M3dimProject() projects the points of the point cloud on the XY (Z=0) plane of its working coordinate system. To change the resulting depth map, use M3dimRotate() or M3dimMatrixTransform() to rotate or transform (respectively) the 3D points before calling **M3dimProject**().

To define the region of the point cloud or 3D geometry to project, the destination buffer's size and camera calibration information are used to establish a bounding box. You can use M3dimCalculateMapSize() to establish the required destination depth map buffer size in X and Y, and M3dimCalibrateDepthMap() to calibrate the buffer. Note that M3dimCalculateMapSize() recommends a size such that most points map to their own pixel with the greatest ratio of valid pixels to invalid pixels.

If you want multiple points to map to the same pixel, you can use M3dimLattice() to establish the required destination depth map buffer size in X and Y (M3dimGetResult() with M_NUMBER_OF_CELLS_...), and, if the destination is a depth map image buffer, McalUniform() to manually calibrate the image buffer. If the destination is a depth map container, use MbufControlContainer() to specify the required offsets and scales to produce natively calibrated coordinates. Use M3dimGetResult() with M_START_POINT_... and M_CELL_SIZE_... to specify the translation and scale of the transformation between pixel and world units. For more information, see the Steps to create a depth map subsection of the Generating fully corrected depth and intensity maps section of Chapter 35: 3D image processing.

If you want to limit which points are considered for the projection, prior to calling **M3dimProject**(), you can crop or mask the point cloud (using M3dimCrop() or changing the data in the source container's confidence component). For more information, see the Regions of interest subsection of the Working with 3D section of Chapter 2: Key information and building an application.

Note that, when generating an intensity map, the source container's reflectance or intensity component provides the color or intensity for pixels in the destination image. If both components exist, only the reflectance component is considered. You cannot generate an intensity map when projecting a 3D geometry.

Parameters

SrcContainerBufOrGeometry3dgeoId

Specifies a source container or 3D geometry object.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the source container or 3D geometry object identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Source 3D geometry object identifier	Specifies the identifier of a 3D geometry object, except that of a box geometry object. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and must have been successfully defined. Supported 3D geometries include cylinder, line, plane, point, and sphere.	c		e	f	g	a		i		j	k	l	m	n	b
Source container identifier	Specifies the identifier of the source container. The container must be 3D-processable (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE, the function returns M_PROCESSABLE). The container must have been previously allocated using MbufAllocContainer() with M_PROC.	c		e	f	g	a		i		j	k	l	m	n	b

DepthMapImageBufId

Specifies the identifier of the destination depth map container or depth map image buffer, previously allocated using MbufAllocContainer() or MbufAlloc2d(). To establish the required destination depth map buffer size in X and Y, use M3dimCalculateMapSize() or M3dimLattice().

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the depth map																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Destination depth map container identifier	Specifies the identifier of the destination depth map container. The destination container must have been previously allocated using MbufAllocContainer() with M_PROC, must store data in a 3D-processable depth map format (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE_DEPTH_MAP, the function returns M_TRUE), and must not be a child container. See the Steps to create a depth map subsection of the Generating fully corrected depth and intensity maps section of Chapter 35: 3D image processing to set up such a container.	c		e	f	g	a		i		j	k	l	m	n	b
Destination depth map image buffer identifier	Specifies the identifier of the destination depth map image buffer. The destination image buffer must be fully corrected (that is, if you call McalInquire() with M_DEPTH_MAP, the function returns M_TRUE). This means that the buffer must be associated with a calibration context and have a constant pixel size in X and Y; it must also have a valid Z-scale. To establish the required calibration, use M3dimCalibrateDepthMap(), or use McalUniform() to calibrate the buffer and McalControl() with M_GRAY_LEVEL_SIZE_Z to set the Z-scale. Note that, depending on the Z-scale and offset of the destination image buffer, a point's Z-value can place it outside the buffer's range of possible values. By default, the point is ignored, but saturation to either 0 or the buffer's maximum value (minus 1) is possible with the Options parameter set to M_SATURATION. The destination image buffer must be a 1-band, 8-bit, 16-bit or 32-bit unsigned buffer.	c		e	f	g	a		i		j	k	l	m	n	b

IntensityMapImageBufId

Specifies the identifier of the destination intensity map image buffer. The image buffer must have the same dimensions as the buffer specified for DepthMapImageBufId. In addition, the image buffer must be an 8-bit or 16-bit unsigned, 1-band or 3-band buffer. The number of bands must be equal to that of the component M_COMPONENT_REFLECTANCE or M_COMPONENT_INTENSITY. If the container has neither M_COMPONENT_REFLECTANCE nor M_COMPONENT_INTENSITY components, or if projecting a 3D geometry, this parameter must be set to **M_NULL**.

ProjectionMode

Specifies the projection mode.

You must set this parameter to M_DEFAULT if the source is a 3D geometry object.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the projection mode																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_POINT_BASED.	c		e	f	g	a		i		j	k	l	m	n	b
M_MESH_BASED	Specifies to consider the source container's mesh component when extracting 3D points for projection into the depth map. For a meshed point cloud, each triangular face has 3 vertices, and a vertex's Z-value determines the gray level of its corresponding pixel in the depth map. The remaining pixels (in the depth map) are assigned interpolated gray levels, since they correspond to the triangular faces between the vertices (in the point cloud's mesh). The container must have an M_COMPONENT_MESH_AIL component before using the M_MESH_BASED mode.	c		e	f	g	a		i		j	k	l	m	n	b
M_POINT_BASED	Specifies to project the 3D points directly into the depth map.	c		e	f	g	a		i		j	k	l	m	n	b

OverlapMode

Specifies the overlap mode, which determines a pixel's gray level when multiple points correspond to the same destination depth map pixel.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the overlap mode

Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_MAX_Z.	c		e	f	g	a		i		j	k	l	m	n	b
M_AVERAGE	Specifies to use the average gray level of all the 3D points projected on a single depth map pixel.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX_INTENSITY	Specifies to use the gray level of the 3D point with the highest intensity. The source container must have an intensity or reflectance component. Note that M_MAX_INTENSITY is not available when the source object is a 3D geometry.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX_Z	Specifies to use the gray level of the 3D point with the largest Z-value, of all the 3D points projected on a single depth map pixel. Note that this value can be bright or dark, depending on whether the Z-scale (M_GRAY_LEVEL_SIZE_Z) is positive or negative. If you calibrated the buffer using M3dimCalibrateDepthMap(), the sign of the Z-scale depends on whether M3dimCalibrateDepthMap() with ZSign was set to M_POSITIVE or M_NEGATIVE.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_Z	Specifies to use the gray level of the 3D point with the smallest Z-value, of all the 3D points projected on a single depth map pixel. Note that this value can be dark or bright, depending on whether the Z-scale (M_GRAY_LEVEL_SIZE_Z) is positive or negative. If you calibrated the buffer using M3dimCalibrateDepthMap(), the sign of the Z-scale depends on whether M3dimCalibrateDepthMap() with ZSign was set to M_POSITIVE or M_NEGATIVE.	c		e	f	g	a		i		j	k	l	m	n	b
M_OVERWRITE	Specifies to use the last gray level of all the 3D points projected on a single depth map pixel. This mode continually overwrites the previous gray value for the pixel. M_OVERWRITE can be marginally faster than the other overlap modes.	c		e	f	g	a		i		j	k	l	m	n	b

Options

Specifies additional options.

> Note that you can combine the 2 values M_ACCUMULATE and M_SATURATION (that is, you can set the Options parameter to M_ACCUMULATE + M_SATURATION).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the option																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies no saturation; points falling outside the depth map image buffer's grayscale range are ignored.	c		e	f	g	a		i		j	k	l	m	n	b
M_ACCUMULATE	Specifies not to clear the previous contents of the depth map image buffer prior to projection.	c		e	f	g	a		i		j	k	l	m	n	b
M_SATURATION	Specifies saturation. Note that saturation can occur, depending on the Z-scale and offset of the destination depth map buffer. If, for a given pixel of the destination buffer, the corresponding gray level of the 3D geometry or point of the point cloud falls outside the buffer's grayscale range, the pixel will be saturated to either 0 or the buffer's maximum value, minus 1 (the maximum value is reserved to represent missing data).	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dim11.lib.
DLL	Requires ail11.dll; ail3dim11.dll.

M3dimProjectEx

Synopsis

Project a point cloud or 3D geometry onto the XY (Z=0) plane to generate a fully corrected depth map according to the settings of the project 3D image processing context.

Syntax

```
void M3dimProjectEx(
    AIL_ID ProjectContext3dimId,             //in
    AIL_ID SrcContainerBufOrGeometry3dgeoId, //in
    AIL_ID AABox3dgeoId,                     //in
    AIL_ID DstDepthMapBufId,                 //in
    AIL_INT64 Options,                       //in
    AIL_INT64 ControlFlag                    //in
)
```

Description

This function projects a point cloud or a 3D geometry onto the XY (Z=0) plane, generating a fully corrected depth map. A depth map pixel's gray level indicates depth. You can use M3dimControl() to specify settings, such as the pixel aspect ratio and projection mode.

This function can be used instead of M3dimCalculateMapSize(), M3dimCalibrateDepthMap(), and M3dimProject(). If you are projecting into a container, **M3dimProjectEx**() can optionally allocate an M_COMPONENT_RANGE component or change the dimensions of a previously existing M_COMPONENT_RANGE component and assign the 3D properties required to produce natively calibrated coordinates before projecting. If you are projecting into an image buffer, **M3dimProjectEx**() can be used instead of M3dimCalibrateDepthMap() and M3dimProject() to calibrate the destination image buffer and then project.

You can optionally specify a region of the point cloud to project using an axis-aligned 3D box geometry and M_FIT_BOX or M_USE_BOX_WORLD_SIZE.

Note that **M3dimProjectEx**() can also project intensity values from the source container's M_COMPONENT_REFLECTANCE or M_COMPONENT_INTENSITY component, if one exists. Intensity values are projected into the destination container's M_COMPONENT_REFLECTANCE or M_COMPONENT_INTENSITY component (respectively), which is created if not previously existing in the destination container. If both components exist in the source container, only the reflectance component is considered.

M3dimProjectEx() projects the points of the point cloud on the XY (Z=0) plane of its working coordinate system. To change the resulting depth map, use M3dimRotate() or M3dimMatrixTransform() to rotate or transform (respectively) the 3D points before calling **M3dimProjectEx**().

If you want to limit which points are considered for the projection, prior to calling **M3dimProjectEx**(), you can crop or mask the point cloud (using M3dimCrop() or changing the data in the source container's confidence component). For more information, see the Regions of interest subsection of the Working with 3D section of Chapter 2: Key information and building an application.

Parameters

ProjectContext3dimId

Specifies a project 3D image processing context.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the project 3D image processing context identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default project 3D image processing context of the current Aurora Imaging Library application. > The operation will use default values for all project context control types (M3dimControl()).	c		e	f	g	a		i		j	k	l	m	n	b
Project 3D image processing context identifier	Specifies the identifier of a project 3D image processing context, previously allocated using M3dimAlloc() with M_PROJECT_CONTEXT. > If a project context is specified, the function applies the project control settings specified using M3dimControl().	c		e	f	g	a		i		j	k	l	m	n	b

SrcContainerBufOrGeometry3dgeoId

Specifies a source container or 3D geometry object to project.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the source container or 3D geometry object identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Source 3D geometry object identifier	Specifies the identifier of a 3D geometry object, except that of a box geometry object. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and must have been successfully defined. Supported 3D geometries include cylinder, line, plane, point, and sphere.	c		e	f	g	a		i		j	k	l	m	n	b
Source container identifier	Specifies the identifier of the source container. The container must be 3D-processable (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE, the function returns M_PROCESSABLE). The container must have been previously allocated using MbufAllocContainer() with M_PROC.	c		e	f	g	a		i		j	k	l	m	n	b

AABox3dgeoId

Specifies the identifier of the axis-aligned 3D box geometry object to use to limit the calculation to points within the box. For M_FIT_BOX and M_USE_BOX_WORLD_SIZE, the box is also used to define the calibration region. The 3D box geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a box.

If a 3D box geometry object is not required, set this parameter to **M_NULL**.

DstDepthMapBufId

Specifies the identifier of the destination image buffer or container in which to store the depth map.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the depth map																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Destination container identifier	Specifies the identifier of the destination container. The destination container must have been previously allocated using MbufAllocContainer() with M_PROC, and must not be a child container. If you pass M_USE_DESTINATION to the Options parameter, the container must store data in a 3D-processable depth map format (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE_DEPTH_MAP, the function returns M_TRUE). If you pass M_FIT_BOX or M_FIT_SRC_DATA to the Options parameter, an M_COMPONENT_RANGE component must exist in the container.	c		e	f	g	a		i		j	k	l	m	n	b
Destination image buffer identifier	Specifies the identifier of the destination image buffer. The destination image buffer must be a 1-band, 8-bit, 16-bit or 32-bit unsigned image buffer. If you pass M_USE_DESTINATION to the Options parameter, the image buffer must contain a fully corrected depth map (that is, if you call McalInquire() with M_DEPTH_MAP, the function returns M_TRUE).	c		e	f	g	a		i		j	k	l	m	n	b

Options

Specifies additional options for the destination.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the option																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO	Specifies to automatically choose the option. If the source is a point cloud and the destination is a container, M_AUTO is the same as M_USE_SRC_DATA. If the source is a point cloud and the destination is an image buffer, or if the source is a finite geometry, M_AUTO is the same as M_FIT_SRC_DATA. If the source is an infinite geometry and an axis-aligned box is provided, M_AUTO is the same as M_FIT_BOX. If the source is an infinite geometry and no axis-aligned box is provided, M_AUTO is the same as M_USE_DESTINATION.	c		e	f	g	a		i		j	k	l	m	n	b

M_FIT_BOX	Specifies to reuse the X-size and Y-size of the destination image buffer or container's range component and to choose the scales and offsets so that the axis-aligned box fits inside the destination depth map. This option is not available when the AABox3dgeoId parameter is set to **M_NULL** or the destination is a container that does not have an M_COMPONENT_RANGE component.	c		e	f	g	a		i		j	k	l	m	n	b
M_FIT_SRC_DATA	Specifies to reuse the X-size and Y-size of the destination image buffer or container's range component and to choose the scales and offsets so that the source data fits inside the destination depth map. Note that if an axis-aligned box is provided, only points within the box are considered. This option is not available when the source is an infinite geometry or the destination is a container that does not have an M_COMPONENT_RANGE component.	c		e	f	g	a		i		j	k	l	m	n	b
M_USE_BOX_WORLD_SIZE	Specifies to use the axis-aligned box to determine the X-size, Y-size, scales, and offsets of the destination container's range component. Note that the points within the box are used to determine the destination pixel sizes. This option is not available when the AABox3dgeoId parameter is set to **M_NULL**, the source is a geometry, or the destination is an image buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_USE_DESTINATION +	Specifies to reuse the X-size, Y-size, and calibration information of the destination image buffer or 3D properties of the destination container's range component. This option is only available when the destination is a fully corrected depth map image buffer or a 3D-processable depth map container.	c		e	f	g	a		i		j	k	l	m	n	b
M_USE_SRC_DATA	Specifies to use the source data to determine the X-size, Y-size, scales, and offsets of the destination container's range component. Note that if an axis-aligned box is provided, only points within the box are considered. This option is not available when the source is a geometry or the destination is an image buffer.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_USE_DESTINATION.

You can add the following value to the above-mentioned value to specify to accumulate the data.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For accumulating data																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ACCUMULATE	Specifies not to clear the previous contents of the destination depth map prior to projection. This is useful to project additional data.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dim11.lib.
DLL	Requires ail11.dll; ail3dim11.dll.

M3dimRemapDepthMap

Synopsis

Remap a depth map to fit within a specified data range, or convert the depth map to a different bit depth.

Syntax

```
void M3dimRemapDepthMap(
    AIL_ID RemapContext3dimId, //in
    AIL_ID SrcImageBufId,      //in
    AIL_ID DstImageBufId,      //out
    AIL_INT64 ControlFlag      //in
)
```

Description

This function linearly remaps source pixels to destination pixels according to the specified mode, which determines the range of Z-values (Z-range) for the destination depth map. Use M3dimControl() to set the remap mode (M_REMAP_MODE) and other remap options. Note that the remap operation maintains the same calibration in X and Y as the source buffer.

You can use **M3dimRemapDepthMap**() to remap a depth map to a different data type. This allows you to pass the remapped depth map to functions that only accept image buffers of certain bit depths (for example, MmodFind()).

You can limit this function's results to a region of an image buffer using a region of interest (ROI) set using MbufSetRegion(). The ROI must be defined in raster format (M_RASTER or M_VECTOR_AND_RASTER). An error is generated if the ROI is only in vector format (M_VECTOR). If both the source and the destination image buffers have an ROI, results are limited to the portion of the ROIs that intersect.

Parameters

RemapContext3dimId

Specifies a remap 3D image processing context.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the remap 3D image processing context identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_REMAP_CONTEXT_BUFFER_LIMITS	Specifies a predefined remap 3D image processing context with all remap context control types (M3dimControl()) set to their default, including M_REMAP_MODE, which is set to M_BUFFER_LIMITS. Use this predefined context to remap the source depth map using its M_MIN and M_MAX as the values to define the destination's Z-range.	c		e	f	g	a		i		j	k	l	m	n	b
M_REMAP_CONTEXT_DATA_EXTREMES	Specifies a predefined remap 3D image processing context with all remap context control types (M3dimControl()) set to their default, except for M_REMAP_MODE, which is set to M_DATA_EXTREMES. Use this predefined context to remap the source depth map using its minimum and maximum pixel values to define the destination's Z-range.	c		e	f	g	a		i		j	k	l	m	n	b
M_REMAP_CONTEXT_USE_DESTINATION	Specifies a predefined remap 3D image processing context with all remap context control types (M3dimControl()) set to their default, except for M_REMAP_MODE, which is set to M_USE_DESTINATION_CALIBRATION. Use this predefined context to remap the source depth map using the destination image buffer's calibration information or destination container's scale, offset, and matrix information to define the destination's Z-range.	c		e	f	g	a		i		j	k	l	m	n	b
Remap 3D image processing context identifier	Specifies the identifier of a remap 3D image processing context, previously allocated using M3dimAlloc() with M_REMAP_CONTEXT. > In this case, the function applies the remap control settings specified using M3dimControl().	c		e	f	g	a		i		j	k	l	m	n	b

SrcImageBufId

Specifies the identifier of the source depth map image buffer or depth map container.

The image buffer must be a 1-band, 8-bit, 16-bit, or 32-bit unsigned buffer, and must contain a fully corrected depth map (that is, if you call McalInquire() with M_DEPTH_MAP, the function returns M_TRUE).

The container must have been previously allocated using MbufAllocContainer() with M_PROC, and must store data in a 3D-processable depth map format (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE_DEPTH_MAP, the function returns M_TRUE).

DstImageBufId

Specifies the identifier of the destination depth map image buffer or container.

The image buffer must be a 1-band, 8-bit, 16-bit, or 32-bit unsigned buffer, and must be the same size as the source image buffer.

The container must have been previously allocated using MbufAllocContainer() with M_PROC, and must not be a child container.

> Note that when the remap mode is set to M_USE_DESTINATION_CALIBRATION, the destination buffer must have the same camera calibration information for X and Y as the source buffer. If the destination is a container, the container must store data in a 3D-processable depth map format (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE_DEPTH_MAP, the function returns M_TRUE).

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dim11.lib.
DLL	Requires ail11.dll; ail3dim11.dll.

M3dimRemovePoints

Synopsis

Remove 3D points from a point cloud, based on a specified condition.

Syntax

```
void M3dimRemovePoints(
    AIL_ID SrcContainerBufId, //in
    AIL_ID DstContainerBufId, //out
    AIL_INT64 Option,         //in
    AIL_INT64 ControlFlag     //in
)
```

Description

This function removes 3D points from a point cloud, based on a specified condition (for example, all invalid points).

The specified points are removed from the point cloud, not just invalidated with the confidence component. The mesh component (if existing) is recalculated for the remaining points. If the point cloud has both intensity and reflectance components, the points' color or intensity data is assumed to be stored in the reflectance component. Consequently, the specified points' color or intensity data is removed from the reflectance component, while the intensity component is copied unchanged into the destination container. This is because, when both components exist in a point cloud, the intensity component often stores a photographic image of the source scene. A data correlation might or might not exist between the image and the container's 3D points, depending on the type of 3D sensor used to generate the data. If the intensity component or the reflectance component exists but not both, it is modified and copied to the destination container.

> Note that, if an M_COMPONENT_NORMALS_AIL component exists in the source container, normal vectors of removed points are also removed; remaining point normals are not recalculated.

The destination point cloud is always unorganized, meaning its data is stored in the destination container with 1D components.

Parameters

SrcContainerBufId

Specifies the identifier of the source point cloud container. The container must have been previously allocated using MbufAllocContainer() with M_PROC, and must be 3D-processable (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE, the function returns M_PROCESSABLE).

DstContainerBufId

Specifies the identifier of the destination container in which to place the modified point cloud. The container must have been previously allocated using MbufAllocContainer() with M_PROC, and must not be a child container.

If, after the operation, there are no valid points remaining, the destination container's range component is reduced to 1x1 and its corresponding confidence equals 0.

Option

Specifies which points to remove.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the option																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_INVALID_NORMALS	Specifies to remove points whose confidence is equal to zero, as well as points whose normal vector values are zero. Note that other points with non-unit normal vectors are not removed. This option is available only when the component M_COMPONENT_NORMALS_AIL exists in the source point cloud container.	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID_POINTS_ONLY	Specifies to remove points whose confidence is equal to zero.	c		e	f	g	a		i		j	k	l	m	n	b
M_NOT_MESH_POINTS	Specifies to remove points whose confidence is equal to zero, as well as points that are not part of the mesh. This option is available only when the component M_COMPONENT_MESH_AIL exists in the source point cloud container.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Specifies which components will be affected by the operation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying whether to remove points from all components																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that the operation will only affect the core components of a 3D-processable container. These core components are listed in the description of MbufInquireContainer() with M_3D_PROCESSABLE. All other components are copied to the destination container unmodified.	c		e	f	g	a		i		j	k	l	m	n	b
M_APPLY_TO_ALL_COMPONENTS	Specifies that the operation will affect the core components of a 3D-processable container, as well as any other component (including custom components) that has the same dimension as M_COMPONENT_RANGE. These components must not have a region of interest (ROI) associated with them. For example, using a component that is an image buffer with an ROI will cause an error. > Note that if a custom component has an associated calibration context, the component has points removed, but its context is discarded. If the resulting operation yields an empty container with 0 valid points, the components with the same dimensions are not created and will not exist in the destination container.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dim11.lib.
DLL	Requires ail11.dll; ail3dim11.dll.

M3dimRestore

Synopsis

Restores a 3D image processing context from disk.

Syntax

```
AIL_ID M3dimRestore(
    AIL_CONST_TEXT_PTR FileName, //in
    AIL_ID SysId,                //in
    AIL_INT64 ControlFlag,       //in
    AIL_ID *Context3dimIdPtr     //out
)
```

Description

This function restores a 3D image processing context that was previously saved to a file, using M3dimSave() or M3dimStream(). This function restores all of the 3D image processing context's settings that were in effect when it was saved.

When the restored 3D image processing context is no longer required, release it using M3dimFree() unless M_UNIQUE_ID was specified during restoration; if M_UNIQUE_ID was specified, the smart identifier manages the 3D image processing context's lifetime and you must not manually free it.

Parameters

FileName

Specifies the name and path of the file from which to restore the 3D image processing context. The function handles (internally) the opening and closing of the file.

This parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the file name and path																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_INTERACTIVE	[This is only applicable to Windows] Opens the **File Open** dialog box from which you can interactively specify the drive, directory, and name of the file.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"). Typically, 3D image processing context files have an M3DIM extension. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

SysId

Specifies the system on which to restore the 3D image processing context.

This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Context3dimIdPtr

Specifies the address of the variable in which to write the 3D image processing context identifier or specifies the data type that the function should use to return the 3D image processing context identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated 3D image processing context ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated 3D image processing context ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_3DIM_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the 3D image processing context (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the calculate map size 3D image processing context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the restored map size 3D image processing context. If restoration fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the fill gaps 3D image processing context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the restored fill gaps 3D image processing context. If restoration fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the filter 3D image processing context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the restored filter 3D image processing context. If restoration fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the lattice 3D image processing context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the restored lattice 3D image processing context. If restoration fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the mesh 3D image processing context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the restored mesh 3D image processing context. If restoration fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the normals 3D image processing context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the restored normals 3D image processing context. If restoration fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the profile 3D image processing context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the restored profile 3D image processing context. If restoration fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the project 3D image processing context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the restored project 3D image processing context. If restoration fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Address in which to write the remap 3D image processing context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the restored remap 3D image processing context. If restoration fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the statistics 3D image processing context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the restored statistics 3D image processing context. If restoration fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the stitch 3D image processing context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the restored stitch 3D image processing context. If restoration fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the subsample 3D image processing context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the restored subsample 3D image processing context. If restoration fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the surface sample 3D image processing context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the restored surface sample 3D image processing context. If restoration fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the 3D image processing context identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_3DIM_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dim11.lib.
DLL	Requires ail11.dll; ail3dim11.dll.

M3dimRotate

Synopsis

Rotate a specified point cloud or 3D geometry.

Syntax

```
void M3dimRotate(
    AIL_ID SrcContainerBufOrGeometry3dgeoId, //in
    AIL_ID DstContainerBufOrGeometry3dgeoId, //out
    AIL_INT64 RotationType,                  //in
    AIL_DOUBLE Param1,                       //in
    AIL_DOUBLE Param2,                       //in
    AIL_DOUBLE Param3,                       //in
    AIL_DOUBLE Param4,                       //in
    AIL_DOUBLE CenterX,                      //in
    AIL_DOUBLE CenterY,                      //in
    AIL_DOUBLE CenterZ,                      //in
    AIL_INT64 ControlFlag                    //in
)
```

Description

This function applies the specified rotation to the 3D points in a point cloud container or to the geometry in a 3D geometry object.

If the source is a point cloud container, **M3dimRotate**() applies the specified rotation to the coordinates in the M_COMPONENT_RANGE component. If an M_COMPONENT_NORMALS_AIL component exists in the source container, it is recalculated and added to the destination container, along with the modified M_COMPONENT_RANGE component. The source container's M_COMPONENT_CONFIDENCE component is copied to the destination container. If M_COMPONENT_REFLECTANCE and M_COMPONENT_MESH_AIL components exist in the source, they are also copied to the destination. Any previously existing M_COMPONENT_NORMALS_AIL, M_COMPONENT_REFLECTANCE, or M_COMPONENT_MESH_AIL components are removed from the destination container.

Note that all angles should be specified in degrees. However, unlike most other Aurora Imaging Library functions, angles are interpreted using the right-hand grip rule around the axis of rotation; if you wrap your right hand around the axis of rotation, pointing your thumb in the positive direction of the axis, your fingers wrap in the direction of rotation. For example, a positive rotation around the Z-axis corresponds to a rotation that turns the positive X-axis toward the positive Y-axis.

All coordinates are expressed in world units in the working coordinate system.

> Note that, when rotating a point cloud, this function affects the coordinates of the points and not their storage location in the container.

Parameters

SrcContainerBufOrGeometry3dgeoId

Specifies the source point cloud container or 3D geometry object.

> Note that the source and destination must match so that both are containers or both are 3D geometries.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the source container or 3D geometry object identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_XY_PLANE	Specifies the XY (Z=0) plane.	c		e	f	g	a		i		j	k	l	m	n	b
Source 3D geometry object identifier	Specifies the identifier of the source 3D geometry object. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and must have been successfully defined.	c		e	f	g	a		i		j	k	l	m	n	b
Source container identifier	Specifies the identifier of the source point cloud container. The container must be 3D-processable (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE, the function returns M_PROCESSABLE). The container must have been previously allocated using MbufAllocContainer() with M_PROC.	c		e	f	g	a		i		j	k	l	m	n	b

DstContainerBufOrGeometry3dgeoId

Specifies the destination container or 3D geometry object.

> Note that the source and destination must match so that both are containers or both are 3D geometries.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the destination container or 3D geometry object identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Destination 3D geometry object identifier	Specifies the identifier of the destination 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY.	c		e	f	g	a		i		j	k	l	m	n	b
Destination container identifier	Specifies the identifier of the destination container, previously allocated using MbufAllocContainer() with M_PROC. The destination container must not be a child container.	c		e	f	g	a		i		j	k	l	m	n	b

RotationType

Specifies the type of rotation to apply to the point cloud or 3D geometry.

See the Parameter associations section for possible values that can be specified.

Param1

Specifies an attribute of the rotation. Its definition is dependent on the type of rotation selected. Set this parameter to **M_DEFAULT** if not used.

See the Parameter associations section for possible values that can be specified.

Param2

Specifies an attribute of the rotation. Its definition is dependent on the type of rotation selected. Set this parameter to **M_DEFAULT** if not used.

See the Parameter associations section for possible values that can be specified.

Param3

Specifies an attribute of the rotation. Its definition is dependent on the type of rotation selected. Set this parameter to **M_DEFAULT** if not used.

See the Parameter associations section for possible values that can be specified.

Param4

Specifies an attribute of the rotation. Its definition is dependent on the type of rotation selected. Set this parameter to **M_DEFAULT** if not used.

See the Parameter associations section for possible values that can be specified.

CenterX

Specifies the X-coordinate of the point around which the rotation is performed.

To specify the volumetric center point when rotating a 3D geometry, set this parameter to M_GEOMETRY_CENTER. To specify the volumetric center point or center of mass when rotating a point cloud, calculate (using M3dimStat()) and then retrieve (using M3dimGetResult()) the required bounding box (M_BOX_CENTER_...) or centroid (M_CENTROID_...) statistics (respectively), and then pass these coordinates to the CenterX, CenterY and CenterZ parameters.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the X-coordinate value																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_GEOMETRY_CENTER	Specifies the 3D geometry's center point. This setting is only applicable if the geometry is finite (for example, a box, sphere, or finite cylinder). > Note that, if M_GEOMETRY_CENTER is specified, you must set CenterY and CenterZ to M_DEFAULT.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the X-coordinate value.	c		e	f	g	a		i		j	k	l	m	n	b

CenterY

Specifies the Y-coordinate of the point around which the rotation is performed. If the CenterX parameter is set to M_GEOMETRY_CENTER, set this parameter to M_DEFAULT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the Y-coordinate value																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the Y-coordinate value.	c		e	f	g	a		i		j	k	l	m	n	b

CenterZ

Specifies the Z-coordinate of the point around which the rotation is performed. If the CenterX parameter is set to M_GEOMETRY_CENTER, set this parameter to M_DEFAULT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the Z-coordinate value																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the Z-coordinate value.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

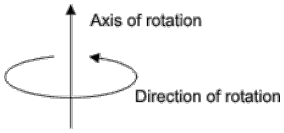
Reserved for future expansion and must be set to **M_DEFAULT**.

Parameter associations

The table below lists possible values for the **RotationType**, **Param1**, **Param2**, **Param3**, and **Param4** parameters.

- For specifying the rotation type

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the rotation type																	
RotationType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	Param1																
	Param2																
	Param3																
	Param4																
M_ROTATION_AXIS_ANGLE		Specifies a rotation described by an axis and angle of rotation. The axis of rotation is defined by a vector. The angle of rotation is measured in the counter-clockwise direction around the axis of rotation. 															
	Param1	Specifies the X-component of the vector that defines the axis of rotation.	c		e	f	g	a		i		j	k	l	m	n	b
	Param2	Specifies the Y-component of the vector that defines the axis of rotation.	c		e	f	g	a		i		j	k	l	m	n	b
	Param3	Specifies the Z-component of the vector that defines the axis of rotation.	c		e	f	g	a		i		j	k	l	m	n	b

	Param4	Specifies the angle of the rotation, in degrees, around the axis of rotation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ROTATION_AXIS_X		Specifies the minimal rotation that can align the working coordinate system's X-axis with the specified vector.	c		e	f	g	a		i		j	k	l	m	n	b
	Param1	Specifies the X-component of the vector.	c		e	f	g	a		i		j	k	l	m	n	b
	Param2	Specifies the Y-component of the vector.	c		e	f	g	a		i		j	k	l	m	n	b
	Param3	Specifies the Z-component of the vector.	c		e	f	g	a		i		j	k	l	m	n	b
M_ROTATION_AXIS_Y		Specifies the minimal rotation that can align the working coordinate system's Y-axis with the specified vector.	c		e	f	g	a		i		j	k	l	m	n	b
	Param1	Specifies the X-component of the vector.	c		e	f	g	a		i		j	k	l	m	n	b
	Param2	Specifies the Y-component of the vector.	c		e	f	g	a		i		j	k	l	m	n	b
	Param3	Specifies the Z-component of the vector.	c		e	f	g	a		i		j	k	l	m	n	b
M_ROTATION_AXIS_Z		Specifies the minimal rotation that can align the working coordinate system's Z-axis with the specified vector.	c		e	f	g	a		i		j	k	l	m	n	b
	Param1	Specifies the X-component of the vector.	c		e	f	g	a		i		j	k	l	m	n	b
	Param2	Specifies the Y-component of the vector.	c		e	f	g	a		i		j	k	l	m	n	b
	Param3	Specifies the Z-component of the vector.	c		e	f	g	a		i		j	k	l	m	n	b
M_ROTATION_QUATERNION		Specifies a rotation operation that is described by a rotation quaternion.	c		e	f	g	a		i		j	k	l	m	n	b
	Param1	Specifies the scalar component of the quaternion.	c		e	f	g	a		i		j	k	l	m	n	b
	Param2	Specifies the X-component of the quaternion.	c		e	f	g	a		i		j	k	l	m	n	b
	Param3	Specifies the Y-component of the quaternion.	c		e	f	g	a		i		j	k	l	m	n	b
	Param4	Specifies the Z-component of the quaternion.	c		e	f	g	a		i		j	k	l	m	n	b
M_ROTATION_X		Specifies a rotation operation around the X-axis. If you displace the center of rotation (using the CenterX, CenterY, and CenterZ parameters), the rotation occurs around an axis that is parallel to the X-axis and runs through the center point.	c		e	f	g	a		i		j	k	l	m	n	b
	Param1	Specifies the X-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_ROTATION_XYZ		Specifies a rotation that is described by three distinct rotations about the axes of the working coordinate system in the following order: a rotation about the X-axis, a rotation about the Y-axis, and a rotation about the Z-axis rotation.	c		e	f	g	a		i		j	k	l	m	n	b
	Param1	Specifies the X-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
	Param2	Specifies the Y-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
	Param3	Specifies the Z-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b

M_ROTATION_XZY		Specifies a rotation that is described by three distinct rotations about the axes of the working coordinate system in the following order: a rotation about the X-axis, a rotation about the Z-axis, and a rotation about the Y-axis rotation.	c		e	f	g	a		i		j	k	l	m	n	b
	Param1	Specifies the X-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
	Param2	Specifies the Z-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
	Param3	Specifies the Y-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_ROTATION_Y		Specifies a rotation operation around the Y-axis. If you displace the center of rotation (using the CenterX, CenterY, and CenterZ parameters), the rotation occurs around an axis that is parallel to the Y-axis and runs through the center point.	c		e	f	g	a		i		j	k	l	m	n	b
	Param1	Specifies the Y-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_ROTATION_YXZ		Specifies a rotation that is described by three distinct rotations about the axes of the working coordinate system in the following order: a rotation about the Y-axis, a rotation about the X-axis, and a rotation about the Z-axis rotation.	c		e	f	g	a		i		j	k	l	m	n	b
	Param1	Specifies the Y-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
	Param2	Specifies the X-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
	Param3	Specifies the Z-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_ROTATION_YZX		Specifies a rotation that is described by three distinct rotations about the axes of the working coordinate system in the following order: a rotation about the Y-axis, a rotation about the Z-axis, and a rotation about the X-axis rotation.	c		e	f	g	a		i		j	k	l	m	n	b
	Param1	Specifies the Y-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
	Param2	Specifies the Z-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
	Param3	Specifies the X-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_ROTATION_Z		Specifies a rotation operation around the Z-axis. If you displace the center of rotation (using the CenterX, CenterY, and CenterZ parameters), the rotation occurs around an axis that is parallel to the Z-axis and runs through the center point.	c		e	f	g	a		i		j	k	l	m	n	b
	Param1	Specifies the Z-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_ROTATION_ZXY		Specifies a rotation that is described by three distinct rotations about the axes of the working coordinate system in the following order: a rotation about the Z-axis, a rotation about the X-axis, and a rotation about the Y-axis rotation.	c		e	f	g	a		i		j	k	l	m	n	b
	Param1	Specifies the Z-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
	Param2	Specifies the X-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
	Param3	Specifies the Y-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_ROTATION_ZYX		Specifies a rotation that is described by three distinct rotations about the axes of the working coordinate system in the following order: a rotation about the Z-axis, a rotation about the Y-axis, and a rotation about the X-axis rotation.	c		e	f	g	a		i		j	k	l	m	n	b

	Param1	Specifies the Z-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
	Param2	Specifies the Y-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
	Param3	Specifies the X-axis rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dim11.lib.
DLL	Requires ail11.dll; ail3dim11.dll.

M3dimSample

Synopsis

Subsample a point cloud to generate a less dense point cloud, sample a meshed point cloud to generate a denser point cloud, or sample a finite 3D geometry to generate a point cloud.

Syntax

```
void M3dimSample(
    AIL_ID SampleContext3dimId,              //in
    AIL_ID SrcContainerBufOrGeometry3dgeoId, //in
    AIL_ID DstContainerBufId,                //out
    AIL_INT64 ControlFlag                    //in
)
```

Description

This function reduces the number of points in a point cloud (subsampling), increases the number of points in a meshed point cloud (surface sampling), or creates a point cloud from a 3D geometry.

When subsampling a point cloud, the function subsamples the source container's point cloud (M_COMPONENT_RANGE and M_COMPONENT_CONFIDENCE components), and places the resulting less dense point cloud into the destination container. If an M_COMPONENT_NORMALS_AIL component exists in the source container, it is subsampled and the new normals component is added to the destination container. Typically, the source container's M_COMPONENT_REFLECTANCE component (if it exists) is also subsampled. Note that subsampling always invalidates the mesh; regardless of the existence of a mesh structure (M_COMPONENT_MESH_AIL component) associated with the source, the destination point cloud will not have a mesh structure.

You can choose the type of subsampling operation to perform, using M3dimControl() with M_SUBSAMPLE_MODE. The grid operation (M_SUBSAMPLE_GRID) uses a lattice structure with a specified cell size to divide points into subgroups and selects a single point per cell. You can use M3dimLattice() to calculate the lattice required to retain a particular fraction of points from the source point cloud. You can copy lattice results into a subsample 3D image processing context, using M3dimCopyResult() with M_SUBSAMPLE_CONTEXT. When you call **M3dimSample**() with a subsample 3D image processing context containing lattice results, a grid operation is performed using the cell sizes of the lattice results (M3dimGetResult() with M_CELL_SIZE_...).

When surface sampling a point cloud, the function samples the surface of a meshed point cloud, and places a denser point cloud into the destination container. New M_COMPONENT_RANGE and M_COMPONENT_CONFIDENCE components are generated for the destination container, as well as a new M_COMPONENT_NORMALS_AIL component (regardless of whether an M_COMPONENT_NORMALS_AIL component exists in the source container). In addition, if the source container has an M_COMPONENT_REFLECTANCE component, destination reflectance values are interpolated for a destination M_COMPONENT_REFLECTANCE component.

You can choose the mode of the surface sampling operation, using M3dimControl() with M_SAMPLE_MODE. When using the grid mode (M_SURFACE_GRID), regardless of the existence of a mesh structure (M_COMPONENT_MESH_AIL component) associated with the source, the destination point cloud will not have a mesh structure; surface sampling using the grid mode invalidates the mesh. If an M_COMPONENT_MESH_AIL component exists in the destination container, it is deleted. When using the tessellation mode (M_TESSELLATION), the source container's M_COMPONENT_MESH_AIL is surface sampled and the new mesh component is added to the destination container.

When surface sampling a finite 3D geometry, the function samples the geometry's surface, and generates the components M_COMPONENT_RANGE and M_COMPONENT_CONFIDENCE. An M_COMPONENT_NORMALS_AIL component is also generated for box, cylinder, and sphere 3D geometries, but not for a point or line. Note that infinite 3D geometries (such as an infinite cylinder, line, or plane) are not supported.

Note that, if the source container has both M_COMPONENT_REFLECTANCE and M_COMPONENT_INTENSITY components, the reflectance component is sampled or subsampled and the intensity component is copied to the destination container unmodified, unless the reflectance component is not in the right format. In this case, the intensity component is subsampled or surface sampled, with the resulting M_COMPONENT_INTENSITY component added to the destination container. If only 1 component of these types exist (reflectance or intensity), that component is subsampled or surface sampled.

> Note that by default, any component in the source container that is not listed above is copied to the destination unmodified, when subsampling or surface sampling a point cloud. For subsampling, you can use M_APPLY_TO_ALL_COMPONENTS to apply the operation to all other components in the container, including custom components. The components must have the same dimensions as M_COMPONENT_RANGE; otherwise, they will be copied to the destination unmodified.

Parameters

SampleContext3dimId

Specifies a subsample or surface sample 3D image processing context.

For specifying the subsample or surface sample 3D image processing context identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Subsample 3D image processing context identifier	Specifies the identifier of a subsample 3D image processing context, previously allocated using M3dimAlloc() with M_SUBSAMPLE_CONTEXT. The function applies the subsample control settings specified using M3dimControl().	c		e	f	g	a		i		j	k	l	m	n	b
Surface sample 3D image processing context identifier	Specifies the identifier of a surface sample 3D image processing context, previously allocated using M3dimAlloc() with M_SURFACE_SAMPLE_CONTEXT. The function applies the surface sample control settings specified using M3dimControl().	c		e	f	g	a		i		j	k	l	m	n	b

SrcContainerBufOrGeometry3dgeoId

Specifies the identifier of the source container containing a 3D-processable point cloud, or specifies the identifier of the source 3D geometry object.

The container must have been previously allocated using MbufAllocContainer() with M_PROC, and must be 3D-processable (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE, the function returns M_PROCESSABLE). The container must have the component M_COMPONENT_MESH_AIL for surface sampling.

The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and must have been successfully defined.

> Note that 3D geometries are supported for a surface sampling context only; the geometry cannot be infinite (such as an infinite cylinder, line, or plane).

DstContainerBufId

Specifies the identifier of the destination container in which to place the generated point cloud. The container must have been previously allocated using MbufAllocContainer() with M_PROC, and must not be a child container.

ControlFlag

Specifies which components will be affected by the operation.

For a surface sample operation, this parameter must be set to M_DEFAULT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying whether to apply subsampling to all components																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that the operation will only affect the core components of a 3D-processable container. These core components are listed in the description of MbufInquireContainer() with M_3D_PROCESSABLE. All other components are copied to the destination container unmodified.	c		e	f	g	a		i		j	k	l	m	n	b
M_APPLY_TO_ALL_COMPONENTS	Specifies that the operation will affect the core components of a 3D-processable container, as well as any other component (including custom components) that has the same dimension as M_COMPONENT_RANGE. These components must not have a region of interest (ROI) associated with them. For example, using a component that is an image buffer with an ROI will cause an error. Note that if a custom component has an associated calibration context, the component is subsampled, but its context is discarded. If the resulting operation yields an empty container with 0 valid points, the components with the same dimensions are not created and will not exist in the destination container. This option is only available for a subsample 3D image processing context.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dim11.lib.
DLL	Requires ail11.dll; ail3dim11.dll.

M3dimSave

Synopsis

Save a 3D image processing context to a file.

Syntax

```
void M3dimSave(
    AIL_CONST_TEXT_PTR FileName, //in
    AIL_ID Context3dimId,        //in
    AIL_INT64 ControlFlag        //in
)
```

Description

This function saves all the information about a previously allocated 3D image processing context to disk.

To load a saved context, use either M3dimRestore() or M3dimStream().

Parameters

FileName

Specifies the name and path of the file in which to save the 3D image processing context; it is recommended that you use the M3DIM file extension for easier use with other Aurora Imaging software products. The function internally handles the opening and closing of this file. If this file already exists, it will be overwritten.

This parameter can be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the file name and path																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_INTERACTIVE	[This is only applicable to Windows] Opens the **File Save As** dialog box from which you can interactively specify the drive, directory, and name of the file.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"). The recommended extension is M3DIM. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

Context3dimId

Specifies the identifier of the 3D image processing context to save. The context must have been successfully allocated (using M3dimAlloc()) prior to calling this function.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dim11.lib.
DLL	Requires ail11.dll; ail3dim11.dll.

M3dimScale

Synopsis

Scale a specified point cloud or 3D geometry.

Syntax

```
void M3dimScale(
    AIL_ID SrcContainerBufOrGeometry3dgeoId, //in
    AIL_ID DstContainerBufOrGeometry3dgeoId, //out
    AIL_DOUBLE ScaleX,                       //in
    AIL_DOUBLE ScaleY,                       //in
    AIL_DOUBLE ScaleZ,                       //in
    AIL_DOUBLE CenterX,                      //in
    AIL_DOUBLE CenterY,                      //in
    AIL_DOUBLE CenterZ,                      //in
    AIL_INT64 ControlFlag                    //in
)
```

Description

This function applies the specified scale factors to the points in the source point cloud container or to the geometry in the source 3D geometry object. The scaling is applied to each point's distance from the origin or specified center point, along X, Y, or Z. Note that you can specify negative scale factors.

If the source is a point cloud container, **M3dimScale**() applies the specified scale factors to the coordinates in the M_COMPONENT_RANGE component. If an M_COMPONENT_NORMALS_AIL component exists in the source container, it is recalculated and added to the destination container, along with the modified M_COMPONENT_RANGE component. The source container's M_COMPONENT_CONFIDENCE component is copied to the destination container. If M_COMPONENT_REFLECTANCE and M_COMPONENT_MESH_AIL components exist in the source container, they are also copied to the destination container. Any previously existing normals, reflectance, or mesh components are removed from the destination container.

> Note that sphere, box, and cylinder 3D geometries do not accept non-uniform scaling. That is, you must apply the same scale factor in X, Y, and Z for a source sphere, box, or cylinder.

> Note that, when scaling a point cloud, this function affects the coordinates of the points and not their storage location in the container.

Parameters

SrcContainerBufOrGeometry3dgeoId

Specifies a source point cloud container or 3D geometry object.

> Note that the source and destination must match so that both are containers or both are 3D geometry objects.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the source container or 3D geometry object identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Source 3D geometry object identifier	Specifies the identifier of the source 3D geometry object. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and must have been successfully defined.	c		e	f	g	a		i		j	k	l	m	n	b
Source container identifier	Specifies the identifier of the source point cloud container. The container must be 3D-processable (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE, the function returns M_PROCESSABLE). The container must have been previously allocated using MbufAllocContainer() with M_PROC.	c		e	f	g	a		i		j	k	l	m	n	b

DstContainerBufOrGeometry3dgeoId

Specifies the destination container or 3D geometry object.

> Note that the source and destination must match so that both are containers or both are 3D geometry objects.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the destination container or 3D geometry object identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Destination 3D geometry object identifier	Specifies the identifier of the destination 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY.	c		e	f	g	a		i		j	k	l	m	n	b
Destination container identifier	Specifies the identifier of the destination container, previously allocated using MbufAllocContainer() with M_PROC. The destination container must not be a child container.	c		e	f	g	a		i		j	k	l	m	n	b

ScaleX

Specifies the factor by which to scale in X.

ScaleY

Specifies the factor by which to scale in Y.

> Note that sphere, box, and cylinder 3D geometries do not accept non-uniform scaling.

ScaleZ

Specifies the factor by which to scale in Z.

> Note that sphere, box, and cylinder 3D geometries do not accept non-uniform scaling.

CenterX

Specifies the X-coordinate of the point that defines the center of the scaling operation.

To specify the volumetric center point when scaling a 3D geometry, set this parameter to M_GEOMETRY_CENTER. To specify the volumetric center point or center of mass when scaling a point cloud, calculate (using M3dimStat()) and then retrieve (using M3dimGetResult()) the required bounding box (M_BOX_CENTER_...) or centroid (M_CENTROID_...) statistics (respectively), and then pass these coordinates to the CenterX, CenterY and CenterZ parameters.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the X-coordinate value																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_GEOMETRY_CENTER	Specifies the 3D geometry object's center point for a finite 3D geometry (for example, a box, sphere, or finite cylinder). > Note that, if you specify M_GEOMETRY_CENTER, you must set CenterY and CenterZ to M_DEFAULT.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the X-coordinate value.	c		e	f	g	a		i		j	k	l	m	n	b

CenterY

Specifies the Y-coordinate of the point that defines the center of the scaling operation. If the CenterX parameter is set to M_GEOMETRY_CENTER, set this parameter to M_DEFAULT.

> Note that sphere, box, and cylinder 3D geometries do not accept non-uniform scaling.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the Y-coordinate value																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b

Value	Specifies the Y-coordinate value. If scaling a sphere, box, or cylinder 3D geometry object, this parameter must be set to the same value as CenterX and CenterZ.	c		e	f	g	a		i		j	k	l	m	n	b

CenterZ

Specifies the Z-coordinate of the point that defines the center of the scaling operation. If the CenterX parameter is set to M_GEOMETRY_CENTER, set this parameter to M_DEFAULT.

> Note that sphere, box, and cylinder 3D geometries do not accept non-uniform scaling.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the Z-coordinate value																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the Z-coordinate value. If scaling a sphere, box, or cylinder 3D geometry object, this parameter must be set to the same value as CenterX and CenterY.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dim11.lib.
DLL	Requires ail11.dll; ail3dim11.dll.

M3dimStat

Synopsis

Compute a variety of statistics on a point cloud, depth map, or 3D geometry.

Syntax

```
void M3dimStat(
    AIL_ID StatContext3dimId,               //in
    AIL_ID AilObjectId,                     //in
    AIL_ID StatResult3dimOrGeometry3dgeoId, //out
    AIL_INT64 ControlFlag                   //in
)
```

Description

This function calculates a variety of statistics for the 3D points in a point cloud, for the real-world coordinates that correspond to pixels in a depth map, or for a 3D geometry. The specified statistics 3D image processing context establishes which statistics to calculate and how to calculate them. If you need to calculate one type of statistic and use default settings, you can use a predefined context; otherwise, use M3dimAlloc() to set up a custom context that you can control using M3dimControl(). For example, to calculate bounding box statistics for a point cloud or depth map, you can first enable the calculation (using M3dimControl() with M_BOUNDING_BOX and M_ENABLE), and then have the algorithm compute the axis-aligned box that contains most of the points but rejects outliers (M_BOUNDING_BOX_ALGORITHM set to M_ROBUST). Note that, if you use the default M_STAT_CONTEXT_BOUNDING_BOX context, **M3dimStat**() computes the bounding box of all valid points. Results are stored in the specified 3D statistics result buffer.

For a 3D geometry, you can calculate bounding box and centroid statistics only.

Note that, when you specify a destination 3D geometry, bounding box and centroid statistics are written directly to the specified 3D geometry object, which establishes the geometry as a 3D box or 3D point, respectively.

After calling **M3dimStat**(), you can obtain the statistics from the result buffer using M3dimGetResult().

> Note that there must be a sufficient number of valid points in the source container or image buffer for the statistics to be calculated. This means that there must be at least 2 valid points for M_STAT_CONTEXT_DISTANCE_TO_NEAREST_NEIGHBOR calculations, and at least 1 valid point for all other statistics types. In the case of an insufficient number of valid points, the result is typically 0; however, PCA or fixturing matrices return the identity matrix, and PCA axes return an axis-aligned coordinate system with vector coordinates (1,0,0), (0,1,0), (0,0,1). Also note that, for moments, if the specified powers in X, Y, and Z are all 0, the result is 1; otherwise, the result is 0.

Parameters

StatContext3dimId

Specifies a previously allocated statistics 3D image processing context (used to evaluate multiple statistical calculations), or a predefined statistics 3D image processing context (used to evaluate a single statistical calculation).

Set this parameter to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the statistics context																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_STAT_CONTEXT_BOUNDING_BOX	Specifies a predefined statistics 3D image processing context with the M_BOUNDING_BOX control type (M3dimControl()) set to M_ENABLE. For a point cloud or depth map, use this predefined context to calculate bounding box statistics using the default setting (M_ALL_POINTS), which includes all valid points; no valid points exist outside the bounding box. To exclude outliers, use a custom context and compute a bounding box that contains most of the valid points instead of all of them, using M3dimControl() with M_BOUNDING_BOX_ALGORITHM set to M_ROBUST. Note that, if the destination object (StatResult3dimOrGeometry3dgeoId) is a 3D geometry, this setting writes the calculated bounding box into the specified 3D geometry object directly.	c		e	f	g	a		i		j	k	l	m	n	b

M_STAT_CONTEXT_CENTROID	Specifies a predefined statistics 3D image processing context with the M_CENTROID control type (M3dimControl()) set to M_ENABLE. Use this predefined context to calculate centroid statistics. The centroid is the center of mass of a point cloud, depth map, or 3D geometry. Centroid statistics include the X-, Y-, and Z-coordinates of the centroid. Note that, if the destination object (StatResult3dimOrGeometry3dgeoId) is a 3D geometry, this setting writes the calculated centroid into the specified 3D geometry object directly.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_CONTEXT_DISTANCE_TO_NEAREST_NEIGHBOR	Specifies a predefined statistics 3D image processing context with the M_DISTANCE_TO_NEAREST_NEIGHBOR control type (M3dimControl()) set to M_ENABLE. Use this predefined context to calculate distance-to-nearest-neighbor statistics, which include the average distance, the maximum distance, and the minimum distance of 3D points or real-world coordinates to their nearest neighbor. > This predefined context is not available for 3D geometries.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_CONTEXT_NUMBER_OF_POINTS	Specifies a predefined statistics 3D image processing context with the M_NUMBER_OF_POINTS control type (M3dimControl()) set to M_ENABLE. Use this predefined context to calculate number-of-points statistics, which include the total number of points, the number of points that are missing data (invalid points), and the number of valid points. > This predefined context is not available for 3D geometries.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_CONTEXT_PCA	Specifies a predefined statistics 3D image processing context with the M_PCA control type (M3dimControl()) set to M_ENABLE. Use this predefined context to calculate principle component analysis (PCA) statistics. > This predefined context is not available for 3D geometries.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_CONTEXT_PCA_NORMALS	Specifies a predefined statistics 3D image processing context with the M_PCA control type set to M_ENABLE, the M_PCA_MODE control type set to M_ORDINARY, and the M_COMPONENT_OF_INTEREST control type set to M_COMPONENT_NORMALS_AIL (M3dimControl()). Use this predefined context to calculate principle component analysis (PCA) statistics on a point cloud's unit normal vectors. > This predefined context is not available for depth maps or 3D geometries.	c		e	f	g	a		i		j	k	l	m	n	b

M_STAT_CONTEXT_SEMI_ORIENTED_BOX	Specifies a predefined statistics 3D image processing context with the M_BOUNDING_BOX control type set to M_ENABLE and the M_BOX_ORIENTATION control type set to M_SEMI_ORIENTED (M3dimControl()). Use this predefined context to calculate semi-oriented bounding box statistics, whereby the minimum volume bounding box is axis-aligned in Z but not in X and Y. To change the axis with which the semi-oriented bounding box is aligned, use a custom context and specify the axis using M_BOX_SEMI_ORIENTED_ROTATION_AXIS. For a point cloud or depth map, specifying this predefined context calculates semi-oriented bounding box statistics using the default algorithm (M_ALL_POINTS), which includes all valid points; no valid points exist outside the semi-oriented bounding box. To exclude outliers, use a custom context and compute a semi-oriented bounding box that contains most of the valid points instead of all of them, using M3dimControl() with M_BOUNDING_BOX_ALGORITHM set to M_ROBUST.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_CONTEXT_SURFACE_AREA	Specifies a predefined statistics 3D image processing context with the M_SURFACE_AREA control type (M3dimControl()) set to M_ENABLE. Use this predefined context to calculate the surface area of a mesh. > This predefined context is only available for 3D-processable point cloud and depth map containers that have an M_COMPONENT_MESH_AIL component.	c		e	f	g	a		i		j	k	l	m	n	b
Statistics 3D image processing context identifier	Specifies the identifier of a statistics 3D image processing context, previously allocated using M3dimAlloc() with M_STATISTICS_CONTEXT. > In this case, use M3dimControl() to enable the statistics type(s) and specify additional control settings.	c		e	f	g	a		i		j	k	l	m	n	b

AilObjectId

Specifies the identifier of the source container containing a 3D-processable point cloud or depth map, the identifier of the source depth map image buffer, or the identifier of the source 3D geometry object.

> Note that when M_COMPONENT_OF_INTEREST is set to M_COMPONENT_NORMALS_AIL (using M3dimControl()), the source object must have normals. This means that the source cannot be a depth map image buffer, a point or line 3D geometry object, or a container without a normals component.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the source container, image buffer, or 3D geometry object identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Source 3D geometry object identifier	Specifies the identifier of the source 3D geometry object. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and must have been successfully defined. Supported 3D geometries include box, cylinder, line, point, and sphere. Note that infinite 3D geometries (such as an infinite cylinder, an infinite line, or a plane) are not supported.	c		e	f	g	a		i		j	k	l	m	n	b
Source depth map container identifier	Specifies the identifier of the source depth map container. The container must have been previously allocated using MbufAllocContainer() with M_PROC, and must store data in a 3D-processable depth map format (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE_DEPTH_MAP, the function returns M_TRUE).	c		e	f	g	a		i		j	k	l	m	n	b

Source image buffer identifier	Specifies the identifier of the source image buffer. The source image buffer must be a 1-band, 8-, 16-, or 32-bit unsigned buffer, and must be fully corrected (that is, if you call McalInquire() with M_DEPTH_MAP, the function returns M_TRUE). The source image buffer can have a region of interest (ROI) associated with it, but it must be a raster region. Using an image buffer with a non-raster type of ROI will cause an error.	c		e	f	g	a		i		j	k	l	m	n	b
Source point cloud container identifier	Specifies the identifier of the source point cloud container. The container must be 3D-processable (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE, the function returns M_PROCESSABLE), and must have been previously allocated using MbufAllocContainer() with M_PROC.	c		e	f	g	a		i		j	k	l	m	n	b

StatResult3dimOrGeometry3dgeoId

Specifies the identifier of the statistics 3D image processing result buffer, or the identifier of a 3D geometry object. The result buffer must have been allocated using M3dimAllocResult() with M_STATISTICS_RESULT. The 3D geometry object must have been allocated using M3dgeoAlloc() with M_GEOMETRY.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dim11.lib.
DLL	Requires ail11.dll; ail3dim11.dll.

M3dimStream

Synopsis

Load, restore, or save a 3D image processing context from/to a file or memory stream.

Syntax

```
void M3dimStream(
    AIL_TEXT_PTR MemPtrOrFileName, //in-out
    AIL_ID SysId,                  //in
    AIL_INT64 Operation,           //in
    AIL_INT64 StreamType,          //in
    AIL_DOUBLE Version,            //in
    AIL_INT64 ControlFlag,         //in
    AIL_ID *Context3dimIdPtr,      //in-out
    AIL_INT *SizeByteVarPtr        //out
)
```

Description

This function can load, restore, or save a 3D image processing context from/to a file or memory stream.

To inquire the number of bytes necessary to save a 3D image processing context to a memory stream, you should first call this function (**M3dimStream**()) with M_INQUIRE_SIZE_BYTE.

The content saved to memory stream is equivalent to the content saved to file. In addition, any file saved using this function is equivalent to a file saved with M3dimSave().

You can use this and other Aurora Imaging Library stream functions, for example, to save all required Aurora Imaging Library objects, as well as any other custom data, for your application to a memory stream. Once in a memory stream, you can write the stream to a single file or transfer it over a network. You are responsible for concatenating the streams and for saving the stream to file.

This function restores all of the object's settings that were in effect when the object was saved.

Using **M3dimStream**(), you can choose to save a backwards-compatible version of the 3D image processing context, which will work using a version of Aurora Imaging Library that is up to one major release older than the current version (depending on which version is specified). However, all settings and features unique to the higher version will be ignored when restored using the lower version. Besides saving backwards-compatible versions, you can also load or restore 3D image processing contexts saved using any Aurora Imaging Library version with which this function was available. Settings that do not exist in the lower version will be filled with default values when the 3D image processing context is loaded or restored.

Parameters

MemPtrOrFileName
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- AIL_TEXT_PTR [optionally, in C++: a reference to an AIL_STRING]
- Optionally, in C++: a reference to a std::vector of type AIL_UINT8

Specifies the file or memory stream.

See the Parameter associations section for possible values that can be returned/specified.

SysId

Specifies the system on which to restore the 3D image processing context.

See the Parameter associations section for possible values that can be specified.

Operation

Specifies the operation to perform.

See the Parameter associations section for possible values that can be specified.

StreamType

Specifies the type of stream in which to store/from which to restore the 3D image processing context. This parameter must be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of stream

Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FILE	Specifies a file stream.	c		e	f	g	a		i		j	k	l	m	n	b
M_MEMORY	Specifies a memory stream. You are responsible for allocating a block of memory for the stream.	c		e	f	g	a		i		j	k	l	m	n	b

Version

Specifies the Aurora Imaging Library version of the 3D image processing context.

See the Parameter associations section for possible values that can be specified.

ControlFlag

Reserved for future expansion. This parameter must be set to **M_DEFAULT**.

Context3dimIdPtr

Specifies the address of the variable in which to write or from which to read the identifier of the 3D image processing context. If the M_RESTORE operation is not successful, **M_NULL** is returned.

See the Parameter associations section for possible values that can be returned/specified.

SizeByteVarPtr

Specifies the address of the variable in which to write the size of the 3D image processing context, in bytes.

If the size is not required, you can set this parameter to **M_NULL**.

> Note that the size of the 3D image processing context will vary depending on the Aurora Imaging Library version specified.

Parameter associations

The table below lists possible values for the **MemPtrOrFileName**, **SysId**, **Operation**, **Version**, and **Context3dimIdPtr** parameters.

- For performing the stream operation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For performing the stream operation.																
Operation	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
MemPtrOrFileName - Possible values passed or returned																
SysId																
Version																
Context3dimIdPtr - Possible values passed or returned																
M_INQUIRE_SIZE_BYTE	Inquires the number of bytes required to save a 3D image processing context to memory stream. This operation is not supported when the StreamType parameter is set to M_FILE.															
MemPtrOrFileName	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
SysId	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
Version	Specifies the Aurora Imaging Library version with which the 3D image processing context must be compatible when saved. The size of a 3D image processing context might differ from one Aurora Imaging Library version to another.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the current version of Aurora Imaging Library.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP4	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 4.	c		e	f	g	a		i		j	k	l	m	n	b

M_PROC_VERSION_100_SP5	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 5.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP6	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 6.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP7	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 7.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_110	Specifies the version as being Aurora Imaging Library 11.0.	c		e	f	g	a		i		j	k	l	m	n	b
Context3dimIdPtr	Specifies the address of the variable containing the identifier of the 3D image processing context.	c		e	f	g	a		i		j	k	l	m	n	b
M_LOAD	Loads the content of a specified file or memory stream into a previously allocated 3D image processing context.	c		e	f	g	a		i		j	k	l	m	n	b
MemPtrOrFileName	Specifies the file or memory stream from which to load the 3D image processing context.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"), when the StreamType parameter is set to M_FILE. 3D image processing contexts typically have an M3DIM file extension. The function handles (internally) the opening and closing of the file. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b
MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which the object is located, and the block of memory must contain the entire object. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b
SysId	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
Version	This parameter must be set to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
Context3dimIdPtr	Specifies the address of the variable containing the identifier of the 3D image processing context. All previous settings are overwritten with those of the loaded 3D image processing context, or are set to the default value if the settings didn't exist in the version of Aurora Imaging Library from which the 3D image processing context is being loaded.	c		e	f	g	a		i		j	k	l	m	n	b
M_RESTORE	Restores a 3D image processing context from a file or memory stream and assigns it an Aurora Imaging Library identifier.	c		e	f	g	a		i		j	k	l	m	n	b
MemPtrOrFileName	Specifies the file or memory stream from which to restore the 3D image processing context.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b

AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "C:*mydirectory**myfile*"), when the StreamType parameter is set to M_FILE. 3D image processing contexts typically have an M3DIM file extension. The function handles (internally) the opening and closing of the file. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///*C:*mydirectory**myfile*").	c		e	f	g	a		i		j	k	l	m	n	b
MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which the object is located, and the block of memory must contain the entire object. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b
SysId	Specifies the system on which to allocate the 3D image processing context.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b
Version	This parameter must be set to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
Context3dimIdPtr	Specifies the address of the variable in which to write the identifier of the 3D image processing context. If the operation is not successful, **M_NULL** is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAVE	Saves a 3D image processing context to a specified file or memory stream.	c		e	f	g	a		i		j	k	l	m	n	b
MemPtrOrFileName	Specifies the file or memory stream.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "C:*mydirectory**myfile*"), when the StreamType parameter is set to M_FILE. For easier use with other Aurora Imaging software products, when saving a 3D image processing context to a file, use the M3DIM file extension. The function handles (internally) the opening and closing of the file. If the file already exists, it will be overwritten. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///*C:*mydirectory**myfile*").	c		e	f	g	a		i		j	k	l	m	n	b
MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which to write, and the block of memory must be large enough to stream the entire context. To determine the required size, call this function with M_INQUIRE_SIZE_BYTE. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b
SysId	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
Version	Specifies the Aurora Imaging Library version with which the 3D image processing context must be compatible when saved.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the current version of Aurora Imaging Library.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP4	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 4.	c		e	f	g	a		i		j	k	l	m	n	b

		M_PROC_VERSION_100_SP5	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 5.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROC_VERSION_100_SP6	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 6.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROC_VERSION_100_SP7	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 7.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROC_VERSION_110	Specifies the version as being Aurora Imaging Library 11.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Context3dimIdPtr		Specifies the address of the variable containing the identifier of the 3D image processing context to save.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dim11.lib.
DLL	Requires ail11.dll; ail3dim11.dll.

M3dimTranslate

Synopsis

Translate a specified point cloud or 3D geometry.

Syntax

```
void M3dimTranslate(
    AIL_ID SrcContainerBufOrGeometry3dgeoId, //in
    AIL_ID DstContainerBufOrGeometry3dgeoId, //out
    AIL_DOUBLE TranslationX,                 //in
    AIL_DOUBLE TranslationY,                 //in
    AIL_DOUBLE TranslationZ,                 //in
    AIL_INT64 ControlFlag                    //in
)
```

Description

This function applies the specified translation to the points in the source point cloud container or to the geometry in the source 3D geometry object. You can specify a translation (displacement) along the X, Y, and/or Z-axis of the working coordinate system, in world units.

If the source is a point cloud container, **M3dimTranslate**() applies the specified translation to the coordinates in the container's M_COMPONENT_RANGE component, and the modified component is added to the destination container. The source container's M_COMPONENT_CONFIDENCE component is copied to the destination container. If M_COMPONENT_REFLECTANCE, M_COMPONENT_NORMALS_AIL, and M_COMPONENT_MESH_AIL components exist in the source container, they are also copied to the destination container. Any previously existing reflectance, normals, and mesh components are removed from the destination container.

> Note that, when translating a point cloud, this function affects the coordinates of the points and not their storage location in the container.

Parameters

SrcContainerBufOrGeometry3dgeoId

Specifies a source point cloud container or 3D geometry object.

> Note that the source and destination must match so that both are containers or both are 3D geometry objects.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the source container or 3D geometry object identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_XY_PLANE	Specifies the XY (Z=0) plane.	c		e	f	g	a		i		j	k	l	m	n	b
Source 3D geometry object identifier	Specifies the identifier of the source 3D geometry object. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and must have been successfully defined.	c		e	f	g	a		i		j	k	l	m	n	b
Source container identifier	Specifies the identifier of the source container. The container must be 3D-processable (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE, the function returns M_PROCESSABLE). The container must have been previously allocated using MbufAllocContainer() with M_PROC.	c		e	f	g	a		i		j	k	l	m	n	b

DstContainerBufOrGeometry3dgeoId

Specifies the destination container or 3D geometry object.

> Note that the source and destination must match so that both are containers or both are 3D geometry objects.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the destination container or 3D geometry object identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

Destination 3D geometry object identifier	Specifies the identifier of the destination 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY.	c		e	f	g	a		i		j	k	l	m	n	b
Destination container identifier	Specifies the identifier of the destination container, previously allocated using MbufAllocContainer() with M_PROC. The destination container must not be a child container.	c		e	f	g	a		i		j	k	l	m	n	b

TranslationX

Specifies the displacement in the X-direction.

TranslationY

Specifies the displacement in the Y-direction.

TranslationZ

Specifies the displacement in the Z-direction.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dim11.lib.
DLL	Requires ail11.dll; ail3dim11.dll.